

智慧製造導論

Introduction to
Smart
Manufacturing

編著：李世淵
(機器人叫獸／ロボット教授)



TRADITION
MEETS
TECHNOLOGY
FOR A BRIGHTER
ISLAND

第四篇

營運、主線與學生 | 第 17-21 章

篇章導讀：第四篇 智慧工廠營運、數位主線與數位學生

第三篇從「被製造的東西」出發，談產品如何被定義、設計與做出來。第四篇把視角拉回工廠的營運：資訊如何在企業系統與現場之間流動，不同型態的工廠如何智慧化，以及如何為工廠、設備與產品建立數位學生，以數位主線把它們串起來，並以模擬讓工廠在實體存在之前就先被看見、先被驗證。本篇由五章組成，是全書從「製造」走向「數位製造」的樞紐。

本篇要回答的問題

1. 一家製造企業需要哪些資訊系統？ERP、MES、PLM、QMS 如何分工？ISA-95 如何定義企業與現場之間的介面？訂單、工單、配方、途程與主資料如何串成生產的資訊流？如何追溯一件產品實際是怎麼做出來的？
2. 離散型與流程型工廠有哪些根本差異？兩者的智慧化各有什麼重點？縱向、橫向與端到端整合是什麼？燈塔工廠的經驗告訴我們什麼？
3. 什麼才是真正的數位學生？產品、設備、製程、產線與工廠學生有何不同？ISO 23247 如何定義製造數位學生？學生需要多快的同步、多高的保真度，又如何估測看不見的狀態？
4. 如何以數位主線串連設計、規劃、生產與測試？多個學生如何組合？資產管理殼、知識圖譜與 OpenUSD 如何讓系統說同一種語言？如何讓模型值得信任，並治理模型與資料？
5. 離散事件、系統動態與代理人模擬各適合什麼問題？如何以虛擬試車在上線前找出控制程式的錯誤？虛擬量測、工業元宇宙與 Sim-to-Real 如何讓模擬產生實際價值？

五章的分工

第 17 章〈企業系統與製造營運管理〉說明工廠的資訊如何流動。本章介紹 ERP、SCM、CRM、PLM、MES / MOM 與 QMS 的分工，以程式實作 MRP 與產能需求規劃；說明 ISA-95 的層級、四類資訊與營運活動模型；討論訂單、工單、配方、途程與主資料治理；以批次族譜與雜湊鏈示範追溯與電子批記錄；最後以台灣半導體的虛擬晶圓廠為例，說明製造營運如何成為對客戶的服務。

第 18 章〈離散型與流程型智慧工廠〉比較兩種製造型態。本章以產品—製程矩陣說明製造型態

的選擇，以牌號切換、裝配線平衡與看板示範兩種工廠的典型問題；介紹流程工廠的 DCS、營運決策階層、模型預測控制的「壓縮與移位」與資料協調；說明縱向、橫向與端到端整合；最後比較電子、石化、鋼鐵與半導體的燈塔工廠，並以成本結構說明投資優先順序。

第 19 章〈製造數位孿生基礎〉釐清數位孿生的定義。本章以熱變位補償比較數位模型、數位影子與數位孿生，介紹產品、設備、製程、產線與工廠孿生及其組合，說明 ISO 23247 的參考架構與可觀測製造元素；討論同步頻率、資料新鮮度與適合用途的保真度，以卡爾曼濾波器估測刀具磨耗；最後以蒙地卡羅模擬評估孿生專案的商業價值與導入風險。

第 20 章〈數位主線、孿生組合與語意互通〉把孿生串起來並讓它可信。本章以有向圖實作工程變更的影響分析與向後追溯，介紹 2026 年發布的 ISO 23247 第 5、6 部分與共同模擬；以資產管理殼、知識圖譜與 OpenUSD 示範語意互通；以驗證、校正、不確定性量化與確認建立模型的可信度；最後談身分、版本、資料血緣與模型治理。

第 21 章〈模擬、虛擬試車、虛擬機台與工業元宇宙〉是本篇的收尾。本章比較四種模擬典範，以軟體在迴路示範 PLC 虛擬試車，以虛擬量測示範漂移修正與可信度檢查，以 OpenUSD 圖層組合檢查工廠佈置，並以領域隨機化說明 Sim-to-Real，展望 AI 驅動的孿生。



圖 IV-1 第四篇五章的學習主線

本篇在全書中的位置

如果說第二篇是工廠的肌肉與神經、第三篇是產品的數位基因，第四篇就是工廠的「數位分身」與「記憶」。對照第 5 章的參考架構，第 17 章對應 ISA-95 的第 3、4 層，第 19、20 章對應 RAMI

4.0 的資產管理殼與 ISO 23247，第 21 章則讓這些架構「動起來」。第 11 章的工業通訊與邊緣運算，是孿生同步的基礎；第 12、13 章的產品資料與模擬，是數位主線的上游。

本篇也為後面各篇鋪路：孿生與系統累積的資料，是第五篇製造資料科學與工業 AI 的原料；虛擬量測、預測維護與排程，將在第 25 到 27 章深入；知識圖譜與模型治理，是第六篇企業 AI 與代理式 AI 的基礎；燈塔工廠與數位供應鏈，則與第七、八篇的價值網路、資安與台灣案例相連。貫穿本篇的主題是「可信」：資料要正確、模型要經過驗證、決策要能追溯，這樣數位化的工廠才值得依靠。

閱讀建議

第四篇橫跨資訊管理、控制工程、建模與模擬，背景不同的讀者可以有不同的重點。工業工程與管理背景的讀者，第 17、18 章較為熟悉，可以把時間留給第 19 到 21 章的孿生與模擬；機械與控制背景的讀者，建議仔細閱讀第 17 章的 ISA-95 與主資料，這是孿生能否落地的關鍵；資訊與 AI 背景的讀者，則可從第 20 章的語意互通與模型治理切入，再回頭理解工廠的營運。課堂教學時建議安排五到六週，每章約一週，第 20 章可視需要延長。

實作方面，建議至少完成以下五項：第 17 章的 MRP 與批次族譜、第 18 章的裝配線平衡或資料協調、第 19 章的熱變位孿生與卡爾曼濾波、第 20 章的 AAS 語意查詢或 VVUQ、第 21 章的 PLC 虛擬試車。這五項串起來，正好走過「資訊流、工廠、孿生、主線、模擬」一條完整的智慧工廠營運路線。

對高中 AI 機器人社團而言，第 21 章的代理人模擬與虛擬試車最容易上手：只要一台電腦，就能模擬 AMR 車隊或輸送分流，並在虛擬環境中測試控制邏輯；第 19 章的熱變位孿生，也可以用 3D 印表機或小型馬達的溫度實作簡化版本。

讀完本篇後，建議你挑選一條你熟悉的產線或實驗室設備，試著回答：它的資訊從哪裡來、流到哪裡去？它的數位表示是模型、影子還是孿生？它的資料有沒有身分、版本與血緣？它的模型經過驗證與確認了嗎？在改變它之前，能否先在模擬中試一次？能回答這些問題，你就已經掌握了智慧工廠營運的核心能力。

表 IV-1 第四篇目錄

章	節
第 17 章 企業系統與製造營運管理	17.1 ERP、SCM、CRM、PLM、MES / MOM 與 QMS
	17.2 ISA-95 層級與企業—控制介面
	17.3 訂單、工單、配方、途程與主資料
	17.4 WIP、製造履歷、追溯與電子批記錄
	17.5 虛擬晶圓廠與製造服務案例
第 18 章 離散型與流程型智慧工廠	18.1 離散型與流程型製造的特徵與差異
	18.2 離散型智慧工廠：物理系統、資訊系統與供應鏈
	18.3 流程型智慧工廠：DCS、APC、資料協調與營運決策階層
	18.4 智慧集成製造系統：縱向、橫向與端到端整合
	18.5 燈塔工廠案例比較：電子、石化、鋼鐵、半導體
第 19 章 製造數位孿生基礎	19.1 數位模型、數位影子與數位孿生
	19.2 產品、設備、製程、產線與工廠孿生
	19.3 ISO 23247 參考架構
	19.4 即時同步、模型保真度與狀態估測
	19.5 平台生態、商業價值與導入風險
第 20 章 數位主線、孿生組合與語意互通	20.1 從設計、規劃、生產到測試的數位主線
	20.2 ISO 23247-5 數位主線與 23247-6 孿生組合
	20.3 AAS、知識圖譜與 OpenUSD 共同資料層
	20.4 驗證、確認、校正與不確定性量化
	20.5 身分、版本、資料血緣與模型治理
第 21 章 模擬、虛擬試車、虛擬機台與工業元宇宙	21.1 離散事件、物理、系統動態與代理人模擬

章	節
	21.2 SIL / MIL / HIL 與 PLC 虛擬試車
	21.3 虛擬機台與虛擬量測
	21.4 AR / VR、OpenUSD 與工業元宇宙
	21.5 合成資料、Sim-to-Real 與 AI 驅動孿生

第四篇 智慧工廠營運、數位主線與數位學生

第 17 章

企業系統與製造營運管理

編著 李世淵 (機器人叫獸) | 助教 李天宇、李宇晴

本章重點

- ERP、SCM、CRM、PLM、MES / MOM、QMS 的分工與發展
- MRP 計算、批量與前置時間倒推
- ISA-95 第 4 層與第 3 層的四類資訊與營運活動模型
- 訂單、工單、途程、配方、產能需求規劃與主資料治理
- WIP、製造履歷、批次族譜、電子批記錄與 21 CFR Part 11
- 虛擬晶圓廠、X 因子與製造即服務

叫獸開講：電腦說有五百個，倉庫一個也沒有

有一次，一家中型機械零件廠的總經理找我去看他們的「數位化」。他很自豪地說，公司導入 ERP 已經五年，也上了 MES，報表很漂亮。剛好那天下午，業務接到一位大客戶的電話，問一張急單什麼時候能出貨。

業務打開 ERP，系統顯示關鍵零件庫存有 500 個，於是答應客戶下週出貨。結果生管一查，倉庫裡那個零件一個也沒有。追查之後才知道：上個月有一批零件在產線上被判定不良報廢，MES 裡有紀錄，但報廢沒有回寫到 ERP；另外有兩張工單已經做完，卻因為現場人員忙，晚了一週才在系統上結案，所以耗用的料也沒有扣帳。ERP 裡的 500 個，其實是「帳上的庫存」，不是「架上的庫存」。

更麻煩的是，業務想知道急單在產線上走到哪裡，得先打電話給生管，生管再打電話問現場組長，組長走到機台旁邊看一看，再一路回報上來。一個簡單的問題，花了一個多小時才得到答案，而且答案還不太確定。

總經理苦笑：「我們花了很多錢買系統，為什麼最基本的問題還是答不出來？」

問題不在系統本身，而在於系統之間的「接縫」：ERP 與 MES 之間的資料什麼時候交換、交換什麼、誰負責、出錯了怎麼辦，沒有人設計清楚；主資料在兩個系統裡各維護一份，版本不一致；現場的實際狀況沒有即時、可靠地回到系統。後來，我們一起梳理了 ERP 與 MES 的介面，依照 ISA-95 定義每一種交換資訊的時機與內容，把報廢、完工與耗用改成在現場即時回報，並建立主資料的單一來源與審核流程。三個月後，業務可以在 ERP 上直接看到每張訂單在產線上的進度，帳上庫存與架上庫存的差異也降到可以接受的範圍。

這一章，我們要談企業的資訊系統：ERP、SCM、CRM、PLM、MES / MOM 與 QMS 各自負責什麼；ISA-95 如何定義企業與控制系統之間的介面；訂單、工單、配方、途程與主資料如何串成生產的資訊流；WIP、製造履歷、追溯與電子批記錄如何記錄「實際發生了什麼」；最後以台灣半導體產業的「虛擬晶圓廠」為例，看看製造營運如何延伸為對客戶的服務。

(說明：本節情境由作者協助企業導入系統的經驗綜合改寫，人物與細節已調整。)

學習目標

讀完本章後，你應該能夠：

1. 說明 ERP、SCM、CRM、PLM、MES / MOM、QMS 等企業系統的功能與分工，以及它們的發展脈絡。
2. 說明 MRP 的邏輯，並以程式計算淨需求、批量與前置時間倒推的計畫訂單。
3. 說明 ISA-95 第 3 層與第 4 層之間交換的資訊類別，以及第 3 層的營運活動模型，並能規劃 ERP 與 MES 的介面。
4. 區分訂單、工單、配方、途程與主資料，說明工單的生命週期，並以程式進行產能需求規劃與主資料一致性檢查。
5. 說明 WIP 追蹤、製造履歷與批次族譜，並以程式進行向前與向後追溯。
6. 說明電子批記錄與 21 CFR Part 11 等法規對電子紀錄的要求，並以雜湊鍵示範防竄改的稽核軌跡。
7. 以虛擬晶圓廠為例，說明製造營運資訊如何成為對客戶的服務。

核心概念

表 17-1 本章核心名詞中英對照

中文名詞	英文	一句話說明
企業資源規劃	Enterprise Resource Planning, ERP	整合財務、採購、銷售、庫存與生產規劃的企業核心系統
物料需求規劃	Material Requirements Planning, MRP	依主生產排程與 BOM 計算零件需求量與下單時間的方法
供應鏈管理	Supply Chain Management, SCM	規劃與協調從供應商到客戶的物流、資訊流與金流
客戶關係管理	Customer Relationship Management, CRM	管理客戶、商機、報價與服務的系統
製造執行系統	Manufacturing Execution System, MES	管理與記錄從工單下達到產品完成之間現場活動的系統

中文名詞	英文	一句話說明
製造營運管理	Manufacturing Operations Management, MOM	ISA-95 第 3 層的生產、品質、維護與庫存營運活動的總稱
品質管理系統	Quality Management System, QMS	管理檢驗、不合格、矯正預防與文件的系統
先進規劃與排程	Advanced Planning and Scheduling, APS	考慮產能與限制的規劃與排程系統
工單	Work Order / Production Order	指示在何時、以何資源、生產何種產品多少數量的指令
途程	Routing	產品製造所經過的工序、工作中心與標準工時
配方	Recipe	流程與批次製造中的原料、參數與步驟定義
主資料	Master Data	料號、BOM、途程、資源等相對穩定、被多個系統共用的基礎資料
在製品	Work in Process, WIP	已投入生產但尚未完成的產品
批次族譜	Lot Genealogy	記錄批次之間分批、合批與投料關係的資料
電子批記錄	Electronic Batch Record, EBR	以電子方式記錄批次製造過程與核准的紀錄

17.1 ERP、SCM、CRM、PLM、MES / MOM 與 QMS

17.1.1 企業為什麼需要這麼多系統

一家製造業公司每天要回答各式各樣的問題：客戶要什麼、什麼時候要（銷售）？要買哪些料、什麼時候到（採購）？要生產什麼、生產多少、什麼時候開始（生產規劃）？產線現在在做什麼、做得怎麼樣（製造執行）？產品合不合格、有沒有異常（品質）？產品怎麼設計、規格是什麼（研發）？賺了多少錢、成本多少（財務）？

每一類問題，都發展出專門的資訊系統。這些系統各有專長，但也各自掌握一部分資料；它們之間如何分工與整合，決定了企業能不能快速、正確地回答問題。叫獸開講中的工廠，就是系統之間的整合出了問題。

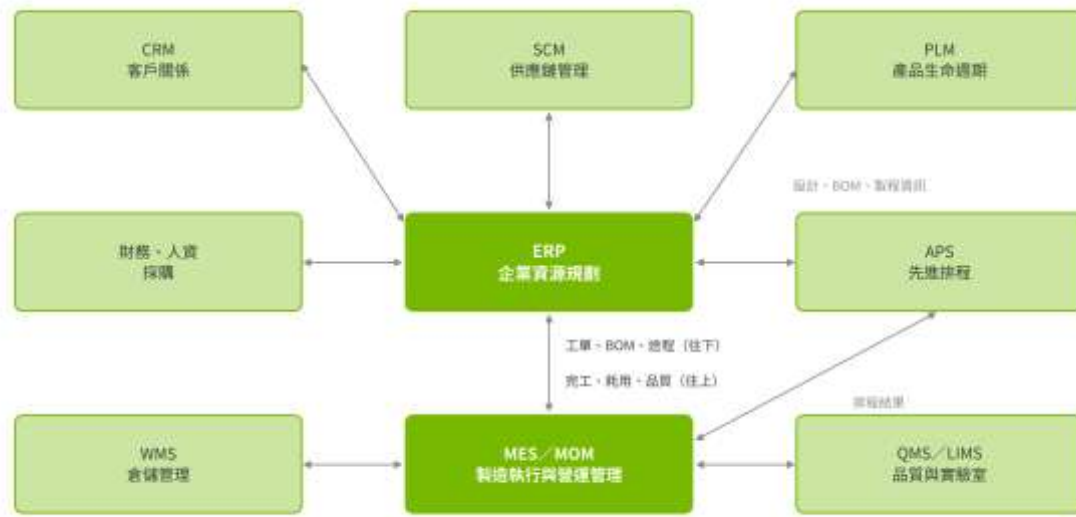


圖 17-1 製造企業的主要資訊系統與資料流

表 17-2 製造企業的主要資訊系統

系統	主要功能	主要使用者	時間尺度
ERP	銷售、採購、庫存、生產規劃、成本、財務	業務、採購、生管、財會	月、週、日
SCM	需求預測、供應規劃、物流與供應商協同	供應鏈、採購、物流	月、週
CRM	客戶資料、商機、報價、客服	業務、行銷、客服	日、週
PLM	產品資料、BOM、圖面、工程變更（第 12 章）	研發、工程	專案期間
APS	考慮產能與限制的規劃與排程（第 27 章）	生管	週、日、時
MES / MOM	派工、生產執行、資料蒐集、追蹤、績效	生管、現場、製造工程	班、時、分
QMS / LIMS	檢驗計畫、不合格品、矯正預防、實驗室資料	品保、實驗室	日、批
WMS	儲位、揀貨、出入庫（第 10 章）	倉儲	時、分

17.1.2 從 MRP 到 ERP

企業系統的發展，可以從生產規劃的演進看起：

- MRP（1960 至 1970 年代）：IBM 的 Joseph Orlicky 於 1964 年前後建立了物料需求規劃的基本原理，1975 年出版了第一本專門介紹 MRP 的著作，成為後來標準化 MRP 系統的藍圖。MRP 回答的是：依照主生產排程與 BOM，每一種零件在什麼時候需要多少、什麼時候要下單。

- MRP II (1980 年代) : 把 MRP 擴展為「製造資源規劃」, 加入產能規劃、車間控制與成本, 讓生產計畫與財務結合; 「MRP II」這個名稱一般歸功於顧問 Oliver Wight。
- ERP (1990 年代以後) : 一般認為「企業資源規劃」(ERP) 一詞由研究機構 Gartner 於 1990 年代初期提出, 用來描述整合財務、人資、銷售、採購、庫存與生產等企業全部功能的單一系統。隨著網際網路與雲端的發展, ERP 又陸續延伸出供應鏈、客戶關係與電子商務等模組, 並逐漸走向雲端服務。

在 ERP 的發展過程中, 大家發現它擅長「計畫」, 卻不擅長「執行」: ERP 以天或週為單位規劃, 但現場的狀況以分鐘為單位變化。於是, 介於 ERP 與現場控制之間的 MES 逐漸受到重視。1992 年成立的 MESA 國際組織推動 MES 的定義與推廣, 並於 1997 年提出 MES 的 11 項功能模型 (常稱 MESA-11 ; 2004 年再擴充為協同式 MES , 2008 年延伸到企業營運與策略層面) , 包括資源配置、詳細排程、派工、文件控制、資料蒐集、人員管理、品質管理、製程管理、維護管理、產品追蹤與族譜、績效分析。2005 年發布的 ISA-95 第 3 部分把這一層的活動擴大定義為「製造營運管理」(MOM) , 涵蓋生產、品質、維護與庫存四大領域, MES 則常被視為其中與生產最相關的核心。

17.1.3 MRP 的邏輯

MRP 是理解 ERP 生產規劃的鑰匙。它的計算可以分成幾個步驟:

- 毛需求: 成品的毛需求來自主生產排程; 零件的毛需求, 則來自母件的計畫下單量乘以 BOM 用量 (第 12 章的 BOM 展開) 。
- 淨需求: 毛需求減去現有庫存與預計收料 (已下單、尚未到貨) , 若還不夠, 就是淨需求。可以再加上安全庫存。
- 批量規則: 淨需求依批量規則調整為計畫收料量, 例如逐批 (需要多少買多少) 或固定批量 (每次一定買 250 個) 。
- 前置時間倒推: 計畫收料的那一週, 減去前置時間, 就是計畫下單的那一週。

下面以一個簡單的例子示範: 成品 X 在第 3、5、7 週各需要 100、150、120 個, 現有庫存 40 個, 前置時間 1 週; 每個 X 需要 2 個零件 C, C 的現有庫存 120 個, 第 2 週會有一筆 100 個的預計收料, 前置時間 2 週, 採購批量固定 250 個:

```

import math
# 物料需求規劃 (MRP)：成品 X 的主生產排程，展開到零件 C 的計畫訂單
weeks = range(1, 9)
mps_X = {3: 100, 5: 150, 7: 120}          # 成品 X 各週的需求 (主生產排程)
bom = {"C": 2}                             # 每個 X 需要 2 個 C
lead = {"X": 1, "C": 2}                    # 前置時間 (週)
def mrp(gross, on_hand, sched, lt, lot=None, ss=0):
    """逐週計算淨需求，依批量規則產生計畫訂單收料，並依前置時間倒推下單週"""
    rows, inv = [], on_hand
    receipts, releases = {}, {}
    for w in weeks:
        inv += sched.get(w, 0)
        need = gross.get(w, 0) + ss - inv
        rec = 0
        if need > 0:
            rec = need if lot is None else math.ceil(need / lot) * lot # 逐批或固定批量
            receipts[w] = rec; releases[w - lt] = rec
        inv = inv + rec - gross.get(w, 0)
        rows.append((w, gross.get(w, 0), sched.get(w, 0), max(need, 0), rec, inv))
    return rows, releases

rows_X, rel_X = mrp(mps_X, on_hand=40, sched={}, lt=lead["X"])
gross_C = {w: q * bom["C"] for w, q in rel_X.items()} # 子件的毛需求 = 母件下單量 × 用量
rows_C, rel_C = mrp(gross_C, on_hand=120, sched={2: 100}, lt=lead["C"], lot=250)
print("X 的計畫下單:", rel_X)
print("C 的毛需求:", gross_C)
print("C 的計畫下單:", rel_C)
for r in rows_C: print("C 第 %d 週: 毛需求 %d、預計收料 %d、淨需求 %d、計畫收料 %d、期末庫存 %d" % r)

```

執行結果：X 在第 3 週需要 100 個，扣掉庫存 40 個後淨需求 60 個，前置時間 1 週，所以第 2 週要下單 60 個；第 5、7 週的需求則分別在第 4、6 週下單 150、120 個。這些母件的下單量乘以 2，就成為 C 在第 2、4、6 週的毛需求 120、300、240 個。

表 17-3 零件 C 的 MRP 計算結果

週	毛需求	預計收料	淨需求	計畫收料	期末庫存
1	0	0	0	0	120
2	120	100	0	0	100
3	0	0	0	0	100
4	300	0	200	250	50
5	0	0	0	0	50
6	240	0	190	250	60
7	0	0	0	0	60
8	0	0	0	0	60

C 在第 2 週有庫存 120 個加預計收料 100 個，足夠應付 120 個的需求；第 4 週淨需求 200 個，依固定批量收料 250 個，倒推前置時間 2 週，第 2 週要下單；第 6 週同理，第 4 週要下單。

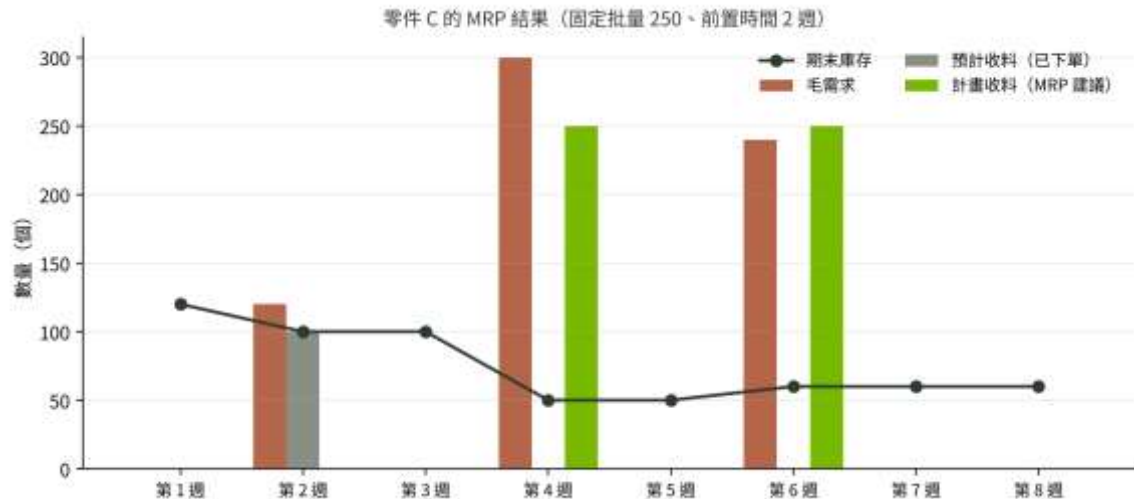


圖 17-2 零件 C 的 MRP 結果

這個例子也說明了 MRP 的幾個重要假設與限制：它假設 BOM、庫存、前置時間與預計收料都是正確的，資料錯一點，計畫就錯很多（叫獸開講中帳上庫存的錯誤，會直接造成 MRP 少下單）；它假設產能是無限的，不檢查工作中心是否忙得過來（17.3 節的產能需求規劃）；前置時間是固定的，不會隨負荷變化（第 8 章說過，負荷越高，等待時間越長）。這些限制，正是 APS 與 MES 要補足的地方，第 27 章的智慧排程將進一步討論。

17.1.4 MES / MOM 做什麼

如果說 ERP 回答「要做什麼、做多少、什麼時候完成」，MES 則回答「現在正在怎麼做、做得怎麼樣」。MES 的典型功能包括：

- 工單管理與派工：接收 ERP 的工單，依排程把工作派到工作站，並管理工單的狀態。
- 作業指導：在工作站顯示作業步驟、圖面、參數與注意事項。
- 資料蒐集：自動或手動蒐集生產數量、良品與不良、工時、設備狀態、製程參數（第 11 章）。
- 物料與 WIP 追蹤：記錄每一批或每一件在哪個工站、用了哪些料、經過了哪些步驟（17.4 節）。

- 品質管理：執行檢驗計畫、記錄檢驗結果、管制不合格品，並觸發 SPC（第 25 章）。
- 設備與維護：記錄設備狀態與停機原因，計算 OEE（第 6 章），觸發維護工單（第 26 章）。
- 績效分析：產出、良率、週期時間、準時率等指標。

17.1.5 系統整合的挑戰

企業系統的價值，很大一部分取決於它們之間的整合。常見的整合問題包括：

- 主資料不一致：同一個料號在 ERP、PLM、MES 中有不同的版次、單位或描述（17.3.5 節）。
- 交易時機不對：工單完工、耗用、報廢沒有即時回報，造成帳實不符。
- 責任不清：同一筆資料由誰建立、誰維護、誰負責修正不明確。
- 介面脆弱：點對點的客製介面在系統升級時容易斷掉（第 11 章的統一命名空間正是為此而生）。
- 過度客製化：為了配合既有作業習慣大量修改套裝軟體，使升級與維護變得困難。

解決這些問題，靠的不只是技術，更需要清楚的架構與分工。下一節的 ISA-95，就是定義這種分工的國際標準。

17.2 ISA-95 層級與企業—控制介面

17.2.1 從架構到介面

第 5 章介紹了 Purdue 模型與 ISA-95 的層級：第 4 層是企業的業務規劃與物流，第 3 層是製造營運管理，第 2 層以下是控制系統與現場。那時我們關心的是「架構」：哪些功能屬於哪一層。本節則聚焦在「介面」：第 4 層與第 3 層之間、第 3 層與第 2 層之間，要交換哪些資訊，以及第 3 層內部有哪些活動。這些正是 ERP 與 MES 整合時最需要回答的問題。

ISA-95（國際標準為 IEC 62264）由多個部分組成：第 1 部分（2000 年）定義模型與術語；第 2 部分（2001 年）定義物件模型的屬性；第 3 部分（2005 年）定義製造營運管理的活動模型；第 4 部分定義製造營運管理整合的物件與屬性，第 5 部分定義企業與製造之間的交易（兩者現行版本皆為 2018 年）；之後的部分則處理訊息服務與資訊交換的設定檔。為了讓不同軟體廠商可以實作，MESA

國際組織維護了以 XML Schema 表達 ISA-95 資料模型的 B2MML (Business To Manufacturing Markup Language) 實作規格。



圖 17-3 ISA-95 的層級與各層之間的資訊交換

17.2.2 第4層與第3層之間的四類資訊

ISA-95 把第4層 (ERP) 與第3層 (MOM) 之間交換的資訊，歸納為四大類：

表 17-4 ISA-95 第4層與第3層之間交換的資訊

資訊類別	方向	內容	例子
產品定義	往下	如何生產一個產品：所需的物料、設備、人員與步驟	BOM、途程、配方、作業規範
生產能力	往上	可用的資源與產能：設備、人員、物料的狀態與可用性	機台可用時數、人員資格、可用庫存
生產排程	往下	要生產什麼、多少、何時	工單、生產計畫、優先順序
生產績效	往上	實際生產了什麼、用了什麼	完工數量、良品與不良、物料耗用、工時

這四類資訊構成一個循環：ERP 提供產品定義與生產排程，MOM 回報生產能力與生產績效，ERP 再依據實際績效修正下一輪的計畫。叫獸開講中的問題，就是「生產績效」回報得太慢、不完整：報廢與耗用沒有即時回到 ERP，導致 ERP 對「生產能力」(可用庫存) 的認知錯誤。

ISA-95 對同樣的資訊類別，也區分了生產、維護、品質與庫存四個營運領域，例如「維護排程」與「維護績效」、「品質檢驗定義」與「品質檢驗結果」。這讓 ERP 與 MOM 之間的介面，不只限於生產工單，也涵蓋維護工單、檢驗計畫與庫存移轉。

17.2.3 第 3 層的營運活動模型

ISA-95 第 3 部分以活動模型描述第 3 層內部的工作。以生產營運為例，包括以下八項活動：

表 17-5 ISA-95 第 3 部分的生產營運活動

活動	說明	本書相關章節
產品定義管理	管理生產規則、配方與作業指示，並與 PLM、ERP 同步	第 12、13 章
生產資源管理	管理設備、人員、物料的能力與狀態	第 9、10 章
詳細生產排程	把 ERP 的生產排程轉換為詳細的作業順序	第 27 章
生產派工	依詳細排程把工作分派到設備與人員	第 27 章
生產執行管理	確認工作依指示執行，處理異常	本章、第 10 章
生產資料蒐集	從設備與人員蒐集生產資料	第 11 章
生產追蹤	追蹤物料與產品的流動與狀態，彙整成績效回報 ERP	本章 17.4 節
生產績效分析	分析產出、效率、品質等指標並提出改善	第 6、22、25 章

品質、維護與庫存營運也套用同一組八項通用活動，例如品質營運包括品質檢驗定義管理、品質資源管理、詳細檢驗排程、檢驗派工、檢驗執行管理、品質資料蒐集、品質追蹤與品質績效分析；這八項活動首尾相接，績效分析的結果再回饋到定義與排程，形成閉環。這些活動模型的價值，在於提供一份「該做哪些事」的清單：導入 MES 或 MOM 時，可以逐項檢查哪些活動由哪一套系統負責、哪些仍靠紙本或人工，以及活動之間的資料如何流動。

17.2.4 設計 ERP 與 MES 的介面

依據 ISA-95 規劃 ERP 與 MES 的介面時，需要回答以下問題：

表 17-6 ERP 與 MES 介面的設計要點

問題	說明	常見的選擇
系統紀錄的主人	每一類資料以哪個系統為準	料號與 BOM 以 PLM / ERP 為準；製程參數以 MES 為準；實際生產紀錄以 MES 為準
交換的時機	即時、定時批次或事件觸發	工單下達在排程確定時；完工與耗用在作業完成時即時回報；績效彙總每班或每日
交換的顆粒度	以工單、工序、批次或單件為單位	ERP 多以工單與工序為單位；MES 內部可細到單件
錯誤處理	傳送失敗、資料不符時如何處理	重送機制、例外佇列、人工審核與對帳報表
變更管理	工單數量、BOM 版次在執行中變更時	鎖定已開工工單的版次，變更以新工單或工程變更處理（第 12 章）
整合方式	點對點、中介軟體或訊息中樞	以標準化的訊息與中介軟體，逐步走向第 11 章的統一命名空間

一個實用的原則是「讓資料在發生的地方被記錄，並在需要的地方被看見」：現場的事件（完工、報廢、停機）應該在現場被記錄於 MES，並即時或準即時地回報 ERP；計畫的變更（插單、取消）應該在 ERP 決定，並即時下達 MES。介面的設計目標，不是讓兩個系統的資料完全相同，而是讓每一類資料都有清楚的「主人」，並在需要的時間內傳到需要的地方。

17.2.5 第 3 層與第 2 層之間

第 3 層與第 2 層之間的介面，是 MES 與 PLC、SCADA、DCS 的連接。往下傳的是工單的派工、配方與設定值；往上傳的是實際值、事件、警報與設備狀態。這一層的技術，第 10 章的控制階層與第 11 章的 OPC UA、MQTT 已經詳細介紹。本章要補充的是一個管理上的重點：設定值與配方的下載，必須受到版本控制與權限管理；任何人在設備端修改的參數，都應該被記錄並回報 MES，否則系統中的「標準參數」與設備實際使用的參數可能不一致，追溯時就會得到錯誤的結論。

17.3 訂單、工單、配方、途程與主資料

17.3.1 從訂單到工單

生產的資訊流，從客戶的訂單開始。依據產品與市場的特性，企業採用不同的接單生產策略：

表 17-7 常見的生產策略

策略	說明	典型產品	系統的重點
存貨生產 (MTS)	依預測生產，客戶從庫存取貨	標準零件、消費品	需求預測、庫存管理
接單生產 (MTO)	接到訂單才開始生產	工具機、客製零件	交期承諾、產能規劃
接單組裝 (ATO)	預先生產模組，接單後組裝	電腦、選配車輛	模組庫存、組態 (第 12 章)
接單設計 (ETO)	接單後才設計與生產	專用設備、大型機械	專案管理、設計與生產並行

不論哪一種策略，訂單最終都要轉換為工單（也稱製令或生產訂單）。工單是指示生產的指令，至少包含：要生產的料號與版次、數量、預計開工與完工時間、使用的 BOM 與途程版次，以及所屬的訂單或計畫。

工單有自己的生命週期，常見的狀態包括：計畫、已下達、已備料、生產中、暫停、完工、結案。每一次狀態的轉換，都對應到 ERP 與 MES 之間的資訊交換。例如工單從「計畫」轉為「已下達」時，ERP 把工單傳給 MES；工單「完工」時，MES 回報完工數量與耗用，ERP 據以入庫與計算成本；「結案」之後，工單的成本與差異才算確定。叫獸開講中延遲結案的工單，就讓庫存與成本都停留在錯誤的狀態。

17.3.2 途程與工作中心

途程定義了一個產品要經過哪些工序、在哪個工作中心、需要多少時間。一筆途程資料通常包括：工序序號、工作中心、換線（準備）時間、每件加工時間、所需的工具與治具、檢驗點。工作中心則是一組能力相同的設備或人員，是產能規劃的基本單位。

有了工單與途程，就可以計算每個工作中心的負荷，這就是產能需求規劃（Capacity Requirements Planning, CRP）。它補足了 MRP 假設產能無限的缺點：

```
# 產能需求規劃：依工單與途程計算各工作中心的負荷（小時）
routing = { # 料號 → [(工序, 工作中心, 換線時間 h, 每件工時 min)]
    "X": [(10, "CNC", 1.0, 6.0), (20, "研磨", 0.5, 3.0), (30, "組裝", 0.2, 4.0)],
    "Y": [(10, "CNC", 1.5, 9.0), (20, "組裝", 0.2, 5.0)],
}
orders = [("WO-001", "X", 120), ("WO-002", "Y", 80), ("WO-003", "X", 150),
          ("WO-004", "Y", 60), ("WO-005", "X", 200)] # WO-005 為插單
capacity = {"CNC": 2 * 40 * 0.85, "研磨": 40 * 0.85, "組裝": 2 * 40 * 0.9} # 機台數 × 40
h × 稼動率
load = {wc: 0.0 for wc in capacity}
for wo, item, qty in orders:
    for op, wc, setup, run in routing[item]:
        load[wc] += setup + qty * run / 60
for wc in capacity:
    u = load[wc] / capacity[wc]
    print(f"{wc}: 負荷 {load[wc]:.1f} h / 可用 {capacity[wc]:.1f} h, 負荷率 {u:.0%}" + ' ← 超載'
          ' if u > 1 else ''")
```

執行結果：在原本的四張工單下，CNC 的負荷是 53 小時，可用產能 68 小時（2 台 × 40 小時 × 稼動率 85%），負荷率 78%；加入 200 件的插單 WO-005 後，CNC 增加 $1 + 200 \times 6 \div 60 = 21$ 小時，負荷達 74 小時、負荷率 109%，超載了；研磨與組裝的負荷率分別為 74% 與 61%，仍有餘裕。

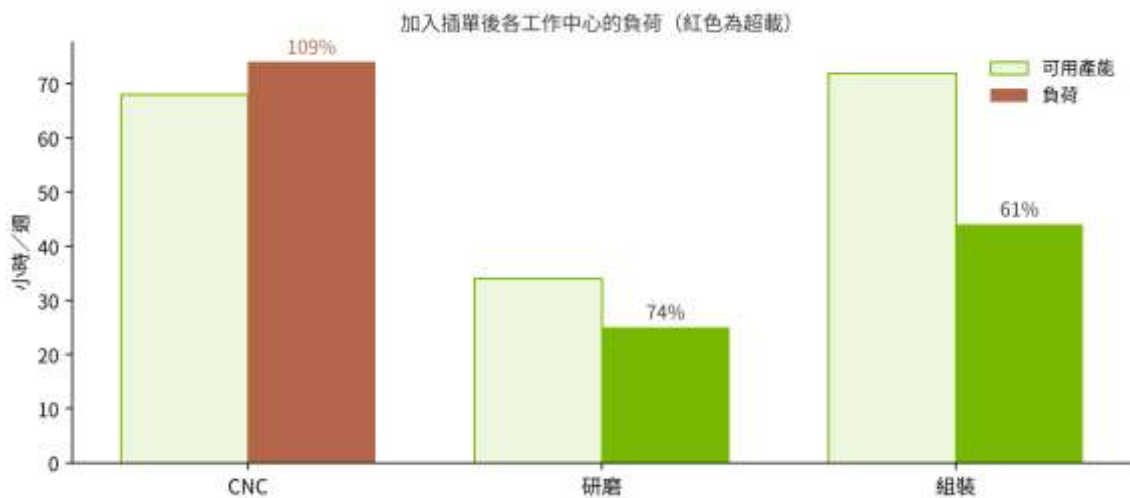


圖 17-4 加入插單後各工作中心的負荷

面對超載，生管可以選擇：加班或增加班次、把部分工作外包、把插單或其他工單往後延、或者把部分工序轉到其他機台。產能需求規劃讓這個決策有了數字的依據，而第 8 章的教訓也提醒我們：負荷率接近 100% 時，等待時間會急遽增加，所以實務上不應該把規劃負荷排到 100%。這類考慮產能限制的排程問題，將在第 27 章深入討論。

17.3.3 配方：流程與批次製造的「途程」

在化工、食品、製藥、半導體等流程與批次製造中，生產的定義常以「配方」表示：用什麼原料、多少比例、經過哪些步驟、每一步的溫度、壓力、時間等參數。ISA-88 批次控制標準 (ISA 於 1995 年核定第 1 部分，1997 年被採納為 IEC 61512-1，2010 年改版) 把配方分為四個層級：

- 通用配方：與設備無關，描述產品的化學或物理處理過程，通常由研發定義。
- 廠區配方：依廠區的原料與條件調整後的配方。
- 主配方：針對特定製程單元與設備類型的配方，可以直接用於生產。
- 控制配方：主配方針對某一批次實例化後的版本，包含實際的批量與設備，是真正被執行的配方。

這種分層的好處是：研發定義的「怎麼做」與工廠實際的「在哪裡做」分開管理，同一個通用配方可以在不同廠區、不同設備上展開。第 18 章將進一步比較離散型與流程型製造在資訊系統上的差異。半導體的「製程配方」(recipe) 則多半指設備上的製程參數組合，由製程工程師管理，並受到嚴格的版本與權限控制。

17.3.4 主資料：一切的基礎

主資料是相對穩定、被多個系統與交易共用的基礎資料，例如料號主檔、BOM、途程、配方、工作中心、設備、人員資格、客戶與供應商。交易資料則是日常營運產生的資料，例如訂單、工單、出入庫與生產紀錄。

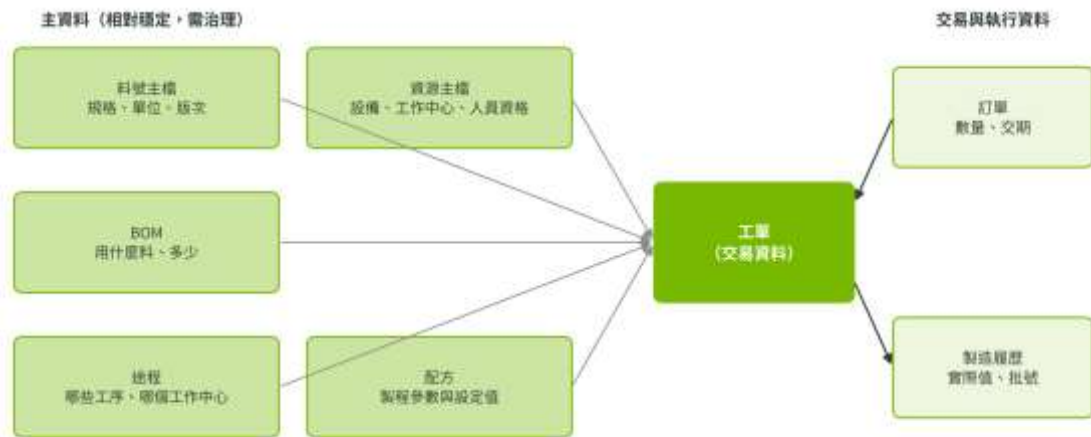


圖 17-5 主資料與交易資料的關係

主資料的品質，決定了所有交易的品質。料號單位錯了，每一次耗用都錯；途程工時錯了，每一次排程與成本都錯；BOM 版次錯了，每一次備料都錯。而且，主資料的錯誤往往「靜靜地」存在，直到某一天造成缺料、停線或錯誤的報價才被發現。

17.3.5 主資料治理

主資料治理的目標，是確保每一筆主資料都有明確的來源、負責人、審核流程與品質指標。一個常見的問題是同一份主資料在多個系統中各自維護，例如 PLM、ERP 與 MES 各有一份 BOM 與途程，久而久之就不一致了。下面的程式比對 ERP 與 MES 中同一料號的版次與單位：

```
# 主資料一致性檢查：比對 ERP 與 MES 中同一料號的 BOM 與途程版次
erp = {"X": {"bom_rev": "C", "routing_rev": "3", "uom": "PCS"},
        "Y": {"bom_rev": "B", "routing_rev": "2", "uom": "PCS"},
        "Z": {"bom_rev": "A", "routing_rev": "1", "uom": "KG"}}
mes = {"X": {"bom_rev": "C", "routing_rev": "3", "uom": "PCS"},
        "Y": {"bom_rev": "A", "routing_rev": "2", "uom": "PCS"},
        "Z": {"bom_rev": "A", "routing_rev": "1", "uom": "G"}}
issues = []
for item in sorted(set(erp) | set(mes)):
    if item not in mes or item not in erp:
        issues.append((item, "只存在於一個系統")); continue
    for field in erp[item]:
        if erp[item][field] != mes[item][field]:
            issues.append((item, f"{field}: ERP={erp[item][field]}, MES={mes[item][field]}"))
for i in issues: print("不一致:", *i)
print(f"共 {len(erp)} 個料號, {len([i[0] for i in issues])} 個有差異")
```

執行結果找出兩個問題：料號 Y 的 BOM 版次在 ERP 是 B、在 MES 還是 A，可能是工程變更只更新了 ERP；料號 Z 的單位在 ERP 是公斤、在 MES 是公克，每一次耗用回報都可能差了一千倍。這

類檢查應該定期自動執行，並把差異列入主資料品質的管理指標。

更根本的做法，是建立「單一來源」的原則：每一類主資料只在一個系統中維護（例如料號與 BOM 以 PLM 為源頭、經 ERP 發布；設備主檔以 MES 為源頭），其他系統透過介面同步，並且不允許在下游系統直接修改。第 12 章的工程變更流程、第 20 章的數位主線，都建立在這個原則之上。

17.4 WIP、製造履歷、追溯與電子批記錄

17.4.1 WIP：產線上的「現在」

在製品 (WIP) 是已經投入生產、尚未完成的產品。WIP 追蹤回答的問題是：每一張工單、每一批或每一件產品，現在在哪裡、做到哪一道工序、狀態如何（加工中、等待、暫停、待檢、重工）？叫獸開講中業務一路打電話問進度，就是缺少可靠的 WIP 追蹤。

WIP 追蹤的顆粒度依產業而不同：

- 以工單或批次追蹤：同一批產品一起移動、一起記錄，適合大量生產與流程產業，例如半導體以晶圓盒中的一批晶圓為單位。
- 以單件序號追蹤：每一件產品都有唯一的序號，記錄自己的歷程，適合高價值、需要個別追溯的產品，例如汽車、工具機、醫療器材與航太零件（第 12 章的組態管理、第 16 章的金屬積層件）。

WIP 的記錄方式，從人工報工、條碼與 RFID 掃描（第 11 章），到設備自動回報，越往後越即時、越可靠。WIP 資料也是許多管理指標的基礎：第 8 章的利特爾定律告訴我們，WIP、產出率與週期時間三者息息相關，WIP 過高通常意味著週期時間過長；掌握 WIP 的分布，就能看出瓶頸在哪裡。

17.4.2 製造履歷

製造履歷記錄一件或一批產品「實際上是怎麼做出來的」：用了哪些原料批號、經過哪些工序、在哪台設備上、由誰操作、使用哪一版的配方與參數、量測與檢驗結果如何、發生過哪些異常與處置。製造履歷的用途包括：

- 品質分析：比較良品與不良品的歷程，找出差異（第 22、25 章）。
- 客訴與召回：產品出問題時，迅速界定原因與影響範圍。
- 法規與客戶要求：醫藥、食品、汽車、航太等產業要求保存完整的生產紀錄。
- 持續改善：累積的製造履歷是資料分析與 AI 模型的原料。

製造履歷與第 12 章的「製造組態」（As-Built）是同一件事的兩個角度：製造組態著重於產品由哪些零件組成，製造履歷則還包括過程與條件。

17.4.3 批次族譜與雙向追溯

在流程與批次製造中，批次之間常有複雜的關係：一批原料分成幾批投入不同的工單（分批）；幾批半成品合成一批（合批）；一批半成品又分裝成許多成品。記錄這些關係的資料稱為批次族譜。有了批次族譜，就能做第 10 章介紹過的雙向追溯：

- 向前追溯：某一批原料有問題，它最終進入了哪些成品？這決定了召回的範圍。
- 向後追溯：某一件成品出問題，它用了哪些批次與原料？這是找出根本原因的起點。

批次族譜可以看成一張有向圖，追溯就是在圖上搜尋：

```
from collections import deque
# 批次族譜：每個邊代表「父批 → 子批」（分批、合批、投料）
edges = [("粉末 P1", "LotA"), ("粉末 P2", "LotB"), ("LotA", "LotA-1"), ("LotA", "LotA-2"),
         ("LotA-2", "LotC"), ("LotB", "LotC"), ("LotA-1", "成品 S1001"), ("LotA-1", "成品 S1002"),
         ("LotC", "成品 S1003"), ("LotC", "成品 S1004"), ("LotC", "成品 S1005")]
down, up = {}, {}
for p, c in edges:
    down.setdefault(p, []).append(c); up.setdefault(c, []).append(p)

def trace(start, graph):
    """廣度優先搜尋：找出所有可到達的節點"""
    seen, q = set(), deque([start])
    while q:
        n = q.popleft()
        for m in graph.get(n, []):
            if m not in seen:
                seen.add(m); q.append(m)
    return sorted(seen)

print("向前追溯（粉末 P1 影響哪些成品）：", [n for n in trace("粉末 P1", down) if n.startswith("成品")])
print("向後追溯（成品 S1004 用了哪些材料與批次）：", trace("成品 S1004", up))
```

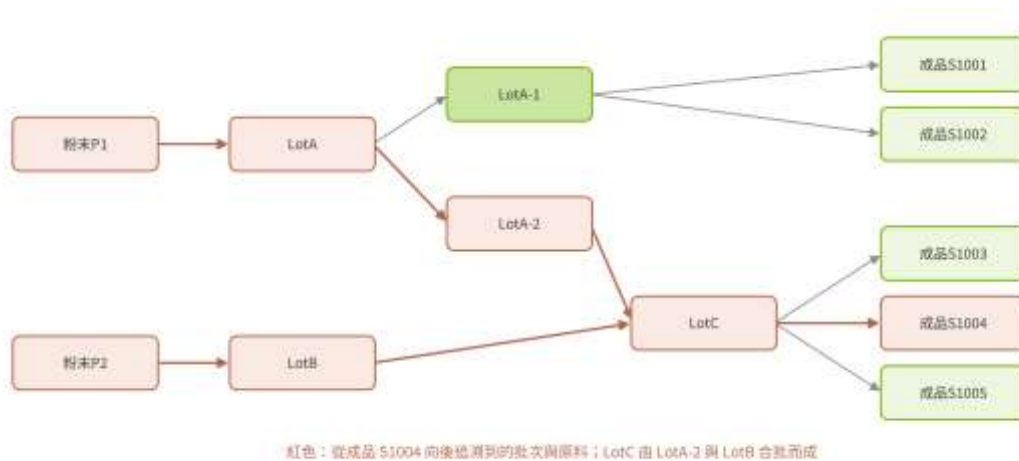


圖 17-6 批次族譜與向後追溯

執行結果：粉末 P1 經過 LotA 分成 LotA-1 與 LotA-2，LotA-2 又與 LotB 合批成 LotC，因此 P1 影響了全部五件成品 S1001 到 S1005；向後追溯成品 S1004，則找到 LotC、LotA-2、LotA、LotB 以及兩批原料粉末 P1 與 P2。這個例子顯示，合批會讓追溯的範圍擴大：如果 P1 有問題，連原本只用 P2 的 LotB 所製成的產品，也因為合批而受到影響。因此，有些產業會限制合批，或在合批時保留更細的紀錄，以縮小未來可能的召回範圍。

17.4.4 電子批記錄

在製藥、生技、食品與醫療器材等受高度管制的產業，每一批產品都必須有完整的批記錄：每一個步驟由誰在何時執行、使用什麼原料與設備、量測值多少、是否有偏差、由誰複核與核准。傳統上這些紀錄以紙本完成，一批產品可能有數百頁，填寫、複核與保存都非常耗時，而且常有漏填、筆誤與事後補寫的問題。

電子批記錄 (EBR) 把批記錄數位化，並與 MES 整合，帶來許多好處：

- 防錯：系統依配方引導操作，自動檢查數值是否在規格內，未完成前一步就無法進行下一步。
- 自動擷取：秤重、溫度等數值直接從設備讀取，避免抄寫錯誤 (第 11 章)。
- 即時可見：主管與品保可以即時看到每一批的進度與偏差。
- 例外審查：品保不必逐頁審查，只需審查系統標示出的偏差與例外，大幅縮短放行時間。
- 檢索與分析：批記錄可以被搜尋與分析，支援第 25 章的品質改善。

17.4.5 電子紀錄的可信度：21 CFR Part 11 與稽核軌跡

電子紀錄容易修改，這既是優點也是風險。為了確保電子紀錄與電子簽章的可信度，美國食品藥物管理局於 1997 年 8 月 20 日施行了 21 CFR Part 11 (電子紀錄與電子簽章)，歐盟的 GMP 規範也有對應的附則 11 (電腦化系統，現行版本於 2011 年發布；2025 年公布的修訂草案大幅強化資安、存取控制與稽核軌跡的要求)。這些規範的核心要求包括：

- 系統確效：證明系統在整個生命週期中準確、可靠地運作。
- 存取控制：每位使用者有唯一的帳號，依職務授予最小權限。
- 稽核軌跡：系統自動記錄誰在何時建立、修改或刪除了紀錄，以及修改前後的值；稽核軌跡不能被關閉或修改，並與紀錄保存同樣久。
- 電子簽章：每個簽章唯一對應一個人，顯示姓名、時間與簽章的意義 (執行、複核、核准)，並與紀錄連結，無法複製到其他紀錄。

稽核軌跡的關鍵是「防竄改」：任何修改都應留下痕跡。下面以雜湊鏈示範一種防竄改的設計：每一筆紀錄都包含前一筆紀錄的雜湊值 (以 SHA-256 計算的數位指紋)，一旦有人修改了某一筆紀錄的內容，它的雜湊值就會改變，與下一筆紀錄中保存的「前一筆雜湊」對不上，竄改立刻被發現：

```
import hashlib, json
# 電子批記錄的稽核軌跡：每筆紀錄包含前一筆的雜湊值，形成「雜湊鏈」，任何竄改都會被發現
def add(chain, record):
    prev = chain[-1]["hash"] if chain else "0" * 64
    body = json.dumps(**record, "prev": prev, ensure_ascii=False, sort_keys=True)
    chain.append(**record, "prev": prev, "hash":
hashlib.sha256(body.encode()).hexdigest())

def verify(chain):
    prev = "0" * 64
    for i, r in enumerate(chain):
        rec = {k: v for k, v in r.items() if k not in ("prev", "hash")}
        body = json.dumps(**rec, "prev": prev, ensure_ascii=False, sort_keys=True)
        if r["prev"] != prev or hashlib.sha256(body.encode()).hexdigest() != r["hash"]:
            return f"第 {i} 筆紀錄驗證失敗"
        prev = r["hash"]
    return "全部紀錄完整"

ebr = []
add(ebr, {"t": "2026-10-05T08:00", "step": "秤料", "value": "原料 A 25.02 kg", "by":
"OP017", "sign": "執行"})
add(ebr, {"t": "2026-10-05T08:20", "step": "複核", "value": "原料 A 25.02 kg", "by":
"QA003", "sign": "確認"})
add(ebr, {"t": "2026-10-05T09:10", "step": "混合", "value": "轉速 60 rpm、20 min", "by":
```

```
"OP017", "sign": "執行"}}
print(verify(ebr))
ebr[0]["value"] = "原料 A 24.50 kg"           # 事後偷偷修改第一筆
print(verify(ebr))
```



圖 17-7 以雜湊鏈保護稽核軌跡

執行結果：三筆紀錄建立後驗證為「全部紀錄完整」；把第一筆的秤料重量從 25.02 公斤改成 24.50 公斤後，驗證立刻指出「第 0 筆紀錄驗證失敗」。要注意，雜湊鏈能「偵測」竊改，但不能「防止」竊改；有權限的人若重新計算整條鏈的雜湊，仍然可能偽造，因此實務上還會把雜湊值定期保存到另一個獨立的系統或時間戳記服務，並搭配權限控管與系統確效。區塊鏈技術在供應鏈追溯中的應用，本質上也是這個想法的延伸（第 33 章）。

17.5 虛擬晶圓廠與製造服務案例

17.5.1 晶圓代工的資訊挑戰

台灣的晶圓代工模式，讓 IC 設計公司不必自建晶圓廠，就能把設計交給代工廠製造。但這也帶來資訊上的挑戰：客戶的產品在別人的工廠裡生產，他們需要知道自己的晶圓走到哪裡、良率如何、什麼時候能出貨，才能規劃後續的封裝、測試與出貨給終端客戶。對代工廠而言，一座晶圓廠同時為數百個客戶生產數千種產品，每一批晶圓要經過數百到上千道製程，週期時間長達數月，如何讓每一位客戶都覺得「這座工廠好像是我自己的」，是一個資訊系統與服務設計的問題。

17.5.2 虛擬晶圓廠的理念

依《遠見》雜誌的報導，台積電創辦人張忠謀於 1997 年提出「虛擬晶圓廠」的理念，希望透過網路「把晶圓廠搬到 IC 設計客戶的後院」，讓客戶就像擁有自己的晶圓廠一樣，隨時掌握生產狀

況。具體的做法包括：

- 線上資訊系統：建立客戶專用的線上系統，讓客戶隨時追蹤晶片的生產進度與良率分析。依台積電官網的介紹，其 eFoundry 服務以 TSMC-Online 等管道，提供設計協同（各設計層級的完整資訊）、工程協同（試產批、晶圓良率、晶圓允收測試分析與品質可靠度資料）與物流協同（晶圓批在製造、封裝、測試與出貨各階段的即時狀態）。
- 專人服務：為客戶指派專屬的服務窗口，客戶只要聯繫一個人就能得到專業的協助。
- 多專案晶圓：讓多個設計共用同一組光罩上的不同區域，分攤動輒上百萬美元的光罩費用，降低客戶的試產成本。

從本章的角度來看，虛擬晶圓廠的本質，是把 MES 中的 WIP、良率與品質資料，經過篩選與權限控管後，提供給客戶使用；把第 4 層與第 3 層之間的資訊交換，延伸到企業與客戶之間。這正是第 12 章「製造者與產品的關係延伸到交貨之後」的另一面：在代工產業，製造服務本身就是產品。

17.5.3 客戶視角的 WIP 與出貨預估

客戶最常問的問題是：「我的晶圓什麼時候會好？」回答這個問題，需要 WIP 進度與週期時間的統計。半導體業常用 X 因子描述週期時間的效率：X 因子等於實際週期時間除以純加工時間（不含等待的理論加工時間）。X 因子為 3，表示產品在廠內的時間有三分之二是在等待。下面以合成資料示範：

```
import numpy as np
# 虛擬晶圓廠的客戶視角：批次進度、X 因子與預估出貨
rng = np.random.default_rng(5)
steps_total, rpt = 600, 30.0          # 600 道製程、純加工時間 30 天
lots = {f"N{100+i}": int(s) for i, s in enumerate(rng.integers(120, 590, 6))}
x_factor = 3.0                        # 週期時間 ÷ 純加工時間（歷史統計）
for lot, done in lots.items():
    remain_rpt = rpt * (steps_total - done) / steps_total
    eta = remain_rpt * x_factor        # 預估剩餘天數
    print(f"{lot}: 完成 {done}/{steps_total} 道（{done/steps_total:.0%}），預估再 {eta:.0f} 天出貨")
ct = rng.normal(90, 6, 200)          # 過去 200 批的週期時間（天）
print(f"歷史週期時間：平均 {ct.mean():.1f} 天，X 因子 {ct.mean()/rpt:.2f}，第 90 百分位 {np.percentile(ct, 90):.1f} 天")
```

執行結果：以 X 因子 3 估算，完成 83% 的批次約再 15 天出貨，只完成 22% 的批次約再 70 天；過去 200 批的平均週期時間約 89.6 天，X 因子約 2.99，第 90 百分位約 98 天。對客戶而言，第 90

百分位比平均值更有用：它表示「九成的批次會在 98 天內完成」，可以作為承諾交期的依據。真實的預估會更細緻，考慮每一批目前所在的站點、該站點的排隊狀況、批次的優先等級，甚至以機器學習預測剩餘的週期時間（第 22、27 章）。

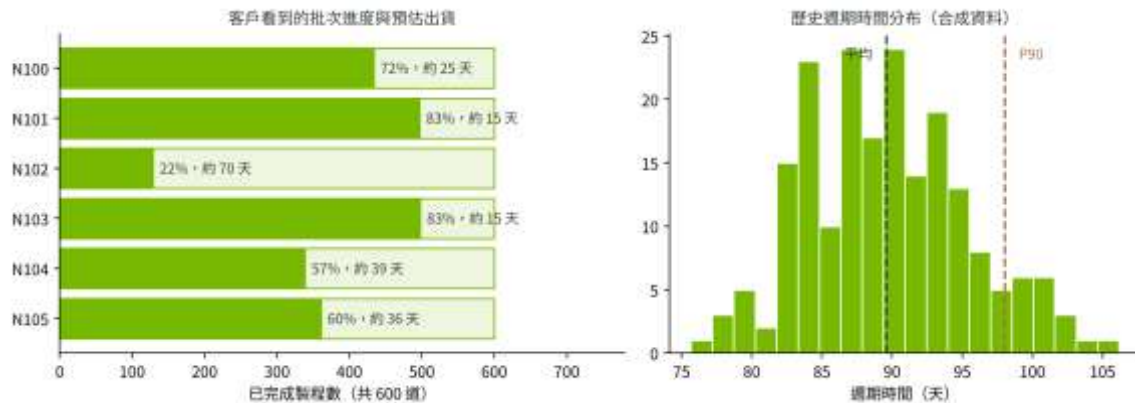


圖 17-8 客戶視角的批次進度與週期時間分布（合成資料）

17.5.4 從虛擬晶圓廠到製造即服務

虛擬晶圓廠的理念，正逐漸擴散到其他產業：

- 線上製造平台：客戶上傳 CAD 模型（第 13 章），平台自動報價、選擇合適的工廠與製程（CNC、射出、積層製造），並讓客戶追蹤進度，稱為「製造即服務」。
- 雲端 MES：中小企業以訂閱的方式使用雲端的 MES 與 ERP，降低導入門檻，第 6 章的低成本路線即可從這裡切入。
- 供應鏈協同：品牌商與代工廠、供應商共享預測、庫存與生產進度，減少第 8 章的長鞭效應，第 33 章將深入討論。

這些模式的共同挑戰，在於資料共享與保護的平衡：提供給客戶的資訊越完整，服務價值越高，但也越可能洩漏其他客戶的資訊或自己的製程機密。因此，權限控管、資料去識別化與合約規範，與系統功能同樣重要（第 35、36 章）。

案例

案例 17-1 教學情境：電腦說有五百個，倉庫一個也沒有

背景：叫獸開講中的機械零件廠（綜合情境），ERP 帳上庫存與實際庫存不符，業務無法即時回答訂單進度。

做法：依 ISA-95 盤點 ERP 與 MES 之間交換的資訊，為每一類資料指定系統紀錄的主人與交換時機；把完工、耗用與報廢改為在現場以條碼即時回報 MES，再由 MES 準即時回寫 ERP；建立主資料的單一來源與審核流程；每日自動產生帳實差異與介面錯誤報表，由專人處理。

成果：三個月後，帳實差異降到可接受範圍，業務可以直接在系統上查詢訂單在產線上的進度，回覆客戶的時間從一個多小時縮短到幾分鐘。

啟示：系統的價值取決於系統之間的接縫；資料的主人、時機與錯誤處理，比功能清單更重要。

討論：（1）為什麼延遲結案的工單會造成庫存與成本的錯誤？（2）帳實差異報表應該包含哪些欄位？

案例 17-2 產業：插單與產能需求規劃

背景：一家精密加工廠常接到客戶的急單，業務習慣先答應再說，結果 CNC 工作中心經常超載，其他訂單跟著延遲。

做法：在 ERP 中完善途程與工時資料，接單時先以產能需求規劃檢查各工作中心的負荷；超過 85% 時，業務必須與生管協商：加班、外包、調整優先順序或與客戶協商交期。

成果：準時交貨率明顯提升，業務也能依產能狀況提出合理的交期與急件加價。

啟示：交期承諾需要以產能為依據；途程與工時的主資料品質，是產能規劃的基礎。

討論：（1）為什麼規劃負荷的上限常設在 85% 左右而非 100%？（可參考第 8 章）（2）急件加價的依據可以如何計算？

案例 17-3 產業：食品原料的快速追溯

背景：一家食品工廠接到供應商通知，某一批原料可能含有不合格的成分，需要在數小時內界定受影響的產品並通知通路下架。

做法：工廠的 MES 記錄了每一批原料的投料與分批、合批關係，以批次族譜向前追溯，找出所有使用該原料的半成品與成品批號，再對照出貨紀錄，找出已出貨的客戶與通路。

成果：在兩小時內完成受影響範圍的界定，只下架了真正受影響的批號，避免了全面下架的損失。

啟示：完整的批次族譜，讓追溯從「翻箱倒櫃找紙本」變成「幾分鐘的查詢」；合批的紀錄方式會影響召回範圍的大小。

討論：(1) 若工廠沒有記錄合批關係，召回範圍會如何變化？(2) 追溯的資料應保存多久？

案例 17-4 產業：電子批記錄縮短放行時間

背景：一家藥廠的紙本批記錄每批數百頁，品保審查與放行需要數天，常因漏填或筆誤需要調查與補正。

做法：導入電子批記錄並與秤重設備、混合設備連線，系統依主配方引導操作，自動擷取數值並檢查規格，具備符合 21 CFR Part 11 的稽核軌跡與電子簽章；品保改以「例外審查」方式，只審查系統標示的偏差。

成果：批記錄的填寫錯誤大幅減少，放行時間從數天縮短到一天以內；稽核時可以快速提出完整的電子紀錄。

啟示：電子批記錄的價值不只是無紙化，更在於防錯、即時與例外審查；系統確效與稽核軌跡是合規的關鍵。

討論：(1) 例外審查需要哪些前提？(2) 電子簽章與一般的登入有什麼不同？

案例 17-5 台灣情境：虛擬晶圓廠

背景：依公開報導，台積電於 1997 年提出「虛擬晶圓廠」的理念，希望讓 IC 設計客戶如同擁有自己的晶圓廠。

做法：透過 TSMC-Online 等線上系統，客戶可以追蹤晶圓的生產進度、良率與品質資料；並以

專屬窗口與多專案晶圓等服務，降低客戶的溝通與試產成本。

成果：客戶得以掌握生產狀況並規劃後續流程，代工廠也透過服務強化與客戶的長期關係。

啟示：製造營運的資料，經過篩選與權限控管後，可以成為對客戶的服務；在代工產業，服務本身就是競爭力。

討論：（1）代工廠提供給客戶的資料，應如何避免洩漏其他客戶的資訊？（2）你的產業可以如何借鏡虛擬晶圓廠的理念？

案例 17-6 教學情境：主資料的千倍誤差

背景：一個學生專題團隊協助一家小型化工廠整理系統資料，發現某原料的耗用量在 MES 報表中異常地高。

做法：團隊撰寫程式比對 ERP 與 MES 的料號主檔，發現該原料在 ERP 以公斤為單位，在 MES 以公克為單位，介面直接傳送數值而沒有換算；進一步檢查，又找出數個 BOM 版次不一致的料號。

成果：工廠修正了介面的單位換算，建立每週自動比對的機制，並規定料號主檔只能在 ERP 建立與修改。

啟示：主資料的錯誤常常靜靜地存在，需要主動、定期、自動化的檢查。

討論：（1）除了單位與版次，主資料還有哪些常見的錯誤？（2）如何設計主資料品質的指標？

實作活動

活動 17-1 系統地圖

選擇一家熟悉的企業（或你的學校實驗室、社團），畫出它使用的資訊系統、每個系統負責的功能與主要使用者，以及系統之間的資料流，標示出哪些交換是人工的、哪些容易出錯。

活動 17-2 MRP 與 CRP 試算

以 17.1.3 節與 17.3.2 節的程式為基礎，為一個三階 BOM 的產品建立主生產排程，計算所有零件的計畫訂單，並把結果轉成工單計算工作中心負荷；嘗試改變批量規則與安全庫存，觀察對庫存與負荷的影響。

活動 17-3 ISA-95 介面設計

以一個簡單的產品為例，依 17.2 節的四類資訊，設計 ERP 與 MES 之間的介面清單：每一筆交換的資料內容、方向、時機、顆粒度、主人與錯誤處理方式，並以 JSON 寫出工單下達與完工回報兩個訊息的範例。

活動 17-4 批次族譜與召回模擬

以 17.4.3 節的程式為基礎，建立一個包含至少 20 個批次、有分批與合批的族譜，隨機指定一批原料有問題，計算受影響的成品數；比較「限制合批」與「自由合批」兩種規則下的平均召回範圍。

活動 17-5 電子批記錄雛形

以 17.4.5 節的程式為基礎，設計一個簡單的電子批記錄：依配方步驟引導輸入、檢查數值是否在規格內、記錄執行者與複核者，並以雜湊鍵保護稽核軌跡；嘗試修改紀錄，觀察驗證結果。

活動 17-6 交期預估

以 17.5.3 節的程式為基礎，利用第 8 章的等候模擬產生各站的等待時間，建立一個依目前所在站點與排隊長度預估剩餘週期時間的模型，並與只用 X 因子的簡單估計比較。相關範例程式將於本書的實驗倉庫中維護。

延伸閱讀

- Joseph Orlicky, Material Requirements Planning (1975 年初版，後續版本由 Ptak 與 Smith 修訂)：MRP 的經典著作。
- ANSI / ISA-95 (IEC 62264) 系列標準，以及 MESA 國際組織的白皮書與 B2MML 實作規格。

- ANSI / ISA-88 (IEC 61512)：批次控制標準，包含配方模型。
- 美國食品藥物管理局 21 CFR Part 11 及其指引文件；歐盟 GMP 附則 11 (2011 年版及 2025 年修訂草案)。
- Wallace J. Hopp 與 Mark L. Spearman，Factory Physics (1996 年初版)：從工廠物理的角度理解 MRP 與產能 (第 8 章已介紹)。

表 17-8 本章思考地圖

本章概念	關鍵提問	後續延伸章節
企業系統分工	每一類資料由哪個系統負責？	第 18、20 章
MRP 與 CRP	計畫考慮了產能嗎？主資料正確嗎？	第 27 章
ISA-95 介面	資訊交換的主人、時機與錯誤處理清楚嗎？	第 11、20、23 章
工單、途程、配方與主資料	主資料是否有單一來源與治理？	第 12、20 章
WIP 與製造履歷	能否即時回答「現在在哪裡、做得怎樣」？	第 19、22 章
追溯與電子批記錄	紀錄是否完整、可信、可追溯？	第 25、33、35 章
虛擬晶圓廠與製造服務	製造資料能否成為對客戶的服務？	第 32、33 章

章末小結

- 製造企業以 ERP、SCM、CRM、PLM、APS、MES / MOM、QMS、WMS 等系統分工；系統的價值取決於它們之間的整合，主資料不一致、交易時機不對與責任不清是常見問題。
- MRP 由 Orlicky 於 1960 年代建立原理，依主生產排程與 BOM 計算毛需求、淨需求、批量與前置時間倒推的計畫訂單；它假設資料正確與產能無限，需要 CRP、APS 與 MES 補足。MES 的功能由 MESA 於 1997 年提出的 11 項功能模型定義，ISA-95 第 3 部分則擴大為涵蓋生產、品質、維護與庫存的 MOM。
- ISA-95 定義第 4 層與第 3 層之間的四類資訊：產品定義與生產排程往下，生產能力與生產績效往上；第 3 部分的生產營運包括產品定義管理、資源管理、詳細排程、派工、執行管理、資料蒐集、追蹤與績效分析八項活動。介面設計要明確資料的主人、時機、顆粒度與錯誤處理。

- 訂單依 MTS、MTO、ATO、ETO 策略轉為工單；途程與工作中心支撐產能需求規劃，本章範例中插單使 CNC 負荷率升到 109%。ISA-88 把配方分為通用、廠區、主與控制四層。主資料需要單一來源、負責人與自動化的一致性檢查。
- WIP 追蹤回答「現在在哪裡」，製造履歷記錄「實際怎麼做」，批次族譜支援向前與向後追溯，合批會擴大召回範圍。電子批記錄帶來防錯、即時與例外審查；21 CFR Part 11 自 1997 年起要求系統確效、存取控制、稽核軌跡與電子簽章，雜湊鍵可偵測竄改。
- 台積電於 1997 年提出的虛擬晶圓廠，透過線上系統讓客戶追蹤進度、良率與品質，把製造營運資料延伸為對客戶的服務；X 因子與週期時間分布支撐交期預估。製造即服務、雲端 MES 與供應鏈協同延續了這個理念，資料共享與保護的平衡是關鍵。

習題

一、選擇題

1. 依主生產排程與 BOM 計算零件需求量與下單時間的方法是？(A) CRP (B) MRP (C) SPC (D) OEE
2. MRP 的原理一般歸功於哪一位在 IBM 工作的人？(A) Oliver Wight (B) Joseph Orlicky (C) Eliyahu Goldratt (D) Taiichi Ohno
3. 推動 MES 定義、於 1992 年成立的組織是？(A) ISA (B) MESA (C) OPC 基金會 (D) IEC
4. ISA-95 中由第 3 層往第 4 層回報「實際生產了什麼、用了什麼」的資訊類別是？(A) 產品定義 (B) 生產排程 (C) 生產能力 (D) 生產績效
5. 接單後才開始設計與生產的策略是？(A) MTS (B) MTO (C) ATO (D) ETO
6. ISA-88 中，針對某一批次實例化、實際被執行的配方是？(A) 通用配方 (B) 廠區配方 (C) 主配方 (D) 控制配方
7. 某一批原料有問題，找出它進入了哪些成品，稱為？(A) 向前追溯 (B) 向後追溯 (C) 產能規劃 (D) 主資料治理

8. 美國 21 CFR Part 11 規範的是？(A) 食品標示 (B) 電子紀錄與電子簽章 (C) 職業安全 (D) 環境排放
9. 半導體業的 X 因子定義為？(A) 良率 ÷ 產能 (B) 週期時間 ÷ 純加工時間 (C) $WIP \div \text{產出率}$ (D) 光罩數 ÷ 層數
10. 依公開報導，台積電提出「虛擬晶圓廠」理念的年份是？(A) 1987 (B) 1992 (C) 1997 (D) 2007

二、簡答題

11. 說明 ERP 與 MES 在功能與時間尺度上的差異。
12. 說明 MRP 的三個主要假設，以及它們在實務上的限制。
13. 說明 ISA-95 第 4 層與第 3 層之間四類資訊的方向與內容。
14. 規劃 ERP 與 MES 的介面時，應回答哪些問題？
15. 說明工單的生命週期，並舉一個狀態轉換時 ERP 與 MES 之間的資訊交換。
16. 什麼是主資料的「單一來源」原則？為什麼重要？
17. 為什麼合批會擴大召回的範圍？
18. 說明電子批記錄的「例外審查」與其前提。

三、計算題

19. 某零件現有庫存 50 個，第 3 週有預計收料 30 個，第 4 週毛需求 200 個，固定批量 100 個，前置時間 2 週。求第 4 週的淨需求、計畫收料量與計畫下單週。
20. 一張工單 300 件，換線時間 2 小時，每件工時 4 分鐘，該工序的負荷為多少小時？
21. 某工作中心有 3 台機器，每週 40 小時，稼動率 80%，可用產能為多少小時？若負荷為 110 小時，負荷率為多少？
22. 某產品的平均週期時間 75 天、純加工時間 25 天，X 因子為多少？表示產品在廠內約有多少比例的時間在等待？

23. 一批晶圓共 600 道製程，已完成 400 道，純加工時間 30 天，X 因子 3，預估還需多少天？
24. 某晶圓廠的 WIP 為 3,000 批，每天產出 100 批，依利特爾定律，平均週期時間約為多少天？
25. 依 17.4.3 節的族譜，若粉末 P2 有問題，向前追溯會影響哪些成品？
26. 依 17.3.2 節的資料，若把插單 WO-005 的數量從 200 件降為 100 件，CNC 的負荷率為多少？

四、申論與設計題

27. 為一家中小型機械加工廠規劃 ERP 與 MES 的整合方案，說明系統分工、介面設計與導入步驟。
28. 評論以下說法：「只要買了最好的 ERP 和 MES，資料就會自動正確。」
29. 選擇一個你熟悉的產業，借鏡虛擬晶圓廠的理念，設計一個提供給客戶的製造資訊服務，並說明資料保護的做法。

五、是非題

30. MRP 假設產能無限，因此需要產能需求規劃補足。
31. ISA-95 只規範 ERP 與 MES 之間的生產工單，不涉及品質與維護。
32. 主資料錯誤通常會立即被發現。
33. 雜湊鍵可以偵測紀錄被竄改，但無法單獨防止竄改。
34. 以單件序號追蹤比以批次追蹤的顆粒度更細。
35. 第 90 百分位的週期時間通常比平均週期時間更適合作為交期承諾的依據。

六、進階挑戰題

36. 擴充 17.1.3 節的 MRP 程式，支援多階 BOM (例如 X 由 B 與 C 組成，B 又由 C 與 D 組成)，並以低階碼確保共用零件在最低階才計算。
37. 以 17.4.3 節的程式為基礎，建立一個能同時處理分批比例 (例如 LotA 有 60% 進入 LotA-1) 的族譜，計算每一件成品中某批原料的「含量比例」。
38. 設計一個 ERP 與 MES 之間的訊息佇列模擬：MES 發送完工與耗用訊息，部分訊息延遲或失敗，ERP 定期對帳；統計在不同失敗率與重送策略下的帳實差異。

七、補充題

39. 列出 MESA 的 11 項 MES 功能。

40. 列出 21 CFR Part 11 的四項核心要求。

提示與參考解答

1. B。

2. B。

3. B。

4. D。

5. D。

6. D。

7. A。

8. B。

9. B。

10. C。

11. 參考：ERP 負責銷售、採購、庫存、生產規劃與財務，時間尺度為月、週、日；MES 負責派工、執行、資料蒐集與追蹤，時間尺度為班、時、分。

12. 參考：假設資料（BOM、庫存、前置時間）正確、產能無限、前置時間固定。實務上資料常有誤差、產能有限、前置時間隨負荷變化，需要主資料治理、CRP 與 APS 補足。

13. 參考：往下為產品定義（如何生產）與生產排程（生產什麼、多少、何時）；往上為生產能力（可用資源）與生產績效（實際生產與耗用）。

14. 參考：資料的主人、交換時機、顆粒度、錯誤處理、執行中的變更管理、整合方式。

15. 參考：計畫、已下達、已備料、生產中、暫停、完工、結案。例如完工時 MES 回報完工數量與耗用，ERP 據以入庫與計算成本。

16. 參考：每一類主資料只在一個系統中建立與維護，其他系統同步而不直接修改，避免多份資料不一致造成錯誤。
17. 參考：合批後，不同來源的原料混在同一批中，任一來源有問題，整個合批及其下游產品都受影響。
18. 參考：品保只審查系統標示的偏差與例外。前提是系統經過確效、能可靠檢查規格與流程、稽核軌跡完整、例外規則經過核准。
19. 第 3 週後可用 $50 + 30 = 80$ ，第 4 週淨需求 $200 - 80 = 120$ ；固定批量 100，計畫收料 200；前置時間 2 週，第 2 週下單。
20. $2 + 300 \times 4 \div 60 = 22$ 小時。
21. $3 \times 40 \times 0.8 = 96$ 小時；負荷率 $110 \div 96 \approx 115\%$ 。
22. $75 \div 25 = 3$ ；等待約占 $(75 - 25) \div 75 \approx 67\%$ 。
23. $30 \times (600 - 400) \div 600 \times 3 = 30$ 天。
24. $3,000 \div 100 = 30$ 天。
25. 粉末 P2 → LotB → LotC → 成品 S1003、S1004、S1005。
26. CNC 負荷 $53 + 1 + 100 \times 6 \div 60 = 64$ 小時，負荷率 $64 \div 68 \approx 94\%$ 。
27. 評分重點：系統分工清楚，依 ISA-95 定義介面的主人、時機與錯誤處理，主資料有單一來源，導入分階段並有評估指標。
28. 評分重點：能指出資料正確性取決於主資料治理、交易時機、介面設計、現場紀律與持續對帳，而非軟體本身。
29. 評分重點：服務內容對客戶有價值，資料範圍與權限設計合理，能說明去識別化、存取控制與合約規範。
30. ✓。
31. X。ISA-95 涵蓋生產、品質、維護與庫存四個營運領域。
32. X。主資料錯誤常靜靜存在，直到造成問題才被發現。

33. ✓。

34. ✓。

35. ✓。

36. 評分重點：正確計算低階碼，依階層順序展開並彙總共用零件的毛需求，結果可驗算。

37. 評分重點：在邊上記錄比例，沿路徑相乘並加總，驗證所有成品中該原料的總量等於投入量。

38. 評分重點：模擬合理，能顯示失敗率與重送延遲對帳實差異的影響，並提出對帳頻率與例外處理的建議。

39. 資源配置與狀態、詳細排程、生產單位派工、文件控制、資料蒐集、人員管理、品質管理、製程管理、維護管理、產品追蹤與族譜、績效分析。

40. 系統確效、存取控制、稽核軌跡、電子簽章。

第 18 章

離散型與流程型智慧工廠

編著 李世淵 (機器人叫獸) | 助教 李天宇、李宇晴

本章重點

- 離散型、批次型與連續型製造，產品—製程矩陣
- 牌號切換、裝配線平衡與看板
- 流程工廠的 DCS、SIS 與營運決策階層
- 模型預測控制的壓縮與移位、資料協調與軟感測器
- 縱向、橫向與端到端整合的智慧集成製造系統
- 電子、石化、鋼鐵與半導體燈塔工廠比較

叫獸開講：哪一座工廠比較「智慧」？

有一年，我帶學生做兩次工廠參訪，前後只隔一週。

第一次參訪的是一座電子組裝廠。走進廠房，映入眼簾的是一條條閃著燈號的產線：機械手臂在貼片機旁上下料，自主移動機器人載著料架在走道上穿梭，每一站的螢幕都顯示著即時的產量、良率與節拍時間。學生們拿著手機拍個不停，有人小聲說：「這就是智慧工廠吧！」

第二次參訪的是一座石化廠。我們穿上防護裝備，先聽了半小時的工安說明，才被帶進中央控制室。控制室很安靜，只有幾位值班工程師坐在一整排螢幕前，螢幕上是密密麻麻的流程圖與趨勢曲線。窗外是綿延好幾公里的管線、塔槽與反應器，卻幾乎看不到人，也看不到機械手臂。

回程的車上，我問學生：「這兩座工廠，哪一座比較智慧？」大多數人選了電子廠，理由是「機器人比較多」、「看起來比較先進」。

這時，一位化工系的同學舉手說：「可是石化廠的那位工程師說，整座工廠有好幾萬個量測點，大部分迴路都是自動控制，還有一套模型預測控制在幫他們節省能源。一個班只要幾個人，就能管一座比電子廠大好幾倍的工廠。」

車上安靜了一會兒。我接著說：「你們看到的差異，不是智慧程度的差異，而是製造型態的差異。電子廠做的是一個一個可以數的產品，所以它的智慧化看起來像機器人、自動搬運與逐件追蹤；石化廠做的是連續流動的物料，它很早就把控制自動化了，所以它的智慧化看起來像模型、最佳化與資料協調。兩者面對的問題不同，走的路也不同。」

這一章，我們要比較離散型與流程型製造：它們在產品、製程、設備、控制、品質與經濟上有哪些根本差異？離散型智慧工廠如何整合物理系統、資訊系統與供應鏈？流程型智慧工廠如何以 DCS、先進製程控制、資料協調與分層決策營運？縱向、橫向與端到端整合如何把兩種工廠串成一個系統？最後，我們以世界經濟論壇的燈塔工廠為例，比較電子、石化、鋼鐵與半導體四個產業的智慧製造路線。

（說明：本節情境由作者帶領學生參訪的經驗綜合改寫，人物與細節已調整。）

學習目標

讀完本章後，你應該能夠：

1. 說明離散型、批次型與連續型製造的特徵，並以產品—製程矩陣分析不同產業的製造型態。
2. 比較離散型與流程型製造在產品定義、設備、控制、品質、換線與成本結構上的差異。
3. 說明離散型智慧工廠在物理系統、資訊系統與供應鏈三個層面的關鍵技術，並以程式進行裝配線平衡與看板計算。
4. 說明流程型智慧工廠的 DCS、先進製程控制與營運決策階層，並以程式示範牌號切換、資料協調與「壓縮與移位」的效益估算。
5. 說明縱向整合、橫向整合與端到端整合的意義，並估算不同整合架構的介面數量與資訊延遲。
6. 以燈塔工廠為例，比較電子、石化、鋼鐵與半導體產業的智慧製造重點，並說明成本結構如何影響投資優先順序。

核心概念

表 18-1 本章核心名詞中英對照

中文名詞	英文	一句話說明
離散型製造	Discrete Manufacturing	生產可以逐件計數的產品，例如汽車、手機、工具機
流程型製造	Process Manufacturing	以化學或物理轉換生產連續或批次的物料，例如石化、鋼鐵、食品
批次製造	Batch Manufacturing	依配方一批一批生產，介於離散與連續之間
產品—製程矩陣	Product-Process Matrix	以產品的量與變化程度，對應適合的製程結構
節拍時間	Takt Time	依需求計算出的每件產品可用的生產時間
裝配線平衡	Line Balancing	把作業分配到工作站，使各站負荷接近節拍時間
分散式控制系統	Distributed Control System, DCS	流程工業中以分散的控制器與集中的操作介面管理大量迴路的系統

中文名詞	英文	一句話說明
先進製程控制	Advanced Process Control, APC	在基本控制之上，以模型與最佳化改善控制品質的技術
模型預測控制	Model Predictive Control, MPC	以模型預測未來，在限制條件下求解最佳控制動作的方法
即時最佳化	Real-Time Optimization, RTO	依經濟目標計算各單元最佳穩態操作點的技術
資料協調	Data Reconciliation	修正量測值使其滿足質量與能量平衡，並找出粗大誤差
牌號切換	Grade Transition	連續製程從一種產品規格轉換到另一種規格的過程
縱向整合	Vertical Integration	從現場設備到企業系統之間的資訊整合
橫向整合	Horizontal Integration	跨部門、跨工廠與跨企業的價值網路整合
端到端整合	End-to-End Integration	貫穿產品從設計、製造到服務整個生命週期的工程整合
燈塔工廠	Lighthouse	世界經濟論壇認可的第四次工業革命技術規模化應用標竿

18.1 離散型與流程型製造的特徵與差異

18.1.1 兩種製造型態

在第 7 章，我們依加工原理介紹了切削、成形、接合與積層等製程。如果把視角從單一製程拉高到整座工廠，會發現製造業大致可以分成兩種型態。

離散型製造生產的是可以逐件計數的產品：一台汽車、一支手機、一顆螺絲。產品由零件組裝而成，結構以 BOM 描述（第 12 章），生產步驟以途程描述（第 17 章）。產品在不同工作站之間移動，每一件都可以被拆解回原來的零件，至少在原則上是如此。

流程型製造生產的是物料：汽油、塑膠粒、鋼捲、麵粉、藥品。原料經過化學反應、相變、混合、分離或熱處理，變成另一種物質，通常無法還原。產品以配方或規格描述（第 17 章的 ISA-88），量的單位是噸、公升或批，而不是「件」。

流程型製造又可以細分為兩種：

- 連續型：物料連續不斷地流過設備，一年運轉 300 天以上，例如煉油、乙烯裂解、造紙、水泥。
設備一旦停機，重新開機可能需要數天，並產生大量不合格品。
- 批次型：依配方一批一批生產，每一批有明確的開始與結束，例如製藥、特用化學品、食品、啤酒。批次之間可能需要清洗與換線。

許多產業兼具兩種特徵，稱為混合型。例如半導體：晶圓以「批」(lot)為單位、可以逐片追蹤，看起來像離散型；但每一道製程(沉積、蝕刻、擴散)本質上是化學與物理反應，以配方控制，又像流程型。鋼鐵廠的煉鐵、煉鋼是連續或批次的流程，到了軋延與裁剪後的鋼捲、鋼板，又變成可以逐件追蹤的產品。食品廠的烹煮是批次流程，包裝之後則是一盒一盒的離散產品。

18.1.2 產品—製程矩陣

1979 年，Robert Hayes 與 Steven Wheelwright 在《哈佛商業評論》發表〈Link Manufacturing Process and Product Life Cycles〉等兩篇文章，提出產品—製程矩陣，用兩個維度描述製造型態的選擇：橫軸是產品結構，從少量、獨特的產品，經過少量多種、大量標準化，到極大量的大宗商品；縱軸是製程結構，從零工式 (Job Shop)、批量式、裝配線到連續流程。



圖 18-1 產品—製程矩陣與典型產業的位置

矩陣的對角線代表「產品與製程相匹配」：少量、獨特的客製化產品適合零工式生產，例如模具

與航太零件；少量多種的產品適合批量生產，例如工具機與特用化學品；大量標準化的產品適合裝配線，例如汽車與家電；極大量的大宗商品適合連續流程，例如石化與水泥。偏離對角線的位置，通常意味著成本過高（用連續流程做客製化產品，設備閒置）或彈性不足（用零工式做大量標準品，效率低落）。

智慧製造帶來的一個重要改變，是讓企業可以「離開對角線而不付出太多代價」。例如，彈性製造系統與數位化的換線，讓裝配線也能生產多樣的產品，這就是第 32 章要討論的大量客製化；模組化、小型化的連續製程設備，則讓化工業也能做中小批量的特用化學品。換句話說，智慧製造正在拉平這張矩陣。

18.1.3 離散型與流程型的比較

表 18-2 離散型與流程型製造的比較

面向	離散型	流程型
產品單位	件、台，可逐件計數與序號追蹤	噸、公升、批，以批號或時間區段追蹤
產品定義	BOM 與途程，結構為「組裝樹」	配方與流程圖，常有副產品與回收流
物料流	在工作站之間移動，可暫存、可重排順序	在管線與設備中連續流動，緩衝有限
設備	工具機、機器人、輸送線，彈性高、單機價值中等	反應器、塔槽、爐、軋機，專用性高、資本密集
控制	PLC、運動控制，以順序與邏輯為主	DCS、迴路控制，以連續變數的調節為主
品質	尺寸、外觀、功能測試，常可逐件檢驗	成分、物性，常需取樣化驗，結果有延遲
換線	換模、換治具、換程式，數分鐘到數小時	牌號切換、清洗，可能產生大量過渡料
主要變異來源	人員、設備狀態、零件公差	原料性質、觸媒活性、環境溫度
成本結構	材料與人工占比較高	原料、能源與折舊占比較高
安全風險	機械傷害、搬運	火災、爆炸、毒性物質外洩
自動化起點	1980 年代以後的機器人與 CNC 普及	1970 年代 DCS 問世後即已高度自動化

這張表的最後一列特別值得注意。流程工業的自動化起步很早：1975 年，Honeywell 的 TDC 2000 與橫河電機的 CENTUM 先後問世，兩家公司都以「全球第一套分散式控制系統」來介紹自己的

產品。從那時起，煉油廠與化工廠就已經能以少數人力監控數千個控制迴路。所以，流程工業的智慧化不是從「自動化」開始，而是從「最佳化」開始：怎麼讓已經自動運轉的工廠，更省能源、更穩定、更快切換、更少不合格品。

相對地，離散製造的自動化較晚、也較不完整：許多組裝、檢驗與搬運工作至今仍仰賴人力。因此，離散型智慧工廠的重點，常落在自動化、彈性化與逐件追蹤。

18.1.4 牌號切換：連續製程的「換線」

離散工廠換線，是換模具、換治具與換程式，第 8 章介紹的快速換模 (SMED) 可以把換線時間從數小時縮短到數分鐘。連續製程的「換線」則完全不同：設備不停機，只是逐步改變進料與操作條件，讓產品從一種牌號（例如某種熔融指數的塑膠）轉換到另一種牌號。在轉換的過程中，產品的性質介於兩種規格之間，兩邊都不合格，稱為過渡料，只能降級出售或回收。

反應器內的產品性質，可以近似為一階混合：新條件下的產品，要逐漸「沖淡」舊的產品。時間常數大約等於反應器的滯留時間。下面的程式比較兩種切換策略：直接把條件改成新牌號，以及先「過衝」到更高的條件、接近新規格時再切回：

```
import numpy as np
# 連續製程的牌號切換：反應器內的產品性質近似一階混合（時間常數 tau = 滯留時間）
tau, dt, F = 2.0, 0.01, 10.0 # 時間常數 2 小時、時間步 0.01 小時、產量 10 噸/小時
specA, specB = (9.5, 10.5), (19.0, 21.0) # 舊牌號 A 與新牌號 B 的規格範圍
def run(policy, T=12.0):
    t = np.arange(0, T, dt); y = np.empty_like(t); yk = 10.0
    for i, ti in enumerate(t):
        y[i] = yk
        u = policy(ti, yk) # 進料條件對應的穩態性質
        yk += dt / tau * (u - yk) # 一階混合：dy/dt = (u - y) / tau
        inA = (y >= specA[0]) & (y <= specA[1]); inB = (y >= specB[0]) & (y <= specB[1])
        off = (~inA) & (~inB)
        return t, y, off.sum() * dt
state = {"hit": False}
def overshoot(ti, yk): # 先「過衝」到 25，接近 B 規格下限時切回 20
    if yk >= 19.0: state["hit"] = True
    return 20.0 if state["hit"] else 25.0
t, y1, h1 = run(lambda ti, yk: 20.0)
t, y2, h2 = run(overshoot)
print(f"直接切換：不合格時間 {h1:.2f} 小時，不合格品 {h1*F:.1f} 噸")
print(f"過衝切換：不合格時間 {h2:.2f} 小時，不合格品 {h2*F:.1f} 噸，減少 {1-h2/h1:.0%}")
```

執行結果：直接切換時，產品性質要約 4.5 小時才進入新規格，產生約 45 噸過渡料；過衝切換只需約 1.8 小時，過渡料約 18 噸，減少約 61%。這個結果也可以用解析解驗算：直接切換的不合格

時間為 $\tau [\ln(10 / 1) - \ln(10 / 9.5)] \approx 2.25\tau \approx 4.5$ 小時。

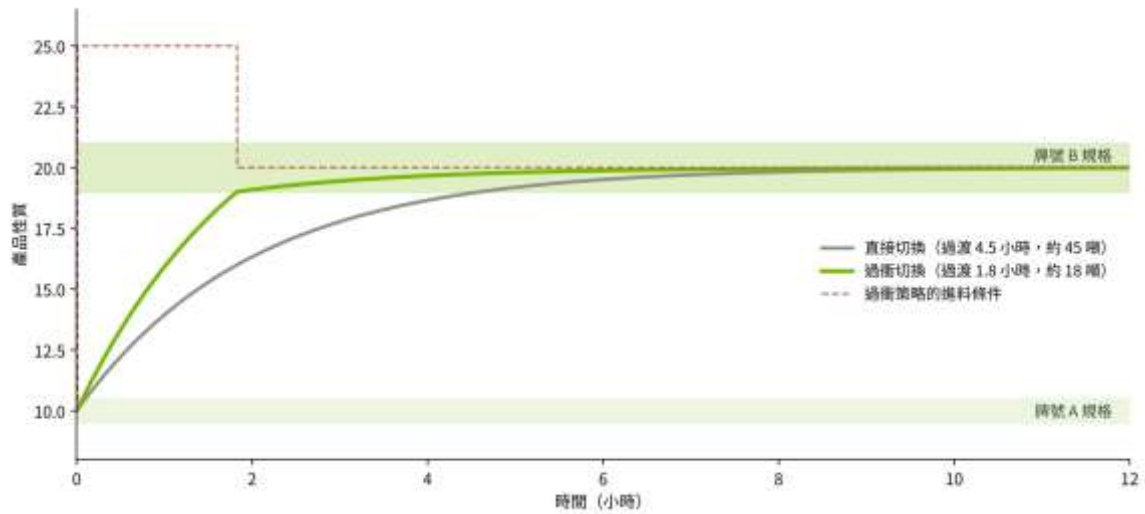


圖 18-2 牌號切換的兩種策略與過渡料

這個簡單的例子說明了流程工業智慧化的典型模式：效益不是來自更多的機器人，而是來自更好的模型與控制策略。實務上，過衝的幅度受到設備與安全限制，也要考慮反應器的非線性與多個性質同時切換，因此需要第 18.3 節的模型預測控制，以及第 27 章的排程：把相近的牌號排在一起，讓切換的「距離」越短越好。

18.1.5 兩種工廠的智慧化重點

綜合以上的比較，兩種工廠的智慧化重點可以歸納如下：

- 離散型：自動化與彈性化（機器人、自主移動機器人、彈性產線）、逐件追蹤與品質（序號、機器視覺、第 24 章）、排程與換線（第 27 章）、供應鏈協同（多階 BOM 帶來大量的供應商）。
- 流程型：先進製程控制與即時最佳化、資料協調與軟感測器（以模型估計難以量測的品質）、能源與排放管理（第 34 章）、設備可靠度與預測維護（第 26 章，非計畫停機的代價極高）、製程安全。

兩者也有越來越多的交集：資料平台、數位孿生（第 19 章）、AI 品質預測與知識管理（第 28 章），在兩種工廠都是重點。差別在於，同樣的技術要套用在不同的問題上。

18.2 離散型智慧工廠：物理系統、資訊系統與供應鏈

18.2.1 三個層面

離散型智慧工廠可以從三個層面來看：物理系統負責「做」，資訊系統負責「想」與「記」，供應鏈負責「接」，把工廠與上游供應商、下游客戶連在一起。三個層面之間以資料串連，形成第 4 章的資訊物理系統。



圖 18-3 離散型智慧工廠的物理系統、資訊系統與供應鏈

表 18-3 離散型智慧工廠的三個層面

層面	主要元素	智慧化重點	本書相關章節
物理系統	工具機、機器人、輸送線、自主移動機器人、自動倉儲、檢測站	彈性化、模組化、人機協作、設備連網	第 9、10、30、31 章
資訊系統	ERP、MES、APS、QMS、WMS、PLM	逐件追蹤、即時派工、數位孿生、閉環品質	第 17、19、25、27 章
供應鏈	供應商、物流、通路、客戶	需求感知、看板與 JIT、供應商協同、韌性	第 33、34 章

18.2.2 物理系統：從專用線到彈性產線

傳統的離散產線是為單一產品設計的專用線：輸送帶把工件依固定順序送過每一站，效率很高，

但換產品時要重新規劃整條線。智慧化的物理系統有三個趨勢：

- 彈性製造單元：數台工具機與機器人組成一個單元，可以加工一族相似的零件，換產品時只要換程式與治具（第 9 章）。
- 矩陣式生產：以自主移動機器人取代固定的輸送帶，工件依自己的途程在工作站之間「選路」，不同產品可以走不同的路線。一些汽車廠已經在試驗這種沒有固定裝配線的生產方式。
- 模組化產線：工作站以標準化的機械、電氣與通訊介面連接，可以像積木一樣重新排列。

彈性的代價是複雜度：路線越自由，排程與交通管理就越困難，這正是第 27 章智慧排程與第 10 章智慧物流要處理的問題。

18.2.3 裝配線平衡

無論產線多麼智慧，基本的工業工程問題依然存在。裝配線平衡是其中最經典的一個：已知產品的組裝作業、各作業的時間與先後順序，如何把作業分配到最少的工作站，並讓每一站的負荷都不超過節拍時間？

節拍時間（Takt Time）由需求決定：

$$\text{節拍時間} = \text{可用生產時間} \div \text{需求數量}$$

例如每班可用 8 小時（28,800 秒），需求 480 台，節拍時間就是 60 秒：每 60 秒必須有一台產品下線。下面的程式以排序位置權重法（Ranked Positional Weight, RPW）進行平衡：先計算每個作業的「位置權重」，也就是它自己和所有後續作業的時間總和，權重越大代表越「卡」在前面；再依權重由大到小，把前置作業已完成、而且放得進目前工作站的作業依序分配：

```
import math
# 裝配線平衡：以排序位置權重法（RPW）把作業分配到工作站，並計算看板數量
tasks = { # 作業：(時間 秒, 前置作業)
    "A": (20, []), "B": (35, ["A"]), "C": (15, ["A"]), "D": (40, ["B"]), "E": (25, ["C"]),
    "F": (30, ["B", "C"]), "G": (20, ["D"]), "H": (35, ["E", "F"]), "I": (25, ["G", "H"]), "J": (25, ["I"])}
avail, demand = 8 * 3600, 480 # 每班可用 8 小時、需求 480 台
takt = avail / demand # 節拍時間（秒/台）
succ = {k: [j for j, (_, p) in tasks.items() if k in p] for k in tasks}
def weight(k): # 位置權重 = 自己 + 所有後續作業的時間
    seen, stack = set(), [k]
    while stack:
        x = stack.pop()
        if x not in seen: seen.add(x); stack += succ[x]
```

```

    return sum(tasks[x][0] for x in seen)
order = sorted(tasks, key=lambda k: -weight(k))
stations, done = [], set()
while len(done) < len(tasks):
    load, st = 0, []
    while True:
        # 依權重挑選「前置已完成且放得下」的作業
        cand = [k for k in order if k not in done and all(p in done for p in tasks[k][1])
                and load + tasks[k][0] <= takt]
        if not cand: break
        k = cand[0]; st.append(k); done.add(k); load += tasks[k][0]
    stations.append((st, load))
total = sum(t for t, _ in tasks.values())
for i, (st, load) in enumerate(stations, 1):
    print(f"工作站 {i}: {'、'.join(st)}, 負荷 {load} 秒 ({load/takt:.0%}) ")
n = len(stations)
print(f"工作站數 {n}, 平衡效率 {total/(n*takt):.1%}, 平衡延遲 {1-total/(n*takt):.1%}")

```

執行結果：作業總時間 270 秒，理論最少工作站數為 $[270 / 60] = 5$ 。程式分配出 5 個工作站：A、B (55 秒)；C、F (45 秒)；D、G (60 秒)；E、H (60 秒)；I、J (50 秒)，平衡效率 90.0%，平衡延遲 10.0%。工作站 3 與 4 的負荷恰好等於節拍時間，是這條線的瓶頸；工作站 2 只有 75% 的負荷，是改善的空間。



圖 18-4 裝配線平衡的結果與各站負荷

RPW 是一種啟發式方法，不保證找到最佳解，但簡單、快速，適合教學與初步規劃。實務上，裝配線平衡還要考慮更多限制：某些作業必須在同一站（使用同一個治具）、某些作業不能在同一站（一個在車頂、一個在車底）、多種產品混線生產時各產品的作業時間不同、人員的熟練度與學習曲線等。這些問題可以用整數規劃或第 27 章的最佳化方法求解。

智慧化為這個古老的問題帶來兩個新的可能。第一，作業時間不再只能靠碼表量測：第 24 章的機器視覺可以自動辨識每個動作的時間，持續更新標準工時。第二，平衡不再是一次性的規劃：當需

求改變、新產品導入或人員異動時，系統可以重新計算並建議調整，甚至讓協作型機器人（第 31 章）動態分擔瓶頸站的作業。

18.2.4 資訊系統：逐件追蹤與閉環

離散型智慧工廠的資訊系統，以第 17 章的 ERP 與 MES 為骨幹，並有三個特別重要的能力：

- 逐件追蹤：每一件產品（或關鍵零件）都有唯一的序號，記錄它在哪一站、由哪台設備與哪位人員、使用哪一批零件、量測結果如何。汽車與醫療器材的召回，常需要精確到單一序號。
- 防錯與引導：系統依產品的序號與組態，在每一站顯示正確的作業指示，並以條碼、扭力槍的連線或視覺檢查確認作業正確，避免裝錯零件或漏鎖螺絲。這在多樣少量的混線生產中尤其重要。
- 閉環品質：測試站的不良資料即時回饋到上游工作站與設備，觸發第 25 章的統計製程管制與根本原因分析；嚴重時自動停線，就是豐田「自働化」（Jidoka）精神的數位化。

18.2.5 供應鏈：看板與需求拉動

離散型產品的 BOM 常有數百到數萬個零件，來自數十到數百家供應商。供應鏈的協同，是離散型智慧工廠不可或缺的一環。第 8 章介紹過豐田生產方式的看板：下游用掉多少，上游才補多少，以「拉」取代「推」。看板的張數可以用以下公式估算：

$$\text{看板數量} = \text{需求率} \times \text{補貨前置時間} \times (1 + \text{安全係數}) \div \text{每箱容量}$$

以前面的裝配線為例，每小時需求 60 台，每台用一個某零件，補貨前置時間 0.5 小時，安全係數 10%，每箱 20 個，看板數量為 $60 \times 0.5 \times 1.10 \div 20 = 1.65$ ，取整數為 2 張。

智慧化的看板，是把實體的卡片變成電子訊號：線邊料架的重量感測器或 RFID 偵測到庫存低於設定點，自動發出補料訊號給倉庫的自主移動機器人，或直接傳給供應商的系統。更進一步，品牌商可以把需求預測、生產計畫與庫存資訊分享給供應商，讓供應商提早準備，這就是第 33 章的網路協同製造。

不過，看板與 JIT 的前提是需求穩定、供應可靠。2020 年以後的疫情與晶片短缺，讓許多企業發現，過度精實的供應鏈缺乏韌性：當某一個關鍵零件斷貨，整條產線就停擺。因此，現代的離散型智慧工廠需要在效率與韌性之間取得平衡，例如對關鍵零件建立策略庫存、開發第二供應來源，並以數

位工具即時掌握多階供應商的風險（第 34 章）。

18.3 流程型智慧工廠：DCS、APC、資料協調與營運決策階層

18.3.1 流程工廠的控制架構

走進一座石化廠的中央控制室，看到的是第 10 章控制階層在流程工業的樣貌：

- 現場儀表：溫度、壓力、流量、液位等傳送器，以及控制閥、馬達與分析儀。一座大型煉油廠的量測點可以多達數萬個。
- 分散式控制系統（DCS）：控制器分散在廠區各處的機櫃中，每台控制器負責數十到數百個迴路；操作站集中在控制室，讓操作員以流程畫面監控全廠。DCS 的設計重點是可靠度：控制器、電源與網路通常都是冗餘配置，可以在不停機的情況下更換故障模組。
- 安全儀表系統（SIS）：與 DCS 獨立，在偵測到危險狀況時自動將製程導入安全狀態，例如緊急停車或洩壓。SIS 依 IEC 61511 等功能安全標準設計，與控制系統分開，是為了避免「控制系統出問題時，連安全保護也一起失效」。
- 歷史資料庫：以秒或分鐘為單位，長期儲存數萬個點位的資料，是流程工業資料分析的基礎（第 23 章）。
- 實驗室資訊管理系統（LIMS）：管理取樣、化驗與品質結果。許多產品性質（成分、黏度、熔融指數）無法線上即時量測，只能定時取樣化驗，結果常延遲數小時。

這個架構在 1970 年代 DCS 問世後就已大致成形。流程工業的挑戰，不是「有沒有自動化」，而是在高度自動化的基礎上，如何做出更好的決策。

18.3.2 營運決策階層

流程工廠的決策，依時間尺度分為數個層級，上層決定下層的目標，下層把結果回報上層：

表 18-4 流程工廠的營運決策階層

層級	時間尺度	主要決策	典型工具
規劃	月、週	買什麼原油或原料、生產哪些產品、各產品多少	線性規劃模型、ERP

層級	時間尺度	主要決策	典型工具
排程	日、班	各單元何時處理哪一批原料、牌號切換順序、儲槽調度	排程系統、APS
即時最佳化	時	依目前價格與設備狀態，計算各單元的最佳穩態操作點	第一原理模型、非線性最佳化
先進製程控制	分	在限制條件下，把製程推向最佳操作點並抑制干擾	模型預測控制、軟感測器
基本控制	秒	維持溫度、壓力、流量、液位於設定值	DCS 的 PID 迴路
安全保護	毫秒到秒	偵測危險並將製程導入安全狀態	SIS、緊急停車系統

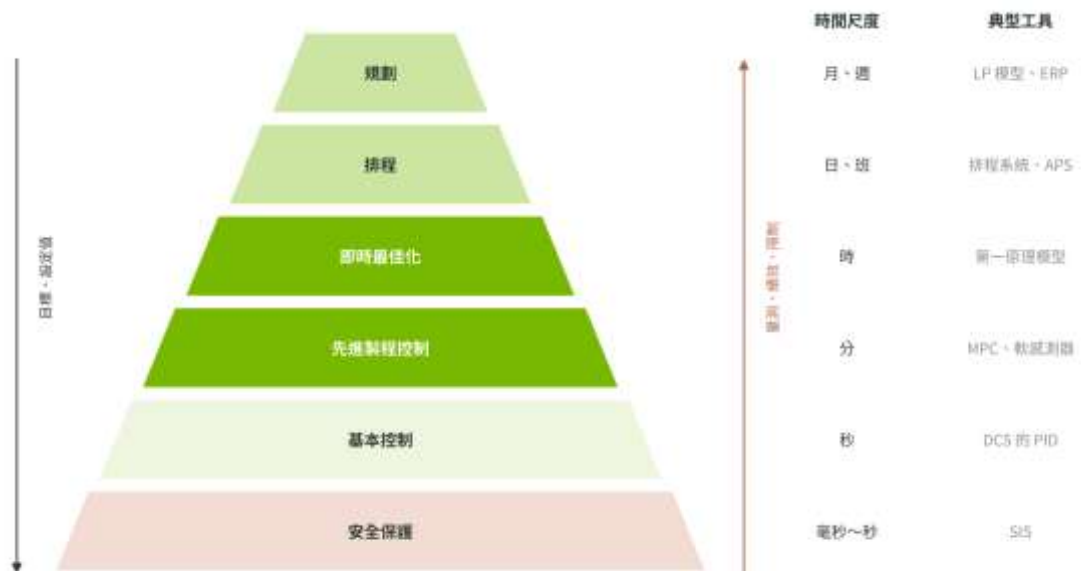


圖 18-5 流程工廠的營運決策階層

這個階層與第 5 章 ISA-95 的層級相對應：規劃與排程屬於第 4 層與第 3 層，即時最佳化與先進製程控制屬於第 3 層與第 2 層之間，基本控制與安全保護屬於第 2 層與第 1 層。它也體現了第 4 章資訊物理系統的分層閉環：每一層都是一個「感測、決策、執行」的迴路，只是時間尺度與決策範圍不同。

階層的價值在於分工：上層不必處理秒級的擾動，下層不必理解市場價格。但階層也有代價：如果上層的模型與下層的實際能力不一致，例如規劃模型以為某個單元可以處理 100 噸，實際上設備結垢後只能處理 90 噸，計畫就無法執行。因此，智慧化的重點之一是讓各層之間的資訊更快、更一致

地流動，這正是 18.4 節縱向整合要解決的問題。

18.3.3 先進製程控制與模型預測控制

DCS 的 PID 迴路各自獨立調節一個變數，難以處理三種情況：變數之間彼此耦合（調整一個會影響另一個）、製程有長的死時間（調整後要好幾分鐘才看得到效果）、以及必須同時遵守多個限制（溫度上限、閥門開度、產品規格）。模型預測控制（MPC）正是為這些情況而生。

MPC 的基本想法很直觀，就像開車時看著前方的路況規劃方向盤與油門：

- 以製程的動態模型（例如步階響應），預測在目前的控制動作下，未來一段時間（預測時域）的輸出會如何變化。
- 在滿足各種限制的前提下，求解未來幾步（控制時域）的最佳控制動作，使預測輸出盡量接近目標，同時避免動作過大。
- 只執行第一步，下一個取樣時間重新量測、重新預測、重新求解，稱為滾動時域。
- 以量測值與模型預測之間的偏差，修正對未量測干擾的估計，讓控制器能夠補償模型誤差。

MPC 起源於流程工業。依 Qin 與 Badgwell 2003 年的工業 MPC 技術回顧：Richalet 等人於 1976 年的研討會上發表 IDCOM 的工業應用，並於 1978 年正式刊出；Shell 石油公司的 Cutler 與 Ramaker 於 1979 年發表動態矩陣控制（DMC）。到 1999 年中，該回顧調查的供應商合計回報了 4,542 個 MPC 應用，其中煉油業約占 44%，石化業次之。

MPC 為什麼值得投資？Qin 與 Badgwell 的回顧引述 Richalet 等人的觀點：真正的經濟效益，來自更好的動態控制讓設定值可以更靠近限制而不違反它。這個概念常被稱為「壓縮與移位」（squeeze and shift）：先壓縮變異，再把目標移向限制。

下面的程式以一個蒸餾塔的產品雜質為例。雜質的規格上限為 1.00%；雜質越低，代表迴流越多、能源消耗越大，所以在不違反規格的前提下，目標越接近上限越省能源。製程以一階加死時間模型描述，原料性質的變動以緩慢漂移的未量測干擾表示。程式比較保守調諧的 PI 控制器與動態矩陣控制（DMC）：

```
import numpy as np
# 先進製程控制（APC）的「壓縮與移位」：比較保守 PI 與模型預測控制（MPC）下的產品雜質變異
```

```

K, tau, theta, dt = 2.0, 10.0, 3, 1.0          # 增益、時間常數（分）、死時間（步）、取樣 1
分鐘
a = np.exp(-dt / tau); b = K * (1 - a)          # 離散模型： $x[k+1] = a \cdot x[k] + b \cdot u[k-\theta]$ 
N = 3000; rng = np.random.default_rng(7)
d = np.zeros(N)
for k in range(1, N): d[k] = 0.97 * d[k-1] + rng.normal(0, 0.03)  # 未量測的原料變動（緩慢
漂移）
def plant(ctrl, sp):
    x, y, u = 0.0, np.zeros(N), np.zeros(N)
    for k in range(N):
        y[k] = x + d[k]                                # 雜質相對於目標的偏差（百分點）
        u[k] = np.clip(ctrl(k, y, u, sp), -1.5, 1.5)    # 操作變數（例如迴流比）上下限
        x = a * x + b * (u[k - theta] if k >= theta else 0.0)
    return y, u
def make_pi(tc):                                       # SIMC 調諧：tc 越大越保守
    Kc = tau / (K * (tc + theta)); Ti = min(tau, 4 * (tc + theta)); s = {"I": 0.0}
    def c(k, y, u, sp):
        err = sp - y[k]; s["I"] += err * dt / Ti
        return Kc * (err + s["I"])
    return c
def make_mpc(P=25, M=5, lam=0.3):                     # 動態矩陣控制（DMC）
    step = np.array([b * (1 - a**(i - theta)) / (1 - a) if i > theta else 0.0 for i in
range(1, P + 1)])
    S = np.array([[step[i - j] if i >= j else 0.0 for j in range(M)] for i in range(P)])
    Kg = np.linalg.solve(S.T @ S + lam * np.eye(M), S.T)[0]  # 只執行第一步（滾動時域）
    s = {"xm": 0.0}
    def c(k, y, u, sp):
        if k: s["xm"] = a * s["xm"] + b * (u[k - 1 - theta] if k - 1 >= theta else 0.0)
        bias = y[k] - s["xm"]                                # 量測與模型的差，視為持續的干擾
        ulast = u[k - 1] if k else 0.0; xf, free = s["xm"], []
        for i in range(1, P + 1):                            # 自由響應：未來輸入維持不變
            j = k + i - 1 - theta
            xf = a * xf + b * (u[j] if 0 <= j < k else (ulast if j >= k else 0.0))
            free.append(xf + bias)
        return ulast + Kg @ (sp - np.array(free))
    return c
yp, up = plant(make_pi(30.0), 0.0); ym, um = plant(make_mpc(), 0.0)
s1, s2 = yp[300:].std(), ym[300:].std()
print(f"雜質標準差：保守 PI {s1:.3f}，MPC {s2:.3f}，降低 {1 - s2/s1:.0%}（壓縮）")
limit = 1.00                                           # 雜質規格上限 1.00%
t1, t2 = limit - 2 * s1, limit - 2 * s2               # 保留兩倍標準差的安全餘裕
print(f"可設定的雜質目標：PI {t1:.2f}%，MPC {t2:.2f}%，向上限移動 {t2 - t1:.2f} 個百分點（移
位）")
v = 15                                                # 假設：雜質目標每提高 0.01 個百
分點，每年省能源 15 萬元
print(f"每年效益約 {(t2 - t1) / 0.01 * v:.0f} 萬元（假設值）")

```

執行結果：保守 PI 下雜質的標準差約 0.097 個百分點，MPC 約 0.067，降低約 31%，這是「壓縮」。若保留兩倍標準差的安全餘裕，PI 的雜質目標只能設在 0.81%，MPC 可以設在 0.87%，向上限移動約 0.06 個百分點，這是「移位」。以假設的能源價值換算，每年效益約 89 萬元。

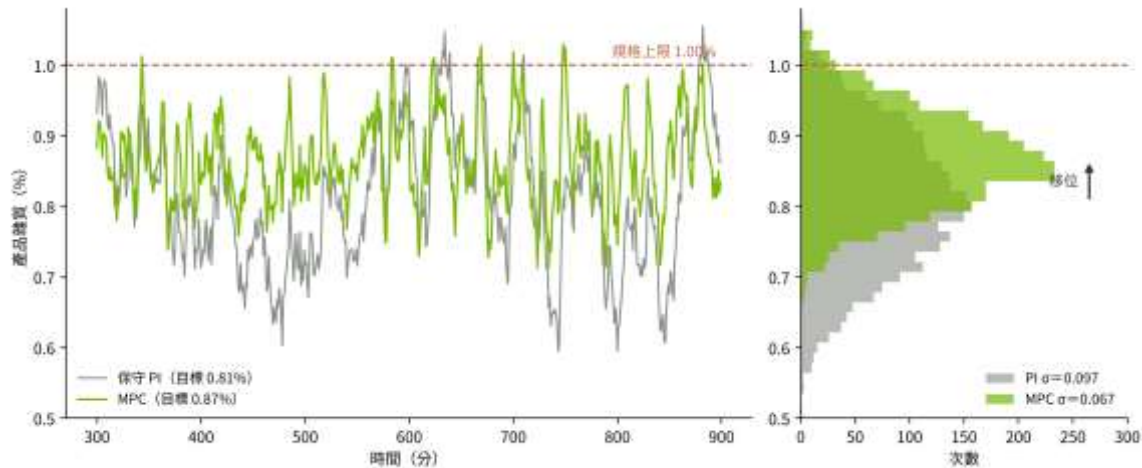


圖 18-6 壓縮與移位：變異降低後，目標可以更靠近規格上限

這個例子也提醒我們兩件事。第一，若把 PI 好好重新調諧（程式中 $t_c = 5$ ），標準差可以降到約 0.074，已經拿到 MPC 大部分的效益。對單一迴路而言，「先把基本控制調好」往往是最便宜的第一步；MPC 真正的優勢，在於多變數耦合、長死時間與多重限制同時存在的場合，例如一座蒸餾塔同時要控制塔頂與塔底的品質、塔壓與再沸器負荷。第二，MPC 的操作變數變動較大（程式中標準差約為 PI 的兩倍），這代表閥門與設備的負擔增加，實務上要以移動懲罰權重（程式中的 lam ）取得平衡。

MPC 的效益很大程度上取決於模型的品質。當製程特性隨時間改變（觸媒老化、設備結垢），模型就會失準，控制效果逐漸下降。許多工廠的 APC 上線幾年後，因為缺乏維護而被操作員關掉，這是流程工業常見的「APC 退化」問題。因此，模型的監控與更新，與 APC 本身同樣重要，第 20 章的模型治理與第 26 章的製程最佳化將進一步討論。

18.3.4 資料協調與粗大誤差偵測

流程工廠的決策仰賴量測，但量測永遠有誤差。更麻煩的是，量測之間常常互相矛盾：流進一個分離塔的流量，與流出的兩股流量加總，通常不會剛好相等。如果直接用這些數字計算物料平衡、產率或能源效率，結果可能差很多，甚至讓即時最佳化走錯方向。

資料協調以質量平衡（或能量平衡）作為約束，用加權最小平方法找出「最小幅度」的修正，讓修正後的數值滿足平衡。量測越不準（標準差越大），被修正得越多。這個方法在化工界已有數十年的歷史，例如 Kuehn 與 Davidson 於 1961 年就在《化學工程進展》（Chemical Engineering

Progress) 發表了以最小平方方法調整量測的解法。

設量測值為 y ，量測的共變異矩陣為 V ，平衡方程式為 $Ax = 0$ ，協調後的估計值為：

$$\hat{x} = y - VA^T(AVA^T)^{-1}Ay$$

協調還有另一個重要功能：找出粗大誤差，也就是儀表故障、校正偏移或管線洩漏造成的大誤差。若量測只有隨機誤差，平衡殘差 $r = Ay$ 的加權平方和 $\gamma = r^T(AVA^T)^{-1}r$ 應服從卡方分布；若 γ 超過臨界值，就表示某處有粗大誤差。再比較各量測的「標準化修正量」，修正幅度相對最大的量測最可疑。

下面的程式以一個三節點的流程為例：節點 1 把兩股進料混合，節點 2 把混合物分離成兩股，節點 3 再把其中一股分流。七個流量都有量測，其中 x_4 的流量計帶有約 +8 的偏移：

```
import numpy as np
from scipy.stats import chi2
# 資料協調：以加權最小平方修正流量量測，使其滿足質量平衡，並以統計檢定找出粗大誤差
# 節點 1：x1 + x2 = x3（混合）；節點 2：x3 = x4 + x5（分離）；節點 3：x5 = x6 + x7（分流）
A = np.array([[1, 1, -1, 0, 0, 0, 0],
              [0, 0, 1, -1, -1, 0, 0],
              [0, 0, 0, 0, 1, -1, -1]], float)
names = ["x1", "x2", "x3", "x4", "x5", "x6", "x7"]
true = np.array([100, 50, 150, 90, 60, 35, 25], float)
y = np.array([101.5, 49.2, 148.9, 98.3, 59.1, 35.6, 24.2]) # 量測值（x4 含約 +8 的粗大誤差）
sd = 0.02 * true # 量測標準差：讀值的 2%
V = np.diag(sd**2)
def reconcile(y, V):
    r = A @ y # 平衡殘差
    S = A @ V @ A.T
    adj = V @ A.T @ np.linalg.solve(S, r) # 最小修正量
    xh = y - adj
    gamma = r @ np.linalg.solve(S, r) # 整體檢定統計量 ~ 卡方（自由度 = 約束數）
    W = V @ A.T @ np.linalg.solve(S, A @ V) # 修正量的共變異
    z = adj / np.sqrt(np.diag(W) + 1e-12) # 各量測的標準化修正量
    return xh, r, gamma, z
xh, r, g, z = reconcile(y, V)
print(f"整體檢定  $\gamma = \{g:.2f\}$ ，臨界值（95%，自由度 3）= {chi2.ppf(0.95, 3):.2f}")
k = int(np.argmax(np.abs(z))); print("最可疑的量測：", names[k])
V2 = V.copy(); V2[k, k] = 1e8 # 剔除可疑量測（視為未量測），重新協調
xh2, r2, g2, z2 = reconcile(y, V2)
print(f"剔除 {names[k]} 後  $\gamma = \{g2:.2f\}$ ，臨界值（自由度 2）= {chi2.ppf(0.95, 2):.2f}")
```

執行結果：三個節點的平衡殘差分別為 1.8、-8.5、-0.7，整體檢定 $\gamma \approx 9.23$ ，超過自由度 3、95% 信心水準的臨界值 7.81，表示有粗大誤差。各量測的標準化修正量中， x_4 最大（約 2.96），被判定為最可疑。把 x_4 視為未量測（給它極大的變異）後重新協調， γ 降到約 0.46，低於自由度 2 的臨界值 5.99，數據恢復一致；協調後的 x_4 約為 90.5，接近真值 90。七個流量與真值的絕對誤差總

和，從原始量測的 14.0 降到 4.2。

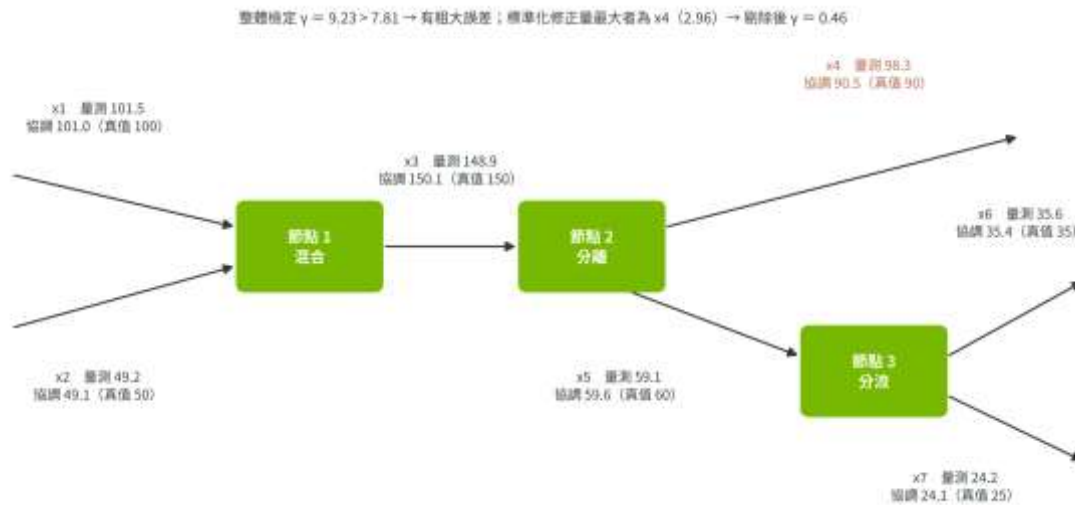


圖 18-7 流程網路的資料協調與粗大誤差偵測

值得注意的是，如果不先剔除粗大誤差就直接協調， x_4 的誤差會被「分攤」到其他流量上：程式的第一次協調把 x_1 修正為 103.9、 x_3 修正為 153.7，反而比原始量測更偏離真值。這說明資料協調必須與粗大誤差偵測搭配使用。實務上，工廠會每天或每班自動執行協調，把被判定為可疑的儀表列入校正清單，這也是第 26 章預測維護的一部分；協調後的數據，再提供給即時最佳化、產率計算與第 34 章的碳排放盤查使用。

18.3.5 軟感測器：量測不到的品質

流程工業的另一個難題，是最重要的品質指標往往無法即時量測。聚合物的熔融指數、汽油的辛烷值、鋼的機械性質，通常要取樣送實驗室化驗，數小時後才有結果。等結果出來，這段時間生產的產品可能早已不合格。

軟感測器以容易量測的變數（溫度、壓力、流量）建立模型，即時估計難以量測的品質。建立軟感測器的方法有兩大類：一是第一原理模型，依質量、能量平衡與反應動力學推導；二是資料驅動模型，以歷史資料訓練迴歸或機器學習模型（第 22 章）。實務上兩者常結合：以物理模型提供結構，以資料修正參數。每次實驗室化驗結果出爐，再用它來校正軟感測器的偏差。

軟感測器的估計值可以作為 MPC 的受控變數，讓品質控制從「事後化驗」變成「即時調整」。這是流程型智慧工廠中，AI 與控制結合最成熟的應用之一。

18.3.6 批次流程的智慧化

批次型的流程工廠，例如製藥、食品與特用化學品，有它自己的智慧化重點：

- 配方管理：依第 17 章的 ISA-88，把通用配方、廠區配方、主配方與控制配方分層管理，確保研發的配方正確地轉換到不同的設備上執行。
- 黃金批次比對：把過去品質最好的批次的溫度、壓力等軌跡當作基準，即時比對目前批次的軌跡，偏離時提早示警。由於每批的時間長短不一，常需要以時間校準或以反應進度作為比對的基準。
- 電子批記錄與放行：第 17 章的電子批記錄，讓品保以例外審查加速放行。
- 設備使用率與排程：批次設備常被多種產品共用，清洗與換線時間可觀，排程與第 27 章的方法密切相關。

近年來，製藥業也逐漸從批次生產走向連續生產：把原本一批一批進行的反應、結晶、乾燥與壓錠，串成連續的流程，以減少中間儲存與等待，並以製程分析技術（Process Analytical Technology，PAT）即時監控品質。這正是「離開產品—製程矩陣對角線」的例子：用連續流程的方式生產原本屬於批次的產品。

18.4 智慧集成製造系統：縱向、橫向與端到端整合

18.4.1 從電腦整合製造到智慧集成製造

「整合」是製造資訊化的老問題。1970 年代初期，Joseph Harrington 在《電腦整合製造》（Computer Integrated Manufacturing，一般引用為 1973 年）一書中提出以電腦整合設計、生產與管理的構想，即 CIM。1980 年代，CIM 成為許多國家推動製造業升級的口號，但在當時的技術條件下，系統之間的連接昂貴而脆弱，許多計畫並未達到預期的效果。

智慧製造重新提出整合的目標，但有了新的工具：標準化的通訊與資訊模型（第 5、11 章）、低成本的感測與運算、雲端與資料平台，以及數位孿生與 AI。德國工業 4.0 工作小組在 2013 年 4 月提交的實施建議中，以三種整合描述工業 4.0 的特徵：透過價值網路的橫向整合、貫穿整個價值鏈的端到端數位工程整合，以及縱向整合與網路化的製造系統。這三種整合，也常被用來描述「智慧集成製

造系統」的範圍。

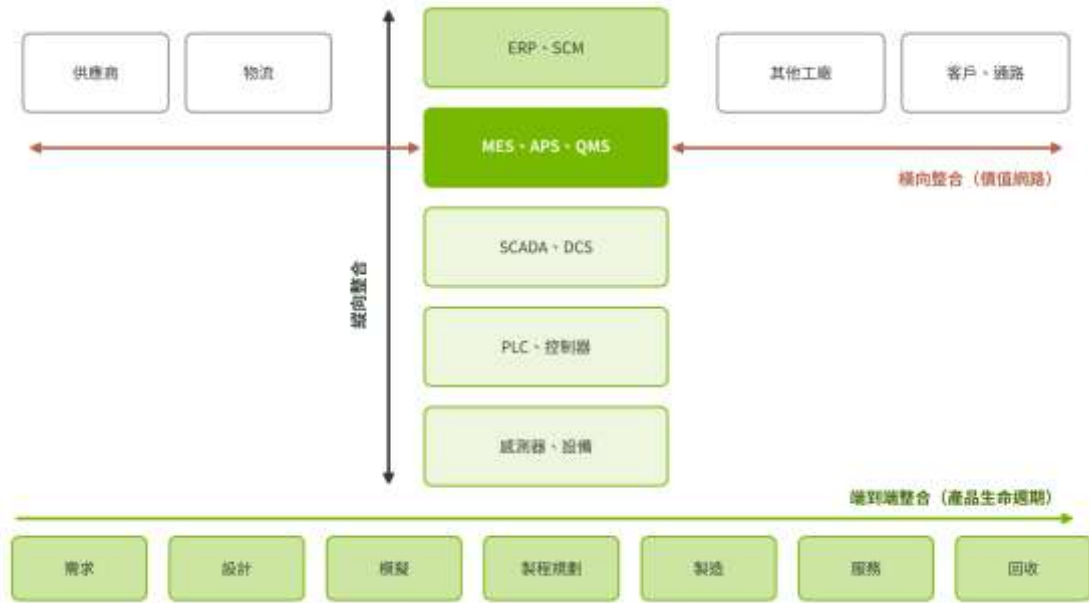


圖 18-8 縱向、橫向與端到端整合

表 18-5 三種整合的比較

整合	方向	整合的對象	關鍵技術與標準	本書相關章節
縱向整合	由下而上	感測器、控制器、SCADA、MES、ERP	ISA-95、OPC UA、統一命名空間	第 5、11、17 章
橫向整合	由左而右	部門、工廠、供應商、物流、客戶	EDI、供應鏈平台、資料空間	第 33 章
端到端整合	沿生命週期	需求、設計、規劃、製造、服務、回收	PLM、數位主線、AAS、數位孿生	第 12、19、20 章

18.4.2 縱向整合：讓現場與企業說同一種語言

縱向整合把第 5 章 ISA-95 的各層串起來：現場事件能夠即時反映到企業系統，企業的決策也能夠正確地傳到現場。第 17 章叫獸開講中「帳上庫存與架上庫存不符」的問題，就是縱向整合不足的典型症狀。

縱向整合的一個關鍵指標是資訊延遲。下面的程式比較兩種架構：傳統的逐層傳遞（SCADA 每 5 分鐘彙總一次給 MES，MES 每 4 小時以批次介面傳給 ERP），以及第 11 章的事件驅動架構（現場

事件經由邊緣閘道發布到共同資料層，各系統即時訂閱）：

```
# 系統整合的介面數量：點對點連接 vs. 經由共同資料層（中樞）
def p2p(n): return n * (n - 1) // 2      # 每兩個系統之間一條介面
def hub(n): return n                    # 每個系統只接到共同資料層
for n in [4, 8, 12, 20]:
    print(f"{n}>2} 個系統：點對點 {p2p(n)>3} 條，中樞 {hub(n)>2} 條")
# 縱向整合的資訊延遲：現場事件要經過幾層、每層多久，才會反映到決策
chain = [("感測器→PLC", 0.01), ("PLC→SCADA", 1), ("SCADA→MES", 300), ("MES→ERP（批次介面）", 4 * 3600)]
chain2 = [("感測器→PLC", 0.01), ("PLC→邊緣閘道", 1), ("閘道→共同資料層（事件發布）", 2), ("資料層→MES、ERP 訂閱", 5)]
tot, tot2 = sum(s for _, s in chain), sum(s for _, s in chain2)
print(f"逐層傳遞約 {tot/3600:.1f} 小時；事件驅動約 {tot2:.0f} 秒")
```

執行結果：逐層傳遞時，一個現場事件（例如某批產品報廢）要約 4.1 小時才會反映到 ERP；改為事件驅動後，約 8 秒內即可被上層系統取得。程式的前半段計算介面數量：12 個系統若兩兩點對點連接，最多需要 66 條介面，20 個系統更多達 190 條；經由共同資料層（中樞）則只需 12 條與 20 條。新增一個系統時，點對點架構最多要新建 12 條介面，中樞架構只要 1 條。

當然，實務上不會每兩個系統都直接相連，點對點的數字是最壞的情況。但它說明了為什麼第 11 章的統一命名空間（UNS）與共同資料層受到重視：介面數量隨系統數量平方成長，是許多工廠「系統越多、整合越亂」的根本原因。不過，中樞架構也有它的風險：中樞本身成為單點故障與資安的焦點，資料的語意與品質也需要集中治理（第 20、35 章）。

縱向整合在兩種工廠的重點不同。離散工廠的縱向整合，常卡在設備的多樣性：不同年代、不同廠牌的工具機與機器人，通訊協定各不相同，第 11 章的 OPC UA 與邊緣閘道是解方。流程工廠的 DCS 與歷史資料庫已經相當成熟，縱向整合的瓶頸則常在第 3 層與第 4 層之間：即時最佳化的結果如何與規劃排程一致、實驗室的品質資料如何即時回到控制層。

18.4.3 橫向整合：從工廠到價值網路

橫向整合有兩個範圍。在企業內部，是部門之間與工廠之間的整合：研發、生管、品管、採購、倉儲與財務共用一致的資料，多個廠區之間可以互相調度產能。在企業之間，是價值網路的整合：品牌商、代工廠、零件供應商、物流與客戶共享預測、訂單、庫存與品質資訊。

橫向整合最大的障礙通常不是技術，而是信任與利益：企業不願意分享自己的庫存與產能，擔心被壓價或洩密。近年來，歐洲推動的工業資料空間（例如 2021 年啟動、2023 年上線的 Catena-X 汽

車產業資料生態系，被稱為第一個工業化、符合 Gaia-X 原則的資料生態系），嘗試以共同的規則、身分驗證與使用控制，讓企業在「保有資料主權」的前提下分享資料。第 33 章將深入討論網路協同製造與數位供應鏈。

18.4.4 端到端整合：沿著生命週期

端到端整合沿著產品的生命週期：從客戶需求、設計、模擬、製程規劃、生產、品質、服務到回收，讓每一個階段的資料都能追溯到上一個階段，並回饋給下一代的設計。第 12 章的 PLM 與產品數位孿生、第 13 章的模擬驅動設計，都是端到端整合的一部分；第 20 章的數位主線，則是把這條線完整串起來的方法。

端到端整合在兩種工廠也有不同的面貌。離散型產品的端到端整合，重點是設計與製造之間的一致：工程變更如何即時反映到產線的作業指示與檢驗計畫。流程型產品的端到端整合，重點常是配方與製程之間的一致：研發實驗室的配方如何放大到試產與量產，量產的經驗如何回饋到配方的改良，這與 ISA-88 的配方分層密切相關。

18.4.5 智慧集成製造系統的架構

綜合三種整合，一個智慧集成製造系統的架構大致包含：

- 邊緣層：設備與控制系統，以標準化的協定（OPC UA、MQTT）提供資料，並在邊緣進行即時的處理與控制（第 11 章）。
- 資料層：共同資料層或統一命名空間，以一致的語意模型（第 20 章的資產管理殼、知識圖譜）描述資產、物料與事件。
- 應用層：ERP、MES、APS、QMS、PLM 等系統，以及數位孿生、分析與 AI 應用，透過資料層互相溝通。
- 協同層：與供應商、客戶與服務夥伴的資料交換，在權限與資料主權的控管下進行。
- 治理層：資料品質、資安、模型管理與組織分工（第 35、36 章）。

這個架構對離散與流程工廠都適用，差別在於各層的重點與既有的基礎。流程工廠的邊緣層與控制層已很成熟，投資重點常在資料層與最佳化；離散工廠的設備連網常是第一步，接著才是資料層與

應用。第 6 章介紹的成熟度評估，可以幫助工廠找出自己目前最弱的一環，決定整合的優先順序。

18.5 燈塔工廠案例比較：電子、石化、鋼鐵、半導體

18.5.1 全球燈塔網絡

2018 年，世界經濟論壇與麥肯錫公司共同成立「全球燈塔網絡」(Global Lighthouse Network)，從全球數千座工廠中，挑選出大規模應用第四次工業革命技術、並在營運與財務上取得顯著成效的工廠，稱為「燈塔」，意思是為其他工廠照亮前進的方向。第一批 9 座燈塔於 2018 年 9 月公布；2019 年 1 月新增 7 座，網絡擴大為 16 座。此後逐年擴充：依世界經濟論壇 2025 年 9 月的新聞稿，網絡已有 201 座；2026 年 1 月再新增 23 座，總數超過 220 座，分布在 30 多個國家。

燈塔的評選類別也隨時間擴充。早期以「生產力」為主，後來陸續加入端到端價值鏈、永續等類別；2026 年 1 月公布的名單，分為生產力、供應鏈韌性、永續、以客戶為中心與人才五類。這個變化本身就反映了智慧製造的演進：從單一工廠的效率，擴展到價值鏈、環境與人。

燈塔工廠的價值，不在於它們用了多新奇的技術，而在於它們把技術「規模化」：許多企業的智慧製造停留在零星的試點，做了很多概念驗證，卻始終無法推廣到全廠，常被稱為「試點煉獄」(pilot purgatory)。燈塔工廠則證明了技術可以在整座工廠、甚至整個企業推廣，並產生可衡量的效益。

18.5.2 四個產業的比較

下表整理四個產業中幾座燈塔工廠的公開資訊。成效數字為企業或媒體公布的數據，計算基準與期間各不相同，只能作為趨勢參考，不宜直接比較。

表 18-6 四個產業燈塔工廠的比較 (依公開報導整理)

產業	例子 (入選時間)	製造型態	公開提及的重點應用	公開提及的成效
電子	鴻海深圳廠區 (2019 年 1 月)	離散型，大量組裝	全自動化製程、機器學習與 AI 最佳化、即時生產監控	生產效率提升約 30%，庫存週期降低約 15%

產業	例子 (入選時間)	製造型態	公開提及的重點應用	公開提及的成效
電子 (面板)	友達台中廠、群創高雄廠 (2021 年 9 月)	混合型，前段 為流程、後段 為組裝	智慧製造與 AI 品質管理	友達：產能提升約 32%、 生產成本降低約 15%；群 創：最終不良率降低約 33%、品質調查時間降低 約 90%
石化 與能 源	沙烏地阿美 Uthmaniyah 天然氣處理廠 (2019 年 1 月)	連續型流程	第四次工業革命技術的整 合應用	依媒體報導，為當時中東 唯一的燈塔；該公司後續 累計五座設施入選
鋼鐵	POSCO 浦項廠、Tata Steel Kalinganagar 廠 (2019 年 7 月)	流程型為主， 後段為離散產 品	POSCO：AI 智慧高爐、連 續鍍鋅線鍍層控制； Tata：大規模預測維護、 數位化與 AI	Tata：將預測維護推廣到 約 5 萬台機器
半導體	美光台灣廠區 (2020 年 9 月)	混合型，批次 流程與逐片追 蹤	AI 影像分析、大數據、自 動化	員工生產力提升約 18%、 非計畫停機降低約三成 (各報導為 30% 至 34%)、不合格品降低約 40%、良率爬升時間縮短 約 20%、總用電降低約 15%

從這張表可以看出幾個規律：

- 電子業的重點在自動化、彈性與品質：產品生命週期短、換線頻繁、人工占比高，所以自動化與 AI 檢測帶來直接效益。
- 石化與能源業的重點在設備可靠度、最佳化與安全：設備資本密集、非計畫停機代價極高，所以預測維護、遠端監控與製程最佳化是主軸。
- 鋼鐵業同時具有流程與離散的特徵：前段的高爐與煉鋼著重能源與製程控制（例如以 AI 預測高爐爐溫與鐵水品質），後段的軋延與鍍層著重品質與追溯；設備數量龐大，預測維護的規模化是一大挑戰。

- 半導體業的重點在良率、設備效率與資料分析：一座晶圓廠每天產生大量的量測與影像資料，設備折舊占成本比重高，良率與設備稼動率的每一點提升都有很大的價值。

依公開報導，美光台灣廠區於 2020 年 9 月入選，是台灣第一個入選燈塔網絡的廠區；隔年，友達台中廠與群創高雄廠也入選。這些案例說明，台灣的半導體與面板產業在智慧製造上已具備國際競爭力，第 37 章將有更多台灣案例的深入分析。

18.5.3 成本結構決定投資優先順序

為什麼不同產業的燈塔工廠，重點如此不同？一個重要的原因是成本結構。同樣是「改善 5%」，改善的是人工、能源還是設備產出，對總成本的影響可能差好幾倍。

下面的程式以四個產業的假設成本結構（教學用的示意數值，不代表任何特定企業），計算四種典型槓桿可以降低的單位成本：

```
import numpy as np
# 不同產業的成本結構，決定了智慧製造「該先投資哪裡」（成本結構為教學用的假設值）
industries = ["電子組裝", "鋼鐵", "石化", "半導體"]
items = ["材料", "能源", "人工", "折舊", "其他"]
cost = np.array([[75, 2, 12, 6, 5],      # 電子組裝：材料占比高、人工次之
                 [55, 20, 8, 10, 7],    # 鋼鐵：原料與能源占比高
                 [75, 12, 3, 6, 4],     # 石化：原料（進料）為主，能源次之
                 [15, 8, 15, 45, 17]],   # 半導體：設備折舊占比最高
                 float)
# 四種典型的智慧製造槓桿（教學用假設值）
levers = {
    "材料利用率 +1%": lambda c: c[0] * 0.01,
    "能源效率 +5%": lambda c: c[1] * 0.05,
    "人工生產力 +10%": lambda c: c[2] * (1 - 1 / 1.10),
    "OEE（產出）+5%": lambda c: (c[3] + c[4] * 0.5) * (1 - 1 / 1.05), # 固定成本被更多產出分攤
}
res = np.array([[f(c) for f in levers.values()] for c in cost])
for name, r in zip(industries, res):
    print(f"{name:<8}" + " ".join(f"{v:>8.2f}" for v in r), " 最大：",
          list(levers)[int(np.argmax(r))])
```

執行結果（各槓桿可降低的單位成本，占總成本的百分比）：

表 18-7 不同成本結構下四種槓桿的效益（假設值）

產業	材料利用率 +1%	能源效率 +5%	人工生產力 +10%	OEE +5%	效益最大的槓桿
電子組裝	0.75	0.10	1.09	0.40	人工生產力
鋼鐵	0.55	1.00	0.73	0.64	能源效率

產業	材料利用率 +1%	能源效率 +5%	人工生產力 +10%	OEE +5%	效益最大的槓桿
石化	0.75	0.60	0.27	0.38	材料利用率
半導體	0.15	0.40	1.36	2.55	OEE (產出)



圖 18-9 不同產業成本結構下的智慧製造槓桿 (假設值)

結果與燈塔工廠的實際重點大致吻合：電子組裝的人工生產力槓桿最大，所以自動化是主軸；鋼鐵的能源效率槓桿最大，所以 AI 高爐與能源管理受到重視；石化的原料占比極高，即使只提升 1% 的原料利用率（例如提高產率、減少過渡料與燃燒排放），效益也超過其他槓桿；半導體的折舊占比最高，提升設備的有效產出（OEE）效益最大。

這個簡化的模型有兩個值得討論的地方。第一，它把各槓桿視為獨立，實際上它們互相關聯：在半導體業，「良率」不只節省材料，而是讓所有成本都由更多的良品分攤，效果類似提升產出，所以半導體業對良率的重視遠超過模型中的「材料利用率」。第二，它只考慮成本，沒有考慮營收：產出提升的前提是市場有需求，否則多做的產品只會變成庫存（第 8 章）。儘管如此，它說明了一個重要的原則：智慧製造的投資，應從自己的成本結構與瓶頸出發，而不是從技術出發。第 6 章的價值樹分析，正是這個原則的系統化方法。

18.5.4 燈塔工廠的共同經驗

雖然各產業的重點不同，燈塔工廠的經驗也有許多共通之處：

- 從問題出發：先找出對營運與財務影響最大的問題，再選擇技術，而不是先買技術再找問題。

- 建立資料基礎：統一的資料平台與資料治理，讓使用案例可以快速複製到其他產線與工廠。這正是 18.4 節整合架構的價值。
- 重視人與組織：燈塔工廠普遍投入大量資源在員工的數位技能培訓與跨部門的組織調整。新增的「人才」類別，也反映了這一點。
- 規模化的機制：以標準化的解決方案範本、內部的推廣團隊與成效追蹤，避免試點煉獄。
- 持續演進：許多燈塔工廠入選後持續擴充使用案例，近年的重點延伸到永續、供應鏈韌性與生成式 AI。

對台灣的中小企業而言，燈塔工廠的規模與投資或許遙不可及，但它們的方法是可以借鏡的：從一個高價值的問題開始，建立可以複製的資料基礎，培養自己的人才，再逐步擴大。第 6 章的低成本路線與第 37 章的台灣案例，將提供更貼近中小企業的做法。

案例

案例 18-1 教學情境：哪一座工廠比較「智慧」？

背景：叫獸開講中的兩次參訪（綜合情境），學生以「機器人多寡」判斷工廠的智慧程度，認為電子廠比石化廠先進。

做法：課堂上請學生分組，以表 18-2 的面向比較兩座工廠，並各自列出三個「最值錢的問題」：電子廠是換線、檢驗與人力；石化廠是能源、過渡料與非計畫停機。再請學生為每個問題找出對應的智慧製造技術。

成果：學生發現兩座工廠都很「智慧」，只是智慧化的方向不同；石化廠的模型預測控制與資料協調，在外觀上看不見，效益卻非常可觀。

啟示：判斷智慧化的程度，要看它是否解決了該製造型態最重要的問題，而不是看機器人的數量。

討論：（1）如果要評估一座食品工廠的智慧化程度，你會看哪些面向？（2）「看不見的智慧」如何向管理者證明它的價值？

案例 18-2 產業：裝配線重新平衡

背景：一家家電組裝廠導入新機種後，產線的瓶頸站經常堆積在製品，其他站則有閒置，整體產出低於需求。

做法：以影像辨識自動量測各作業的時間，更新標準工時；以裝配線平衡重新分配作業，並在瓶頸站加入協作型機器人分擔鎖螺絲的作業；導入電子看板，讓線邊補料依實際耗用自動觸發。

成果：平衡效率明顯提升，瓶頸站的在製品堆積消失，產出達到需求，線邊庫存也同時下降。

啟示：裝配線平衡是經典的工業工程問題，智慧化讓它從一次性的規劃變成持續的調整。

討論：（1）為什麼作業時間的量測方式會影響平衡的結果？（2）協作型機器人適合分擔哪些作業？

案例 18-3 產業：聚合物工廠的牌號切換

背景：一家塑膠原料廠每月要進行數十次牌號切換，每次產生數十噸過渡料，只能降級出售，是很大的損失。

做法：以歷史資料建立反應器的動態模型與熔融指數的軟感測器；以模型預測控制設計切換軌跡，在設備限制內縮短過渡時間；並與排程系統配合，把相近的牌號排在一起，減少切換的幅度。

成果：過渡時間與過渡料明顯減少，軟感測器也讓操作員不必等待實驗室化驗結果就能調整。

啟示：流程工業的「換線」改善，仰賴模型、控制與排程的結合；這是看不見的智慧化。

討論：（1）過衝切換的幅度受到哪些限制？（2）排程如何影響過渡料的總量？

案例 18-4 產業：煉油廠的資料協調

背景：一家煉油廠每天的物料平衡總是對不上，各單元的產率報表互相矛盾，管理者不知道該相信哪個數字。

做法：建立全廠的質量平衡模型，每天自動執行資料協調與粗大誤差偵測；把被判定為可疑的流

量計列入校正清單，並追蹤校正後的改善；協調後的數據提供給產率計算與即時最佳化使用。

成果：物料平衡的差異大幅縮小，數個長期偏移的流量計被找出並校正，產率報表的可信度提高，最佳化的建議也更容易被操作人員接受。

啟示：資料協調是流程工業資料品質的基礎；沒有可信的數據，再好的最佳化也無法發揮。

討論：（1）為什麼不先剔除粗大誤差就協調，可能讓結果更糟？（2）資料協調的結果可以如何支援預測維護？

案例 18-5 台灣情境：半導體與面板的燈塔工廠

背景：依公開報導，美光台灣廠區於 2020 年 9 月成為台灣第一個入選世界經濟論壇燈塔網絡的廠區；2021 年，友達台中廠與群創高雄廠也相繼入選。

做法：依各企業公布的資訊，這些工廠大量應用 AI 影像分析、大數據與自動化，處理晶圓與面板的品質檢測、設備維護與製程最佳化。

成果：依公開報導，美光台灣廠區在生產力、非計畫停機、不合格品、良率爬升時間與用電上都有明顯改善；友達與群創則在產能、成本、不良率與品質調查時間上取得成效。

啟示：半導體與面板產業兼具流程與離散的特徵，資料量大、設備昂貴，是 AI 與大數據最能發揮的場域之一。

討論：（1）為什麼半導體業對良率特別重視？請以 18.5.3 節的成本結構說明。（2）面板與半導體的智慧製造重點有何異同？

案例 18-6 教學情境：成本結構決定先做什麼

背景：一個學生專題團隊協助兩家企業規劃智慧製造：一家是小型電子組裝廠，一家是金屬熱處理廠。兩家老闆都說「想導入 AI」。

做法：團隊先依 18.5.3 節的方法，整理兩家企業的成本結構，再估算各種槓桿的效益。電子組裝廠的人工占比高，團隊建議從自動化檢驗與電子作業指示開始；熱處理廠的能源占比高，團隊建

議先建立爐溫與能耗的監測，再以模型最佳化升溫曲線與裝爐方式。

成果：兩家企業都從效益最大的問題開始，第一年的改善成果讓老闆願意繼續投資。

啟示：智慧製造的起點應是成本結構與瓶頸，而不是熱門的技術名詞。

討論：（1）若企業的瓶頸是市場需求不足，提升 OEE 還有價值嗎？（2）如何為中小企業設計簡單的價值樹？

實作活動

活動 18-1 製造型態分析

選擇三種不同的產品（例如手機、瓶裝飲料、鋼筋），依表 18-2 的面向分析它們的製造型態，並把它們標示在產品—製程矩陣上，說明哪些環節屬於離散型、哪些屬於流程型。

活動 18-2 裝配線平衡

以 18.2.3 節的程式為基礎，選擇一個你熟悉的組裝產品（例如桌上型風扇或機器人社團的底盤），列出作業、時間與先後順序，計算節拍時間並進行平衡；再嘗試「最大作業時間優先」等其他規則，比較工作站數與平衡效率。

活動 18-3 牌號切換策略

以 18.1.4 節的程式為基礎，加入「過衝幅度不得超過 23」的設備限制，比較不同過衝幅度與切回時機下的過渡料；再嘗試把一階模型改為二階模型（兩個串聯的反應器），觀察策略是否需要調整。

活動 18-4 資料協調實驗

以 18.3.4 節的程式為基礎，以隨機數產生 1,000 組量測（其中 10% 隨機加入粗大誤差），統計整體檢定與標準化修正量找出粗大誤差的成功率；比較量測誤差為 1%、2% 與 5% 時的結果。

活動 18-5 壓縮與移位

以 18.3.3 節的程式為基礎，改變 PI 的調諧參數與 MPC 的移動懲罰權重，畫出「受控變數標準差」對「操作變數標準差」的權衡曲線；說明你會如何選擇控制器的參數。

活動 18-6 燈塔工廠研究

選擇一座燈塔工廠，查閱世界經濟論壇與該企業公開的資料，整理它的製造型態、主要使用案例、成效與推廣方式，並以 18.5.3 節的方法，推測它的成本結構如何影響投資重點。相關範例程式將於本書的實驗倉庫中維護。

延伸閱讀

- Robert H. Hayes 與 Steven C. Wheelwright，〈Link Manufacturing Process and Product Life Cycles〉，Harvard Business Review，1979：產品—製程矩陣的原始文章。
- S. Joe Qin 與 Thomas A. Badgwell，〈A Survey of Industrial Model Predictive Control Technology〉，Control Engineering Practice，2003：工業 MPC 的發展與應用回顧。
- 德國工業 4.0 工作小組，Recommendations for Implementing the Strategic Initiative INDUSTRIE 4.0 (2013)：提出橫向、端到端與縱向三種整合。
- 世界經濟論壇與麥肯錫公司，全球燈塔網絡歷年報告：燈塔工廠的使用案例與經驗。
- Wallace J. Hopp 與 Mark L. Spearman，Factory Physics：離散製造系統的產能、變異與庫存（第 8 章已介紹）。

表 18-8 本章思考地圖

本章概念	關鍵提問	後續延伸章節
離散型與流程型	這座工廠最重要的問題是什麼？	第 6、37 章
裝配線平衡與看板	瓶頸在哪裡？補料由需求拉動嗎？	第 8、27、33 章
牌號切換	過渡料可以如何以模型與排程減少？	第 25、27 章
DCS、APC 與決策階層	各層的目標與模型一致嗎？	第 19、26 章
資料協調與軟感測器	數據可信嗎？看不到的品質如何估計？	第 22、23 章

本章概念	關鍵提問	後續延伸章節
縱向、橫向與端到端整合	資訊延遲多久？介面能否複製與治理？	第 11、20、33 章
燈塔工廠與成本結構	投資是否從成本結構與瓶頸出發？	第 6、34、37 章

章末小結

- 製造可分為離散型、批次型與連續型；半導體、鋼鐵與食品等產業兼具兩種特徵。Hayes 與 Wheelwright 於 1979 年提出的產品—製程矩陣，說明產品結構與製程結構的匹配；智慧製造讓企業可以較低的代價離開對角線。
- 離散型與流程型在產品定義、設備、控制、品質、換線、成本結構與安全上有根本差異。流程工業自 1975 年 DCS 問世後即已高度自動化，智慧化的重點在最佳化；離散製造的重點則常在自動化、彈性化與逐件追蹤。本章範例中，過衝切換讓牌號切換的過渡料減少約 61%。
- 離散型智慧工廠整合物理系統、資訊系統與供應鏈。本章以排序位置權重法把 10 個作業分配到 5 個工作站，平衡效率 90%；看板數量由需求率、補貨前置時間、安全係數與每箱容量決定。
- 流程型智慧工廠以 DCS、SIS、歷史資料庫與 LIMS 為基礎，決策分為規劃、排程、即時最佳化、先進製程控制、基本控制與安全保護。MPC 起源於 1970 年代後期的流程工業，效益來自「壓縮與移位」；本章範例中變異降低約 31%，目標可向上限移動約 0.06 個百分點。資料協調以質量平衡修正量測，並以卡方檢定與標準化修正量找出粗大誤差。
- 智慧集成製造系統包含縱向、橫向與端到端三種整合；點對點介面數量隨系統數平方成長，共同資料層與事件驅動架構可以降低介面數量與資訊延遲。
- 全球燈塔網絡於 2018 年成立，至 2026 年初已超過 220 座。電子、石化、鋼鐵與半導體的燈塔工廠重點各不相同，成本結構是重要原因；智慧製造的投資應從成本結構與瓶頸出發，並重視資料基礎、人才與規模化。

習題

一、選擇題

1. 以噸或公升計量、產品通常無法還原為原料的製造型態是？(A) 離散型 (B) 流程型 (C) 零工式 (D) 專案式
2. 產品—製程矩陣由誰提出？(A) Hayes 與 Wheelwright (B) Hopp 與 Spearman (C) Porter 與 Heppelmann (D) Qin 與 Badgwell
3. 1975 年問世、常被稱為最早的分散式控制系統的是？(A) Honeywell TDC 2000 與橫河 CENTUM (B) Modicon 084 (C) SAP R/3 (D) IBM System/360
4. 節拍時間的計算方式是？(A) 作業總時間 $\div$ 工作站數 (B) 可用生產時間 $\div$ 需求數量 (C) 週期時間 $\div$ 加工時間 (D) 需求數量 $\div$ 可用生產時間
5. 獨立於 DCS、在危險時把製程導入安全狀態的系統是？(A) LIMS (B) MES (C) SIS (D) APS
6. 以模型預測未來、在限制下求解最佳動作，並只執行第一步的控制方法是？(A) PID (B) MPC (C) 開關控制 (D) 順序控制
7. Shell 的 Cutler 與 Ramaker 於 1979 年發表的 MPC 演算法是？(A) IDCOM (B) DMC (C) LQR (D) SIMC
8. 資料協調所依據的約束條件通常是？(A) 質量與能量平衡 (B) 產品規格 (C) 設備折舊 (D) 人員排班
9. 世界經濟論壇與麥肯錫成立全球燈塔網絡的年份是？(A) 2011 (B) 2013 (C) 2018 (D) 2022
10. 依公開報導，美光台灣廠區入選燈塔網絡的年份是？(A) 2018 (B) 2020 (C) 2022 (D) 2025

二、簡答題

11. 舉例說明為什麼半導體製造兼具離散型與流程型的特徵。
12. 為什麼流程工業的智慧化常從「最佳化」而非「自動化」開始？
13. 說明裝配線平衡中的位置權重，以及排序位置權重法的步驟。
14. 說明連續製程的牌號切換與離散製程的換線有何不同。
15. 說明流程工廠營運決策階層的各層，以及階層的價值與代價。
16. 說明「壓縮與移位」的意義。
17. 為什麼資料協調必須與粗大誤差偵測搭配使用？
18. 說明縱向、橫向與端到端整合的差異。

三、計算題

19. 每班可用生產時間 7.5 小時，需求 900 件，節拍時間為多少秒？
20. 某產品的作業總時間 210 秒，節拍時間 45 秒，理論最少工作站數為多少？若實際分配為 6 站，平衡效率為多少？
21. 某零件每小時需求 120 個，補貨前置時間 0.25 小時，安全係數 20%，每箱 15 個，需要幾張看板？
22. 某反應器的時間常數 3 小時，產品性質從 10 直接切換到 20，舊規格 9.5 ~ 10.5，新規格 19 ~ 21，產量每小時 8 噸。求過渡時間與過渡料。
23. 一個節點的平衡為 $x_1 = x_2 + x_3$ ，量測值為 100、61、41，三者的量測變異相同。求協調後的數值。
24. 某產品性質的規格上限為 5.0，控制改善前的標準差 0.20，改善後 0.12。若保留兩倍標準差的餘裕，目標可向上限移動多少？
25. 某工廠有 15 個資訊系統，若兩兩點對點連接，最多需要幾條介面？經由共同資料層需要幾條？
26. 依 18.5.3 節的假設，鋼鐵的能源占總成本 20%。若某鋼廠每噸成本 2 萬元、年產 300 萬噸，能源

效率提升 5%，每年可節省多少成本？

四、申論與設計題

27. 選擇一家你熟悉的離散型工廠與一家流程型工廠，比較它們的智慧化重點，並各提出三個優先導入的項目。
28. 評論以下說法：「機器人越多的工廠，智慧化程度越高。」
29. 為一家中小型食品工廠（前段為批次烹煮、後段為包裝）規劃縱向、橫向與端到端整合的藍圖，說明各階段的重點。

五、是非題

30. 流程工業的自動化起步比離散製造晚。
31. 半導體製造兼具離散型與流程型的特徵。
32. 排序位置權重法保證找到工作站數最少的最佳解。
33. 資料協調時若存在粗大誤差，該誤差可能被分攤到其他量測上。
34. 對單一控制迴路而言，重新調諧 PID 有時就能取得 MPC 大部分的效益。
35. 點對點整合的介面數量隨系統數量線性成長。

六、進階挑戰題

36. 以整數規劃（例如 Python 的 PuLP 或 OR-Tools）求解 18.2.3 節的裝配線平衡，與排序位置權重法比較；再加入「D 與 H 不得在同一站」的限制，觀察結果的變化。
37. 擴充 18.1.4 節的程式，讓產品同時有兩個性質需要切換（例如熔融指數與密度），兩者由同一個操作變數影響但時間常數不同，設計使總過渡時間最短的切換策略。
38. 擴充 18.3.3 節的程式為兩個輸入、兩個輸出的 MPC（例如蒸餾塔的塔頂與塔底品質），加入操作變數的上下限與變化率限制，並與兩個獨立的 PI 迴路比較。

七、補充題

39. 列出德國工業 4.0 實施建議中的三種整合。

40. 依時間尺度由長到短，列出流程工廠營運決策階層的六個層級。

提示與參考解答

1. B。

2. A。

3. A。

4. B。

5. C。

6. B。

7. B。

8. A。

9. C。

10. B。

11. 參考：晶圓以批為單位、可逐片追蹤，像離散型；但沉積、蝕刻、擴散等製程本質上是化學與物理反應，以配方控制，像流程型。

12. 參考：流程工業自 1970 年代 DCS 問世後即已高度自動化，大部分迴路自動控制；剩下的效益在於能源、產率、穩定與切換的最佳化。

13. 參考：位置權重為作業本身與所有後續作業的時間總和。步驟：計算權重並排序；依序把前置已完成且放得下的作業分配到目前工作站；放不下時開新工作站，直到所有作業分配完畢。

14. 參考：離散換線是停機換模具、治具與程式；牌號切換時設備不停機，逐步改變條件，過程中產生兩邊都不合格的過渡料，時間受反應器滯留時間影響。

15. 參考：規劃、排程、即時最佳化、先進製程控制、基本控制、安全保護。價值在於分工與時間尺度的分離；代價是上下層模型若不一致，計畫可能無法執行。

16. 參考：先以更好的控制降低變異（壓縮），再把目標移向限制（移位），在不違反規格的前提下

取得經濟效益。

17. 參考：協調以最小平方分攤誤差，若有粗大誤差未被剔除，會被分攤到其他量測，使結果更偏離真值。

18. 參考：縱向整合串連現場到企業系統；橫向整合串連部門、工廠與企業之間的價值網路；端到端整合沿產品生命週期串連設計、製造與服務。

19. $7.5 \times 3,600 \div 900 = 30$ 秒。

20. $\lceil 210 \div 45 \rceil = \lceil 4.67 \rceil = 5$ 站； $210 \div (6 \times 45) \approx 77.8\%$ 。

21. $120 \times 0.25 \times 1.2 \div 15 = 2.4$ ，取 3 張。

22. 過渡時間 $= 3 \times [\ln(10) - \ln(10 / 9.5)] \approx 3 \times 2.251 \approx 6.75$ 小時；過渡料 $\approx 6.75 \times 8 \approx 54$ 噸。

23. 殘差 $r = 100 - 61 - 41 = -2$ ；變異相同時，修正量依約束係數平均分攤，每個量測修正 $2 / 3$ ：

$$x_1 \approx 100.67, x_2 \approx 60.33, x_3 \approx 40.33, \text{滿足 } 100.67 = 60.33 + 40.33。$$

24. 改善前目標 $5.0 - 0.40 = 4.60$ ，改善後 $5.0 - 0.24 = 4.76$ ，可移動 0.16。

25. $15 \times 14 \div 2 = 105$ 條；共同資料層 15 條。

26. 每噸節省 $2 \text{ 萬} \times 20\% \times 5\% = 200$ 元；每年 $200 \times 300 \text{ 萬} = 6$ 億元。

27. 評分重點：能從製造型態與成本結構出發，指出各自最重要的問題，提出的項目具體、可行並有評估指標。

28. 評分重點：能指出智慧化程度應以是否解決該製造型態的關鍵問題來判斷；以流程工業的 APC、資料協調為例說明「看不見的智慧」。

29. 評分重點：前段批次與後段離散的特徵都有考慮；縱向整合（設備連網、MES）、橫向整合（供應商與通路）、端到端整合（配方與產品開發）分階段規劃，並說明資料與人才的配套。

30. X。流程工業自 1970 年代 DCS 問世後即已高度自動化，起步較早。

31. ✓。

32. X。排序位置權重法是啟發式方法，不保證最佳解。

33. ✓。

34. ✓。

35. ✗。點對點介面數量最多為 $n(n-1) \div 2$ ，隨系統數量的平方成長。

36. 評分重點：正確建立指派變數與前置、節拍限制，求得最佳站數並與 RPW 比較；能正確加入互斥限制並解釋結果。

37. 評分重點：能模擬兩個不同時間常數的性質，說明兩者無法同時最快到達規格的權衡，提出合理的策略並以過渡料比較。

38. 評分重點：正確建立多變數步階響應矩陣與限制，能顯示 MPC 在耦合與限制下的優勢，並討論操作變數的變動。

39. 透過價值網路的橫向整合、貫穿整個價值鏈的端到端數位工程整合、縱向整合與網路化的製造系統。

40. 規劃、排程、即時最佳化、先進製程控制、基本控制、安全保護。

第 19 章

製造數位孿生基礎

編著 李世淵（機器人叫獸） | 助教 李天宇、李宇晴

本章重點

- 數位模型、數位影子與數位孿生的資料流與閉環
- 產品、設備、製程、產線與工廠孿生及其組合
- ISO 23247 參考架構與可觀測製造元素
- 即時同步、資料新鮮度與適合用途的模型保真度
- 以卡爾曼濾波器進行狀態估測與剩餘壽命預測
- 平台生態、商業價值的蒙地卡羅評估與導入風險

叫獸開講：會動的 3D 模型，就是數位孿生嗎？

前幾年，「數位孿生」成了展覽會場上最熱門的名詞。有一次，一家工廠的主管很興奮地邀我去看他們剛完成的「數位孿生工廠」。

會議室的大螢幕上，是一座做得非常漂亮的 3D 工廠：每一台機台、每一條輸送帶、每一個料架都栩栩如生，滑鼠一拖，就可以從空中俯瞰整座廠房，再飛進產線，看到機器手臂在上下料。主管說：「這是我們請廠商花了半年建的，未來新產線的規劃、客戶參觀，都可以用它來展示。」

我問：「畫面上這台 CNC 現在顯示運轉中，實際上它現在在做什麼？」

主管愣了一下，請工程師去現場看。結果那台機台其實已經停機兩個小時，正在等待維修。工程師解釋，畫面上的狀態是「示意動畫」，機台的移動資料每週由人員從報表整理後匯入一次。

我又問：「如果這個模型算出某台機器的參數需要調整，它會怎麼做？」

工程師說：「它不會算，它只是畫面。」

我沒有潑冷水，而是請他們換個方式想：這個 3D 模型是一個很好的起點，但嚴格來說，它是「數位模型」，還不是「數位孿生」。真正的數位孿生，要和實體保持同步，要能用模型理解與預測實體的狀態，還要能把決策送回實體，形成閉環。後來，他們挑了廠內最常出問題的一台精密加工機，從主軸溫度與熱變位補償做起：感測器每分鐘把溫度送進模型，模型即時估算熱變位，並自動把補償值寫回控制器。三個月後，這台機器的尺寸超差件數大幅下降。那座漂亮的 3D 工廠也沒有浪費：它成了把一台台設備孿生組合起來的視覺化介面。

這一章，我們要釐清數位模型、數位影子與數位孿生的差異；介紹產品、設備、製程、產線與工廠等不同層級的孿生，以及它們如何組合；說明國際標準 ISO 23247 為製造數位孿生定義的參考架構；討論即時同步、模型保真度與狀態估測這三個讓孿生「可信」的關鍵技術；最後談數位孿生的平台生態、商業價值與導入風險。

（說明：本節情境由作者協助企業導入數位孿生的經驗綜合改寫，人物與細節已調整。）

學習目標

讀完本章後，你應該能夠：

1. 說明數位孿生的起源，並依資料流的自動化程度區分數位模型、數位影子與數位孿生，以程式比較三者在此變位補償上的效果。
2. 說明產品、設備、製程、產線與工廠孿生的範圍與用途，以及孿生的組合，並以程式進行產線孿生的假設分析。
3. 說明 ISO 23247 系列標準的組成、參考模型中的各個實體，以及可觀測製造元素的類型，並能以簡化程式實作其資料流。
4. 說明即時同步的需求與資料新鮮度，並以卡爾曼濾波器進行狀態估測與剩餘壽命預測。
5. 說明模型保真度與「適合用途」的原則，並以程式分析模型精度與計算成本的權衡。
6. 說明數位孿生的平台生態、商業價值與導入風險，並以蒙地卡羅模擬評估專案的淨現值與風險。

核心概念

表 19-1 本章核心名詞中英對照

中文名詞	英文	一句話說明
數位模型	Digital Model	實體的數位表示，與實體之間沒有自動的資料流
數位影子	Digital Shadow	實體的資料自動流入數位表示，但回饋到實體仍需人工
數位孿生	Digital Twin	實體與數位表示之間雙向自動同步，並以模型支援決策
可觀測製造元素	Observable Manufacturing Element, OME	ISO 23247 中可被觀測、被孿生的製造元素，例如設備、物料、人員
同步	Synchronization	讓數位表示的狀態與實體保持一致的機制與頻率
資料新鮮度	Data Freshness	孿生中的資料距離實際發生的時間有多久
模型保真度	Model Fidelity	模型描述實體行為的精確與完整程度

中文名詞	英文	一句話說明
適合用途	Fit for Purpose	模型的保真度與同步頻率以應用需求為準，而非越高越好
狀態估測	State Estimation	融合模型與雜訊量測，估計無法直接量測的狀態
卡爾曼濾波器	Kalman Filter	以預測與修正兩步驟，遞迴估計線性系統狀態的最佳估測器
剩餘使用壽命	Remaining Useful Life, RUL	設備或零件到達失效門檻前還能使用的時間或次數
孿生組合	Twin Composition	把多個孿生組合成更大範圍孿生的方法，例如設備組成產線
假設分析	What-if Analysis	在孿生上嘗試不同的決策，比較結果後再用於實體
平台生態	Platform Ecosystem	支撐孿生的軟體、資料、模型與服務供應商所形成的生態系

19.1 數位模型、數位影子與數位孿生

19.1.1 數位孿生的起源

數位孿生的想法並不新。在阿波羅計畫中，NASA 在地面保留與太空中相同的飛行器與模擬系統，用來分析太空中發生的問題、測試解決方案，這常被視為「孿生」概念的前身。

在製造領域，一般認為 Michael Grieves 於 2002 年在密西根大學一場關於產品生命週期管理的報告中，提出了實體空間、虛擬空間與兩者之間資訊流的概念模型，最初稱為「鏡像空間模型」(Mirrored Spaces Model)，後來改稱「資訊鏡像模型」。「數位孿生」(Digital Twin) 這個名稱，則常被追溯到 NASA 2010 年的技術路線圖草案，其中把數位孿生描述為：一個整合多物理、多尺度與機率的模擬，使用最好的物理模型、感測器更新與機隊歷史等資料，來映照其飛行中的「孿生」的生命。

這段描述點出了數位孿生的三個要素：一是實體，例如一架飛機或一台機器；二是數位表示，包括模型與資料；三是兩者之間持續的資訊流。缺少任何一個，就不是完整的數位孿生。

2010 年代中期以後，隨著感測器、物聯網、雲端運算與模擬軟體的成熟，數位孿生從航太擴展到製造、能源、建築與醫療等領域，成為智慧製造最常被提及的概念之一。也因為熱門，「數位孿生」一詞常被用得很寬鬆：一個 3D 模型、一個監控看板、一套模擬軟體，都可能被稱為數位孿生。這正是叫獸開講中的情境。

19.1.2 三種整合程度

為了釐清這個名詞，Kritzinger 等人於 2018 年發表在 IFAC-PapersOnLine 的文獻回顧，依實體與數位物件之間資料流的自動化程度，把它們分為三類：

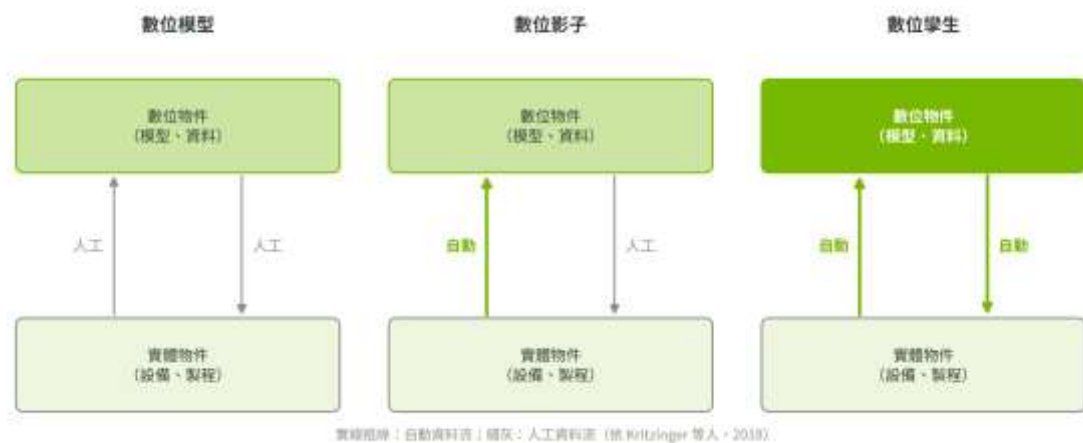


圖 19-1 數位模型、數位影子與數位孿生的資料流

表 19-2 數位模型、數位影子與數位孿生的比較

類別	實體 → 數位	數位 → 實體	典型例子
數位模型	人工	人工	CAD 模型、離線模擬、人工匯入資料的 3D 工廠
數位影子	自動	人工	即時監控看板、狀態監測、自動更新的模擬
數位孿生	自動	自動	自動補償、閉環最佳化、自主調整的生產系統

這個分類簡單而有力，它提醒我們：判斷是不是數位孿生，不是看畫面有多漂亮，而是看資料怎麼流動。叫獸開講中的 3D 工廠，資料每週由人員匯入，結果也不會自動回到實體，所以是數位模型；如果機台狀態自動即時更新，它就成了數位影子；只有當模型的決策能自動送回實體，例如自動補償或自動調整參數，才是數位孿生。

不過，「自動送回實體」並不表示完全不需要人。許多應用中，孿生的建議需要人員確認後才執

行，這在安全攸關或責任重大的場合是必要的（第 31、36 章）。實務上，重點在於回饋的路徑是否已經數位化、是否能在需要的時間內完成，而不是一律追求全自動。

19.1.3 以熱變位補償比較三種整合程度

第 9 章介紹過，工具機的主軸在運轉時會發熱膨脹，造成工件尺寸的誤差，稱為熱變位。精密加工中，熱變位常是最大的誤差來源之一。下面的程式模擬一台工具機一個班次（8 小時）的加工：主軸的溫升以一階模型描述，熱變位與主軸溫升及環境溫度有關；每 2 分鐘加工一件，公差為 $\pm 10 \mu\text{m}$ 。實際情況與計畫有兩個差異：第 200 到 260 分鐘發生意外停機，以及下午環境溫度從 20°C 升到 26°C 。

程式比較三種做法：數位模型依「計畫」與標準環境離線算好補償表；數位影子即時取得感測資料、估算熱變位，但由人員每 60 分鐘手動更新一次補償值；數位孿生則即時估算，並自動把補償值寫回控制器。為了更貼近現實，孿生內的模型參數刻意設定為有 5% 的誤差。

```
import numpy as np
# 數位模型、數位影子與數位孿生：以主軸熱變位補償比較三種資料流
rng = np.random.default_rng(3)
dt, T = 1.0, 480 # 時間步 1 分鐘，一班 8 小時
t = np.arange(T)
tau, kT, aL, beta = 40.0, 25.0, 1.7, 3.0 # 熱時間常數（分）、滿載溫升（ $^\circ\text{C}$ ）、 $\mu\text{m}/^\circ\text{C}$ 、環境影響  $\mu\text{m}/^\circ\text{C}$ 
duty_plan = np.where((t % 120) < 100, 1.0, 0.2) # 計畫：每 2 小時運轉 100 分鐘、休息 20 分鐘
duty = duty_plan.copy(); duty[200:260] = 0.0 # 實際：第 200~260 分鐘意外停機
amb = 20 + 6 * np.clip((t - 60) / 300, 0, 1) # 實際環境溫度：上午  $20^\circ\text{C}$ ，下午升到  $26^\circ\text{C}$ 
def thermal(duty, amb):
    Ts, out = 0.0, []
    for k in range(T):
        Ts += dt / tau * (kT * duty[k] - Ts) # 主軸相對溫升（一階）
        out.append(Ts)
    Ts = np.array(out)
    return Ts, aL * Ts + beta * (amb - 20) # 熱變位（ $\mu\text{m}$ ）
Ts, delta = thermal(duty, amb)
Ts_meas = Ts + rng.normal(0, 0.2, T) # 主軸溫度感測器
amb_meas = amb + rng.normal(0, 0.1, T)
model = lambda Tsm, am: 0.95 * aL * Tsm + beta * (am - 20) # 孿生內的模型（參數有 5% 誤差）
parts = t[::2] # 每 2 分鐘加工一件
tol = 10.0 # 公差  $\pm 10 \mu\text{m}$ 
_, comp_plan = thermal(duty_plan, np.full(T, 20.0)) # 1) 數位模型：離線補償表
est = model(Ts_meas, amb_meas)
comp_shadow = np.array([est[(k // 60) * 60] for k in range(T)]) # 2) 數位影子：每 60 分鐘人工更新
comp_twin = est # 3) 數位孿生：自動、每件更新
for name, comp in [("不補償", np.zeros(T)), ("數位模型", comp_plan), ("數位影子",
```

```
comp_shadow), ("數位孿生", comp_twin)]:
    e = (delta - comp)[parts]
    print(f"{name:<6} RMS 誤差 {np.sqrt(np.mean(e**2)):5.1f} μm，最大
{np.abs(e).max():5.1f} μm，超出公差 {np.sum(np.abs(e) > tol):3d} 件／{len(parts)}")
```

執行結果：

表 19-3 熱變位補償的模擬結果 (240 件)

做法	RMS 誤差	最大誤差	超出公差件數
不補償	42.6 μm	59.2 μm	235
數位模型 (離線補償表)	10.4 μm	17.9 μm	86
數位影子 (每 60 分鐘人工更新)	11.0 μm	31.0 μm	82
數位孿生 (自動、每件更新)	1.7 μm	3.0 μm	0

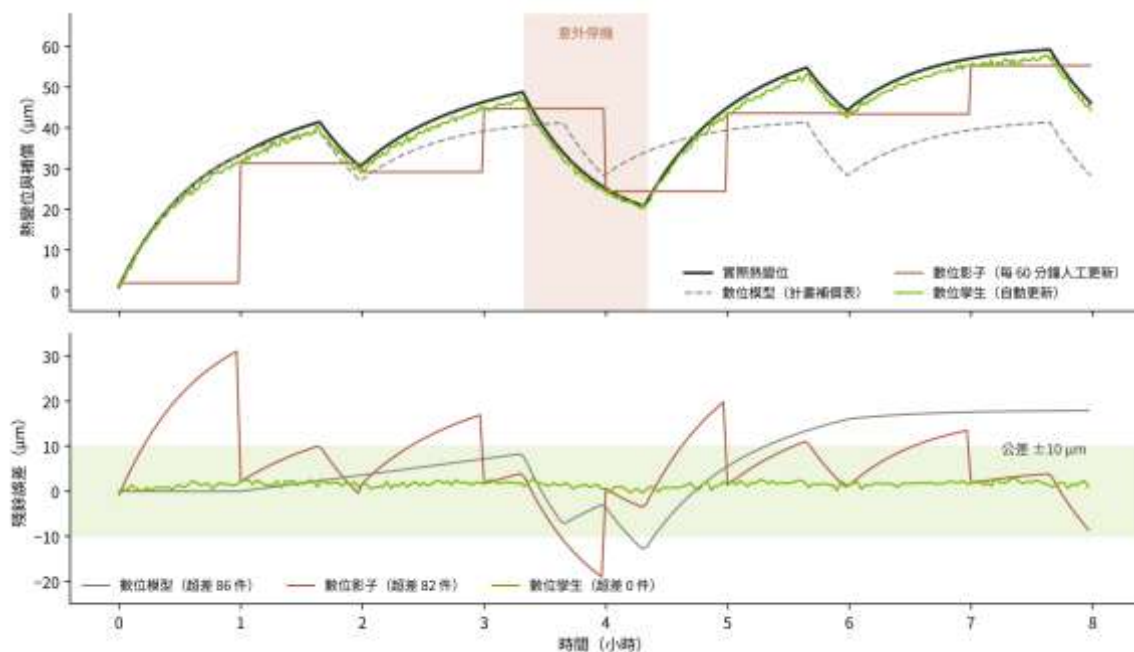


圖 19-2 三種整合程度下的熱變位與補償誤差

這個結果有幾點值得細看。第一，數位模型在「一切照計畫」時表現不錯，但遇到意外停機與環境變化就失準，因為它不知道實際發生了什麼。第二，數位影子雖然「知道」實際狀況，模型估算的熱變位也很準，但回饋靠人工，每小時才更新一次；在停機後重新開機、主軸快速升溫的階段，補償值跟不上，最大誤差反而比數位模型更大。第三，數位孿生即使模型參數有 5% 的誤差，因為持續以實際量測修正、並自動回饋，誤差仍遠小於公差。

這說明了數位孿生的價值來自「閉環」：模型不必完美，但資料要新、回饋要快。這也呼應了第 4 章資訊物理系統的核心概念：感測、計算、決策與執行形成閉環。

19.2 產品、設備、製程、產線與工廠孿生

19.2.1 孿生的層級

數位孿生的對象可以小到一顆軸承，大到一座工廠，甚至一條跨國的供應鏈。依製造系統的層級，可以分為以下幾種：

表 19-4 製造數位孿生的層級

層級	對象	典型模型與資料	典型用途	本書相關章節
產品孿生	單一產品或零件的設計與實例	CAD、BOM、模擬模型、序號與使用資料	設計驗證、使用監測、服務與回收	第 12、13 章
設備孿生	工具機、機器人、泵浦、壓縮機	運動學、動力學、熱模型、狀態資料	狀態監測、熱補償、預測維護、虛擬試車	第 9、21、26 章
製程孿生	切削、射出、熔接、積層、化學反應	物理模型、製程參數、品質量測	參數最佳化、品質預測、虛擬量測	第 16、21、25 章
產線孿生	多台設備、輸送與緩衝	離散事件模擬、週期時間、故障資料	產能分析、瓶頸、排程與假設分析	第 8、18、27 章
工廠孿生	整座工廠的產線、物流、能源與人員	佈置、物流、能源、MES 與 ERP 資料	佈置規劃、能源管理、營運決策	第 17、34 章

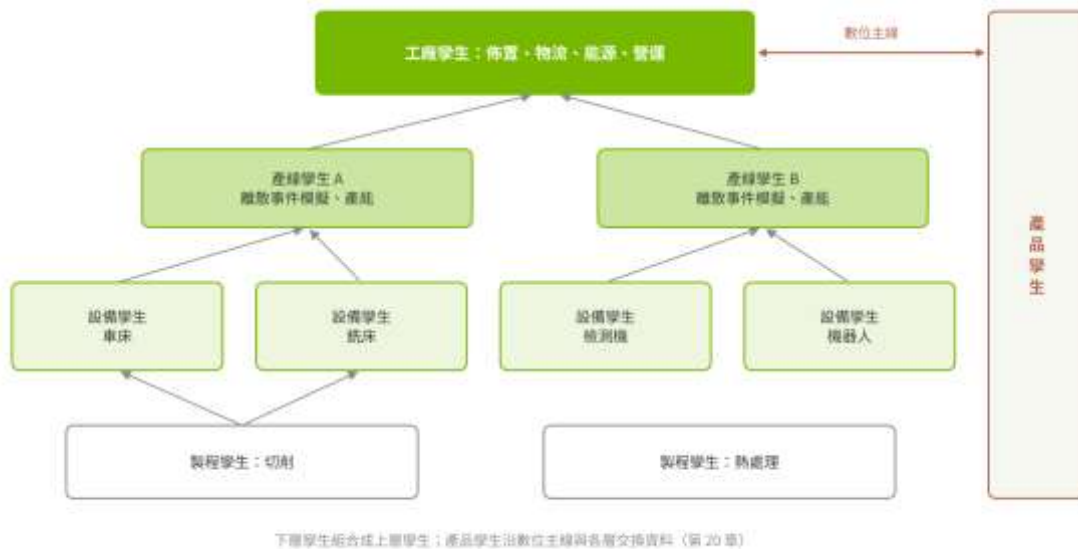


圖 19-3 數位學生的層級與組合

這些層級不是互相獨立的。第 12 章的產品學生，描述的是「做什麼」；設備與製程學生描述的是「用什麼做、怎麼做」；產線與工廠學生描述的是「在哪裡做、做多少」。它們之間透過第 20 章的數位主線串連：產品設計的變更，會影響製程參數與產線排程；設備的狀態，會影響產品的品質與交期。

19.2.2 產品學生

產品學生有兩種：一是「類型」學生，也就是產品的設計，包含 CAD、BOM、模擬與規格，對應第 5 章 RAMI 4.0 生命週期軸上的「類型」；二是「實例」學生，也就是每一件出廠產品的個別紀錄，包含序號、製造履歷（第 17 章）、使用資料與維修紀錄。第 12 章的換電式電動機車與智慧工具機，就是產品實例學生的例子：每一顆電池、每一台機器的使用狀況都被記錄並回饋到設計。

19.2.3 設備學生

設備學生是製造業最早普及的學生之一，因為設備的價值高、停機的代價大，又有現成的控制器與感測資料。19.1.3 節的熱變位補償就是設備學生的例子。其他常見的應用包括：

- 狀態監測與預測維護：以振動、溫度、電流估計設備的健康狀態與剩餘壽命（第 26 章與本章 19.4 節）。

- 虛擬試車：在設備或產線實際安裝前，以學生連接真實的 PLC 程式進行測試，縮短現場試車時間（第 21 章）。
- 加工模擬與碰撞檢查：以工具機的運動學模型與刀具路徑，在實際加工前檢查干涉與加工時間（第 13 章）。

19.2.4 製程學生

製程學生描述加工過程本身：材料如何被切削、熔化、成形或反應。它的模型常是物理與資料的混合，例如第 16 章的熔池模型加上監測資料，或第 18 章的反應器模型加上軟感測器。製程學生最重要的應用之一是虛擬量測：以製程資料即時估計產品的品質，減少實際量測的次數與延遲（第 21 章）。

19.2.5 產線學生與假設分析

產線學生把多台設備的學生組合起來，回答產線層級的問題：產能是多少？瓶頸在哪裡？如果增加緩衝區、改變排程或加開一台機器，產出會增加多少？由於這些問題牽涉到隨機的故障、等待與阻塞，通常以第 8 章介紹的離散事件模擬來回答。

下面的程式模擬一條三台機器串聯的產線。每一台機器的「設備學生」提供三個參數：週期時間、平均故障間隔（MTBF）與平均修復時間（MTTR），這些參數可以從設備的歷史資料估計並持續更新。產線學生把它們組合起來，模擬 40 小時的生產，並比較不同緩衝區大小下的產出：

```
import numpy as np
# 產線學生：由三台設備學生（週期時間、MTBF、MTTR）組成串聯產線，做緩衝區大小的假設分析
machines = [ # 名稱, 週期時間（秒），平均故障間隔 MTBF（分），平均修復時間 MTTR（分）
    ("M1 車削", 50, 120, 10), ("M2 銑削", 55, 90, 15), ("M3 檢測", 45, 200, 8)]
def simulate(buffers, hours=40, seed=1):
    rng = np.random.default_rng(seed)
    n = len(machines); H = hours * 3600
    up = [True] * n; t_next = [rng.exponential(m[2] * 60) for m in machines] # 下次狀態
    切換時間
    prog = [0.0] * n; holding = [False] * n # 加工進
    度、已完成待送出
    buf = [0] * (n - 1); done = 0
    for s in range(H):
        for i, (name, ct, mtbf, mttr) in enumerate(machines):
            if s >= t_next[i]: # 故障或修
            復
                up[i] = not up[i]
                t_next[i] = s + rng.exponential((mtbf if up[i] else mttr) * 60)
            for i in reversed(range(n)): # 由下游往
```

上游更新

```

    if holding[i]:
        # 嘗試把成品送到下游緩衝
        if i == n - 1: done += 1; holding[i] = False
        elif buf[i] < buffers[i]: buf[i] += 1; holding[i] = False
        elif buffers[i] == 0 and not holding[i + 1] and prog[i + 1] == 0 and up[i + 1]:
            # 無緩衝：直接交給下一台
            prog[i + 1] = 1e-9; holding[i] = False
        else: continue
        # 阻塞
    if not up[i]: continue
    if prog[i] == 0:
        # 取料
        if i == 0: prog[i] = 1e-9
        elif buf[i - 1] > 0: buf[i - 1] -= 1; prog[i] = 1e-9
        else: continue
        # 缺料（飢餓）
    prog[i] += 1
    if prog[i] >= machines[i][1]: prog[i] = 0; holding[i] = True
    return done / hours
avail = [m[2] / (m[2] + m[3]) for m in machines]
for (name, ct, mtbf, mttr), A in zip(machines, avail):
    print(f"{name}: 週期 {ct} 秒，可用率 {A:.1%}，有效產能 {3600 / ct * A:.1f} 件/時")
for b in [0, 2, 5, 10, 20]:
    th = np.mean([simulate([b, b], seed=s) for s in range(3)])
    print(f"緩衝區 {b:>2} 件：產出 {th:.1f} 件/時")

```

執行結果：三台機器的可用率分別為 92.3%、85.7% 與 96.2%，有效產能分別為每小時 66.5、56.1 與 76.9 件。M2 的有效產能最低，是瓶頸，理論上限為每小時 56.1 件。但模擬的實際產出低於這個上限：沒有緩衝區時只有每小時約 47.1 件；緩衝區增加到 2、5、10、20 件時，產出分別約為 49.0、50.1、51.4 與 52.4 件。

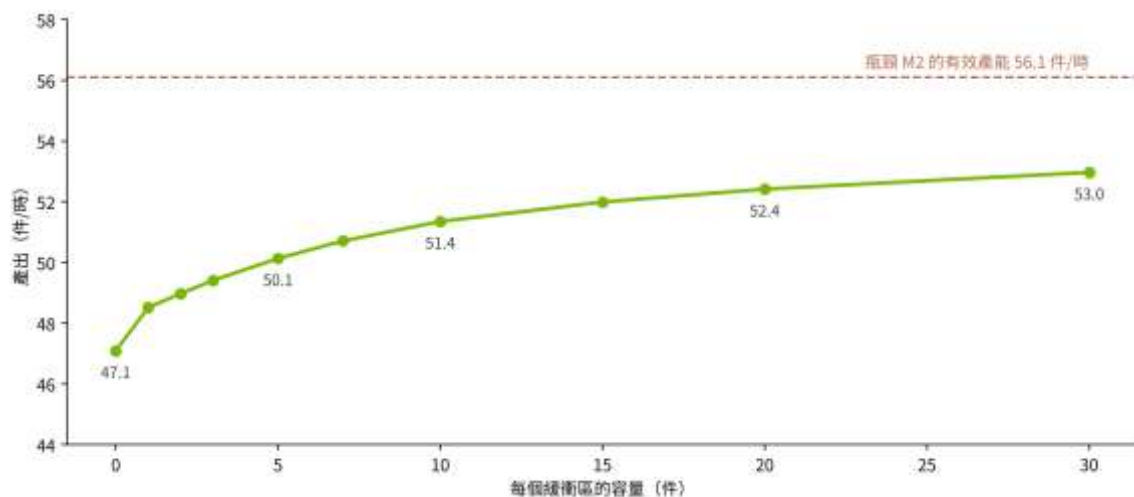


圖 19-4 產線學生的假設分析：緩衝區大小與產出

為什麼實際產出低於瓶頸的有效產能？因為上下游的故障會互相影響：M1 故障時，M2 沒有料可做（飢餓）；M3 故障時，M2 做好的工件無處可放（阻塞）。緩衝區可以吸收這些干擾，但效益

遞減：從 0 增加到 5 件，產出增加約 3 件 / 時；從 10 增加到 20 件，只再增加約 1 件 / 時，卻要多占用空間與在製品（第 8 章的利特爾定律）。

這正是產線孿生的價值：在實際改變產線之前，先在孿生上比較各種方案的效益與代價。如果設備孿生的參數持續由實際資料更新，例如 M2 的故障率在換新主軸後下降，產線孿生的結論也會隨之更新，讓決策跟上現場的變化。

19.2.6 工廠孿生與孿生的組合

工廠孿生涵蓋整座工廠：建築與佈置、產線、物流、能源、人員與資訊系統。它常以 3D 場景作為視覺化的介面，把各個設備與產線孿生的資料整合在一起。叫獸開講中的 3D 工廠，在接上即時資料與底層的設備孿生之後，就成了工廠孿生的入口。

依公開報導，鴻海於 2024 年說明以 NVIDIA Omniverse 與 Isaac Sim，結合西門子的軟體，為其墨西哥瓜達拉哈拉的新工廠建立數位孿生，用來規劃組裝流程、在虛擬環境中訓練機器人，並預期每年可以減少超過 30% 的用電。這類工廠孿生的特點，是在工廠蓋好之前就先存在，讓設計、規劃與試車可以在虛擬環境中先完成，第 21 章將進一步介紹。

把多個孿生組合成更大範圍的孿生，稱為孿生組合。組合的方式有很多種：可以把所有模型整合成一個大模型，也可以讓各個孿生維持獨立、透過標準化的介面交換資料，再由上層的孿生協調。後者較有彈性，也較容易讓不同供應商的孿生互通，但需要共同的語意與介面標準。ISO 23247 第 6 部分就專門處理這個問題，第 20 章將深入討論。

19.3 ISO 23247 參考架構

19.3.1 為什麼需要標準

數位孿生熱門之後，每家軟體廠商、每個研究團隊都有自己的定義與架構。這帶來兩個問題：第一，使用者難以比較不同的方案，也難以判斷一個系統是否真的是數位孿生；第二，不同供應商的孿生彼此無法互通，一座工廠可能同時有工具機廠商的設備孿生、模擬軟體的產線孿生與雲端平台的工廠孿生，卻無法串在一起。

國際標準組織 ISO 的工業資料技術委員會 (ISO / TC 184 / SC 4) 制定了 ISO 23247 系列標準《自動化系統與整合——製造用數位孿生框架》；美國國家標準與技術研究院 (NIST) 的研究人員也發表了多篇分析與應用這套標準的報告。這個標準不規定使用哪一套軟體，而是提供一個中立的框架：製造數位孿生應該包含哪些元素、它們之間如何分工、交換哪些資訊。

19.3.2 標準的組成

表 19-5 ISO 23247 系列標準

部分	名稱	發布	主要內容
第 1 部分	概述與一般原則	2021 年	範圍、名詞、製造數位孿生的需求與原則
第 2 部分	參考架構	2021 年	參考模型的實體與子實體、功能觀點
第 3 部分	實體製造元素的數位表示	2021 年	可觀測製造元素的資訊屬性
第 4 部分	資訊交換	2021 年	各實體之間資訊交換的需求；不規定特定的資料格式與通訊協定
第 5 部分	數位孿生的數位主線	2026 年 6 月	以數位主線串連設計、規劃、生產與測試階段的孿生
第 6 部分	數位孿生組合	2026 年 7 月	多個孿生的通訊、彙整與互通，整合式、統一式與聯邦式組合

ISO 23247 把製造數位孿生定義為：可觀測製造元素的一種「適合用途」的數位表示，並在元素與其數位表示之間進行同步。這個定義有兩個關鍵詞。「適合用途」表示孿生不必、也不可能完美複製實體，只要滿足應用需求即可，這將在 19.4 節進一步討論。「同步」則對應 19.1 節的資料流：沒有同步，就只是數位模型。

第 5、6 部分是 2026 年才發布的新標準，分別處理孿生如何沿著生命週期串連，以及多個孿生如何組合，第 20 章將深入介紹。本節聚焦於第 1 到第 4 部分的核心：參考架構與可觀測製造元素。

19.3.3 可觀測製造元素

ISO 23247 中被孿生的對象，稱為可觀測製造元素 (Observable Manufacturing Element，OME)。標準列出的 OME 類型包括：人員、設備、物料、製程、設施、環境、產品與支援文件。

這份清單比一般人想像的更廣。數位孿生不只是「機器的孿生」：一個工單的製程、一批物料的狀態、一位操作員的資格與位置、廠房的溫溼度，都可以是孿生的對象。19.1.3 節的熱變位例子中，主軸是設備，環境溫度是環境，兩者都是 OME，熱變位模型則把它們的狀態連結在一起。

第 3 部分為 OME 定義了需要記錄的資訊，包括識別碼、特性、排程、狀態、位置、報告與關係七項，其中只有識別碼是必要的，其餘依需要選用；資訊又可分為加工過程中不變的靜態資訊（例如物料的序號）與會改變的動態資訊（例如物料的形狀）。這與第 17 章的主資料、第 20 章的資產管理有密切的關係：OME 的資訊模型，必須與企業既有的資料一致。

19.3.4 參考模型

ISO 23247 第 2 部分的參考模型，由以下幾個實體組成：

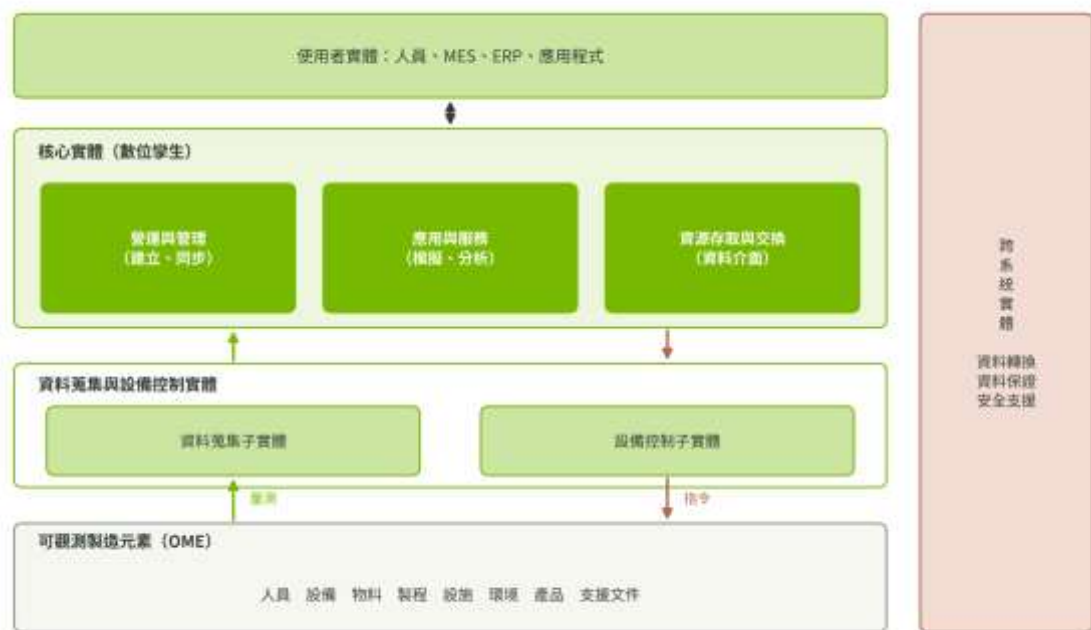


圖 19-5 ISO 23247 參考模型的實體與資料流（依標準內容繪製的示意圖）

表 19-6 ISO 23247 參考模型的實體

實體	子實體或功能	職責	本書相關章節
可觀測製造元素	人員、設備、物料、製程、設施、環境、產品、文件	被觀測與被控制的實體對象	第 9、10 章
資料蒐集與設備控制實體	資料蒐集子實體、設備控制子實體	從 OME 蒐集資料、把指令送回 OME	第 10、11 章

實體	子實體或功能	職責	本書相關章節
核心實體	營運與管理、應用與服務、資源存取與交換子實體	建立與同步孿生、執行模擬與分析、提供資料存取	本章、第 20、21 章
使用者實體	人員、應用程式、其他系統	使用孿生提供的服務，例如 MES、ERP、操作人員	第 17、27 章
跨系統實體	資料轉換、資料保證、安全支援等功能實體	跨越各實體的共通功能：格式轉換、資料品質、資安	第 20、35 章

這個架構與第 5 章介紹的參考架構一脈相承：資料蒐集與設備控制實體對應 ISA-95 的第 1、2 層，核心實體與使用者實體對應第 3、4 層；跨系統實體則把資料品質與資安當作貫穿各層的共通需求。

下面的程式以一個極簡的例子示範這些實體的分工：OME 是一台 CNC，資料蒐集與設備控制實體負責讀取主軸溫度並寫回補償值，核心實體負責同步孿生、執行熱變位模擬並提供資料匯出，使用者實體則是一個自動補償的應用。程式特別加入兩個實務上很重要的檢查：孿生資料超過 5 秒未更新就視為過期，不執行補償；設備控制寫回前檢查安全限制。

```
import json
# 依 ISO 23247-2 參考模型的簡化實作：可觀測製造元素 → 資料蒐集與設備控制 → 孿生核心 → 使用者
class OME:
    # 可觀測製造元素 (Observable Manufacturing Element)
    def __init__(self, oid, kind): self.oid, self.kind, self.state = oid, kind, {"spindle_temp": 25.0, "offset_um": 0.0}
    def read(self): return dict(self.state)
    def write(self, key, val): self.state[key] = val
class DataCollectionDeviceControl:
    # 資料蒐集與設備控制實體
    def __init__(self, ome, limits): self.ome, self.limits = ome, limits
    def collect(self, ts): return {"oid": self.ome.oid, "ts": ts, **self.ome.read()}
    def control(self, key, val):
        # 設備控制：寫回前檢查安全限制
        lo, hi = self.limits[key]
        if not lo <= val <= hi: return f"拒絕：{key}={val} 超出 [{lo}, {hi}]"
        self.ome.write(key, val); return f"已寫入：{key}={val}"
class DigitalTwinCore:
    # 孿生核心實體（營運管理、應用服務、資源存取交換）
    def __init__(self, max_age): self.twins, self.max_age = {}, max_age
    def sync(self, msg): self.twins[msg["oid"]] = msg
    # 營運與管理：同步
    def fresh(self, oid, now): return now - self.twins[oid]["ts"] <= self.max_age
    def simulate(self, oid):
        # 應用與服務：模擬熱變位
        return round(1.7 * (self.twins[oid]["spindle_temp"] - 20), 1)
    def export(self, oid): return json.dumps(self.twins[oid], ensure_ascii=False)
    # 資源存取與交換
```

```

class UserEntity:                                # 使用者實體：人員或其他應用
    def __init__(self, core, dcdc): self.core, self.dcdc = core, dcdc
    def compensate(self, oid, now):
        if not self.core.fresh(oid, now): return "孿生資料過期，不執行補償"
        return self.dcdc.control("offset_um", -self.core.simulate(oid))
ome = OME("CNC-07", "equipment")
dcdc = DataCollectionDeviceControl(ome, {"offset_um": (-50.0, 50.0)})
core = DigitalTwinCore(max_age=5.0); user = UserEntity(core, dcdc)
ome.state["spindle_temp"] = 38.0; core.sync(dcdc.collect(ts=100.0))
print("同步後的孿生：", core.export("CNC-07"))
print("t=102：", user.compensate("CNC-07", now=102.0), "：控制器偏移量",
ome.state["offset_um"])
print("t=110：", user.compensate("CNC-07", now=110.0))
ome.state["spindle_temp"] = 60.0; core.sync(dcdc.collect(ts=111.0))
print("t=111：", user.compensate("CNC-07", now=111.0))

```

執行結果：在 $t = 100$ 秒時主軸溫度 $38\text{ }^{\circ}\text{C}$ 被同步到孿生； $t = 102$ 秒時資料仍新鮮，模型算出熱變位 $30.6\text{ }\mu\text{m}$ ，補償值 $-30.6\text{ }\mu\text{m}$ 成功寫入控制器。 $t = 110$ 秒時，孿生資料已經 10 秒沒有更新，超過 5 秒的新鮮度門檻，系統拒絕補償。 $t = 111$ 秒時主軸溫度異常升到 $60\text{ }^{\circ}\text{C}$ ，模型算出的補償值 $-68.0\text{ }\mu\text{m}$ 超出設備控制允許的 $\pm 50\text{ }\mu\text{m}$ ，寫入被拒絕。

這個例子雖然簡化，卻說明了參考架構的兩個實務價值：一是分工清楚，資料蒐集、模型與應用可以由不同的團隊或供應商負責，只要介面一致就能替換；二是把安全與資料品質的檢查放在明確的位置，避免「模型說什麼，機器就做什麼」。在真實系統中，溫度 $60\text{ }^{\circ}\text{C}$ 這種異常值，還應該觸發第 26 章的異常偵測與人員確認，而不只是拒絕寫入。

19.3.5 資訊交換

第 4 部分處理各實體之間的資訊交換。作為框架標準，它明言不規定特定的資料格式與通訊協定，而是說明資訊交換的需求，讓實作者依產業與應用選用既有的標準。NIST 研究人員在分析這套標準的論文中也指出，OME 的資訊模型本身並未在 ISO 23247 中標準化，並舉例說明 OME 的「報告」資訊可以取自品質資訊框架 (QIF) 或工具機領域常用的 MTConnect。實務上，第 11 章的 OPC UA、MQTT 與第 13 章的 STEP 等標準，都是常見的選擇。這種「不重新發明、讓既有標準各司其職」的做法，降低了導入的門檻，也意味著前面各章介紹的通訊與資料標準，正是建構數位孿生的基礎。

19.4 即時同步、模型保真度與狀態估測

一個數位孿生是否可信，取決於三件事：它的資料是否夠新（同步）、它的模型是否夠準（保真度）、它能否從不完美的量測中推估出真實的狀態（狀態估測）。

19.4.1 即時同步：多快才夠快？

「即時」是一個相對的概念。數位孿生的同步頻率，應該由應用的決策週期決定：

表 19-7 不同應用對同步頻率的需求

應用	決策週期	同步需求	典型技術
運動控制、伺服	毫秒以下	不適合由孿生直接執行，留在控制器	PLC、運動控制器（第 10 章）
熱補償、製程調整	秒到分	秒級	邊緣運算、OPC UA（第 11 章）
狀態監測、預測維護	分到時	分級	歷史資料庫、串流處理（第 23、26 章）
排程與派工	分到班	事件觸發	MES、APS（第 17、27 章）
產能與佈置規劃	週到月	批次更新	離散事件模擬（第 21 章）

這張表的第一列特別重要：毫秒級的控制迴路應該留在控制器內，而不是交給經過網路、可能有延遲的孿生。孿生的角色是在控制器之上提供更好的設定值與決策，就像第 18 章流程工廠的決策階層。

同步的品質不只看頻率，還要看資料新鮮度與時間一致性。19.3.4 節的程式以時間戳記檢查資料是否過期；而當孿生融合來自多個來源的資料時，各來源的時鐘必須同步，否則可能把不同時間的量測當成同一時刻的狀態。第 11 章介紹的精確時間協定（PTP），就是解決這個問題的工具。

同步的方式有兩種：週期同步，每隔固定時間更新一次；事件同步，只有狀態改變時才更新，例如機台從運轉變成停機。事件同步可以大幅減少資料量，但需要確保事件不遺失。實務上兩者常並用：重要的狀態以事件即時通知，連續的量測以適當的頻率取樣。

19.4.2 模型保真度：適合用途

模型越精細，就越準確嗎？通常是的，但代價是計算時間、開發成本與所需的參數越多。數位孿生必須在有限的時間內算出結果，才能跟上實體的變化，所以保真度必須「適合用途」。

下面的程式以一維熱傳導估算刀把（直徑 20 mm、長 100 mm 的鋼棒）一端受熱 10 分鐘後另一端的溫升。最簡單的模型把整根棒視為一個溫度（集總模型），較精細的模型則把棒分成 N 格以有限差分法計算。以 200 格的結果作為參考解，比較不同格數的誤差與計算時間：

```
import numpy as np, time
# 模型保真度：以一維熱傳導估算刀把（直徑 20 mm、長 100 mm 鋼棒）一端受熱 10 分鐘後另一端的溫升
L, k, rho, cp, h = 0.10, 45.0, 7850.0, 470.0, 15.0 # 長度 m、熱傳導係數、密度、比熱、對流係數
D = 0.02; A = np.pi * D**2 / 4; P = np.pi * D # 直徑 20 mm 的截面積與周長
alpha = k / (rho * cp); q_in = 10.0 # 熱端輸入熱量 10 W
def tip_temp(n, t_end=600.0):
    if n == 1: # 集總模型：整根棒視為一個溫度
        C = rho * cp * A * L; G = h * P * L; Tt = 0.0; dt = 0.5
        for _ in range(int(t_end / dt)): Tt += dt * (q_in - G * Tt) / C
        return Tt
    dx = L / n; dt = 0.4 * dx**2 / alpha; steps = int(np.ceil(t_end / dt)); dt = t_end / steps
    T = np.zeros(n)
    for _ in range(steps):
        lap = np.zeros(n)
        lap[1:-1] = T[2:] - 2 * T[1:-1] + T[:-2]
        lap[0] = T[1] - T[0]; lap[-1] = T[-2] - T[-1]
        src = np.zeros(n); src[0] = q_in / (rho * cp * A * dx) # 熱源加在第一格
        T = T + dt * (alpha * lap / dx**2 + src - h * P / (rho * cp * A) * T)
    return T[-1] # 冷端（量測端）溫升
ref = tip_temp(200)
for n in [1, 2, 5, 10, 20, 50, 100]:
    t0 = time.perf_counter(); v = tip_temp(n); ms = (time.perf_counter() - t0) * 1000
    print(f"{n:>3} 格：{v:6.2f} °C，誤差 {abs(v - ref):5.2f} °C ({abs(v-ref)/ref:5.1%})，計算 {ms:7.1f} ms")
```

執行結果：參考解的冷端溫升約 30.04 °C。集總模型算出約 40.97 °C，誤差約 36%，因為它假設整根棒溫度均勻，忽略了熱從熱端傳到冷端需要時間；分成 2 格時誤差約 12.5%，5 格約 2.0%，10 格約 0.5%，20 格約 0.1%。計算時間則隨格數快速增加：在作者的電腦上，10 格約 1 毫秒，100 格則超過 100 毫秒（計算時間依電腦而異）。由於顯式差分法的時間步必須隨格寬的平方縮小，理論上格數加倍，計算量大約增加為八倍；格數少時，程式的固定開銷占比較大，實測的增幅會小一些。

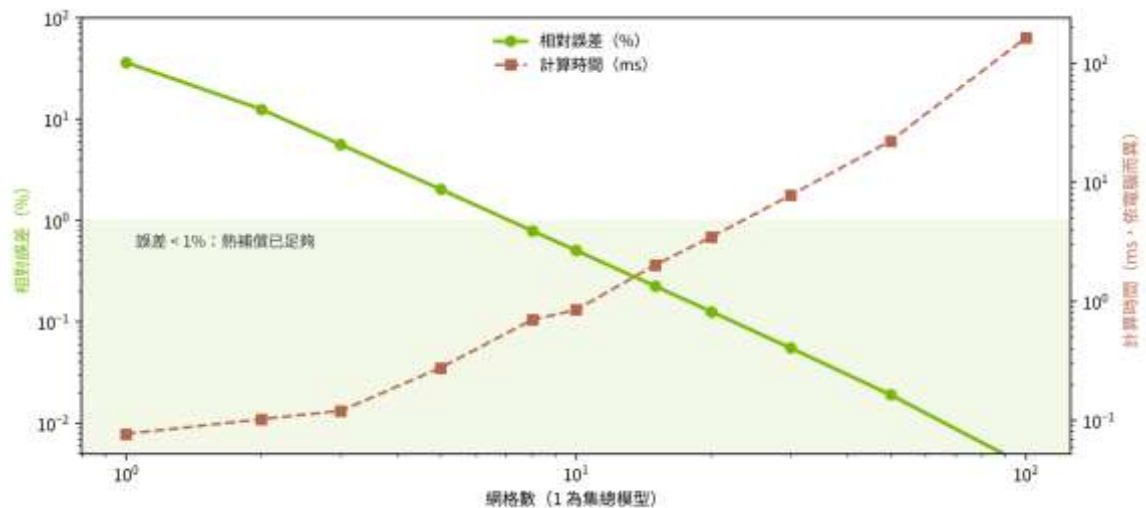


圖 19-6 模型保真度與計算成本的權衡

如果應用是熱變位補償，量測與補償的精度大約是 $0.5\text{ }^{\circ}\text{C}$ 等級，10 格的模型已經足夠，再增加格數只是浪費計算資源；但如果學生要同時處理一座工廠數百台機器，或要在每次決策前模擬數千種情境（例如 19.5 節的蒙地卡羅分析），每一毫秒都很重要。反過來，如果應用是設計新的刀把，需要知道內部的溫度分布與熱應力，就需要第 13 章的三維有限元素模型。

這就是「適合用途」的意義：先確定學生要回答什麼問題、需要多準、多快，再選擇模型。常用的策略包括：

- 降階模型：以高保真度模型的結果訓練較簡單的模型，例如以有限元素模擬的結果建立代理模型，讓學生可以即時計算（第 21 章）。
- 多保真度：平時以快速的低保真度模型運行，需要時再呼叫高保真度模型詳細分析。
- 物理與資料混合：以物理模型提供結構，以資料修正參數或補償模型誤差。19.1.3 節的模型參數有 5% 誤差，仍能靠持續的量測修正達到良好的效果，就是這個原則的體現。

19.4.3 狀態估測：看見看不見的狀態

數位學生需要知道實體的「狀態」，但很多狀態無法直接量測：刀具磨耗了多少、軸承的損傷有多深、反應器內的觸媒還剩多少活性。能量測的往往是間接的訊號，例如主軸功率、振動或溫度，而且帶有雜訊。狀態估測就是融合模型與量測，推估這些看不見的狀態。

第 11 章介紹過以卡爾曼濾波器融合多個感測器。這裡把它用在刀具磨耗的估測：狀態包含磨耗

量與磨耗率兩個變數；每加工一件，就有一筆主軸功率資料，功率與磨耗成正比，但雜訊很大；每 50 件，才以量測儀直接量一次磨耗，精度高但成本也高。為了模擬真實情況，第 150 件之後換成較硬的材料，磨耗率突然變快：

```
import numpy as np
# 狀態估測：以卡爾曼濾波器融合「間接、雜訊大」的主軸功率與「直接、少量」的離線量測，估計刀具磨耗
rng = np.random.default_rng(5)
N = 300 # 加工件數
rate_true = 0.0010 * (1 + 0.6 * (np.arange(N) > 150)) # 真實磨耗率 (mm/件)，第 150 件後換硬料變快
w_true = np.cumsum(rate_true)
power = 2.0 + 8.0 * w_true + rng.normal(0, 0.08, N) # 主軸功率 (kW) = 基準 + 8 kW/mm × 磨耗 + 雜訊
gauge_idx = set(range(49, N, 50)) # 每 50 件以量測儀直接量一次
gauge = w_true + rng.normal(0, 0.005, N)
F = np.array([[1, 1], [0, 1]]); Q = np.diag([1e-8, 1e-8]) # 狀態：[磨耗, 磨耗率]
Hp, Rp = np.array([[8.0, 0.0]]), 0.08**2 # 功率量測 (扣除基準 2 kW)
Hg, Rg = np.array([[1.0, 0.0]]), 0.005**2 # 直接量測
x = np.array([0.0, 0.0008]); P = np.diag([1e-4, 1e-7])
def update(x, P, z, H, R):
    S = H @ P @ H.T + R; K = P @ H.T / S
    return x + (K * (z - H @ x)).ravel(), (np.eye(2) - K @ H) @ P
est, sd = [], []
for k in range(N):
    x = F @ x; P = F @ P @ F.T + Q # 預測
    x, P = update(x, P, power[k] - 2.0, Hp, Rp) # 每件都有功率資料
    if k in gauge_idx: x, P = update(x, P, gauge[k], Hg, Rg)
    est.append(x.copy()); sd.append(np.sqrt(P[0, 0]))
est, sd = np.array(est), np.array(sd)
naive = (power - 2.0) / 8.0 # 只用功率直接換算
limit = 0.30 # 磨耗上限 0.30 mm
print(f"直接換算 RMS 誤差 {np.sqrt(np.mean((naive - w_true)**2))*1000:.1f} μm；卡爾曼 {np.sqrt(np.mean((est[:,0] - w_true)**2))*1000:.1f} μm")
for k in [100, 200]:
    rul = (limit - est[k, 0]) / est[k, 1]
    print(f"第 {k+1} 件：估計磨耗 {est[k,0]:.3f}±{2*sd[k]:.3f} mm，磨耗率 {est[k,1]*1000:.2f} μm/件，預估剩餘 {rul:.0f} 件")
k_est = int(np.argmax(est[:, 0] + 2 * sd > limit)); k_true = int(np.argmax(w_true > limit))
k_naive = int(np.argmax(naive + 2 * 0.01 > limit)) # 直接換算的標準差約 0.01 mm，同樣保留 2σ
print(f"換刀警示：卡爾曼第 {k_est+1} 件，直接換算第 {k_naive+1} 件，真實超限第 {k_true+1} 件")
```

執行結果：只用功率直接換算磨耗，RMS 誤差約 9.6 μm；卡爾曼濾波器融合模型與兩種量測後，誤差降到約 3.6 μm。在第 101 件時，濾波器估計磨耗約 0.096 mm (真值 0.101 mm)，磨耗率約 0.87 μm / 件；換硬料之後，它逐漸「學到」磨耗變快，到第 201 件時估計磨耗約 0.230 mm (真值 0.231 mm)，磨耗率約 1.45 μm / 件，預估剩餘約 48 件，而真實的剩餘件數約為 43 件。

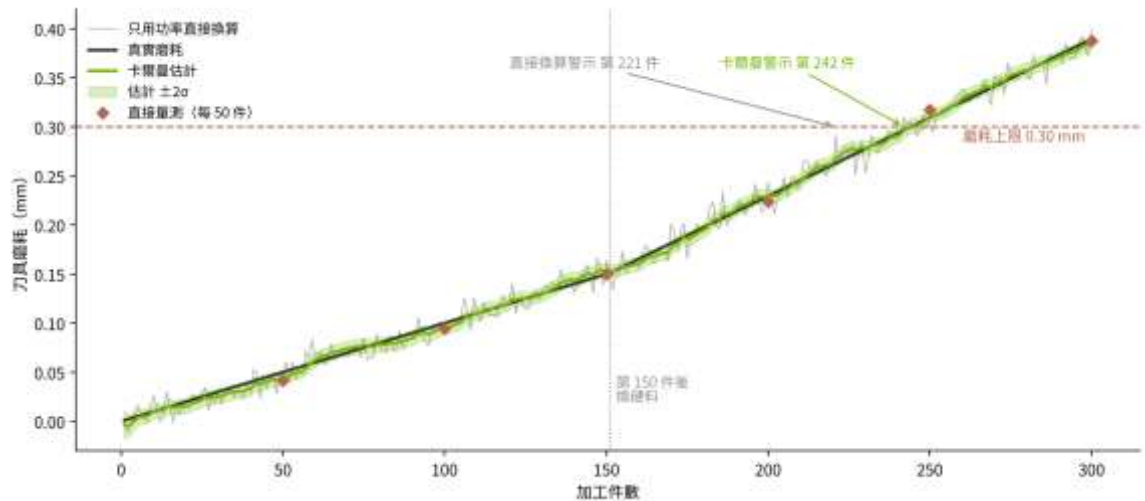


圖 19-7 以卡爾曼濾波器估計刀具磨耗與剩餘壽命

更有實務意義的是換刀警示：若以 0.30 mm 為磨耗上限、保留兩倍標準差的安全餘裕，卡爾曼濾波器在第 242 件發出警示，真實超限發生在第 245 件；只用功率直接換算時，因為雜訊大、需要較大的餘裕，在第 221 件就發出警示，相當於每把刀少用了約 20 件的壽命。更準確的狀態估測，讓學生可以在安全的前提下更充分地利用資源。

這個例子也展示了學生的另一個特性：它會「學習」。磨耗率是模型的參數，也被當成狀態來估計，所以當加工條件改變時，學生會自動調整。更一般地，以實際資料持續修正模型參數，稱為模型校正或學生更新。但要注意，模型持續變動也帶來新的問題：學生的結果是否仍然可信？哪一版模型做了哪個決策？這些是第 20 章驗證、確認與模型治理要處理的問題。

19.5 平台生態、商業價值與導入風險

19.5.1 數位學生的技術堆疊

一個完整的數位學生，很少由單一軟體完成。它通常由多層技術組成：

表 19-8 數位學生的技術堆疊

層	功能	典型技術與標準	本書相關章節
連接與邊緣	從設備蒐集資料、把指令送回設備	OPC UA、MQTT、MTConnect、邊緣閘道	第 10、11 章

層	功能	典型技術與標準	本書相關章節
資料平台	儲存、整理與提供時序、事件與主資料	歷史資料庫、資料湖、統一命名空間	第 17、23 章
語意與資訊模型	描述資產、關係與屬性，讓不同系統看懂彼此	資產管理殼、知識圖譜、ISO 23247 的 OME 資訊	第 5、20 章
模型與模擬	物理模型、離散事件模擬、機器學習模型	CAE、模擬軟體、降階模型、AI 模型	第 13、21、22 章
視覺化與互動	3D 場景、儀表板、AR / VR	OpenUSD、遊戲引擎、網頁儀表板	第 21、31 章
應用	補償、維護、排程、規劃等具體用途	MES、APS、維護系統、客製應用	第 17、26、27 章
治理與資安	權限、版本、模型管理、資料品質	身分管理、稽核、模型註冊	第 20、35、36 章

叫獸開講中的 3D 工廠，只做了「視覺化與互動」這一層；熱變位補償的學生，則只用了連接、簡單的模型與應用三層。兩者都不完整，但後者解決了實際的問題。這提醒我們：技術堆疊是一張地圖，不是一份必須一次買齊的清單。

19.5.2 平台生態

數位學生的市場上有幾類主要的參與者：

- 自動化與工業軟體廠商：提供從控制器、PLM、模擬到 MES 的整套方案，優勢在於與設備及工程資料的緊密整合。
- 模擬與工程軟體廠商：提供高保真度的物理模擬與降階模型工具。這個領域近年也在整併，例如晶片設計軟體公司 Synopsys 於 2025 年 7 月完成對模擬軟體公司 Ansys 的收購。
- 雲端平台：提供資料蒐集、儲存、運算與學生建模的服務，優勢在於規模與彈性。
- 3D 與 GPU 運算平台：提供大規模 3D 場景、物理模擬與機器人訓練的環境，例如 19.2.6 節提到的 NVIDIA Omniverse，常以第 20 章介紹的 OpenUSD 作為場景描述格式。
- 開源社群：提供學生框架、資產管理殼實作等開源工具，讓學校與中小企業可以低成本起步。

平台的選擇是一個長期的承諾。孿生的模型、資料與應用一旦建立在某個平台上，轉移的成本可能很高，這稱為供應商鎖定。而平台本身也可能改變：服務會改版、改價，甚至停止。例如，依 AWS 的官方文件，其 IoT Events 服務自 2025 年 5 月 20 日起不再接受新客戶，並於 2026 年 5 月 20 日終止支援，既有使用者必須在期限內遷移。這不代表雲端服務不可靠，而是提醒我們：孿生的資料與模型應該盡量以開放、標準化的格式保存，讓核心資產不被單一平台綁住。這也是 ISO 23247 與資產管理殼等標準的價值所在。

19.5.3 商業價值

數位孿生的價值可以沿著產品與工廠的生命週期來看：

- 設計與工程：以模擬取代部分實體試作，縮短開發時間（第 13、14 章）。
- 規劃與試車：在產線建造前驗證佈置、產能與控制程式，縮短現場試車時間（第 21 章）。
- 營運：減少非計畫停機、報廢與能源消耗，提升品質與產能（第 25、26、34 章）。
- 服務：以產品孿生提供遠端監測、預測維護與新的服務模式（第 12、32 章）。

然而，數位孿生的投資常常不小，效益也有很大的不確定性：模型做不做得準？現場會不會真的使用？效益多久才會出現？面對這種不確定，單一的投資報酬率數字容易誤導。比較好的做法，是以機率分布描述不確定的成本與效益，再以蒙地卡羅模擬評估結果的分布。

下面的程式評估一個工廠級數位孿生專案五年的淨現值（NPV）。建置成本、每年維運成本與三種效益（停機減少、報廢減少、能源節省）都以三角分布表示（最小值、最可能值、最大值），效益逐年爬升；另外考慮一個常被忽略的風險：導入不一定成功，假設有 25% 的機率孿生最後沒有被現場持續使用，只剩下少量效益。所有金額都是教學用的假設值：

```
import numpy as np
# 數位孿生專案的商業價值：以蒙地卡羅模擬淨現值（NPV）的分布與風險（所有金額皆為教學用假設值，單位：萬元）
rng = np.random.default_rng(11); n = 20000; years = 5; r = 0.08
capex = rng.triangular(800, 1000, 1500, n) # 建置成本：軟硬體、整合、模型開發（常超支）
opex = rng.triangular(120, 150, 220, n) # 每年維運：授權、雲端、模型維護與人力
b_down = rng.triangular(200, 400, 600, n) # 每年效益：非計畫停機減少
b_scrap = rng.triangular(100, 200, 350, n) # 每年效益：報廢與重工減少
b_energy = rng.triangular(30, 70, 120, n) # 每年效益：能源節省
adopt = rng.random(n) < 0.75 # 75% 的機率導入成功（被現場持續
```

使用)

```

ramp = np.array([0.4, 0.8, 1.0, 1.0, 1.0]) # 效益逐年爬升
def npv(capex, opex, benefit, adopt):
    cash = -capex.copy()
    for y in range(years):
        b = benefit * ramp[y] * np.where(adopt, 1.0, 0.15) # 導入失敗時只剩少量效益
        cash += (b - opex) / (1 + r) ** (y + 1)
    return cash
benefit = b_down + b_scrap + b_energy
v = npv(capex, opex, benefit, adopt)
print(f"NPV 平均 {v.mean():.0f} 萬元，中位數 {np.median(v):.0f}，P10 {np.percentile(v,10):.0f}，P90 {np.percentile(v,90):.0f}")
print(f"NPV < 0 的機率 {np.mean(v < 0):.1%}")
v_ok = npv(capex, opex, benefit, np.ones(n, bool))
print(f"若確保導入成功：NPV 平均 {v_ok.mean():.0f} 萬元，虧損機率 {np.mean(v_ok < 0):.1%}")
v_pilot = npv(capex * 0.3, opex * 0.4, benefit * 0.35, adopt) # 先做單一產線試點
print(f"單一產線試點：NPV 平均 {v_pilot.mean():.0f} 萬元，虧損機率 {np.mean(v_pilot < 0):.1%}，P5 {np.percentile(v_pilot,5):.0f}；全廠 P5 {np.percentile(v,5):.0f}")

```

執行結果：

表 19-9 數位學生專案的 NPV 模擬結果 (假設值，單位：萬元)

情境	NPV 平均	虧損機率	說明
全廠導入，成功機率 75%	約 16	約 32%	中位數約 339，P10 約 -1,447，P90 約 917
全廠導入，確保導入成功	約 505	約 9%	移除「沒人用」的風險後，價值大幅提升
單一產線試點	約 28	約 30%	最不利的 5% 情境 (P5) 損失約 522，全廠則約 1,566

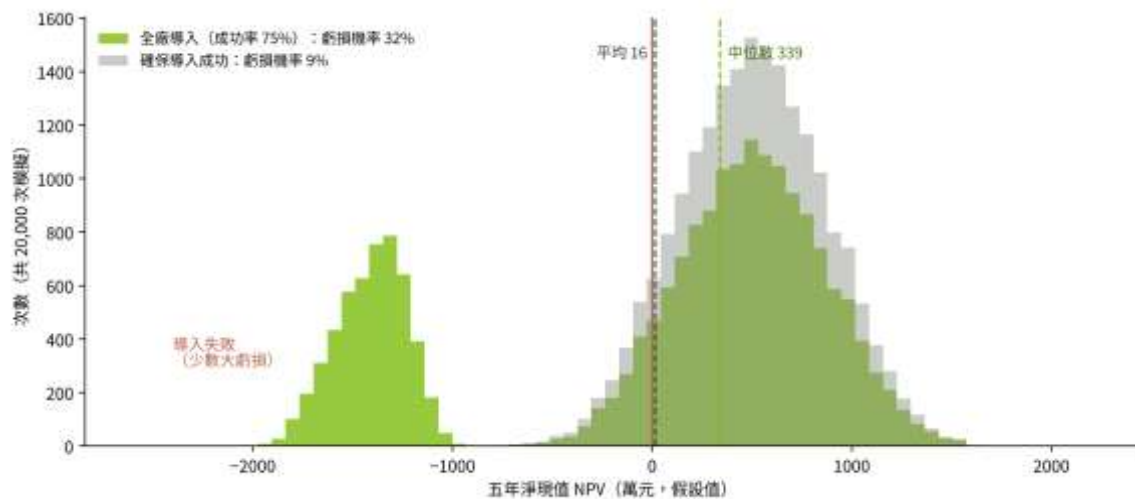


圖 19-8 數位學生專案 NPV 的分布 (假設值)

這個結果有三個啟示。第一，平均值只有約 16 萬元，看起來幾乎不值得做；但中位數約 339 萬元，大多數情境其實是賺錢的，平均值被少數導入失敗的大虧損拉低。單一數字無法呈現這種「多數成功、少數大敗」的分布，這正是蒙地卡羅分析的價值。第二，影響最大的不是技術，而是「現場是

否真的使用」：如果能確保導入成功，平均 NPV 從約 16 萬元提升到約 505 萬元，虧損機率從約 32% 降到約 9%。因此，投資在使用者參與、教育訓練與流程調整上，往往比投資在更精美的模型上更划算。第三，先做試點並不會讓平均報酬更高，但能把最壞情況的損失從約 1,566 萬元限制在約 522 萬元，同時累積經驗、驗證效益，降低全面推廣時的風險。這與第 6 章的轉型方法與第 18 章燈塔工廠「從問題出發、再規模化」的經驗一致。

19.5.4 導入風險

表 19-10 數位學生的常見導入風險

風險	症狀	對策	本書相關章節
只有畫面，沒有閉環	漂亮的 3D 模型，資料靠人工更新，結果不影響決策	從具體決策出發，先做數位影子，再逐步閉環	本章 19.1 節
資料品質不佳	感測器失準、時間不同步、主資料不一致	資料協調、時間同步、主資料治理	第 11、17、18 章
模型失準	製程或設備改變後，模型結果偏離實際	持續校正、監控模型誤差、版本管理	第 20 章
過度追求保真度	模型太複雜，算不快、維護不了	適合用途、降階模型、多保真度	本章 19.4 節
供應商鎖定	資料與模型綁在單一平台，轉移困難	開放標準、資料可攜、合約條款	第 5、20 章
資安與安全	學生可以寫回設備，成為攻擊與誤動作的入口	寫回前的安全限制、權限管理、網路分區	第 35 章
組織與人才	專案結束後無人維護，現場不信任結果	明確的負責人、教育訓練、讓使用者參與設計	第 6、36 章

其中最容易被低估的，是資安與安全。數位影子只讀取資料，風險相對有限；數位學生能把決策寫回設備，一旦模型出錯、資料被竊改或系統被入侵，就可能直接影響生產甚至人員安全。19.3.4 節程式中的安全限制與資料新鮮度檢查，是最基本的防護；完整的做法還包括第 35 章的網路分區、身分驗證與稽核，以及第 36 章的人工覆核機制。

19.5.5 導入路徑

綜合本章的討論，一個務實的數位孿生導入路徑大致如下：

- 選擇問題：挑選一個價值明確、資料可得、決策週期清楚的問題，例如熱變位補償、刀具壽命或瓶頸設備的停機。
- 建立數位影子：先讓實體的資料自動流入模型，以監控與分析驗證模型的準確度，同時建立現場對模型的信任。
- 閉環：在安全限制與人工確認機制下，讓模型的決策自動或半自動地回到實體。
- 擴展與組合：把成功的設備孿生複製到其他設備，再組合成產線與工廠孿生；這時，標準化的資訊模型與介面變得非常重要。
- 治理與持續改善：建立模型的版本管理、效益追蹤與負責人，讓孿生在設備與製程改變時持續保持可信。

這條路徑與第 6 章的成熟度階梯相呼應，也為後面兩章鋪路：第 20 章談如何以數位主線、孿生組合與語意互通把孿生串起來並加以治理；第 21 章談模擬、虛擬試車與工業元宇宙，讓孿生在實體存在之前就開始發揮作用。

案例

案例 19-1 教學情境：會動的 3D 模型

背景：叫獸開講中的工廠（綜合情境）花了半年建立精美的 3D 工廠模型，但機台狀態每週由人工匯入，模型也不會產生任何決策。

做法：以 Kritzing 等人的分類重新檢視，確認它是數位模型；改從問題出發，選定一台常因熱變位超差的精密加工機，加裝溫度感測器，先建立數位影子驗證熱變位模型，再讓補償值自動寫回控制器，並設定寫回的安全限制。

成果：該機台的尺寸超差件數大幅下降；3D 工廠模型後來接上即時資料，成為設備孿生的視覺化入口。

啟示：判斷數位孿生的關鍵是資料流與閉環，而不是畫面；從一個有價值的問題開始，比一次建好整座工廠更有效。

討論：（1）你見過哪些被稱為「數位孿生」的系統？依表 19-2 它們屬於哪一類？（2）讓補償自動寫回控制器之前，需要哪些驗證？

案例 19-2 產業：工具機熱補償孿生

背景：一家精密零件廠的工件尺寸在上下午之間有明顯的差異，冬夏兩季也不同，過去靠操作員依經驗調整刀補。

做法：在主軸、床身與環境加裝溫度感測器，以歷史加工與量測資料建立熱變位模型；模型在邊緣電腦上運行，每件加工前自動計算補償值寫入控制器；每天以首件量測結果校正模型參數。

成果：尺寸變異明顯縮小，首件檢驗的調機次數減少，操作員不再需要憑經驗頻繁調整。

啟示：設備孿生最容易產生效益的場景，是誤差有明確的物理成因、資料容易取得、決策可以自動執行的問題。

討論：（1）為什麼每天要以首件量測校正模型？（2）如果溫度感測器故障，系統應該如何反應？

案例 19-3 產業：產線孿生輔助投資決策

背景：一家汽車零件廠的產出長期低於需求，管理者考慮購買一台新的銑床，預算約數百萬元。

做法：以三台設備的歷史資料建立設備孿生（週期時間、故障與修復時間），組成產線孿生，比較三種方案：增加緩衝區、改善瓶頸機台的維修流程以縮短修復時間、購買新機台。

成果：模擬顯示，縮短瓶頸機台的修復時間並適度增加緩衝區，就能達到大部分的產出目標，投資遠低於購買新機台；工廠先執行低成本方案，再依實際成效決定是否購機。

啟示：產線孿生的價值在於「先在虛擬中試錯」；設備孿生的參數若持續由實際資料更新，產線層級的決策也會跟上現場的變化。

討論：（1）模擬結果與實際產出有差距時，應該先檢查哪些資料？（2）為什麼增加緩衝區的效益會遞減？

案例 19-4 台灣情境：在工廠蓋好之前先建孿生

背景：依 NVIDIA 2024 年 6 月的公開說明，鴻海為其位於墨西哥瓜達拉哈拉、生產 AI 伺服器的新工廠建立數位孿生。

做法：依報導，鴻海結合 NVIDIA Omniverse、Isaac Sim 與西門子的軟體，在虛擬環境中規劃組裝流程與佈置，並讓機器手臂在虛擬環境中學習取放零件，再部署到實體工廠。

成果：依報導，鴻海預期可以提升複雜伺服器的製造效率，並每年減少超過 30% 的用電。

啟示：工廠孿生可以在實體存在之前就開始發揮作用，讓設計、規劃與機器人訓練提前在虛擬環境中完成；這是第 21 章虛擬試車與工業元宇宙的典型應用。

討論：（1）在工廠蓋好之前建立的孿生，如何確保與完工後的實體一致？（2）在虛擬環境中訓練的機器人，部署到實體時可能遇到哪些差異？

案例 19-5 產業：刀具壽命孿生

背景：一家航太零件廠加工難切削材料，過去依固定件數換刀，為了安全常保留很大的餘裕，刀具成本高昂；偶爾仍因材料批次差異發生刀具過度磨耗。

做法：以主軸功率與振動作為間接量測，每隔一定件數以量測儀直接檢查刀具，再以卡爾曼濾波器估計磨耗量與磨耗率，依估計值與不確定性決定換刀時機。

成果：在不增加品質風險的前提下，平均刀具壽命延長；材料批次變硬時，孿生能較早偵測到磨耗加速並提早示警。

啟示：狀態估測讓孿生「看見」無法直接量測的狀態；估計的不確定性與估計值本身同樣重要。

討論：（1）直接量測的頻率應如何決定？（2）如果功率感測器的增益慢慢漂移，濾波器會受到什麼影響？

案例 19-6 教學情境：平台改變時，學生還在嗎？

背景：一個學生團隊為學校的實驗產線建立數位影子，把資料與規則都放在某個雲端平台的專屬服務上。

做法：團隊在課堂上討論一個公開的例子：依 AWS 官方文件，其 IoT Events 服務於 2025 年 5 月停止接受新客戶，並於 2026 年 5 月終止支援。團隊盤點自己的系統，把設備資料改以 OPC UA 與 MQTT 的標準格式送入開源的資料平台，資產的描述以開放的資訊模型保存，雲端服務只負責可替換的運算與視覺化。

成果：系統的核心資料與模型不再綁定單一服務，往後更換平台或延伸到其他產線時，只需改寫少部分的程式。

啟示：平台會改變，學生的核心資產應以開放、標準化的格式保存；選擇平台時，除了功能，也要評估資料可攜性與退場成本。

討論：（1）哪些部分最適合使用雲端服務？哪些部分應該自己掌握？（2）如何在合約中保障資料的可攜性？

實作活動

活動 19-1 分辨三種整合程度

蒐集三個自稱「數位學生」的產品或案例（可從廠商網站、展覽或新聞取得），依表 19-2 判斷它們屬於數位模型、數位影子或數位學生，並說明判斷的依據。

活動 19-2 熱變位學生

以 19.1.3 節的程式為基礎，改變數位影子的人工更新頻率（每 15、30、60、120 分鐘），以及學生模型的參數誤差（0%、5%、10%、20%），畫出超出公差件數的變化，說明「資料新鮮度」與「模型精度」哪一個影響較大。

活動 19-3 產線學生的假設分析

以 19.2.5 節的程式為基礎，在總緩衝區容量固定為 10 件的條件下，比較不同的分配方式（例如 $5 + 5$ 、 $2 + 8$ 、 $8 + 2$ ），並嘗試縮短瓶頸機台的修復時間，比較兩種改善方式的效益。

活動 19-4 ISO 23247 小型實作

以 19.3.4 節的程式為基礎，用第 11 章的 OPC UA 模擬伺服器取代程式中的 OME，讓資料蒐集實體透過 OPC UA 讀取溫度、寫回補償值；並加入一個顯示學生狀態的簡單網頁或終端機儀表板。

活動 19-5 狀態估測

以 19.4.3 節的程式為基礎，改變直接量測的頻率（每 20、50、100 件）與功率量測的雜訊，觀察估計誤差與換刀警示時機的變化；再讓功率感測器的增益每件漂移 0.01%，觀察濾波器的表現，並思考如何修正。

活動 19-6 學生專案的商業評估

選擇一個你感興趣的數位學生應用，列出成本與效益項目，以三角分布估計其範圍，並以 19.5.3 節的程式評估 NPV 的分布；說明哪一個變數對結果的影響最大（可用敏感度分析），以及你會如何降低風險。相關範例程式將於本書的實驗倉庫中維護。

延伸閱讀

- Werner Kritzinger 等人，〈Digital Twin in manufacturing: A categorical literature review and classification〉，IFAC-PapersOnLine，51(11)，2018：數位模型、數位影子與數位學生的分類。
- Michael Grieves 與 John Vickers 等人關於數位學生起源與概念的著作：了解「實體、虛擬與資訊流」的概念模型。
- ISO 23247 系列標準《Automation systems and integration — Digital twin framework for manufacturing》第 1 至 6 部分。
- 美國國家標準與技術研究院（NIST）關於製造數位學生標準與使用案例的技術報告。

- Dan Simon · Optimal State Estimation (Wiley · 2006) : 卡爾曼濾波器與狀態估測的系統性教材。

表 19-11 本章思考地圖

本章概念	關鍵提問	後續延伸章節
數位模型、影子與孿生	資料如何流動？決策是否回到實體？	第 4、20 章
孿生的層級與組合	要孿生的對象是什麼？如何組合成更大的孿生？	第 20、21 章
ISO 23247 參考架構	各實體的分工與介面是否清楚？	第 5、20 章
即時同步	決策週期多長？資料夠新嗎？時鐘一致嗎？	第 11、23 章
模型保真度	模型是否適合用途？能否算得夠快？	第 13、21 章
狀態估測	看不見的狀態如何估計？不確定性多大？	第 22、26 章
商業價值與風險	價值來自哪裡？最大的風險是技術還是組織？	第 6、35、36 章

章末小結

- 數位孿生的概念常追溯到 Grieves 2002 年在密西根大學提出的鏡像空間模型，名稱則見於 NASA 2010 年的技術路線圖草案。Kritzinger 等人依資料流的自動化程度，把它們分為數位模型、數位影子與數位孿生。本章的熱變位範例中，只有雙向自動同步的數位孿生把超差件數從 80 多件降為 0，即使模型參數有 5% 的誤差。
- 製造數位孿生可分為產品、設備、製程、產線與工廠等層級，彼此透過數位主線串連並可組合。產線孿生範例中，瓶頸機台的有效產能為每小時 56.1 件，但上下游故障造成飢餓與阻塞，無緩衝時實際產出約 47.1 件，緩衝區增加的效益遞減。
- ISO 23247 為製造數位孿生提供中立的框架：第 1 到 4 部分於 2021 年發布，第 5、6 部分於 2026 年發布。參考模型包含可觀測製造元素、資料蒐集與設備控制實體、核心實體、使用者實體與跨系統實體；OME 涵蓋人員、設備、物料、製程、設施、環境、產品與支援文件。
- 同步頻率應由決策週期決定，毫秒級控制留在控制器；資料新鮮度與時間一致性同樣重要。模型保真度應適合用途：熱傳導範例中 10 格已將誤差降到約 0.5%，再增加格數只換來有限的精度與

快速增加的計算量。卡爾曼濾波器融合間接與直接量測，把磨耗估計誤差從約 9.6 μm 降到約 3.6 μm ，並讓換刀時機更接近真實極限。

- 數位孿生由多層技術堆疊而成，平台會改變，核心資料與模型應以開放標準保存。蒙地卡羅分析顯示，孿生專案的最大風險常是「現場沒有持續使用」；先試點可以限制最壞情況的損失。務實的導入路徑是：選擇問題、建立數位影子、閉環、擴展與組合、治理與持續改善。

習題

一、選擇題

1. 依 Kritzinger 等人的分類，實體到數位的資料流為自動、數位到實體為人工的是？(A) 數位模型 (B) 數位影子 (C) 數位孿生 (D) 數位主線
2. 「數位孿生」這個名稱常被追溯到？(A) 1990 年的 CAD 標準 (B) 2021 年的 ISO 23247 (C) NASA 2010 年的技術路線圖草案 (D) 2013 年的工業 4.0 實施建議
3. ISO 23247 中被孿生的對象稱為？(A) 資產管理殼 (B) 可觀測製造元素 (C) 工作中心 (D) 控制配方
4. ISO 23247 第 6 部分的主題是？(A) 資訊交換 (B) 參考架構 (C) 數位孿生組合 (D) 一般原則
5. 在 ISO 23247 參考模型中，負責從設備蒐集資料並把指令送回設備的是？(A) 使用者實體 (B) 跨系統實體 (C) 資料蒐集與設備控制實體 (D) 核心實體
6. 毫秒級的伺服控制迴路應該放在？(A) 雲端孿生 (B) 控制器 (C) ERP (D) 3D 視覺化平台
7. 以預測與修正兩步驟，融合模型與雜訊量測估計狀態的方法是？(A) 蒙地卡羅模擬 (B) 卡爾曼濾波器 (C) 有限差分法 (D) 離散事件模擬
8. 「適合用途」的保真度原則是指？(A) 模型越精細越好 (B) 保真度與同步頻率依應用需求決定 (C) 只使用資料驅動模型 (D) 只使用物理模型

9. 串聯產線沒有緩衝區時，上游機台故障會讓下游機台？（A）阻塞 （B）飢餓 （C）加速
（D）不受影響
10. 依公開報導，鴻海以 NVIDIA Omniverse 建立數位孿生的新工廠位於？（A）台灣高雄 （B）美國威斯康辛 （C）墨西哥瓜達哈拉 （D）越南北江

二、簡答題

11. 說明數位模型、數位影子與數位孿生的差異，各舉一例。
12. 為什麼在 19.1.3 節的範例中，數位影子的最大誤差反而比數位模型更大？
13. 說明產品孿生中「類型」與「實例」的差異。
14. 說明 ISO 23247 對製造數位孿生的定義，以及其中的兩個關鍵詞。
15. 列出 ISO 23247 參考模型的核心實體所包含的三個子實體，並說明其職責。
16. 為什麼孿生寫回設備之前，要檢查資料新鮮度與安全限制？
17. 說明週期同步與事件同步的差異與適用時機。
18. 為什麼數位孿生專案的效益，常取決於組織而非技術？

三、計算題

19. 主軸溫升以一階模型描述，時間常數 40 分鐘、滿載溫升 25°C 、熱變位係數 $1.7\ \mu\text{m}/^{\circ}\text{C}$ 。從冷機開始滿載運轉 40 分鐘後，熱變位約為多少？
20. 某機台的 MTBF 為 90 分鐘、MTTR 為 15 分鐘、週期時間 55 秒，求可用率與有效產能（件/時）。
21. 承上題，若改善維修流程使 MTTR 降為 10 分鐘，有效產能變為多少？
22. 孿生每 5 秒同步一次，網路延遲 0.5 秒、處理時間 1 秒。資料最舊可能是多少秒前的？若應用要求資料不超過 5 秒，同步週期最長應為多少？
23. 以顯式有限差分法計算一維熱傳導，網格從 10 格增加到 40 格，理論上計算量約增加為幾倍？
24. 純量卡爾曼濾波：預測值 10、預測變異 4，量測值 12、量測變異 1。求卡爾曼增益、更新後的估

計值與變異。

25. 刀具估計磨耗 0.230 mm、磨耗率 1.45 μm / 件，磨耗上限 0.30 mm，預估剩餘幾件？

26. 某學生專案建置成本 1,000 萬元，之後 5 年每年淨效益 300 萬元，折現率 8%，求 NPV。

四、申論與設計題

27. 為你熟悉的一台設備或一條產線設計數位孿生：說明要解決的問題、OME、需要的資料與同步頻率、模型的保真度、回饋的方式與安全機制，並依 ISO 23247 的參考模型畫出架構。

28. 評論以下說法：「數位孿生就是把工廠做成 3D 模型。」

29. 一家中小企業想導入數位孿生，但預算有限。請為它規劃導入路徑，並說明如何降低供應商鎖定與導入失敗的風險。

五、是非題

30. 畫面精美、可以互動的 3D 工廠模型，就是數位孿生。

31. 數位孿生的模型必須完美，才能產生價值。

32. ISO 23247 規定製造數位孿生必須使用特定的通訊協定。

33. 增加緩衝區可以提升串聯產線的產出，但效益會遞減。

34. 磨耗率這類模型參數，也可以被當成狀態，由卡爾曼濾波器估計。

35. 數位孿生能把決策寫回設備，因此也增加了資安與安全的風險。

六、進階挑戰題

36. 以 SimPy 等離散事件模擬套件重寫 19.2.5 節的產線孿生，驗證結果與原程式一致；並在總緩衝容量固定的條件下，找出使產出最大的緩衝配置。

37. 若主軸功率與磨耗的關係是非線性的（例如功率與磨耗的平方根成正比），請以擴展卡爾曼濾波器或粒子濾波器改寫 19.4.3 節的程式，並比較估計精度。

38. 以 19.4.2 節的高保真度熱傳導模型產生訓練資料，建立一個降階模型（例如以兩個一階系統串聯，或以迴歸模型），比較降階模型與高保真度模型的精度與計算時間。

七、補充題

39. 列出 ISO 23247 中可觀測製造元素的八種類型。

40. 列出 ISO 23247 參考模型的五個實體。

提示與參考解答

1. B。

2. C。

3. B。

4. C。

5. C。

6. B。

7. B。

8. B。

9. B。

10. C。

11. 參考：數位模型與實體之間沒有自動資料流，例如離線模擬；數位影子的資料自動從實體流入，但回饋靠人工，例如即時監控看板；數位孿生雙向自動，例如自動熱補償。

12. 參考：數位影子雖然即時估算熱變位，但補償每 60 分鐘才由人工更新一次；在停機後重新開機、主軸快速升溫時，補償值跟不上變化，最大誤差因此較大。

13. 參考：類型孿生是產品的設計，例如 CAD、BOM 與模擬；實例孿生是每一件產品的個別紀錄，例如序號、製造履歷、使用與維修資料。

14. 參考：可觀測製造元素的適合用途的數位表示，並在元素與其數位表示之間進行同步。關鍵詞為「適合用途」與「同步」。

15. 參考：營運與管理子實體負責孿生的建立、同步與管理；應用與服務子實體提供模擬、分析等服

務；資源存取與交換子實體提供資料與功能的存取介面。

16. 參考：過期的資料可能已不代表實體的狀態，據以寫回會造成錯誤；模型可能出錯或遇到異常輸入，安全限制可以避免危險或不合理的指令。
17. 參考：週期同步每隔固定時間更新，適合連續變化的量測；事件同步只在狀態改變時更新，資料量小，適合狀態切換，但需確保事件不遺失。實務上常並用。
18. 參考：變生的效益要靠現場持續使用並依其結果行動；若現場不信任、無人維護或流程未調整，再好的模型也無法產生效益。蒙地卡羅分析顯示導入成功與否對 NPV 的影響最大。
19. 溫升 $25 \times (1 - e^{-1}) \approx 15.8^\circ\text{C}$ ；熱變位 $\approx 1.7 \times 15.8 \approx 26.9 \mu\text{m}$ 。
20. 可用率 $90 \div (90 + 15) \approx 85.7\%$ ；有效產能 $3,600 \div 55 \times 0.857 \approx 56.1$ 件 / 時。
21. 可用率 $90 \div 100 = 90\%$ ；有效產能 $3,600 \div 55 \times 0.9 \approx 58.9$ 件 / 時。
22. 最舊約 $5 + 0.5 + 1 = 6.5$ 秒；同步週期最長 $5 - 1.5 = 3.5$ 秒。
23. 計算量約與格數的三次方成正比（時間步數隨格數平方增加，每步的計算量隨格數增加）， $4^3 = 64$ 倍。
24. $K = 4 \div (4 + 1) = 0.8$ ；估計值 $10 + 0.8 \times (12 - 10) = 11.6$ ；變異 $(1 - 0.8) \times 4 = 0.8$ 。
25. $(0.30 - 0.230) \div 0.00145 \approx 48$ 件。
26. 年金現值因子 ≈ 3.993 ，現值 $\approx 300 \times 3.993 \approx 1,198$ 萬元，NPV ≈ 198 萬元。
27. 評分重點：問題與價值明確；OME、資料、同步頻率與保真度合理且符合適合用途；回饋路徑有安全限制與人工確認；架構對應 ISO 23247 的實體。
28. 評分重點：能以資料流與閉環區分數位模型與數位變生，指出 3D 只是視覺化層，並說明 3D 模型在接上資料與模型後可以成為變生的介面。
29. 評分重點：從單一高價值問題與數位影子開始，分階段閉環與擴展；以開放標準保存資料與模型、評估退場成本；重視使用者參與與負責人，以試點限制損失。
30. X。若資料靠人工更新、結果不回到實體，只是數位模型。
31. X。模型只要適合用途，並以實際資料持續修正、形成閉環，即可產生價值。

32. X。ISO 23247 是框架標準，第 4 部分明言不規定特定的資料格式與通訊協定，實作時可採用既有標準。
33. ✓。
34. ✓。
35. ✓。
36. 評分重點：模擬邏輯正確（故障、修復、飢餓、阻塞），結果與原程式在統計上一致；能系統性搜尋緩衝配置並解釋最佳配置的原因。
37. 評分重點：正確推導量測方程式的線性化或粒子權重，估計結果合理，能比較不同方法在非線性下的精度與計算量。
38. 評分重點：降階模型結構合理、參數以資料擬合，能量化精度與速度的權衡，並說明適用範圍。
39. 人員、設備、物料、製程、設施、環境、產品、支援文件。
40. 可觀測製造元素、資料蒐集與設備控制實體、核心實體、使用者實體、跨系統實體。

第 20 章

數位主線、孿生組合與語意互通

編著 李世淵 (機器人叫獸) | 助教 李天宇、李宇晴

本章重點

- 數位主線：工程變更的影響分析與問題的向後追溯
- ISO 23247-5 數位主線與 23247-6 孿生組合、共同模擬
- 資產管理殼與語意識別碼、知識圖譜與 SPARQL
- OpenUSD 與共同資料層的分工
- 驗證、校正、不確定性量化與確認域
- 身分、版本、資料血緣與模型治理

叫獸開講：改了一個孔，找了三天

有一次，一家精密零件廠的品保經理打電話給我，語氣很疲憊：「老師，客戶退回一批支架，說有一個孔的位置不對。我們查了三天，才搞清楚是怎麼回事。」

事情是這樣的：兩週前，研發部門因為客戶的要求，把支架上一個孔的位置移動了 0.5 公釐，在 PLM 系統裡發出工程變更，CAD 圖面也從 B 版改成了 C 版。製程工程師收到通知，更新了 CAM 程式，存在自己電腦的資料夾裡；但夜班使用的 NC 程式，是放在機台旁電腦裡的舊版本，沒有人去換。檢驗部門的三次元量測程式也還是依 B 版編寫的，所以用舊程式加工、用舊程式量測，結果「全部合格」。

更麻煩的是追查的過程。品保要回答客戶三個問題：哪些序號的產品受影響？它們是用哪一版程式、在哪台機台、用哪一批材料做的？其他已出貨的產品有沒有同樣的問題？為了回答這些問題，他們翻遍了 PLM 的變更紀錄、工程師電腦裡的 CAM 檔案、MES 的生產紀錄、品保的 Excel 量測報表，以及機台的操作日誌。這些資料都存在，卻彼此沒有連結：PLM 不知道 NC 程式用的是哪一版圖面，MES 不知道量測程式依據的是哪一版規格。

我跟經理說：「你們缺的不是資料，而是把資料串起來的那條線。」

後來，這家工廠花了一年的時間，從工程變更流程開始，讓每一個設計版本、製程規劃、NC 程式、量測程式、生產序號與量測結果，都帶著唯一的識別碼與版本，彼此以明確的關係連結起來。工程變更一發出，系統就能自動列出所有受影響的下游項目，並通知負責人；任何一個序號，都能在幾分鐘內追溯到它的設計版本、程式、機台與材料。這條把產品從設計、規劃、生產到測試串起來的線，就是數位主線。

這一章，我們要介紹數位主線的概念與實作；說明 2026 年新發布的 ISO 23247 第 5 部分（數位主線）與第 6 部分（孿生組合）；比較資產管理殼、知識圖譜與 OpenUSD 等讓不同系統「說同一種語言」的共同資料層；討論如何以驗證、確認、校正與不確定性量化，讓孿生的模型值得信任；最後談身分、版本、資料血緣與模型治理。

（說明：本節情境由作者協助企業改善工程變更流程的經驗綜合改寫，人物與細節已調整。）

學習目標

讀完本章後，你應該能夠：

1. 說明數位主線的起源與定義，以及它如何串連設計、規劃、生產與測試，並以程式進行工程變更的影響分析與問題的向後追溯。
2. 說明 ISO 23247 第 5 部分與第 6 部分的範圍，比較整合式、統一式與聯邦式的孿生組合，並以程式分析共同模擬的資料交換週期與誤差。
3. 說明資產管理殼的結構與語意識別碼的作用，並以程式示範跨供應商的語意互通。
4. 說明知識圖譜與 OpenUSD 的用途，並以 RDF 與 SPARQL 建立及查詢製造知識圖譜。
5. 區分驗證、確認、校正與不確定性量化，並以程式完成一個模型的收斂檢查、貝氏校正、預測區間與確認。
6. 說明身分、版本、資料血緣與模型治理的需求，並以程式實作簡單的血緣追蹤與完整性檢查。

核心概念

表 20-1 本章核心名詞中英對照

中文名詞	英文	一句話說明
數位主線	Digital Thread	串連產品生命週期各階段資料與模型的通訊與追溯框架
模型為基礎的定義	Model-Based Definition, MBD	以帶有尺寸、公差與註記的 3D 模型取代 2D 圖面作為權威資料
工程變更	Engineering Change	對已發行的設計進行修改的正式流程
孿生組合	Digital Twin Composition	把多個數位孿生組合為更大範圍孿生的方法
共同模擬	Co-simulation	多個模擬器各自求解，定期交換資料的模擬方式
資產管理殼	Asset Administration Shell, AAS	資產的標準化數位表示，由多個子模型組成
子模型	Submodel	AAS 中描述資產某一面向的資料集合，例如銘牌、技術資料

中文名詞	英文	一句話說明
語意識別碼	Semantic ID	指向標準字典或概念的全域識別碼，讓不同系統理解屬性的意義
知識圖譜	Knowledge Graph	以實體與關係描述知識、可被查詢與推論的圖形資料
通用場景描述	OpenUSD	描述與組合大型 3D 場景的開放格式與框架
驗證	Verification	確認電腦程式是否正確地求解了數學模型
確認	Validation	確認模型是否足夠準確地描述真實世界
校正	Calibration	以量測資料調整模型參數，使模型與實際一致
不確定性量化	Uncertainty Quantification, UQ	量化並傳遞模型、參數與量測的不確定性
資料血緣	Data Lineage	記錄資料與模型從何而來、經過哪些處理、被誰使用
模型治理	Model Governance	管理模型的版本、核准、監控、退場與責任的制度

20.1 從設計、規劃、生產到測試的數位主線

20.1.1 數位主線的起源

「數位主線」(Digital Thread) 一詞，一般認為源自航太產業。依《Digital Engineering 247》的整理，洛克希德·馬丁公司最早以這個詞描述直接以 3D CAD 資料驅動製造系統，例如 CNC 加工或碳纖維鋪放的程式，從電腦模型到現場機台形成一條不中斷的資料連結。航太與國防產業的產品複雜、生命週期長達數十年，設計變更與追溯的需求特別強烈，因此率先推動這個概念。

隨著概念的擴展，數位主線的範圍從「設計到製造」延伸到整個產品生命週期：需求、設計、模擬、製程規劃、生產、檢驗、測試、使用、維修與回收。它的意義也從單向的資料傳遞，擴展為雙向的回饋：下游的生產與使用資料，回到上游改善設計與規劃。

數位主線與數位孿生的關係常被混淆。可以這樣理解：數位孿生是某一個對象（一台機器、一條產線、一件產品）在某個時間的「數位分身」；數位主線則是把不同對象、不同階段的資料與孿生串起來的「線」。第 19 章的熱變位孿生，知道主軸現在的溫度與熱變位；數位主線則知道這台機器正在加工的是哪一版設計、用的是哪一版程式、補償模型是用哪些資料校正的。沒有主線，孿生就是一

座座孤島。

20.1.2 模型為基礎的定義與模型為基礎的企業

數位主線的一個重要基礎，是模型為基礎的定義（MBD）：以帶有尺寸、公差、材料與製程註記的 3D 模型，取代傳統的 2D 圖面，作為產品的權威資料。第 13 章介紹的 STEP AP242 就支援這類產品製造資訊。當設計以機器可讀的模型表達時，製程規劃、CAM 程式與量測程式就可以直接由模型產生，而不需要人員重新判讀圖面、重新輸入資料。

更進一步，把 MBD 的概念擴展到整個企業，讓需求、設計、製造、品質與服務都以模型和結構化資料溝通，稱為模型為基礎的企業（Model-Based Enterprise，MBE）。數位主線，就是 MBE 中串連這些模型的機制。

20.1.3 主線上的資料

表 20-2 數位主線在各階段串連的資料

階段	主要資料與模型	典型系統	本書相關章節
需求	客戶需求、規格、法規要求	需求管理、PLM	第 12、13 章
設計	CAD 模型與版本、BOM、模擬模型與結果	CAD、CAE、PLM	第 12~14 章
規劃	製程規劃、途程、NC 程式、檢驗計畫、量測程式	CAPP、CAM、MES	第 13、17 章
生產	工單、序號、機台、參數、材料批號、異常	MES、SCADA、歷史資料庫	第 17、19 章
檢驗與測試	量測結果、測試資料、不合格與處置	QMS、CMM、測試系統	第 24、25 章
使用與服務	運轉資料、維修紀錄、退回與回饋	服務系統、產品孿生	第 12、26、32 章



圖 20-1 數位主線串連產品生命週期的各階段

這些資料多半早已存在於各個系統中。數位主線要做的，不是把它們全部搬到同一個資料庫，而是讓每一筆資料有唯一的身分與版本，並記錄它們之間的關係：哪一版 NC 程式是由哪一版製程規劃產生的？哪一個序號是由哪一版 NC 程式、在哪台機台、用哪一批材料加工的？只要關係被明確記錄，就能沿著關係向下游做影響分析，或向上游做追溯。

20.1.4 影響分析與向後追溯

下面的程式以有向圖表示叫獸開講中的數位主線：從需求、CAD、製程規劃、NC 程式、檢驗計畫、量測程式，到序號、機台、材料批號與測試結果。程式做兩件事：一是工程變更的影響分析，從 CAD B 版向下游找出所有受影響的項目；二是問題的向後追溯，從不合格的測試結果向上游找出所有來源，再與合格件的來源比較，找出嫌疑：

```
from collections import deque
# 數位主線：以有向圖串連需求、設計、規劃、生產與測試的資料，做工程變更的影響分析與問題的向後追溯
edges = [ # (上游, 關係, 下游)
    ("REQ-12 孔位精度 ±0.02 mm", "衍生", "CAD 支架 rev B"),
    ("CAD 支架 rev B", "規劃", "製程規劃 v2"), ("CAD 支架 rev B", "定義", "檢驗計畫 v3"),
    ("製程規劃 v2", "產生", "NC 程式 OP20 v5"), ("檢驗計畫 v3", "產生", "CMM 程式 v3"),
    ("NC 程式 OP20 v5", "加工", "SN-001"), ("NC 程式 OP20 v5", "加工", "SN-002"), ("NC 程式 OP20 v5", "加工", "SN-003"),
    ("NC 程式 OP20 v5", "加工", "SN-004"),
    ("CMM 程式 v3", "量測", "SN-001"), ("CMM 程式 v3", "量測", "SN-002"), ("CMM 程式 v3", "量測", "SN-003"),
    ("CMM 程式 v3", "量測", "SN-004"),
    ("機台 CNC-05", "加工", "SN-001"), ("機台 CNC-07", "加工", "SN-002"), ("機台 CNC-05", "加工", "SN-003"),
    ("機台 CNC-07", "加工", "SN-004"),
    ("材料批 M-881", "投料", "SN-001"), ("材料批 M-881", "投料", "SN-002"), ("材料批 M-902", "投料", "SN-003"),
    ("材料批 M-902", "投料", "SN-004"),
```

```

("SN-001", "測試", "振動測試 T-1 合格"), ("SN-003", "測試", "振動測試 T-3 合格"),
("SN-004", "測試", "振動測試 T-4 不合格"),
]
down, up = {}, {}
for a, r, b in edges:
    down.setdefault(a, []).append((r, b)); up.setdefault(b, []).append((r, a))
def trace(start, graph):
    seen, q, order = {start}, deque([start]), []
    while q:
        x = q.popleft()
        for r, y in graph.get(x, []):
            if y not in seen: seen.add(y); q.append(y); order.append((x, r, y))
    return order
imp = sorted({y for _, _, y in trace("CAD 支架 rev B", down)})
print("工程變更影響：", len(imp), "個；已生產序號：", [n for n in imp if
n.startswith("SN")])
bad = {y for _, _, y in trace("振動測試 T-4 不合格", up)}
good = set()
for t in ["振動測試 T-1 合格", "振動測試 T-3 合格"]: good |= {y for _, _, y in trace(t, up)}
suspects = sorted(x for x in bad - good if not x.startswith("SN"))
print("合格件沒有共用的來源（嫌疑）：", suspects)
for s_ in suspects:
    print(f"{s_} 也生產了：", [y for _, y in down.get(s_, []) if y.startswith("SN") and
y != "SN-004"])

```

執行結果：CAD B 版改成 C 版時，向下游共找到 11 個受影響的項目，包括製程規劃 v2、NC 程式 OP20 v5、檢驗計畫 v3、CMM 程式 v3，以及已經生產的 SN-001 到 SN-004 四個序號與它們的測試結果。這份清單正是叫獸開講中的工廠缺少的：如果工程變更發出時就自動產生，夜班的 NC 程式與量測程式就不會被遺漏。

向後追溯時，從振動測試 T-4 不合格向上游找到 9 個來源，包括需求、設計、程式、序號 SN-004、材料批 M-902 與機台 CNC-07。只看這份清單，每一項都可能是原因；但與合格的 T-1、T-3 比較後，程式發現只有「機台 CNC-07」是合格件沒有共用的來源：材料批 M-902 也用在合格的 SN-003 上，所以嫌疑較小。程式接著找出 CNC-07 也生產了 SN-002，建議優先隔離檢查。

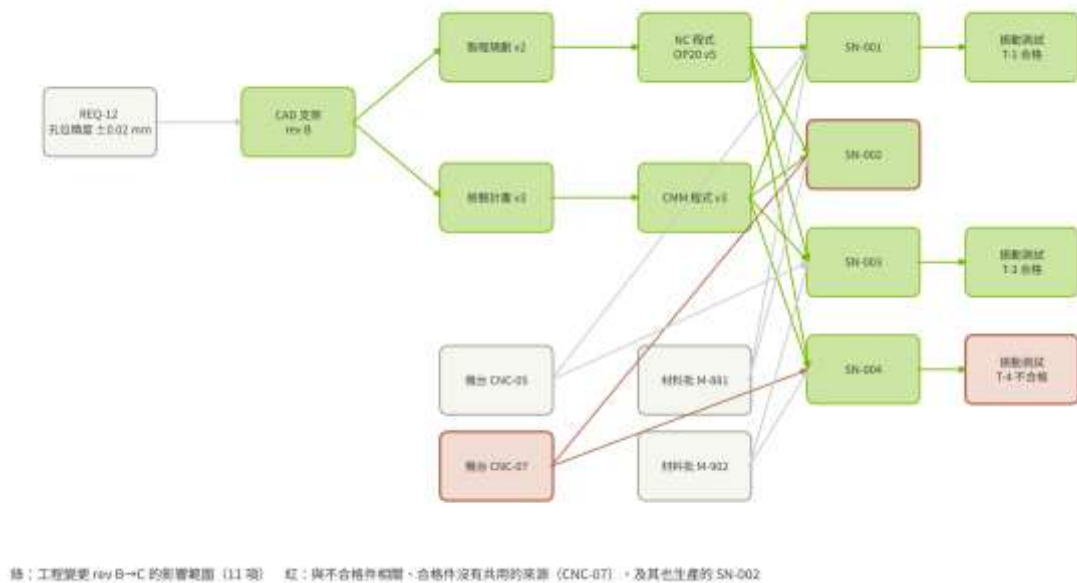


圖 20-2 數位主線上的影響分析與向後追溯

這個比較合格與不合格來源的方法很簡單，卻是許多根本原因分析的起點；第 25 章的品質分析與第 22 章的資料科學方法，可以在更大的資料上做類似但更嚴謹的推論。數位主線的價值，在於讓這些分析所需要的「關係」隨時可得，而不必花三天去翻找。

20.1.5 數位主線的實作挑戰

道理簡單，實作卻不容易。常見的挑戰包括：

- 身分不一致：同一個零件在 PLM、ERP、MES 中有不同的料號或命名規則，關係無從建立。這需要第 17 章的主資料治理與 20.5 節的身分管理。
- 版本斷鏈：檔案被複製到個人電腦或機台旁的電腦後，與原始版本失去連結，就是叫獸開講的情況。程式應由受控的系統發佈到機台，而不是靠人工複製。
- 系統封閉：許多工程與製造軟體以專屬格式保存資料，不易與其他系統連結。開放的標準格式 (STEP、QIF、MTConnect、AAS 等) 與 API 是解方。
- 顆粒度：記錄得太粗 (只知道某一天生產了哪些產品)，無法追溯；記錄得太細 (每一毫秒的所有參數)，又難以管理。顆粒度應由追溯與分析的需求決定。

20.2 ISO 23247-5 數位主線與 23247-6 學生組合

20.2.1 兩個新標準

第 19 章介紹了 ISO 23247 的第 1 到第 4 部分。2026 年，ISO 又發布了兩個新的部分：

表 20-3 ISO 23247 第 5、6 部分

部分	名稱	發布	範圍
第 5 部分	數位學生的數位主線	2026 年 6 月 24 日	說明數位主線如何在產品生命週期（設計、規劃、生產、測試）中，促成製造數位學生的建立、連接、管理與維護；以原則、方法與使用案例說明
第 6 部分	數位學生組合	2026 年 7 月 14 日	定義製造中數位學生組合的原則與方法，涵蓋學生之間的通訊、彙整與互通，並以整合式、統一式與聯邦式三種組合方式說明

這兩個部分回應了第 19 章最後提出的問題：學生建起來之後，如何沿著生命週期串連（第 5 部分），以及多個學生如何組合成更大的學生（第 6 部分）。由於這兩份標準剛發布不久，業界的實作經驗仍在累積中；本節依標準公開的摘要說明其精神，並以範例說明學生組合的實務議題。讀者若要依標準實作，應參閱標準全文。

20.2.2 第 5 部分：數位主線與學生

依標準摘要，第 5 部分把數位主線定位為讓製造數位學生在生命週期中被建立、連接、管理與維護的機制。換句話說，主線不只是追溯的工具，也是學生本身的「生命線」：

- 建立：設計階段的 CAD、BOM 與模擬模型，透過主線成為設備或產品學生的初始內容。
- 連接：規劃階段的製程、程式與檢驗計畫，透過主線與學生的模型連結，讓學生知道「應該怎麼做」。
- 管理：生產階段的實際資料，透過主線回到學生，讓學生知道「實際怎麼做」，並與計畫比較。
- 維護：測試與使用的結果，透過主線回饋到設計與規劃；設計變更時，主線也通知所有相關的學生更新。

20.1.4 節的影響分析，就是主線在「維護」上的一個例子：工程變更發生時，主線讓所有受影響的學生與資料都能被找到並更新。

20.2.3 第 6 部分：學生的組合方式

第 6 部分處理多個學生的組合。依標準摘要，組合方式分為整合式、統一式與聯邦式三種。這三個名詞源自企業整合領域對互通方式的分類，例如 ISO 14258 (1998 年) 把互通分為：整合式，所有系統採用同一種標準格式；統一式，各模型之間有共同的後設 (meta) 層結構，可以建立語意上的對應；聯邦式，沒有預先約定的後設模型，必須在運作中動態地相互調適。以下依這個觀點說明三者在學生組合上的特徵 (第 6 部分的正式定義請參閱標準全文)：



圖 20-3 學生組合的三種方式 (示意)

表 20-4 三種學生組合方式的比較 (示意)

方式	特徵	優點	缺點	類比
整合式	各學生採用同一種格式，模型與資料可以合併成一個整體	一致性高、可做緊密耦合的計算	修改困難、各方須交出模型細節	把幾家公司合併成一家
統一式	各學生保有自己的實作，但遵循共同的後設模型，可建立語意對應	互通容易、可替換	需要大家先同意共同模型	各公司採用同一套會計科目
聯邦式	沒有預先約定的共同模型，各學生獨立運作，執行時動態協調與交換資料	彈性高、保護各方智慧財產	協調與一致性較難保證	各國以條約合作

實務上，組合方式的選擇常取決於「誰擁有模型」。設備廠商不一定願意公開其機台模型的細節；不同部門的學生可能由不同的團隊、在不同的軟體上開發。這時，聯邦式的組合較可行：各學生把模型包裝成黑盒子，只透過介面交換必要的輸入與輸出。

在模擬領域，讓各模型各自求解、定期交換資料的做法稱為共同模擬（co-simulation）；若各模型又遵循共同的介面標準，就同時帶有統一式的特徵。Modelica 協會維護的功能模型介面（Functional Mock-up Interface，FMI）就是共同模擬最常用的開放標準：模型被包裝成功能模型單元（FMU），由主控程式協調各單元的時間推進與資料交換。FMI 源自歐洲的 MODELISAR 研究計畫，1.0 版於 2010 年發布，3.0 版於 2022 年發布。

20.2.4 共同模擬的資料交換週期

聯邦式組合有一個重要的代價：各學生之間只在特定的時間點交換資料，兩次交換之間，每個學生只能假設對方的狀態不變。交換越頻繁越準確，但通訊與協調的成本越高。

下面的程式以第 19 章的熱變位問題延伸：主軸學生與床身學生由不同的團隊開發。主軸發熱，熱量傳到床身，再散到環境；兩者的溫度互相影響。程式比較整合式（兩個方程式一起解）與聯邦式（各自計算，每隔 H 秒交換一次對方的溫度）：

```
import numpy as np
# 學生組合：主軸學生與床身學生由不同團隊開發；比較「整合式」（同一個模型一起解）與「聯邦式」（各自
# 運算、定期交換資料的共同模擬）
C1, C2 = 4000.0, 60000.0          # 主軸、床身的熱容（J/K）
G12, G2a, G1a = 8.0, 25.0, 2.0   # 主軸-床身、床身-環境、主軸-環境的熱導（W/K）
T_end, dt = 4 * 3600, 1.0        # 模擬 4 小時，內部時間步 1 秒
P = lambda t: 300.0 if (t % 3600) < 2700 else 0.0  # 主軸發熱：每小時運轉 45 分鐘
def integrated():
    T1 = T2 = 0.0; out = []
    for k in range(int(T_end / dt)):
        t = k * dt
        q12 = G12 * (T1 - T2)
        T1 += dt * (P(t) - q12 - G1a * T1) / C1
        T2 += dt * (q12 - G2a * T2) / C2
        out.append((T1, T2))
    return np.array(out)
def federated(H):
    # 每 H 秒交換一次對方的溫度，期間視為不變（零階保持）
    T1 = T2 = 0.0; T2_seen = T1_seen = 0.0; out = []; msgs = 0
    for k in range(int(T_end / dt)):
        t = k * dt
        if k % int(H / dt) == 0: T2_seen, T1_seen = T2, T1; msgs += 2
        T1 += dt * (P(t) - G12 * (T1 - T2_seen) - G1a * T1) / C1  # 主軸學生只看到「上次收
        # 到的」床身溫度
```

```

T2 += dt * (G12 * (T1_seen - T2) - G2a * T2) / C2 # 床身學生只看到「上次收
到的」主軸溫度
out.append((T1, T2))
return np.array(out), msgs
ref = integrated()
print(f"整合式：主軸最高溫升 {ref[:,0].max():.2f} K，床身最高溫升 {ref[:,1].max():.2f} K")
for H in [1, 10, 60, 300, 900]:
    y, msgs = federated(H)
    e1 = np.abs(y[:, 0] - ref[:, 0]).max(); e2 = np.abs(y[:, 1] - ref[:, 1]).max()
    print(f"交換週期 {H:>4} 秒：訊息 {msgs:>6} 則，主軸最大誤差 {e1:.3f} K，床身最大誤差 {e2:.3f} K")

```

執行結果：

表 20-5 聯邦式共同模擬的交換週期與誤差 (4 小時模擬)

交換週期	訊息數	主軸最大誤差	床身最大誤差
1 秒	28,800	0.000 K	0.000 K
10 秒	2,880	0.018 K	0.013 K
60 秒	480	0.123 K	0.086 K
300 秒	96	0.675 K	0.476 K
900 秒	32	2.474 K	1.832 K

整合式解出的主軸最高溫升約 35.84 K，床身約 7.63 K。聯邦式在交換週期為 1 秒時與整合式完全相同（兩者的計算方式在這個條件下等價）；週期拉長到 60 秒，誤差約 0.1 K，對熱變位補償而言（約 0.2 μm ）可以忽略，訊息數卻只有每秒交換的六十分之一；週期拉長到 15 分鐘，主軸誤差約 2.5 K，就不可接受了。

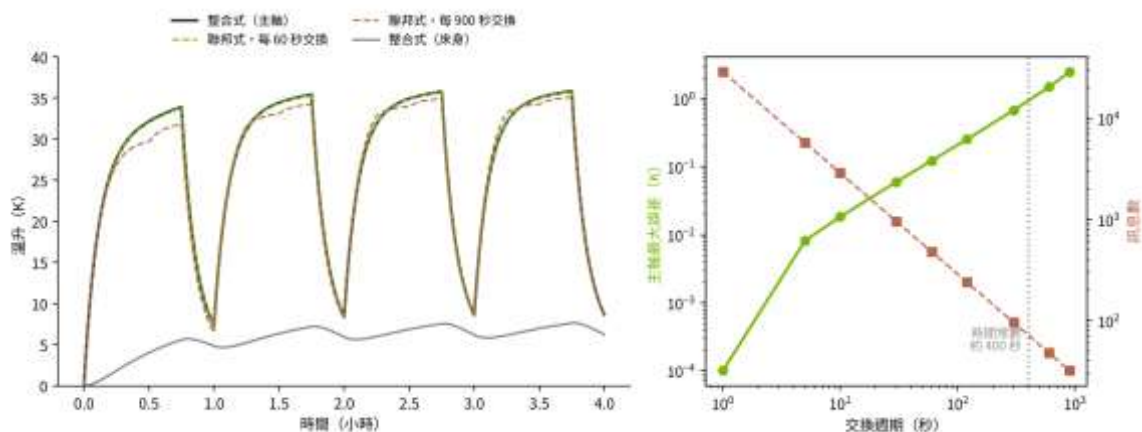


圖 20-4 共同模擬的交換週期與誤差

這個結果呼應了第 19 章「適合用途」的原則：交換週期應該比耦合的動態快得多。這個例子中，主軸的熱時間常數約為 $C1 \div (G12 + G1a) = 4,000 \div 10 = 400$ 秒，交換週期取 60 秒（約時間常數的七分之一）就已足夠；若耦合的現象更快（例如機器人與輸送帶的同步），交換週期就必須相應縮短。實務上，FMI 等標準也支援變動的交換步長與更精緻的插值方法，以在精度與成本之間取得更好的平衡。

20.3 AAS、知識圖譜與 OpenUSD 共同資料層

20.3.1 語意互通：從「看得到」到「看得懂」

第 11 章的 OPC UA 與 MQTT 讓系統之間「連得上」；第 18 章的共同資料層讓資料「找得到」。但要讓不同系統真正合作，還需要第三層：「看得懂」。同樣是主軸的最高轉速，甲廠商的機台稱為 MaxRotationSpeed，單位是 1 / min；乙廠商稱為 Drehzahl_max；丙系統可能只是一個叫 P1023 的參數。人可以從說明書猜出它們是同一件事，程式卻不行。

語意互通，就是讓資料帶著明確、可被機器理解的意義。本節介紹三種常用的技術：資產管理殼描述「資產是什麼、有哪些屬性」；知識圖譜描述「事物之間有什麼關係」；OpenUSD 描述「東西在空間中長什麼樣、放在哪裡」。它們各有專長，常常一起使用。

20.3.2 資產管理殼

資產管理殼（Asset Administration Shell，AAS）是德國工業 4.0 平台提出的概念，第 5 章介紹 RAMI 4.0 時已經提過。它的想法是：每一個資產，無論是一台機器、一個零件、一套軟體，甚至一份合約，都有一個標準化的數位表示，讓任何應用都能以相同的方式存取它的資訊與服務。

推動 AAS 的使用者組織是工業數位孿生協會（Industrial Digital Twin Association，IDTA），由德國機械設備製造業聯合會（VDMA）與電機電子工業協會（ZVEI），偕同資訊通訊產業協會（Bitkom）及 20 家機械與電機企業，於 2020 年 9 月 24 日成立，負責維護 AAS 的技術規範與子模型範本。國際標準方面，IEC 63278-1《工業應用的資產管理殼——第 1 部分：資產管理殼結構》於 2023 年 12 月發布。

依 IEC 63278-1 與 IDTA 的規範，AAS 的結構大致如下：

- 資產管理殼：一個資產的數位表示，有全域唯一的識別碼，並指向它所代表的資產。
- 子模型：描述資產某一面向的資料集合，例如數位銘牌、技術資料、運轉資料、維護資訊、碳足跡。IDTA 發布了許多子模型範本，讓不同廠商以相同的結構描述相同的面向。
- 子模型元素：子模型中的個別資料，例如屬性、範圍、檔案、操作。
- 語意識別碼：每一個子模型與元素都可以帶一個語意識別碼，指向標準字典（例如 ECLASS、IEC 共同資料字典）中的概念。這是語意互通的關鍵。
- 介面：以標準化的 API 或檔案格式（例如 JSON、XML、AASX 套件）存取 AAS。



圖 20-5 資產管理殼的結構 (簡化示意)

下面的程式以簡化的 AAS 結構，描述兩家供應商的機台。兩家使用不同語言、不同名稱的屬性，甚至不同的單位，但每個屬性都帶有語意識別碼。程式不依名稱，而依語意識別碼查詢，並做單位換算，最後找出符合加工需求的機台。為了避免與正式的識別碼混淆，程式中的語意識別碼使用教學用的示例網址：

```
import json
# 資產管理殼 (AAS)：兩家供應商以不同的名稱描述同一個屬性，用語意識別碼 (semanticId) 而非名稱來查詢
# 以下語意識別碼為教學用的示例；實務上應採用 IDTA 子模型範本與 ECLASS、IEC CDD 等字典的正式識別碼
SEM = {"max_speed": "https://example.org/sem/MaxSpindleSpeed", "power":
"https://example.org/sem/SpindlePower",
      "maker": "https://example.org/sem/ManufacturerName", "temp":
```

```

"https://example.org/sem/SpindleTemperature"}
def prop(idShort, sem, vtype, value, unit=None):
    p = {"modelType": "Property", "idShort": idShort, "semanticId": {"type":
"ExternalReference",
    "keys": [{"type": "GlobalReference", "value": sem}]}, "valueType": vtype,
"value": value}
    if unit: p["qualifiers"] = [ # 簡化：以限定詞記錄單位（正式規範中通常由概念描述定義）
        {"type": "unit", "valueType": "xs:string", "value": unit}]
    return p
aas_a = {"id": "urn:vendorA:aas:CNC-07", "assetKind": "Instance", "submodels": [
    {"idShort": "Nameplate", "submodelElements": [prop("ManufacturerName", SEM["maker"],
"xs:string", "廠商甲")]}],
    {"idShort": "TechnicalData", "submodelElements": [prop("MaxRotationSpeed",
SEM["max_speed"], "xs:int", "12000", "1/min"),
                                                    prop("RatedPower", SEM["power"],
"xs:double", "15", "kW")]}],
    {"idShort": "OperationalData", "submodelElements": [prop("SpindleTemp", SEM["temp"],
"xs:double", "38.2", "degC")]}]}
aas_b = {"id": "urn:vendorB:aas:MC-3", "assetKind": "Instance", "submodels": [
    {"idShort": "Typenschild", "submodelElements": [prop("Hersteller", SEM["maker"],
"xs:string", "廠商乙")]}],
    {"idShort": "Leistungsdaten", "submodelElements": [prop("Drehzahl_max",
SEM["max_speed"], "xs:int", "15000", "1/min"),
                                                    prop("Leistung", SEM["power"],
"xs:double", "18500", "W")]}],
    {"idShort": "Betriebsdaten", "submodelElements": [prop("Spindeltemperatur",
SEM["temp"], "xs:double", "101.3", "degF")]}]}
UNIT = {("kW", "kW"): lambda v: v, ("W", "kW"): lambda v: v / 1000,
        ("degC", "degC"): lambda v: v, ("degF", "degC"): lambda v: (v - 32) * 5 / 9}
def find(aas, sem):
    # 依語意識別碼搜尋，不依賴 idShort
    for sm in aas["submodels"]:
        for el in sm["submodelElements"]:
            if el["semanticId"]["keys"][0]["value"] == sem:
                unit = el.get("qualifiers", [{}])[0].get("value")
                return el["idShort"], float(el["value"]) if el["valueType"] !=
"xs:string" else el["value"], unit
    return None
def get(aas, key, target=None):
    name, v, unit = find(aas, SEM[key])
    return (UNIT[(unit, target)](v) if target else v), name
print("同一個語意「最高轉速」，兩家使用的名稱：", [find(a, SEM["max_speed"])[0] for a in
(aas_a, aas_b)])
for aas in (aas_a, aas_b):
    mk, _ = get(aas, "maker"); sp, n1 = get(aas, "max_speed"); pw, n2 = get(aas, "power",
"kW"); tp, n3 = get(aas, "temp", "degC")
    print(f"{aas['id']} ({mk})：最高轉速 {sp:.0f} rpm [{n1}]，額定功率 {pw:.1f} kW
[{n2}]，主軸溫度 {tp:.1f} °C [{n3}]")
capable = [a["id"] for a in (aas_a, aas_b) if get(a, "max_speed")[0] >= 14000 and get(a,
"power", "kW")[0] >= 15]
print("轉速 ≥ 14000 rpm 且功率 ≥ 15 kW 的機台：", capable)

```

執行結果：同一個「最高轉速」的語意，甲廠商稱為 MaxRotationSpeed，乙廠商稱為 Drehzahl_max。依語意識別碼查詢後，程式正確讀出甲廠商機台的最高轉速 12,000 rpm、額定功率 15.0 kW、主軸溫度 38.2 °C；乙廠商機台的最高轉速 15,000 rpm、額定功率由 18,500 W 換算為 18.5 kW、主軸溫度由 101.3 °F 換算為約 38.5 °C。符合「轉速至少 14,000 rpm 且功率至少 15 kW」的只有乙廠商的 MC-3。

這個例子說明了 AAS 與語意識別碼的價值：應用程式不需要知道每一家廠商的命名習慣，只要依標準字典中的概念查詢即可。當工廠有數十家供應商、數千台設備時，這種「依語意、不依名稱」的整合方式，比為每一對系統寫一個轉換程式更可行。若有 n 個系統各自使用自己的詞彙，兩兩對照需要 $n(n-1) \div 2$ 份對照表；若都對應到共同的字典，只需要 n 份，這與第 18 章介面數量的分析是同一個道理。

需要說明的是，程式為了簡潔做了幾處簡化：正式的 AAS 規範中，單位與定義通常由語意識別碼所指向的概念描述提供，而不是寫在每一個屬性上；JSON 的結構也比程式中的更完整。實作時，應採用 IDTA 的規範與開源工具。

20.3.3 知識圖譜

AAS 擅長描述一個資產「有什麼」，但製造中更多的問題是關於「關係」：這個零件需要哪些製程？哪些機台具備這些能力？這台機台目前在做哪一張工單？這張工單屬於哪一個客戶的訂單？這些問題適合用知識圖譜來回答。

知識圖譜以「主詞—述詞—受詞」的三元組描述知識，例如「CNC-07 具備能力 五軸銑削」、「五軸銑削 是一種 銑削」。全球資訊網協會 (W3C) 制定的資源描述框架 (RDF) 是最常用的三元組格式，網路本體語言 (OWL) 用來定義類別與關係的語意，SPARQL 則是查詢 RDF 的標準語言。知識圖譜的一個重要能力是推論：若已知「五軸銑削是一種銑削」、「銑削是一種切削加工」，查詢「哪些機台能做切削加工」時，系統就能自動把具備五軸銑削能力的機台也找出來。

下面的程式以 Python 的 rdflib 套件建立一個小型的製造知識圖譜，描述製程能力的階層、機台的能力、精度與狀態，以及零件的需求，再以 SPARQL 查詢可以加工這個零件的機台：

```
from rdflib import Graph, Namespace, Literal, RDF, RDFS
from rdflib.namespace import XSD
# 知識圖譜：以 RDF 三元組描述機台、能力、零件與排程，再以 SPARQL 查詢「哪些機台可以做這個零件」
EX = Namespace("https://example.org/factory#")
g = Graph(); g.bind("ex", EX)
T = lambda s, p, o: g.add((EX[s], EX[p] if p not in ("a", "sub") else (RDF.type if p == "a" else RDFS.subClassOf), o))
for c, parent in [("五軸銑削", "銑削"), ("三軸銑削", "銑削"), ("銑削", "切削加工"), ("車削", "切削加工")]:
    T(c, "sub", EX[parent])
machines = [(("CNC-05", "三軸銑削", 0.010, "運轉中"), ("CNC-07", "五軸銑削", 0.005, "維修中"),
              ("CNC-09", "五軸銑削", 0.008, "閒置"), ("LT-02", "車削", 0.010, "閒置")]
```

```

for m, cap, acc, st in machines:
    T(m, "a", EX["機台"]); T(m, "具備能力", EX[cap])
    T(m, "定位精度", Literal(acc, datatype=XSD.double)); T(m, "狀態", Literal(st))
T("支架-A1", "a", EX["零件"]); T("支架-A1", "需要製程", EX["銑削"]); T("支架-A1", "需要精度",
Literal(0.010, datatype=XSD.double))
q = """
PREFIX ex: <https://example.org/factory#>
PREFIX rdfs: <http://www.w3.org/2000/01/rdf-schema#>
SELECT ?m ?cap ?acc ?st WHERE {
    ex:支架-A1 ex:需要製程 ?need ; ex:需要精度 ?tol .
    ?m a ex:機台 ; ex:具備能力 ?cap ; ex:定位精度 ?acc ; ex:狀態 ?st .
    ?cap rdfs:subClassOf* ?need .          # 能力是所需製程的子類別（可推論多層）
    FILTER(?acc <= ?tol)
} ORDER BY ?acc"""
print("三元組數量:", len(g))
rows = list(g.query(q))
for m, cap, acc, st in rows:
    print(f"可加工: {m.split('#')[1]} ({cap.split('#')[1]}, 精度 {float(acc)} mm, {st})")
print("排除維修中的機台後:", [r[0].split('#')[1] for r in rows if str(r[3]) != "維修中"])

```

執行結果：圖譜共有 23 個三元組。零件「支架-A1」需要銑削、精度 0.010 mm。查詢找到三台可加工的機台，依精度排序為 CNC-07（五軸銑削，0.005 mm，維修中）、CNC-09（五軸銑削，0.008 mm，閒置）與 CNC-05（三軸銑削，0.010 mm，運轉中）。注意圖譜中並沒有直接寫「CNC-07 具備銑削能力」，是查詢中的 `rdfs:subClassOf*` 沿著「五軸銑削是一種銑削」的關係推論出來的。車床 LT-02 因為能力不符而被排除。排除維修中的機台後，剩下 CNC-09 與 CNC-05。

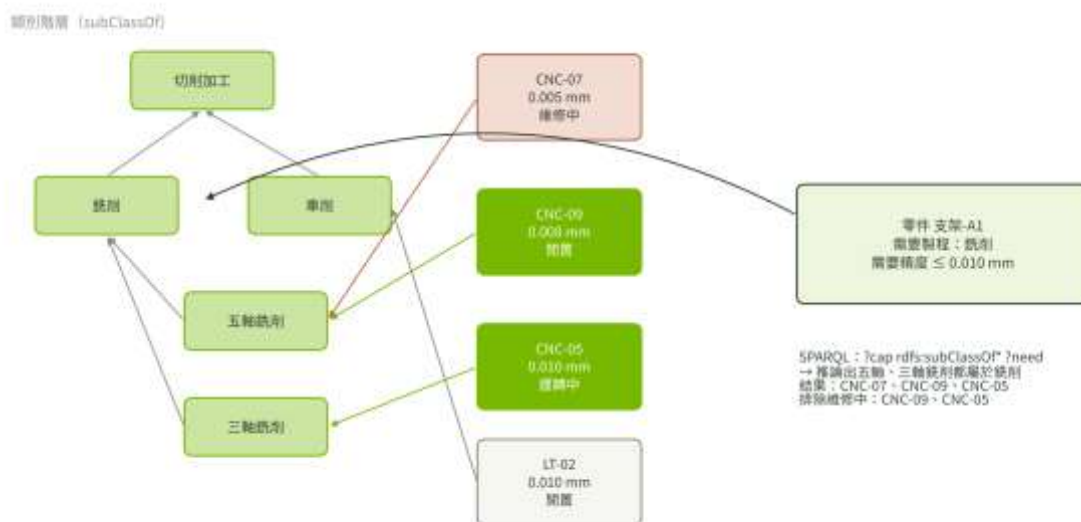


圖 20-6 製造知識圖譜與查詢結果

知識圖譜的優點是彈性：新增一種製程、一台機器或一種關係，只要加入新的三元組，不必修改資料庫的結構。這使它特別適合整合來自不同系統的異質資料，也是第 28 章企業 AI 大腦與工業知識系統的重要基礎：大型語言模型可以把使用者的自然語言問題轉成圖譜查詢，再根據查詢結果回答，

降低「憑空編造」的風險。它的挑戰則在於本體的設計與維護：類別與關係的定義需要領域專家的參與，而且要長期維持一致。

AAS 與知識圖譜並不衝突。AAS 的子模型與語意識別碼，可以轉換成知識圖譜中的節點與關係；知識圖譜則把多個資產的 AAS 串成一張網，回答跨資產的問題。

20.3.4 OpenUSD

數位孿生常需要 3D 場景：工廠佈置、機台外觀、機器人的運動、物流路線。第 13 章介紹的 STEP 與 STL 擅長描述單一零件，但一座工廠的 3D 場景由成千上萬個物件組成，來自不同的軟體與團隊，而且不斷變動。通用場景描述 (Universal Scene Description, USD) 就是為這類大型、多人協作的 3D 場景而設計的。

USD 由皮克斯動畫工作室開發，用於電影製作中的大型 3D 場景，並於 2016 年以開放原始碼釋出，通稱 OpenUSD。2023 年 8 月，皮克斯、Adobe、Apple、Autodesk 與 NVIDIA 共同成立 OpenUSD 聯盟 (Alliance for OpenUSD, AOUSD)，在 Linux 基金會旗下的聯合開發基金會推動其標準化；2025 年 12 月，聯盟發布了 OpenUSD 核心規範 1.0 版。

OpenUSD 對製造業有吸引力的特點，是它的「分層與組合」機制：一個場景可以由多個圖層疊合而成，例如建築團隊負責廠房圖層、設備廠商提供機台圖層、物流團隊負責輸送與 AMR 路線圖層，各自修改自己的圖層而不互相覆蓋，最後組合成完整的工廠場景。這與孿生組合的需求不謀而合。第 19 章提到的工廠孿生，常以 OpenUSD 作為場景描述格式，第 21 章將介紹它在虛擬試車與工業元宇宙中的應用。

需要注意的是，OpenUSD 主要描述「場景」：幾何、外觀、位置與階層。機台的技術資料、即時狀態與語意，則需要與 AAS、OPC UA 等標準搭配：例如在 USD 場景中的機台物件上，以屬性連結到它的 AAS 識別碼，讓使用者在 3D 畫面中點選機台時，就能取得它的資訊與狀態。

20.3.5 共同資料層的組合

表 20-6 語意互通與共同資料層技術的比較

技術	擅長描述	典型問題	主要標準或組織	本書相關章節
統一命名空間	即時資料的位置與結構	「CNC-07 的主軸溫度現在是多少？」	MQTT、OPC UA 等	第 11、18 章
資產管理殼	資產的屬性、文件與服務	「這台機台的額定功率與保養手冊在哪？」	IDTA、IEC 63278	第 5 章、本章
知識圖譜	事物之間的關係與推論	「哪些機台能做這個零件？影響哪些訂單？」	W3C RDF、OWL、SPARQL	本章、第 28 章
OpenUSD	3D 場景的幾何、位置與組合	「新機台放進去會不會和輸送帶干涉？」	AOUSD	本章、第 21 章
數位主線	生命週期中資料的版本與關係	「這個序號用的是哪一版設計與程式？」	ISO 23247-5、PLM	本章 20.1 節

這些技術不是互相取代，而是分工合作。一個實務的共同資料層，可能以統一命名空間傳遞即時資料，以 AAS 描述每一個資產，以知識圖譜連結資產、訂單與製程，以 OpenUSD 呈現 3D 場景，再以數位主線管理所有資料的版本與關係。共同的關鍵，是每一個資產與資料都有唯一、穩定的身分，這是 20.5 節的主題。

20.4 驗證、確認、校正與不確定性量化

20.4.1 為什麼要相信模型？

數位孿生的決策建立在模型上。第 19 章的熱變位補償、產線假設分析與刀具磨耗估計，都假設模型「夠準」。但模型憑什麼值得相信？如果模型錯了，孿生自動寫回設備的補償值，可能讓產品報廢，甚至造成安全問題。

模擬與工程界發展出一套建立模型可信度的方法，常合稱為 VVUQ：驗證（Verification）、確認（Validation）與不確定性量化（Uncertainty Quantification）；其中常常還包含校正

(Calibration) 。美國機械工程師學會 (ASME) 有一系列相關標準：

表 20-7 ASME 的 VVUQ 相關標準

標準	名稱	領域
VVUQ 1-2022	驗證、確認與不確定性量化術語	共通名詞
V&V 10-2019	計算固體力學的驗證與確認	結構與固體力學
V&V 20-2009	計算流體力學與熱傳的驗證與確認	流體與熱傳
V&V 40-2018	以驗證與確認評估計算模型的可信度：醫療器材應用	依風險決定所需的可信度

依 ASME 的說法，驗證是把電腦模型與其背後的數學模型比較；確認則是把電腦模型與實驗資料比較，判斷模擬預測真實的能力。常見的通俗說法是：驗證回答「方程式解對了嗎？」，確認回答「解的是對的方程式嗎？」。

20.4.2 四個步驟

- 驗證：檢查程式是否正確實作數學模型。程式驗證以已知解析解的問題測試程式；解答驗證則估計網格或時間步造成的數值誤差，例如第 13 章的網格收斂與第 19 章的保真度分析。
- 校正：以實驗或現場資料估計模型中不確定的參數，例如熱變位係數與時間常數。校正後的參數本身也有不確定性。
- 不確定性量化：把參數、輸入與量測的不確定性傳遞到預測，得到預測區間，而不只是一個數字。
- 確認：以「沒有用於校正」的新資料，檢查預測是否在可接受的範圍內。確認的結論只在測試過的條件範圍內有效，稱為確認域。

20.4.3 以熱變位模型示範 VVUQ

下面的程式把四個步驟串起來，以第 19 章的主軸熱變位模型為例：

- 驗證：模型以顯式歐拉法求解一階微分方程。把時間步從 4 分鐘逐次減半，與解析解比較，計算觀察到的收斂階數；歐拉法的理論階數為 1。

- 校正：以 8 筆冷機滿載升溫的量測（量測誤差標準差 $1.5\ \mu\text{m}$ ），以貝氏方法在參數網格上計算熱變位係數 a 與時間常數 τ 的後驗分布（均勻先驗）。
- 不確定性量化：從後驗分布抽樣 2,000 組參數，加上量測雜訊，預測一個新的運轉情境（運轉 60 分、停 30 分、再運轉 30 分），得到 95% 預測區間。
- 確認：以兩組新的實驗資料檢查。情境 A 的條件與校正時相同；情境 B 的環境溫度高 $4\ ^\circ\text{C}$ ，而模型中沒有環境溫度這一項。

```
import numpy as np
# 驗證、確認、校正與不確定性量化（VUQ）：以主軸熱變位模型為例
rng = np.random.default_rng(2)
kT = 25.0 # 滿載溫升（K）
def model(a, tau, duty, dt=1.0): # 數值模型：顯式歐拉法解
    dT/dt = (kT*u - T)/tau, delta = a*T
    T = 0.0; out = []
    for u in duty: T += dt * (kT * u - T) / tau; out.append(a * T)
    return np.array(out)
# (1) 驗證（Verification）：程式是否正確地解了方程式？以解析解檢查收斂階數
def euler_error(dt, a=1.7, tau=40.0, t_end=60.0):
    n = int(t_end / dt); T = 0.0
    for _ in range(n): T += dt * (kT - T) / tau
    return abs(a * T - a * kT * (1 - np.exp(-t_end / tau)))
errs = [euler_error(dt) for dt in [4.0, 2.0, 1.0, 0.5]]
orders = [np.log2(errs[i] / errs[i + 1]) for i in range(3)]
# (2) 校正（Calibration）：以 8 筆量測，用貝氏方法估計 a 與 tau 的後驗分布
a_true, tau_true, sig = 1.7, 40.0, 1.5
duty_cal = np.ones(80); t_cal = np.arange(10, 81, 10) - 1
y_cal = model(a_true, tau_true, duty_cal)[t_cal] + rng.normal(0, sig, len(t_cal))
A, TAU = np.meshgrid(np.linspace(1.2, 2.2, 101), np.linspace(20, 70, 101))
loglik = np.zeros_like(A)
for i in range(A.shape[0]):
    for j in range(A.shape[1]):
        r = y_cal - model(A[i, j], TAU[i, j], duty_cal)[t_cal]; loglik[i, j] = -0.5 *
np.sum(r**2) / sig**2
post = np.exp(loglik - loglik.max()); post /= post.sum() # 均勻先驗 → 後驗 ∝ 似然
a_m, tau_m = (post * A).sum(), (post * TAU).sum()
# (3) 不確定性量化（UQ）：從後驗抽樣，傳遞到新的運轉條件，得到預測區間
duty_val = np.r_[np.ones(60), np.zeros(30), np.ones(30)] # 確認情境：運轉 60 分、停
30 分、再運轉 30 分
idx = rng.choice(A.size, 2000, p=post.ravel())
preds = np.array([model(A.ravel()[k], TAU.ravel()[k], duty_val) for k in idx]) +
rng.normal(0, sig, (2000, len(duty_val)))
lo, hi = np.percentile(preds, [2.5, 97.5], axis=0); mid = preds.mean(axis=0)
# (4) 確認（Validation）：與新的實驗資料比較；情境 B 的環境溫度高  $4\ ^\circ\text{C}$ （模型沒有這一項）
t_val = np.arange(15, 120, 15) - 1
truth = model(a_true, tau_true, duty_val)
yA = truth[t_val] + rng.normal(0, sig, len(t_val))
yB = truth[t_val] + 3.0 * 4.0 + rng.normal(0, sig, len(t_val)) # 環境影響  $3\ \mu\text{m}/^\circ\text{C} \times 4\ ^\circ\text{C}$ 
cover = lambda y: np.mean((y >= lo[t_val]) & (y <= hi[t_val]))
print("驗證：觀察收斂階數", [f"{p:.2f}" for p in orders])
print(f"校正：a = {a_m:.2f}  $\mu\text{m}/\text{K}$ , tau = {tau_m:.1f} 分")
print(f"UQ：第 60 分鐘預測 {mid[59]:.1f}  $\mu\text{m}$ , 95% 區間 [{lo[59]:.1f}, {hi[59]:.1f}]")
print(f"確認 A：區間覆蓋 {cover(yA):.0%}, 平均誤差 {np.mean(yA - mid[t_val]):+.1f}  $\mu\text{m}$ ")
```

```
print(f"確認 B：區間覆蓋 {cover(yB):.0%}，平均誤差 {np.mean(yB - mid[t_val]):+.1f} μm")
```

執行結果：

表 20-8 熱變位模型的 VVUQ 結果

步驟	結果	解讀
驗證	時間步 4、2、1、0.5 分鐘的誤差約為 0.733、0.361、0.179、0.089 μm ，觀察收斂階數約 1.01 ~ 1.02	與歐拉法的理論階數 1 相符，程式實作正確
校正	a 約 1.91 $\mu\text{m} / \text{K}$ (標準差約 0.14)， τ 約 49.8 分 (標準差約 6.5)，兩者相關係數約 0.97	兩個參數高度相關，個別難以確定 (真值為 1.7 與 40)
不確定性量化	第 60 分鐘預測約 33.5 μm ，95% 區間約 30.2 ~ 36.8 μm	預測帶有明確的不確定範圍
確認 A (同條件)	7 筆中有 6 筆 (約 86%) 落在 95% 區間內，平均誤差約 -0.5 μm	在確認域內，模型可用
確認 B (環境高 4 $^{\circ}\text{C}$)	0 筆落在區間內，平均誤差約 +11.7 μm	超出確認域，模型缺少環境溫度項

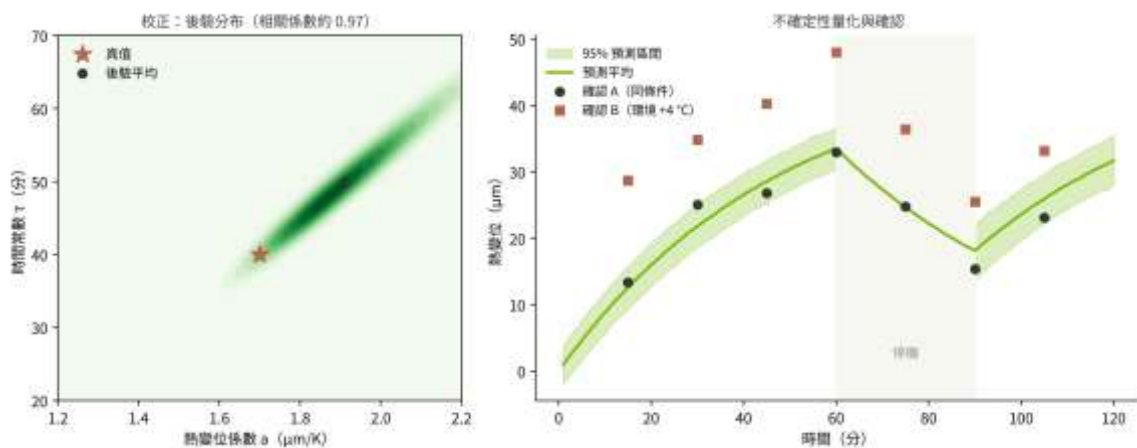


圖 20-7 熱變位模型的校正後驗與確認結果

這些結果有三個值得深思的地方。

第一，校正得到的參數與真值有明顯差距，但這不代表模型不能用。後驗分布顯示 a 與 τ 的相關係數高達 0.97：只用升溫階段前 80 分鐘的資料，「較大的 a 配上較慢的 τ 」與「較小的 a 配上較快

的 τ 」都能解釋量測，兩個參數無法被個別確定，這稱為參數不可辨識。儘管如此，模型的預測仍然準確，因為預測依賴的是參數的組合。若需要個別確定參數，就要設計能區分它們的實驗，例如加入冷卻階段或延長升溫時間。

第二，95% 預測區間只覆蓋了確認 A 中約 86% 的資料，與名目的 95% 有些差距。只有 7 筆資料時，這樣的差距在統計上並不罕見；但若在更多資料上持續偏低，就表示模型低估了不確定性，需要檢討。

第三，情境 B 完全失敗。這不是校正的問題，而是模型的形式缺少環境溫度這一項：無論參數怎麼調，都無法描述環境溫度的影響。這種誤差稱為模型形式誤差。確認的價值，正是在實際使用之前，發現模型在哪些條件下可信、在哪些條件下不可信。數位學生在確認域之外運作時，應該提出警示，而不是靜靜地給出錯誤的結果。第 19 章的熱變位例子把環境溫度納入模型，就是因為現場確實會遇到這種情況。

20.4.4 可信度要多高？

不是每一個模型都需要最高等級的 VVUQ。ASME V&V 40 提出了以風險為基礎的思考方式：模型可信度的要求，取決於模型對決策的影響程度，以及錯誤決策的後果嚴重性。一個用來產生監控看板的模型，錯了只是看錯數字；一個直接寫回機台補償值的模型，錯了可能造成報廢；一個決定醫療器材是否安全的模型，錯了可能危及生命。

對數位學生而言，這意味著：學生的自動化程度越高（從第 19 章的數位影子到數位學生），對 VVUQ 的要求也越高；而在模型的可信度尚不足時，應保留人工確認的環節（第 31、36 章）。

20.5 身分、版本、資料血緣與模型治理

20.5.1 身分：每樣東西都要有名字

數位主線與共同資料層的前提，是每一個資產、文件、資料集與模型都有唯一、穩定的身分。叫獸開講中的工廠，同一支 NC 程式在工程師電腦、伺服器與機台旁電腦上各有一份，名稱相同、內容不同，系統根本無法知道哪一份是哪一版。

製造中的身分有幾個層次：

- 類型與實例：「型號 X 的支架」是類型，「序號 SN-004 的那一件支架」是實例。第 5 章 RAMI 4.0 與第 19 章的產品孿生都區分這兩者，身分也必須區分。
- 全域唯一：身分在整個企業，甚至在供應鏈之間都不能重複。常見的做法是以網址形式的識別碼（URI）、國際註冊資料識別碼（IRDI）或通用唯一識別碼（UUID）表示。第 19 章提到，ISO 23247-3 為可觀測製造元素規定的唯一必要資訊就是識別碼；AAS 與子模型也都要求全域唯一的識別碼。
- 穩定：身分一旦發出就不應改變或重複使用。料號因為產品改名而重新編碼，是許多追溯斷鏈的原因。
- 與實體連結：條碼、QR 碼、RFID 或直接刻印的資料矩陣碼，讓實體上的身分可以被機器讀取，連結到數位世界（第 17 章）。

20.5.2 版本：知道「是哪一版」

設計、程式、配方與模型都會改變。版本管理要回答三個問題：

- 有哪些版本？每一個發行的版本都應該不可修改；要改，就發行新版本。這與軟體工程的版本控制相同。
- 版本之間差在哪裡？變更的內容、原因、核准人與日期都應記錄，這就是第 12 章的工程變更管理。
- 哪個產品用了哪一版？這稱為有效性（effectivity）：從哪一個序號、哪一個日期或哪一張工單開始，改用新版本。20.1 節的影響分析，正是依有效性找出受影響的序號。

模型的版本管理還有一個特別之處：模型不只是程式碼，還包含參數、訓練或校正用的資料、以及執行環境。同樣的程式碼，用不同的資料校正，就是不同的模型。因此，模型的版本必須同時記錄程式、參數與資料的版本，這就是資料血緣。

20.5.3 資料血緣

資料血緣記錄資料與模型「從哪裡來、經過哪些處理、被誰使用」。W3C 於 2013 年發布的

PROV 系列建議標準，提供了描述來源與血緣的通用模型，其核心概念有三個：

- 實體 (Entity)：資料集、模型、文件、決策等「東西」。
- 活動 (Activity)：產生或使用實體的過程，例如校正、訓練、補償。
- 代理人 (Agent)：對活動負責的人、軟體或組織。

實體與活動之間以「由.....產生」、「使用了」、「衍生自」、「歸屬於」等關係連結。下面的程式以這個概念，記錄熱變位變生的資料、模型與決策之間的血緣，並以雜湊值保護內容的完整性。

情境是：模型 v1.3 由工程師以八月的溫度資料與九月的 CMM 量測資料校正；模型 v1.4 由自動重訓排程以九月的資料與 v1.3 為起點重新校正；兩個版本分別被用於線上補償。事後發現，CMM-2 的量測程式有座標系錯誤，九月的 CMM 量測資料不可信：

```
import hashlib, json
# 資料血緣與模型治理：以「實體-活動-代理人」記錄每個資料集、模型與決策的來源（參考 W3C PROV 的概念）
def h(obj): return hashlib.sha256(json.dumps(obj, sort_keys=True,
ensure_ascii=False).encode()).hexdigest()[:10]
registry, links = {}, [] # 實體登錄表、關係（產物，關係，來源）
def entity(name, content, kind):
    registry[name] = {"kind": kind, "hash": h(content), "content": content}; return name
def activity(name, used, generated, agent):
    for u in used: links.append((generated, "derivedFrom", u))
    registry[generated]["generatedBy"] = name; registry[generated]["agent"] = agent
d1 = entity("溫度資料 2026-08", {"rows": 43200, "sensor": "TS-07 校正 2026-07-01"}, "資料集")
d2 = entity("溫度資料 2026-09", {"rows": 43200, "sensor": "TS-07 校正 2026-07-01"}, "資料集")
d3 = entity("量測資料 CMM 2026-09", {"rows": 640, "cmm": "CMM-2 程式 v3"}, "資料集")
m1 = entity("熱變位模型 v1.3", {"a": 1.72, "tau": 41.0, "code": "thermal.py@a1b2c3"}, "模型")
activity("校正作業 #17", [d1, d3], m1, "工程師 王")
m2 = entity("熱變位模型 v1.4", {"a": 1.88, "tau": 47.5, "code": "thermal.py@d4e5f6"}, "模型")
activity("校正作業 #18", [d2, d3, m1], m2, "自動重訓排程")
for k, (m, n) in enumerate([(m1, 120), (m2, 95)]):
    dec = entity(f"補償決策批次 {k+1}", {"model": m, "parts": n}, "決策")
    activity(f"線上補償 {k+1}", [m], dec, "學生核心 CNC-07")
def ancestors(x):
    out, stack = set(), [x]
    while stack:
        y = stack.pop()
        for a, r, b in links:
            if a == y and b not in out: out.add(b); stack.append(b)
    return out
def descendants(x):
    return {n for n in registry if x in ancestors(n)}
print("模型 v1.4 的血緣：", ", ".join(sorted(ancestors(m2))))
print("模型 v1.4 由誰產生：", registry[m2]["generatedBy"], "／", registry[m2]["agent"])
bad = descendants(d3) # 事後發現 CMM-2 程式 v3 有座標系錯誤
```

```
print("受「量測資料 CMM 2026-09」影響的項目：", "、".join(sorted(bad)))
parts = sum(registry[n]["content"]["parts"] for n in bad if registry[n]["kind"] == "決策")
print("需要重新評估的補償件數：", parts)
registry[m2]["content"]["a"] = 1.80 # 有人未經流程直接修改了模型參數
print("模型 v1.4 完整性：", "通過" if h(registry[m2]["content"]) == registry[m2]["hash"]
      else "雜湊不符，參數已被更動")
```

執行結果：模型 v1.4 的血緣包括八月與九月的溫度資料、九月的 CMM 量測資料，以及它的前一版 v1.3；它由「校正作業 #18」產生，負責的代理人是自動重訓排程。當九月的 CMM 量測資料被判定不可信時，程式沿著血緣向下游找出受影響的項目：模型 v1.3、v1.4，以及兩批補償決策，共 $120 + 95 = 215$ 件產品需要重新評估。最後，有人未經流程直接把模型 v1.4 的參數 a 從 1.88 改成 1.80，內容的雜湊值與登錄時不同，完整性檢查立即發現。

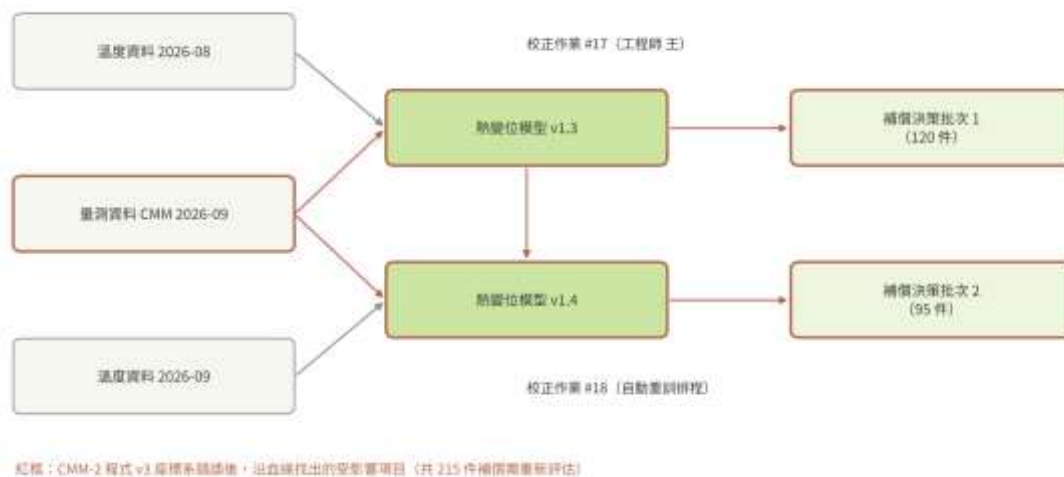


圖 20-8 資料、模型與決策的血緣

這個例子說明了血緣的兩個用途：一是影響分析，當某一筆資料出問題時，快速找出所有受影響的模型與決策，這與 20.1 節數位主線的影響分析是同一個道理，只是對象從產品資料延伸到資料與模型；二是可重現與可稽核，任何一個決策都能追溯到它用的是哪一版模型、模型又是用哪些資料、由誰、以什麼方式產生的。第 17 章的雜湊鍵保護電子批記錄，這裡的雜湊值保護模型與資料的完整性，兩者的原理相同。程式為了易讀，把雜湊值截短為 10 個十六進位字元；實務上應保留完整的 SHA-256 雜湊值，並搭配數位簽章確認是誰簽發的。

20.5.4 模型治理

當工廠中的模型從幾個增加到幾百個，有的由工程師手寫，有的由資料自動訓練，有的直接寫回設備，就需要制度化的管理，稱為模型治理。一個基本的模型治理制度包含以下要素：

表 20-9 模型治理的基本要素

要素	內容	本章與相關章節
登錄	每一個上線的模型都有唯一身分、版本、負責人與用途說明	20.5.1、20.5.2 節
血緣	記錄模型的程式、參數、資料與產生過程	20.5.3 節
可信度	依風險決定所需的驗證、確認與不確定性量化	20.4 節
核准與發佈	新版本經過測試與核准才能上線，並可回復到前一版	第 12 章的變更管理
監控	持續比較模型預測與實際，偵測準確度下降與資料漂移	第 22、26 章
確認域	明確記錄模型在哪些條件下可信，超出時發出警示	20.4.3 節
存取與完整性	權限控管、雜湊與簽章，防止未經授權的修改	第 17、35 章
退場	模型不再適用時正式停用，保留紀錄以供稽核	第 36 章

這些要素並非數位孿生獨有。對於包含機器學習的模型，國際標準 ISO / IEC 42001 (2023 年 12 月發布) 提供了人工智慧管理系統的要求，涵蓋 AI 系統的開發、提供與使用的政策、目標與流程；第 36 章將進一步討論 AI 治理與可信工業 AI。

模型治理最常見的失敗，不是缺少工具，而是缺少責任：模型上線之後，沒有人負責監控它是否仍然準確。第 19 章提到流程工業的先進製程控制常在上線幾年後因缺乏維護而被關掉，就是這種情況。因此，每一個模型都應該有明確的「主人」，就像每一台設備都有負責的保養人員一樣。

案例

案例 20-1 教學情境：改了一個孔，找了三天

背景：叫獸開講中的精密零件廠（綜合情境），工程變更後夜班仍使用舊版 NC 程式與量測程式，客戶退貨後花了三天追查受影響的範圍。

做法：讓設計版本、製程規劃、NC 程式、量測程式、序號、機台與材料批號都帶有唯一身分與版本，並記錄彼此的關係；NC 與量測程式只能由受控的系統發佈到機台；工程變更發出時自動產生影響清單並通知負責人。

成果：工程變更的影響項目可以即時列出，任何序號都能在幾分鐘內追溯到設計版本、程式、機台與材料，類似的版本錯誤不再發生。

啟示：數位主線的核心不是新的系統，而是身分、版本與關係；資料都在，缺的是那條線。

討論：(1) 為什麼「受控的程式發佈」比「提醒大家更新」更有效？(2) 追溯資料的顆粒度應該如何決定？

案例 20-2 產業：工程變更的自動影響分析

背景：一家工具機廠的產品有上千個零件，每月數十件工程變更，常因漏改某一份文件或程式而造成組裝錯誤或售後問題。

做法：在 PLM 中建立零件、圖面、BOM、製程、NC 程式、檢驗計畫與服務手冊之間的關係，並與 MES 的序號紀錄連結；工程變更發出時，系統沿關係自動列出受影響項目、負責人與已出貨的序號，並追蹤每一項的處理狀態。

成果：漏改的情況大幅減少，工程變更的處理時間縮短，售後服務也能依序號判斷每一台機器的組態。

啟示：影響分析的品質取決於關係的完整性；從設計延伸到製造與服務，數位主線的價值才完整。

討論：(1) 哪些關係最容易被遺漏？(2) 已出貨產品的工程變更，應該如何決定是否回收或升級？

案例 20-3 產業：以資產管理殼整合多家設備

背景：一家電子組裝廠的產線有來自五家廠商的設備，每家的資料格式與命名都不同，每接一台新設備，就要寫一套新的轉換程式。

做法：要求新設備的供應商依 IDTA 的子模型範本提供資產管理殼，包括數位銘牌、技術資料與運轉資料，屬性帶有標準字典的語意識別碼；既有設備則由工廠的整合團隊補建資產管理殼；應

用程式一律依語意識別碼存取資料。

成果：新設備的整合時間明顯縮短，維修人員可以從同一個介面查到各廠牌設備的技術資料與文件。

啟示：語意互通需要在採購階段就提出要求；標準化的子模型與語意識別碼，讓整合從「一對一」變成「一對多」。

討論：(1) 在採購規格中，應如何描述資產管理殼的要求？(2) 舊設備補建資產管理殼的成本如何估計？

案例 20-4 產業：模型在確認域外失效

背景：一家工廠的熱變位補償學生在春秋兩季表現良好，到了夏天，下午的工件尺寸卻開始系統性偏大。

做法：工程團隊檢查血緣與確認紀錄，發現模型是以春季的資料校正與確認的，環境溫度的範圍只涵蓋 20~24 °C；夏季下午廠房溫度超過 28 °C，已在確認域之外。團隊補充高溫的資料重新校正與確認，並在學生中加入確認域的檢查：環境溫度超出範圍時發出警示，改由人員確認補償值。

成果：夏季的尺寸偏差消除；往後每次模型更新，都會同時更新並記錄確認域。

啟示：確認的結論只在測試過的條件內有效；學生應知道自己「什麼時候不可信」。

討論：(1) 除了環境溫度，還有哪些條件應該納入確認域？(2) 如何在不中斷生產的情況下蒐集新條件下的確認資料？

案例 20-5 台灣情境：以 OpenUSD 規劃工廠

背景：依公開報導，鴻海等電子製造業者近年以 NVIDIA Omniverse 等平台建立工廠的 3D 數位學生，這類平台以 OpenUSD 作為場景描述格式。

做法：在工廠的 3D 場景中，建築、設備、輸送與機器人等圖層可由不同團隊與供應商分別維

護，再組合成完整的工廠；場景中的設備物件連結到它們的技術資料與即時狀態，用於佈置規劃、機器人訓練與產線模擬（第 19、21 章）。

成果：依報導，這類做法讓工廠可以在實體建造前於虛擬環境中完成部分規劃與驗證工作。

啟示：OpenUSD 的分層與組合機制，適合多方協作的工廠場景；但 3D 場景需要與 AAS、OPC UA 等語意與資料標準搭配，才能成為真正的孿生。

討論：（1）3D 場景中的設備物件，應該與哪些資料連結？（2）不同供應商提供的 3D 模型，如何確保尺寸與座標一致？

案例 20-6 教學情境：誰動了我的模型？

背景：一個學生專題團隊為實驗室的 3D 印表機建立溫度補償模型，模型檔放在共用資料夾中。某天補償效果突然變差，沒有人知道發生了什麼事。

做法：團隊改用版本控制管理程式與參數，為每一次校正記錄使用的資料、執行者與結果，並在模型載入時檢查雜湊值；模型更新必須經過另一位組員的審查與確認測試才能上線。

成果：後來再發生效果變差時，團隊從紀錄中很快發現是一位組員為了測試而修改了參數卻忘了改回來；導入流程後，這種事情沒有再發生。

啟示：模型治理不只是大企業的事；版本、血緣、完整性檢查與審查，是任何團隊都可以落實的基本功。

討論：（1）學生專題中，哪些模型治理的要素最值得優先落實？（2）自動重訓的模型，應該如何安排審查？

實作活動

活動 20-1 畫出一條數位主線

選擇一個你熟悉的產品（例如機器人社團的底盤零件），列出從需求、設計、製程規劃、加工程式、量測到測試的所有資料，標示每一項的身分與版本，並以 20.1.4 節的程式建立關係圖，模擬一次

工程變更的影響分析。

活動 20-2 共同模擬實驗

以 20.2.4 節的程式為基礎，把零階保持改為線性外插（以前兩次收到的值預測對方目前的溫度），比較在相同交換週期下的誤差；再嘗試讓交換週期隨主軸發熱的變化自動調整。

活動 20-3 建立資產管理殼

為實驗室的一台設備（例如 3D 印表機或小型 CNC），依 IDTA 的數位銘牌與技術資料子模型範本，建立一個資產管理殼，並以開源的 AAS 工具檢視與提供 API；再改寫 20.3.2 節的程式，從 API 讀取資料。

活動 20-4 製造知識圖譜

擴充 20.3.3 節的知識圖譜，加入工單、訂單、客戶與材料，回答「若 CNC-09 停機，會影響哪些客戶的訂單？」等問題，並嘗試把一個自然語言問題人工轉換成 SPARQL 查詢。

活動 20-5 VVUQ 實驗

以 20.4.3 節的程式為基礎，在校正資料中加入冷卻階段，觀察 α 與 τ 的相關係數是否下降；再把環境溫度加入模型，重新校正與確認情境 B，比較結果。

活動 20-6 模型登錄表

以 20.5.3 節的程式為基礎，為你的專題或實驗室建立一個模型登錄表，記錄每個模型的版本、負責人、血緣、確認域與雜湊值，並設計一個「資料被判定不可信時」的影響分析與處理流程。相關範例程式將於本書的實驗倉庫中維護。

延伸閱讀

- ISO 23247-5:2026 與 ISO 23247-6:2026：製造數位孿生框架的數位主線與孿生組合。
- IEC 63278-1:2023，以及工業數位孿生協會（IDTA）的資產管理殼規範與子模型範本。
- W3C 的 RDF、OWL、SPARQL 與 PROV 系列建議標準：知識圖譜與資料血緣的基礎。

- OpenUSD 聯盟 (AOUSD) 發布的 OpenUSD 核心規範 1.0 (2025) 。
- ASME V&V 10、V&V 20、V&V 40 與 VVUQ 1 等標準：計算模型的驗證、確認與不確定性量化。
- Marc C. Kennedy 與 Anthony O'Hagan · 〈 Bayesian calibration of computer models 〉 · Journal of the Royal Statistical Society Series B · 2001：電腦模型貝氏校正與模型差異的經典論文。

表 20-10 本章思考地圖

本章概念	關鍵提問	後續延伸章節
數位主線	資料有身分、版本與關係嗎？變更能自動找到影響嗎？	第 12、25、33 章
孿生組合	誰擁有模型？整合、統一或聯邦？交換週期夠嗎？	第 21 章
AAS 與語意識別碼	系統依名稱還是依語意整合？	第 5、11、33 章
知識圖譜	跨系統的關係問題能被查詢與推論嗎？	第 28、29 章
OpenUSD	3D 場景能多方協作，並連結到資料嗎？	第 21、31 章
VVUQ	模型經過驗證與確認嗎？確認域在哪裡？	第 21、22、36 章
血緣與模型治理	每個決策能追溯到哪一版模型與資料嗎？誰負責？	第 35、36 章

章末小結

- 數位主線一詞源自航太產業，指以模型資料直接驅動製造的不中斷資料連結，後來擴展為串連需求、設計、規劃、生產、測試與服務的雙向框架。它的核心是身分、版本與關係；本章範例中，一次工程變更自動找出 11 個受影響項目，比較合格與不合格件的來源，則把嫌疑縮小到單一機台。
- ISO 23247 第 5 部分 (2026 年 6 月) 說明數位主線如何促成孿生的建立、連接、管理與維護；第 6 部分 (2026 年 7 月) 以整合式、統一式與聯邦式說明孿生組合。聯邦式共同模擬保護各方的模型，但需要權衡交換週期：範例中每 60 秒交換一次，誤差約 0.1 K，訊息只有每秒交換的六十分之一。FMI 是共同模擬的常用開放標準。

- 語意互通讓系統「看得懂」彼此。資產管理殼由 IDTA 推動、IEC 63278-1 於 2023 年 12 月發布，以子模型與語意識別碼描述資產；知識圖譜以 RDF 三元組與 SPARQL 描述並推論關係；OpenUSD 由皮克斯於 2016 年開源，AOUSD 於 2025 年 12 月發布核心規範 1.0，擅長多方協作的 3D 場景。它們與統一命名空間、數位主線分工合作。
- VVUQ 讓模型值得信任：驗證檢查方程式解得對不對，校正估計參數，不確定性量化給出預測區間，確認以新資料檢查模型是否描述真實。範例中驗證得到收斂階數約 1，校正揭露參數不可辨識，確認則發現模型在環境溫度改變時失效，說明確認域的重要。可信度的要求應依決策的風險而定。
- 身分要全域唯一且穩定，版本要不可修改並記錄有效性，血緣以實體、活動與代理人記錄資料與模型的來源（W3C PROV）。範例中一筆不可信的量測資料影響了兩個模型版本與 215 件產品的補償決策，雜湊值則發現了未經授權的參數修改。模型治理包括登錄、血緣、可信度、核准、監控、確認域、存取與退場，最關鍵的是明確的負責人。

習題

一、選擇題

1. 「數位主線」一詞一般認為源自哪一個產業？（A）半導體 （B）航太 （C）食品 （D）紡織
2. 以帶有尺寸、公差與註記的 3D 模型取代 2D 圖面作為權威資料，稱為？（A）MBD （B）MES
（C）MRP （D）MPC
3. ISO 23247 中規範數位主線的是第幾部分？（A）第 2 部分 （B）第 4 部分 （C）第 5 部分
（D）第 6 部分
4. 依 ISO 14258 的分類，沒有預先約定的後設模型、必須在運作中動態調適的互通方式是？（A）整合式
（B）統一式 （C）聯邦式 （D）集中式
5. 由 Modelica 協會維護、常用於共同模擬的開放標準是？（A）FMI （B）STEP （C）MQTT
（D）PROV
6. 資產管理殼中，讓不同廠商的屬性可以被正確理解的關鍵是？（A）idShort （B）語意識別碼

(C) 檔案大小 (D) 介面顏色

7. 查詢 RDF 知識圖譜的標準語言是？(A) SQL (B) SPARQL (C) G-code (D) Modelica

8. OpenUSD 最初由哪一家公司開發？(A) 皮克斯 (B) 西門子 (C) IBM (D) ABB

9. 「把電腦模型與實驗資料比較，判斷模擬預測真實的能力」是？(A) 驗證 (B) 確認 (C) 校正 (D) 編譯

10. W3C PROV 的三個核心概念是？(A) 實體、活動、代理人 (B) 訂單、工單、配方 (C) 設計、製造、服務 (D) 感測、決策、執行

二、簡答題

11. 說明數位主線與數位孿生的差異與關係。

12. 為什麼 NC 程式應該由受控的系統發佈到機台，而不是由人員複製？

13. 比較整合式、統一式與聯邦式孿生組合的優缺點。

14. 為什麼聯邦式共同模擬的交換週期應該比耦合的動態快得多？

15. 說明資產管理殼的主要結構，以及語意識別碼的作用。

16. 知識圖譜的「推論」能力是什麼？舉一個製造上的例子。

17. 說明驗證、校正、不確定性量化與確認四個步驟的差異。

18. 什麼是確認域？為什麼孿生在確認域外運作時應該發出警示？

三、計算題

19. 一份製程規劃產生 3 支 NC 程式，每支程式各加工了 120 件產品。若該製程規劃因工程變更而失效，受影響的產品有多少件？

20. 共同模擬 4 小時，兩個孿生每 60 秒互相交換一次溫度（每次 2 則訊息），共需多少則訊息？若改為每 10 秒交換一次呢？

21. 主軸孿生的熱容 $4,000 \text{ J/K}$ ，與床身及環境的總熱導 10 W/K ，熱時間常數為多少秒？若交換週期取時間常數的七分之一，約為多少秒？

22. 乙廠商機台的功率為 18,500 W、主軸溫度為 101.3 °F，換算為 kW 與 °C。
23. 有 8 個系統各自使用自己的詞彙，若兩兩建立對照表需要幾份？若都對應到共同字典需要幾份？
24. 以時間步 2 分鐘與 1 分鐘計算的誤差分別為 0.3609 與 0.1791 μm ，求觀察到的收斂階數。
25. 某量以網格 h 計算得 10.36，以 $h/2$ 計算得 10.18，若方法為一階收斂，以理查森外插估計網格無限細時的值。
26. 若把 SHA-256 雜湊值截短為 10 個十六進位字元（40 位元），登錄 100 萬個實體時，依生日問題估算至少發生一次碰撞的機率約為多少？這說明了什麼？

四、申論與設計題

27. 為一家中小型機械加工廠設計數位主線：列出需要串連的資料、身分與版本規則、關係的記錄方式，以及導入步驟。
28. 評論以下說法：「只要模型和實驗資料吻合，模型就可以用在任何情況。」
29. 為一座使用多家供應商設備的工廠，規劃以 AAS、知識圖譜、OpenUSD 與統一命名空間組成的共同資料層，說明各自的分工與連結方式。

五、是非題

30. 數位主線就是把所有資料搬到同一個資料庫。
31. 聯邦式學生組合可以讓模型擁有者不必公開模型的內部細節。
32. 依名稱整合多家廠商的資料，比依語意識別碼整合更可靠。
33. 校正得到的參數與真值不同，模型的預測就一定不準。
34. 確認的結論只在測試過的條件範圍內有效。
35. 模型的版本只需要記錄程式碼的版本。

六、進階挑戰題

36. 以 Python 的 basyx 等開源套件，依 IDTA 規範建立一個真正的資產管理殼（含數位銘牌子模型），匯出為 AASX 套件，並寫程式依語意識別碼讀取其中的屬性。

37. 擴充 20.4.3 節的程式，在模型中加入「模型差異」項（例如以高斯過程描述），依 Kennedy 與 O'Hagan 的架構重新校正，並比較情境 B 的預測區間。
38. 把 20.1.4 節的數位主線與 20.5.3 節的血緣改寫為 RDF 知識圖譜（可使用 W3C PROV 的詞彙），並以 SPARQL 完成影響分析與向後追溯。

七、補充題

39. 列出表 20-9 中模型治理的八個基本要素。
40. 列出 ISO 23247 第 5 部分摘要中，數位主線所促成的孿生生命週期活動，以及涵蓋的四個產品生命週期階段。

提示與參考解答

1. B。
2. A。
3. C。
4. C。
5. A。
6. B。
7. B。
8. A。
9. B。
10. A。
11. 參考：數位孿生是某個對象的數位分身，反映其狀態；數位主線是串連不同對象、不同生命週期階段的資料與孿生的線，記錄身分、版本與關係。主線讓孿生知道自己對應哪一版設計、程式與資料。
12. 參考：人工複製會讓檔案脫離原始版本，無法確認使用的是哪一版，也無法在工程變更時自動更

- 新；受控發佈確保機台上只有核准的版本，並留下紀錄。
13. 參考：整合式一致性高但修改困難、須交出模型；統一式互通容易但需先同意共同模型；聯邦式彈性高、保護智財，但協調與一致性較難保證。
 14. 參考：兩次交換之間各變生假設對方狀態不變；若對方的狀態在這段時間內變化明顯，誤差就會累積。交換週期遠小於耦合動態的時間常數時，誤差才可忽略。
 15. 參考：資產管理殼代表一個資產，由多個子模型組成，子模型包含子模型元素；語意識別碼指向標準字典中的概念，讓應用程式依意義而非名稱理解屬性。
 16. 參考：依已知的類別階層或規則，推出未明寫的事實。例如已知「五軸銑削是一種銑削」，查詢能做銑削的機台時，自動包含具備五軸銑削能力的機台。
 17. 參考：驗證檢查程式是否正確求解數學模型；校正以資料估計參數；不確定性量化把不確定性傳遞到預測區間；確認以未用於校正的新資料檢查模型是否描述真實。
 18. 參考：確認域是模型經確認可信的條件範圍。超出範圍時，模型可能因缺少某些物理項而失準，卻仍給出看似合理的數字；警示可以讓人員介入，避免錯誤的自動決策。
 19. $3 \times 120 = 360$ 件。
 20. $4 \times 3,600 \div 60 \times 2 = 480$ 則；每 10 秒交換為 2,880 則。
 21. $4,000 \div 10 = 400$ 秒； $400 \div 7 \approx 57$ 秒，約 60 秒。
 22. 18.5 kW； $(101.3 - 32) \times 5 \div 9 \approx 38.5$ °C。
 23. $8 \times 7 \div 2 = 28$ 份；8 份。
 24. $\log_2 (0.3609 \div 0.1791) \approx \log_2 2.015 \approx 1.01$ 。
 25. 一階收斂時 $f \approx 2 \times 10.18 - 10.36 = 10.00$ 。
 26. 機率 $\approx 1 - \exp(-n^2 \div 2^{41}) = 1 - \exp(-10^{12} \div 2.2 \times 10^{12}) \approx 0.37$ 。說明截短的雜湊值不足以作為大量實體的唯一身分或完整性證明，實務上應保留完整的 SHA-256。
 27. 評分重點：資料範圍涵蓋設計到測試；身分全域唯一且穩定、版本不可修改並記錄有效性；關係有明確的記錄方式；導入從工程變更與程式發佈等痛點開始，分階段擴展。

28. 評分重點：能指出吻合只代表在確認過的條件下可信；超出確認域、存在模型形式誤差或參數不可辨識時，預測可能失準；應依風險進行 VVUQ 並記錄確認域。
29. 評分重點：統一命名空間負責即時資料、AAS 描述資產、知識圖譜連結關係、OpenUSD 呈現場景；以共同的身分連結各技術，並說明治理與資安。
30. X。數位主線記錄資料之間的身分、版本與關係，資料可以留在原本的系統中。
31. ✓。
32. X。名稱因廠商與語言而異，依語意識別碼整合才可靠。
33. X。參數不可辨識時，個別參數可能偏離真值，但參數組合仍可給出準確的預測；應以確認判斷。
34. ✓。
35. X。模型版本應同時記錄程式、參數、校正或訓練資料與執行環境。
36. 評分重點：資產管理殼與子模型結構符合規範、語意識別碼正確、AASX 可被其他工具開啟，程式能依語意讀取屬性。
37. 評分重點：正確建立模型差異項與超參數，校正結果合理，能說明加入模型差異後預測區間的變化及其意義。
38. 評分重點：圖譜的類別與關係設計合理，善用 PROV 詞彙，SPARQL 查詢能正確完成向下影響分析與向上追溯。
39. 登錄、血緣、可信度、核准與發佈、監控、確認域、存取與完整性、退場。
40. 活動：建立、連接、管理與維護製造數位孿生；階段：設計、規劃、生產與測試。

第 21 章

模擬、虛擬試車、虛擬機台與工業元宇宙

編著 李世淵 (機器人叫獸) | 助教 李天宇、李宇晴

本章重點

- 離散事件、物理、系統動態與代理人四種模擬典範
- 模型、軟體與硬體在迴路，以及 PLC 虛擬試車
- 虛擬機台與虛擬量測：漂移修正與可信度檢查
- AR / VR、工業元宇宙與 OpenUSD 圖層組合
- 合成資料、領域隨機化與 Sim-to-Real
- 代理模型、物理資訊神經網路與 AI 驅動孿生

叫獸開講：在虛擬工廠裡先把錯誤犯完

我認識一位做自動化整合的工程師，他告訴我一個讓他刻骨銘心的經驗。

那年，他的團隊為客戶建一條全新的自動化包裝線：輸送帶、條碼讀取、分流推桿、機器手臂與自動倉儲。PLC 程式在公司寫好、看起來沒問題，便帶到客戶工廠現場試車。結果，光是試車就花了將近一個月。分流推桿常把箱子推錯道；兩個箱子靠得太近時，系統會「記錯」箱子的類別；機器手臂和輸送帶的交握訊號偶爾對不上，整條線就卡住。每找到一個錯誤，就要停線、修改、重新測試。客戶的生產計畫因此延後，團隊也在現場熬了好幾個通宵。

下一個專案，他們決定換個做法。在設備還沒出廠之前，先在電腦裡建立整條產線的模擬模型：輸送帶怎麼動、箱子在哪裡、感測器什麼時候觸發、推桿多久伸出。然後把真正的 PLC 程式接上這個模型，讓 PLC「以為」自己在控制真的產線。工程師在辦公室裡一次又一次地測試各種情況：箱子間距很小、條碼讀不到、推桿故障、緊急停止。幾十個錯誤在虛擬環境中就被找到並修正。等到真正的設備到了現場，試車只用了原本不到一半的時間。

他笑著說：「以前是在客戶的工廠裡犯錯，現在是在虛擬工廠裡先把錯誤犯完。」

這就是虛擬試車。它是本章的主角之一，但模擬在智慧製造中的用途遠不只於此：從規劃產線產能的離散事件模擬，到分析供應鏈動態的系統動態，再到描述自主移動機器人車隊的代理人模擬；從以製程資料預測晶圓品質的虛擬量測，到讓人在虛擬環境中協作的工業元宇宙；以及近年快速發展、以模擬產生資料來訓練 AI 的合成資料與 Sim-to-Real。

這一章，我們會依序介紹四種模擬典範、軟體與硬體在迴路的虛擬試車、虛擬機台與虛擬量測、AR / VR 與工業元宇宙，最後談合成資料、Sim-to-Real 與 AI 驅動的孿生。

（說明：本節情境由作者與業界工程師交流的經驗綜合改寫，人物與細節已調整。）

學習目標

讀完本章後，你應該能夠：

1. 比較離散事件、物理、系統動態與代理人四種模擬典範的適用問題，並以程式實作排隊、庫存動態

與 AMR 車隊的模擬。

2. 區分模型在迴路、軟體在迴路與硬體在迴路，說明虛擬試車的流程與效益，並以程式示範在上線前找出 PLC 邏輯錯誤。
3. 說明虛擬機台與虛擬量測的概念，並以程式建立虛擬量測模型、修正機台漂移與判斷預測的可信度。
4. 說明 AR、VR、MR 與工業元宇宙的應用，並以 OpenUSD 組合多個圖層的工廠場景、進行佈置檢查。
5. 說明合成資料、領域隨機化與 Sim-to-Real 的概念，並以程式比較固定參數模擬與領域隨機化在真實環境中的表現。
6. 說明 AI 驅動孿生的發展，包括代理模型、物理資訊神經網路與基礎模型，並判斷其適用與限制。

核心概念

表 21-1 本章核心名詞中英對照

中文名詞	英文	一句話說明
離散事件模擬	Discrete-Event Simulation, DES	只在事件發生時更新系統狀態的模擬，適合排隊與產線
系統動態	System Dynamics, SD	以存量、流量與回饋迴路描述系統整體行為的模擬
代理人模擬	Agent-Based Simulation, ABS	以許多依規則行動的個體，觀察整體湧現行為的模擬
模型在迴路	Model-in-the-Loop, MIL	控制演算法與受控對象都以模型表示並一起模擬
軟體在迴路	Software-in-the-Loop, SIL	實際的控制程式在電腦上執行，連接受控對象的模型
硬體在迴路	Hardware-in-the-Loop, HIL	實際的控制器硬體連接即時運算的受控對象模型
虛擬試車	Virtual Commissioning	在實體設備完成前，以模擬模型測試與除錯控制程式
虛擬機台	Virtual Machine Tool	以模型描述機台行為，用於加工模擬、補償與預測
虛擬量測	Virtual Metrology, VM	以製程與設備資料預測產品量測值，減少實際量測
擴增實境	Augmented Reality, AR	把數位資訊疊加在真實世界的影像上

中文名詞	英文	一句話說明
虛擬實境	Virtual Reality, VR	讓使用者沉浸在完全由電腦產生的環境中
工業元宇宙	Industrial Metaverse	以即時 3D、模擬與孿生為基礎，讓人與 AI 協作的虛擬工業空間
合成資料	Synthetic Data	以模擬或生成模型產生、而非實際蒐集的訓練資料
領域隨機化	Domain Randomization	在模擬中隨機改變參數，使模型能適應真實世界的差異
Sim-to-Real	Simulation-to-Reality	把在模擬中訓練的模型或策略移轉到真實世界
代理模型	Surrogate Model	以資料擬合高保真度模擬的快速近似模型

21.1 離散事件、物理、系統動態與代理人模擬

21.1.1 四種模擬典範

「模擬」一詞涵蓋很多不同的方法。依描述系統的方式，常可分為四種典範：

表 21-2 四種模擬典範的比較

典範	描述方式	時間推進	典型問題	本書相關章節
離散事件	實體（工件、訂單）在資源之間流動、排隊	跳到下一個事件	產能、瓶頸、在製品、交期	第 8、19、27 章
物理	微分方程式描述力、熱、流體、電磁	連續、小時間步	結構強度、熱變形、機器人運動	第 13、16、19 章
系統動態	存量、流量與回饋迴路	連續、大時間步	供應鏈波動、產能投資、政策評估	第 8、33 章
代理人	許多個體依規則行動並互動	離散步驟	AMR 車隊、人員行為、市場	第 10、29 章

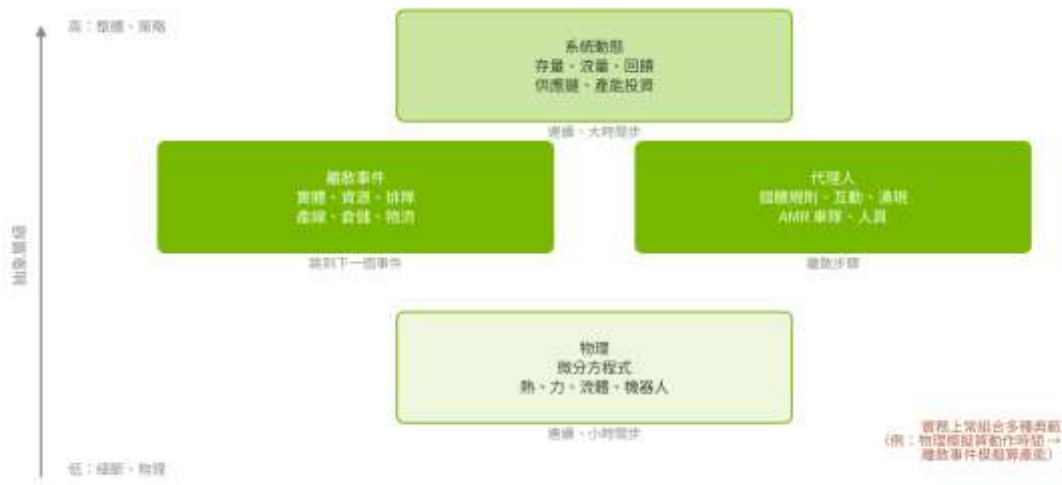


圖 21-1 四種模擬典範與其抽象層級

這四種典範的抽象層級不同：物理模擬看得最細，描述一個零件或一台機器內部的物理現象；離散事件與代理人模擬描述一條產線或一個車隊中的個體；系統動態則從最高的層級看整體，例如整條供應鏈的庫存與訂單。實務上，許多問題需要結合多種典範，例如以物理模擬算出機器人的動作時間，再放進離散事件模擬中計算產線產能，這與第 20 章的孿生組合是同一個思路。

離散事件與系統動態的發展都很早。IBM 的 Geoffrey Gordon 開發的通用模擬系統 GPSS，於 1961 年正式發布，是最早被廣泛使用的離散事件模擬語言之一；同一年，麻省理工學院的 Jay Forrester 出版《工業動態學》（Industrial Dynamics），被視為系統動態領域的開山之作。代理人模擬則隨著計算能力的提升，在 1990 年代以後逐漸普及。

21.1.2 離散事件模擬

離散事件模擬的核心想法是：系統的狀態只在「事件」發生時改變，例如工件到達、開始加工、加工完成。兩個事件之間，什麼都不會變，所以模擬可以直接「跳」到下一個事件，而不必一秒一秒地計算。這使它特別適合模擬產線、倉儲與物流這類由排隊與等待主導的系統。

第 8 章介紹過排隊理論：工作站的利用率越高，等待時間會急劇增加；加工時間的變異越大，等待也越長。下面的程式以事件清單模擬一個工作站：工件隨機到達，依先到先服務加工，記錄每一件的等待時間，並與排隊理論的解析解比較。程式同時比較兩種加工時間：指數分布（變異大，對應 $M/M/1$ 模型）與固定時間（沒有變異，對應 $M/D/1$ 模型）：

```

import heapq, numpy as np
# 離散事件模擬：單一工作站的排隊，以事件清單推進時間，並與排隊理論的解析解比較
def simulate(lam, mu, n=200_000, service="exp", seed=1):
    rng = np.random.default_rng(seed)
    events, t_free, waits, t = [], 0.0, [], 0.0
    for i in range(n):
        # 產生到達事件
        t += rng.exponential(1 / lam); heapq.heappush(events, (t, i))
    while events:
        t_arr, i = heapq.heappop(events)
        # 取出時間最早的事件
        start = max(t_arr, t_free)
        # 機台忙碌就排隊
        s = rng.exponential(1 / mu) if service == "exp" else 1 / mu
        t_free = start + s; waits.append(start - t_arr)
    return np.mean(waits[n // 10:])
# 去掉暖機期
# 每分鐘可加工 1 件
mu = 1.0
for lam in [0.5, 0.8, 0.9, 0.95]:
    rho = lam / mu
    wq_mm1 = rho / (mu - lam); wq_md1 = rho / (2 * mu * (1 - rho))
    print(f"利用率 {rho:.0%}：等待時間 模擬 {simulate(lam, mu):6.2f} 分 (M/M/1 解析 {wq_mm1:6.2f})；"
          f"加工時間固定時 模擬 {simulate(lam, mu, service='det'):6.2f} 分 (M/D/1 解析 {wq_md1:6.2f})")

```

執行結果：

表 21-3 單一工作站的平均等待時間（分）

利用率	M / M / 1 模擬	M / M / 1 解析	M / D / 1 模擬	M / D / 1 解析
50%	1.00	1.00	0.50	0.50
80%	4.11	4.00	2.05	2.00
90%	9.57	9.00	4.63	4.50
95%	21.44	19.00	9.96	9.50

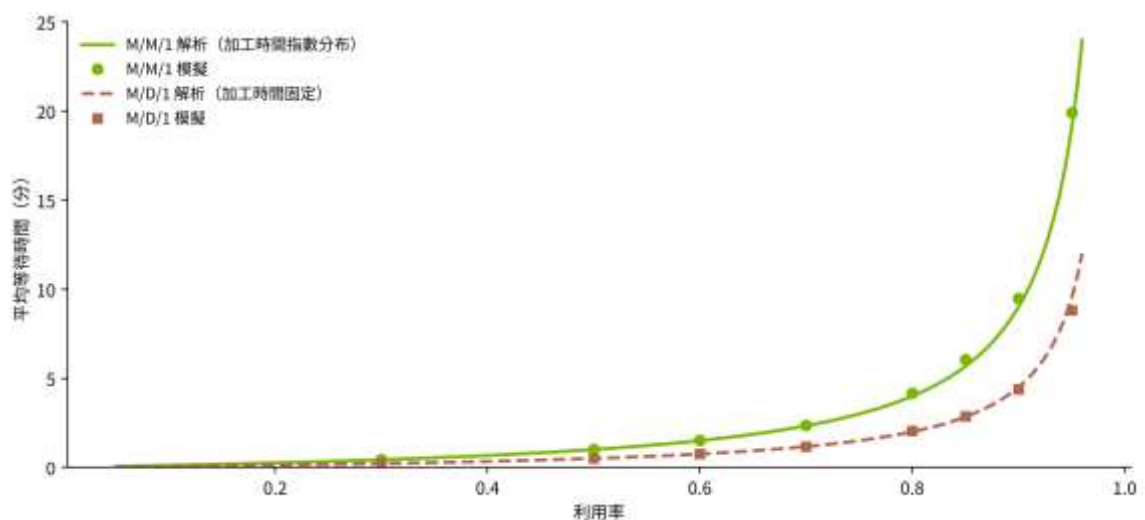


圖 21-2 離散事件模擬與排隊理論：利用率與等待時間

模擬結果與解析解相當接近，這是一種「以已知答案檢查模擬程式」的做法，正是第 20 章所說的驗證。利用率越高，兩者的差距越大，因為系統越接近飽和，等待時間的波動越大，需要越長的模擬才能收斂；在利用率 95% 時，即使模擬了二十萬件，誤差仍有一成以上。這也提醒我們：模擬結果本身是隨機的，應該以多次重複或更長的模擬估計其信賴區間。

這個結果也再一次印證了第 8 章的工廠物理：加工時間沒有變異時，等待時間只有指數分布時的一半；利用率從 80% 升到 95%，等待時間增加為將近五倍。真實的產線比單一工作站複雜得多，有多種產品、多個工作站、故障、換線與人員排班，解析解不再存在，這時離散事件模擬就成為分析產線的主要工具，第 19 章的產線學生就是一例。

21.1.3 物理模擬

物理模擬以微分方程式描述零件或設備內部的物理現象。第 13 章的有限元素分析、第 16 章的熔池模型、第 19 章的熱傳導模型都是物理模擬。對於機器人與自動化設備，常用的還有多體動力學模擬：描述連桿、關節、馬達與夾爪之間的力與運動，以及物體之間的接觸與碰撞。遊戲與電影產業發展出的即時物理引擎，近年也大量應用在機器人的虛擬訓練中，這是 21.5 節的主題。

物理模擬的挑戰在於速度與參數：高保真度的模型算得慢，而摩擦、接觸與材料等參數又常常難以準確得知。這兩個問題，分別對應第 19 章的保真度權衡與 21.5 節的 Sim-to-Real 落差。

21.1.4 系統動態

系統動態從整體的角度看系統：關心的不是個別的工件，而是「存量」（庫存、在途訂單、產能、人力）如何隨「流量」（生產、出貨、訂貨）變化，以及這些變化如何透過回饋迴路影響彼此。Forrester 在《工業動態學》中，就以這種方法分析了企業生產與分銷系統中的庫存波動，這種現象後來被稱為長鞭效應（第 8、33 章）。

下面的程式模擬一個簡單的庫存系統：需求在第 10 週由每週 100 件升到 120 件；企業每週依需求預測、庫存缺口與在途訂單缺口決定訂貨量；訂單要經過平均 6 週才會到貨，以三階延遲表示，也就是貨品依序經過三段管線。程式比較兩種訂貨政策：一種只看手上的庫存，忽略已經下單、還在路上的貨；另一種同時考慮在途訂單：

```

import numpy as np
# 系統動態：庫存與訂貨的存量-流量模型；比較「忽略在途訂單」與「考慮在途訂單」的訂貨政策
dt, T = 0.125, 60.0 # 時間步 0.125 週，模擬 60 週
L, Ti, Tsl, Tf = 6.0, 2.0, 3.0, 2.0 # 交貨延遲、庫存調整時間、在途調整時間、預測平滑時間（週）
def run(w_sl): # w_sl：對在途訂單缺口的重視程度（0 = 忽略）
    t = np.arange(0, T, dt); D = np.where(t >= 10, 120.0, 100.0) # 第 10 週需求由 100 升到 120
    I, F = 400.0, 100.0
    pipe = np.full(3, 100.0 * L / 3) # 在途訂單以三階延遲表示（三段管線）
    out = []
    for k, tk in enumerate(t):
        F += dt * (D[k] - F) / Tf # 指數平滑預測需求
        SL = pipe.sum(); I_star, SL_star = 4 * F, F * L # 目標庫存（4 週需求）、目標在途量
        order = max(0.0, F + (I_star - I) / Ti + w_sl * (SL_star - SL) / Tsl)
        flow = pipe / (L / 3) # 每一段以 L/3 的時間常數往下一段流動
        pipe += dt * (np.r_[order, flow[:-1]] - flow)
        I += dt * (flow[-1] - D[k]) # 最後一段到貨進入庫存
        out.append((tk, I, order, D[k]))
    return np.array(out)
for w, name in [(0.0, "忽略在途訂單"), (1.0, "考慮在途訂單")]:
    r = run(w); t, I, O = r[:, 0], r[:, 1], r[:, 2]
    print(f"{name}：訂單峰值 {O.max():.0f}，庫存 最高 {I.max():.0f}、最低 {I.min():.0f}（目標 480），"
          f"第 40~60 週庫存擺幅 {I[t >= 40].max() - I[t >= 40].min():.0f}")

```

執行結果：需求只從 100 升到 120，忽略在途訂單的政策卻讓訂單衝到每週約 254 件；由於前面訂的貨還在路上，庫存先跌到約 213 件，接著大量到貨又衝到約 708 件，之後持續上下擺盪，第 40 到 60 週的擺幅仍高達約 495 件，始終無法穩定在目標的 480 件。考慮在途訂單的政策，訂單峰值只有約 163 件，庫存在約 332 到 494 件之間調整，第 40 週以後幾乎完全穩定（擺幅約 2 件）。

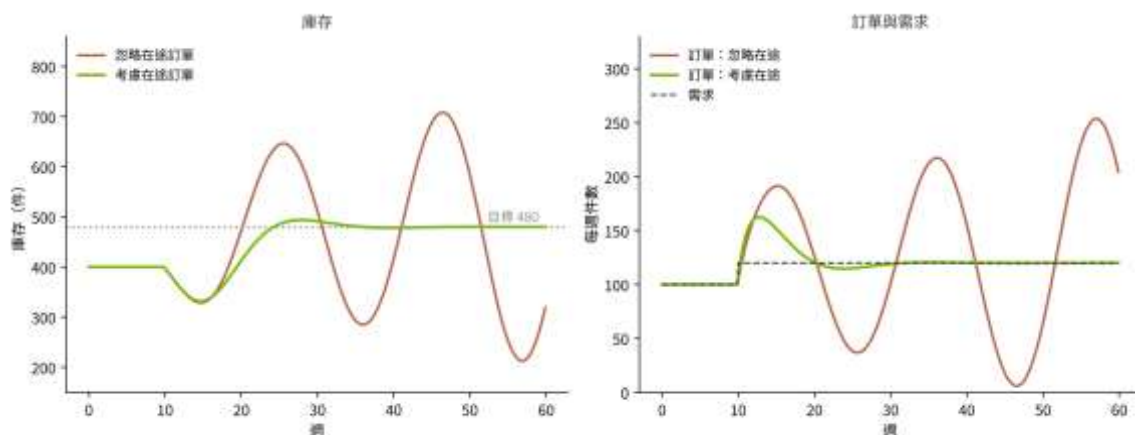


圖 21-3 系統動態：忽略與考慮在途訂單的庫存與訂單

這個結果說明了系統動態的核心洞見：系統的行為常由其結構決定，尤其是延遲與回饋。每一個人都依自己的「合理」規則行事，訂貨的人看到庫存低就多訂，但因為忽略了延遲，整個系統就產生

劇烈的波動。這也是第 8 章長鞭效應的根本原因之一。系統動態不追求個別數字的精確，而在於理解結構、比較政策，常用於供應鏈、產能投資與策略規劃的討論。

21.1.5 代理人模擬

代理人模擬由許多「代理人」組成，每一個代理人依自己的規則感知環境、做出決定並採取行動。整體的行為，例如壅塞、合作或市場價格，是從個體的互動中「湧現」出來的，而不是事先寫好的方程式。它特別適合描述由許多自主個體組成的系統，例如自主移動機器人車隊、工廠中的人員動線，或第 29 章的多代理人製造系統。

下面的程式模擬一條 60 格的環狀走道上的 AMR 車隊。每台 AMR 只依一條簡單的規則行動：前方空格才前進；到了裝貨站（空車時）或卸貨站（載貨時）要停留 4 步裝卸。程式改變車隊規模，觀察送達的件數：

```
import numpy as np
# 代理人模擬：環狀走道上的自主移動機器人（AMR），每台依簡單規則行動，觀察車隊規模與壅塞
CELLS, LOAD, UNLOAD, SERVICE = 60, 0, 30, 4          # 60 格環道、裝貨站、卸貨站、裝卸各需 4 步
def simulate(n, steps=5000, seed=0):
    rng = np.random.default_rng(seed)
    pos = list(rng.choice(CELLS, n, replace=False)); busy = [0] * n; loaded = [False] * n
    occupied = set(pos); delivered = 0; moves = 0
    for _ in range(steps):
        for i in rng.permutation(n):                    # 每台 AMR 依序決定行動（規則：前方空格才前進）
            if busy[i] > 0:
                busy[i] -= 1
                if busy[i] == 0 and pos[i] == UNLOAD and loaded[i]: loaded[i] = False; delivered += 1
                if busy[i] == 0 and pos[i] == LOAD and not loaded[i]: loaded[i] = True; continue
            if pos[i] == LOAD and not loaded[i]: busy[i] = SERVICE; continue
            if pos[i] == UNLOAD and loaded[i]: busy[i] = SERVICE; continue
            nxt = (pos[i] + 1) % CELLS
            if nxt not in occupied:
                occupied.remove(pos[i]); occupied.add(nxt); pos[i] = nxt; moves += 1
    return delivered / steps * 1000, moves / (steps * n)
res = {}
for n in [1, 2, 4, 6, 8, 10, 11, 12, 13, 15, 20, 30, 40]:
    th, v = simulate(n); res[n] = th
    print(f"AMR {n:>2} 台：每 1000 步送達 {th:5.1f} 件，平均每台移動率 {v:.2f}")
top = max(res.values()); n98 = min(k for k, v in res.items() if v >= 0.98 * top)
print(f"最高產出 {top:.1f} 件；達到最高產出 98% 的最小車隊 {n98} 台")
```

執行結果：車隊在 10 台以內時，產出幾乎與台數成正比，每台每 1,000 步送達約 14.2 件，平均移動率約 0.86；超過 11 台之後，產出停在每 1,000 步約 151~153 件，不再增加，平均移動率卻一

路下降，到 40 台時只剩約 0.23，也就是大部分時間都在排隊。程式找出達到最高產出 98% 的最小車隊為 11 台。

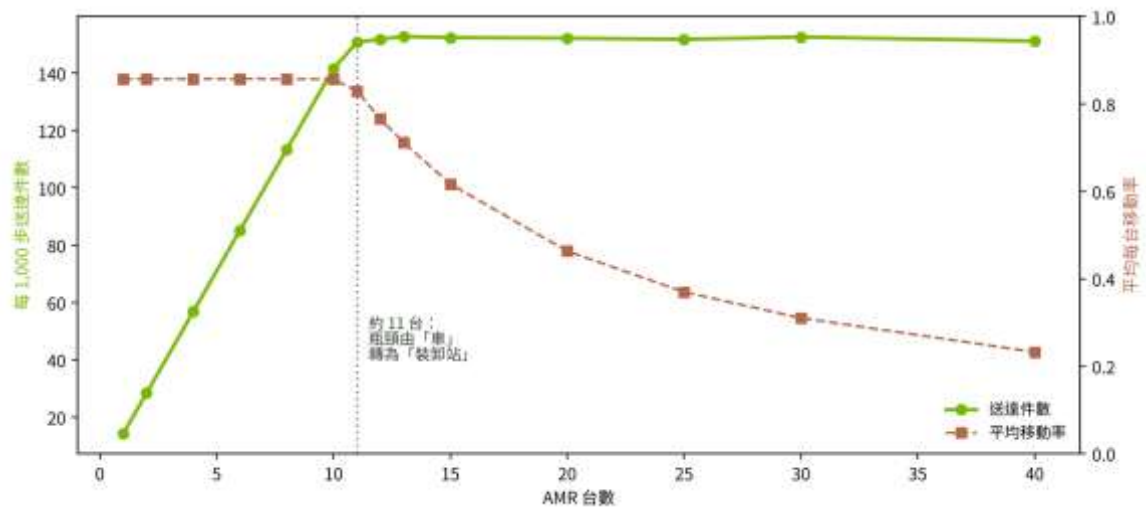


圖 21-4 代理人模擬：AMR 車隊規模、產出與移動率

這個結果可以用簡單的推算驗證：一台 AMR 繞一圈約需 60 步移動加上兩次裝卸，約 70 步，所以每 1,000 步約送達 14.3 件；而每一個站的服務能力約為每 1,000 步 150 件左右。當車隊的總運送能力超過站的服務能力，瓶頸就從「車不夠」移到「站不夠」，再多的車只會在站前排隊、堵住走道。這與第 8 章的瓶頸理論一致，只是這裡的瓶頸是從個體互動中湧現出來的。真實的 AMR 系統還有路徑規劃、交會讓車、充電與任務指派，這些交互作用更難以解析，代理人模擬就成為評估車隊規模與交通規則的重要工具。

21.2 SIL / MIL / HIL 與 PLC 虛擬試車

21.2.1 為什麼需要虛擬試車

傳統的自動化專案，控制程式要等到機械與電氣都安裝完成，才能在現場第一次與真正的設備一起測試。這時，任何程式錯誤都會造成停線、等待與重工，而且專案通常已經接近交期，試車的壓力最大。更危險的是，有些錯誤可能造成設備碰撞或人員受傷。

虛擬試車 (Virtual Commissioning) 把這個過程提前：在實體設備完成之前，先以模擬模型代替設備，讓控制程式與模型連接、測試與除錯。德國工程師協會與電機電子資訊技術協會的 VDI / VDE 3693 系列準則專門討論虛擬試車，其第 1 部分 (2016 年發布、2025 年改版) 定義了虛擬試車

的名詞、模型類型與在自動化設備生命週期中的定位。

21.2.2 在迴路的層級

依「什麼是真的、什麼是模擬的」，在迴路測試可以分為幾個層級：

表 21-4 在迴路測試的層級

層級	控制器	受控對象	時間	典型用途
模型在迴路 (MIL)	控制演算法的模型	模型	可快於或慢於實際時間	控制策略設計、參數調整
軟體在迴路 (SIL)	實際的控制程式，在電腦或虛擬控制器上執行	模型	通常可加速	PLC 邏輯測試、大量情境測試
硬體在迴路 (HIL)	實際的控制器硬體	即時運算的模型	必須與實際時間同步	時序、通訊與 I / O 測試
實機試車	實際的控制器	實際的設備	實際時間	最終驗收



圖 21-5 模型在迴路、軟體在迴路、硬體在迴路與實機試車

越往下層，測試越接近真實，但成本也越高、能測的情境越少。MIL 適合在設計初期比較不同的控制策略，例如第 18 章的 PI 與 MPC；SIL 讓真正的 PLC 程式在電腦上執行，可以快速、大量、自動化地測試各種情境，包括在真實設備上不敢做的故障與極端情況；HIL 則把真正的控制器接上即時模擬器，檢查實際的掃描時間、通訊延遲與 I / O 接線。汽車電子控制單元的開發很早就廣泛採用這種分層測試，自動化產線的虛擬試車也借鏡了這個做法。

21.2.3 虛擬試車需要的模型

虛擬試車的模型，至少要描述控制程式「看得到、碰得到」的部分：

- 感測器：光電開關、近接開關、條碼讀取器、編碼器，何時觸發、輸出什麼訊號。
- 致動器：馬達、氣缸、推桿、閥門，收到指令後多久完成動作、可能卡住或失效。
- 物料流：工件、箱子、托盤在輸送帶與設備之間如何移動、何時到達哪個位置。
- 設備行為：機器人、工具機、其他控制器的交握訊號與狀態。

這些模型不必有完美的物理保真度。一條輸送帶只要能正確地讓箱子以設定的速度移動、在正確的位置觸發感測器即可，不必模擬皮帶的彈性。這正是第 19 章「適合用途」的原則：模型的保真度以能測出控制程式的錯誤為準。很多虛擬試車工具也直接使用第 13 章的 CAD 模型與第 20 章的 OpenUSD 場景，加上運動學與感測器的設定。

21.2.4 軟體在迴路示範：分流推桿

下面的程式重現叫獸開講中的分流問題。一條輸送帶以每秒 0.5 公尺的速度輸送箱子；箱子在 0.5 公尺處經過條碼讀取器，得知它是 A 類還是 B 類；在 1.2 公尺處觸發光電開關；B 類的箱子應該在 1.25 ~ 1.45 公尺的推桿有效區內被推出，A 類的則繼續前進。推桿完全伸出需要 0.3 秒。PLC 程式以 10 毫秒的掃描週期執行，模型則模擬箱子的移動、感測器的觸發與推桿的動作。

程式比較兩個版本的 PLC 邏輯：版本 1 用一個暫存器記住「下一個箱子」的類別；版本 2 以先進先出佇列記住所有已讀取、還沒到達推桿的箱子。測試情境包括箱子間距 3 秒與間距 1.2 秒兩種：

```
from collections import deque
# 虛擬試車（軟體在迴路，SIL）：以 10 毫秒掃描週期執行 PLC 邏輯，連接輸送線與推桿的模擬模型，在上
# 線前找出錯誤
DT, V = 0.01, 0.5 # 掃描與模擬步長 10 ms、輸送速度 0.5 m/s
X_SCAN, X_EYE, PUSH_ZONE = 0.5, 1.2, (1.25, 1.45) # 條碼讀取位置、光電開關位置、推桿有效區
# (m)
EXTEND = 0.30 # 推桿伸出所需時間 (s)
class PLC_v1: # 版本 1：只用一個暫存器記住「下一個箱
# 子」的類別
    def __init__(self): self.next_type, self.eye_prev, self.timer = None, False, 0.0
    def scan(self, inp):
        if inp["scan_event"] is not None: self.next_type = inp["scan_event"]
        rising = inp["eye"] and not self.eye_prev; self.eye_prev = inp["eye"]
        if rising and self.next_type == "B": self.timer = EXTEND + 0.25
        if rising: self.next_type = None
        self.timer = max(0.0, self.timer - DT)
        return {"push": self.timer > 0}
```

```

class PLC_v2(PLC_v1):
    # 版本 2：以先進先出佇列記住所有已讀取、
    # 未到推桿的箱子
    def __init__(self): super().__init__(); self.fifo = deque()
    def scan(self, inp):
        if inp["scan_event"] is not None: self.fifo.append(inp["scan_event"])
        rising = inp["eye"] and not self.eye_prev; self.eye_prev = inp["eye"]
        if rising and self.fifo and self.fifo.popleft() == "B": self.timer = EXTEND +
0.25
        self.timer = max(0.0, self.timer - DT)
        return {"push": self.timer > 0}
def run(plc, boxes):
    # boxes : [(進入時間, 類別)]; 模型負責輸
    # 送、感測與推桿物理
    items = [{"t0": t0, "type": ty, "x": None, "scanned": False, "pushed": False} for t0,
ty in boxes]
    ext, t, out = 0.0, 0.0, {"push": False}
    while t < boxes[-1][0] + 4.0:
        scan_event, eye = None, False
        for b in items:
            if b["pushed"] or t < b["t0"]: continue
            b["x"] = (t - b["t0"]) * V
            if not b["scanned"] and b["x"] >= X_SCAN: b["scanned"] = True; scan_event =
b["type"]
            if abs(b["x"] - X_EYE) < 0.05: eye = True # 箱長 0.1 m
        out = plc.scan({"scan_event": scan_event, "eye": eye})
        ext = min(EXTEND, ext + DT) if out["push"] else max(0.0, ext - DT)
        if ext >= EXTEND:
            # 推桿完全伸出時，推出位於有效區內的箱子
            for b in items:
                if not b["pushed"] and b["x"] is not None and PUSH_ZONE[0] <= b["x"] <=
PUSH_ZONE[1]: b["pushed"] = True
            t += DT
        wrong = sum((b["type"] == "B") != b["pushed"] for b in items)
        return wrong, len(items)
tests = {"間距 3 秒": [(i * 3.0, ty) for i, ty in enumerate("ABBAB")],
        "間距 1.2 秒": [(i * 1.2, ty) for i, ty in enumerate("ABBAB")],
        "間距 1.2 秒，換順序": [(i * 1.2, ty) for i, ty in enumerate("BABBA")]}
for name, boxes in tests.items():
    w1, n = run(PLC_v1(), boxes); w2, _ = run(PLC_v2(), boxes)
    print(f"{name}: 版本 1 分錯 {w1}/{n} 箱; 版本 2 分錯 {w2}/{n} 箱")

```

執行結果：箱子間距 3 秒時，兩個版本都正確分流，5 箱全對。間距縮短為 1.2 秒時，版本 1 在第一組順序 (A、B、B、A、B) 中分錯 4 箱，在第二組順序 (B、A、B、B、A) 中分錯 3 箱；版本 2 在所有情境中都沒有分錯。

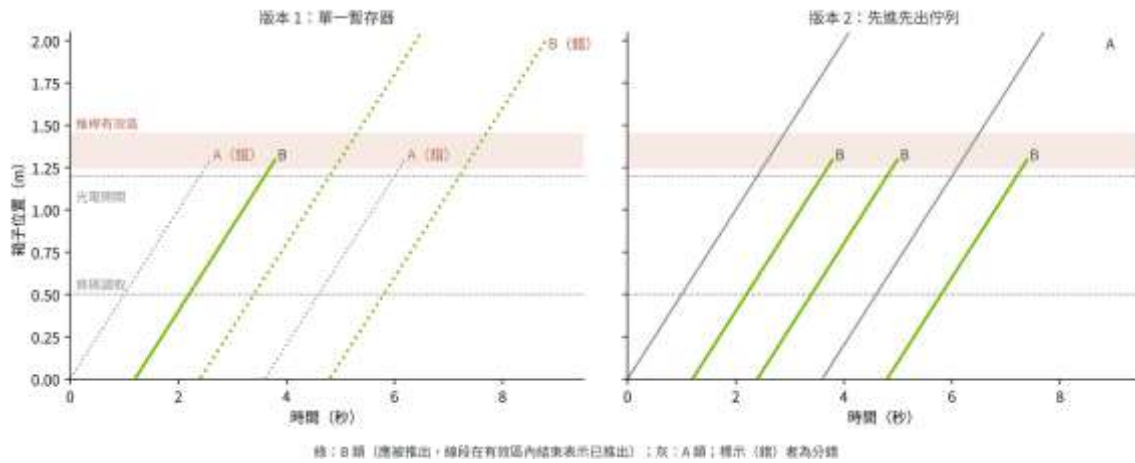


圖 21-6 虛擬試車：箱子間距縮短時，版本 1 的邏輯錯誤

原因並不難理解：條碼讀取器到光電開關的距離是 0.7 公尺，以每秒 0.5 公尺的速度需要 1.4 秒。當箱子間距大於 1.4 秒，任何時刻在兩者之間最多只有一個箱子，一個暫存器就夠用；間距縮短到 1.2 秒時，前一個箱子還沒到推桿，下一個箱子就已經被讀取，暫存器的內容被覆蓋，系統就「記錯」了。這正是叫獸開講中那個在現場花了很多時間才找到的錯誤。

這個例子有幾點值得注意。第一，這類錯誤只在特定條件下發生（箱子靠得很近），如果只用「正常間距」測試，永遠不會發現。虛擬試車的價值，在於可以系統性地、自動化地測試大量的情境與邊界條件，包括在實際設備上難以重現或危險的情況。第二，模型非常簡單，只描述箱子的位置、感測器的觸發與推桿的動作，卻足以找出邏輯錯誤。第三，修正後的版本 2 應該再以同樣的測試案例重新執行，這就是軟體工程中的回歸測試；把測試案例保存下來，往後每次修改 PLC 程式都自動重跑，讓產線的控制程式也能享有軟體工程的品質保證。

21.2.5 虛擬試車的導入

虛擬試車的效益，主要在於縮短現場試車時間、提早發現錯誤、降低風險，以及讓操作人員在設備到廠前就先在虛擬環境中受訓。但它也有成本：建模需要時間與人力，模型與實際設備之間也可能有差異。常見的導入建議包括：

- 從重複性高的設備開始：標準化的模組（輸送段、分流器、機器人工作站）建一次模型可以重複使用，讓建模成本攤提到多個專案。
- 要求設備供應商提供模型：在採購規格中要求提供可用於虛擬試車的行為模型，例如以第 20 章的 FMI 格式封裝的功能模型單元。

- 與 PLC 開發流程整合：把虛擬試車納入程式開發的每一個階段，而不是只在最後才做一次。
- 記錄模型的確認域：模型沒有描述的現象（例如皮帶打滑、感測器受灰塵干擾），仍需要在實機試車中測試。

21.3 虛擬機台與虛擬量測

21.3.1 虛擬機台

虛擬機台以模型描述一台機台的行為：它的運動學與動力學、伺服控制、熱變形、切削力與振動。依用途不同，虛擬機台的保真度可以差很多：

- 加工模擬：以運動學模型與刀具路徑，在實際加工前檢查碰撞、超行程與加工時間，第 13 章的 CAM 模擬就屬於這一類。
- 誤差預測與補償：以熱模型或幾何誤差模型預測加工誤差並補償，例如第 19 章的熱變位學生。
- 加工品質預測：結合伺服控制、機台動態與切削力模型，預測輪廓誤差、表面粗糙度與顫振，以在加工前調整參數（第 9 章）。
- 操作訓練：讓學生或新進人員在虛擬機台上練習操作與程式編寫，不必擔心撞機。

虛擬機台常與 21.2 節的虛擬試車結合：機台的控制器以 SIL 或 HIL 的方式連接虛擬機台，測試加工程式與自動化流程。

21.3.2 虛擬量測

在許多製程中，產品的品質要經過量測才知道，但量測既慢又貴。半導體晶圓廠是最典型的例子：薄膜厚度、線寬與疊對誤差都要以精密儀器量測，量測站的產能有限，所以通常只抽測一部分晶圓。沒有被量測的晶圓，如果品質出了問題，可能要到後面的製程甚至最終測試才會被發現。

虛擬量測的想法是：既然每一片晶圓在製程中都留下了大量的設備與感測資料，例如溫度、壓力、氣體流量與射頻功率，何不以這些資料建立模型，預測每一片晶圓的量測值？這樣，每一片晶圓都有一個「虛擬」的量測值，實際量測只需用來建立與修正模型。

台灣在虛擬量測的研究與應用上有相當的成果。成功大學鄭芳田教授的團隊發展了全自動虛擬量

測 (Automatic Virtual Metrology, AVM) 系統，相關研究包括 2008 年發表於《IEEE 半導體製造彙刊》、提出以「信心指標」 (Reliance Index, RI) 評估虛擬量測可信度的論文，以及 2012 年發表於《IEEE 自動化科學與工程彙刊》的〈Developing an Automatic Virtual Metrology System〉；後者描述的系統曾在台灣一家 TFT-LCD 面板廠的化學氣相沉積機台上實作。這些研究指出，虛擬量測要能實際使用，不只要預測得準，還要能自動判斷「這次的預測不可信」，並在機台保養或製程改變後自動更新模型。

21.3.3 虛擬量測示範

下面的程式模擬一台化學氣相沉積機台：膜厚由溫度、壓力、氣體流量、射頻功率與沉積時間決定，另外隨著反應室逐漸髒污而緩慢變薄，每 600 片保養一次後恢復。先以 200 片全量測的晶圓建立嶺迴歸模型，再預測接下來 1,000 片的膜厚，其中只有每 25 片抽測 1 片。第 700~704 片的氣體流量計故障，讀值偏高，但實際流量正常。程式比較：

- 靜態虛擬量測：只用建模時的模型。
- 抽測偏差修正：以抽測晶圓的實際值與預測值的差，以指數加權平均修正模型的偏差，以追上機台漂移。
- 可信度檢查：以馬氏距離衡量每一片晶圓的製程資料與建模資料的差異，差異過大時標記為「預測不可信」，改以實際量測；並讓偏差修正略過被標記的抽測。

```
import numpy as np
from sklearn.linear_model import Ridge
from sklearn.preprocessing import StandardScaler
# 虛擬量測：以製程感測資料預測每一片晶圓的膜厚；實際量測只抽 1/25，並以抽測結果修正機台漂移
rng = np.random.default_rng(4)
N0, N = 200, 1000                                # 建模用 200 片（全量測），之後量產 1000 片
def wafers(n, age0):
    X = rng.normal(0, 1, (n, 5))                  # 溫度、壓力、氣體流量、射頻功率、沉積時間（標準化）
    age = age0 + np.arange(n)                      # 腔體累計片數（反應室逐漸髒污）
    pm = (age % 600) / 600                          # 每 600 片保養一次，保養後歸零
    y = 500 + X @ np.array([6.0, -4.0, 5.0, 3.0, 8.0]) - 12.0 * pm + rng.normal(0, 1.5, n) # 膜厚（Å）
    return X, y, pm
X0, y0, _ = wafers(N0, 0)
sc = StandardScaler().fit(X0); m = Ridge(alpha=1.0).fit(sc.transform(X0), y0)
X, y, pm = wafers(N, N0)
X[700:705, 2] += 6.0                               # 第 700~704 片：氣體流量計故障，讀值偏高（實際流量正常）
```

```

pred_static = m.predict(sc.transform(X))
# 可信度檢查：輸入與建模資料的馬氏距離過大時，標記「預測不可信」，改以實際量測
Z0 = sc.transform(X0); S_inv = np.linalg.inv(np.cov(Z0.T))
d2 = np.einsum("ij,jk,ik->i", sc.transform(X), S_inv, sc.transform(X))
flag = d2 > 20.5 # 自由度 5 的卡方分布 99.9% 分位數約 20.5
sampled = np.arange(N) % 25 == 0
def update(skip_flagged, alpha=0.3): # 以抽測結果的指數加權平均修正偏差
    pred, bias = np.empty(N), 0.0
    for i in range(N):
        pred[i] = pred_static[i] + bias
        if sampled[i] and not (skip_flagged and flag[i]):
            bias = (1 - alpha) * bias + alpha * (y[i] - pred_static[i])
    return pred
pred_upd, pred_safe = update(False), update(True)
rmse = lambda e: np.sqrt(np.mean(e**2))
print(f"靜態虛擬量測 RMSE {rmse(pred_static - y):.2f} Å；加上抽測偏差修正 RMSE {rmse(pred_upd - y):.2f} Å")
print(f"被標記為不可信的晶圓：{np.where(flag)[0].tolist()}，其預測誤差 {np.round(pred_upd[flag] - y[flag], 1).tolist()} Å")
ok = ~flag
print(f"不可信的晶圓改用實際量測：RMSE {rmse(pred_upd[ok] - y[ok]):.2f} Å；"
      f"再讓偏差修正略過不可信的抽測：RMSE {rmse(pred_safe[ok] - y[ok]):.2f} Å")

```

執行結果：

表 21-5 虛擬量測的結果 (1,000 片晶圓・膜厚單位 Å)

做法	RMSE	說明
只靠抽測	-	只有 4% 的晶圓有實測值，其餘無從得知
靜態虛擬量測	6.53	模型跟不上反應室逐漸髒污造成的漂移
加上抽測偏差修正	3.50	以抽測結果持續修正偏差，追上漂移
不可信的晶圓改用實際量測	2.99	流量計故障的 5 片被標記並改為實測
偏差修正再略過不可信的抽測	2.52	避免故障資料污染偏差修正

被標記為不可信的晶圓共有 7 片：第 700~704 片的預測誤差約 23~34 Å，正是流量計故障的晶圓；另有第 295 與 527 片被標記，但預測誤差只有 1~3 Å，屬於誤報。以 99.9% 的門檻檢查 1,000 片，出現一兩片誤報是可以預期的。

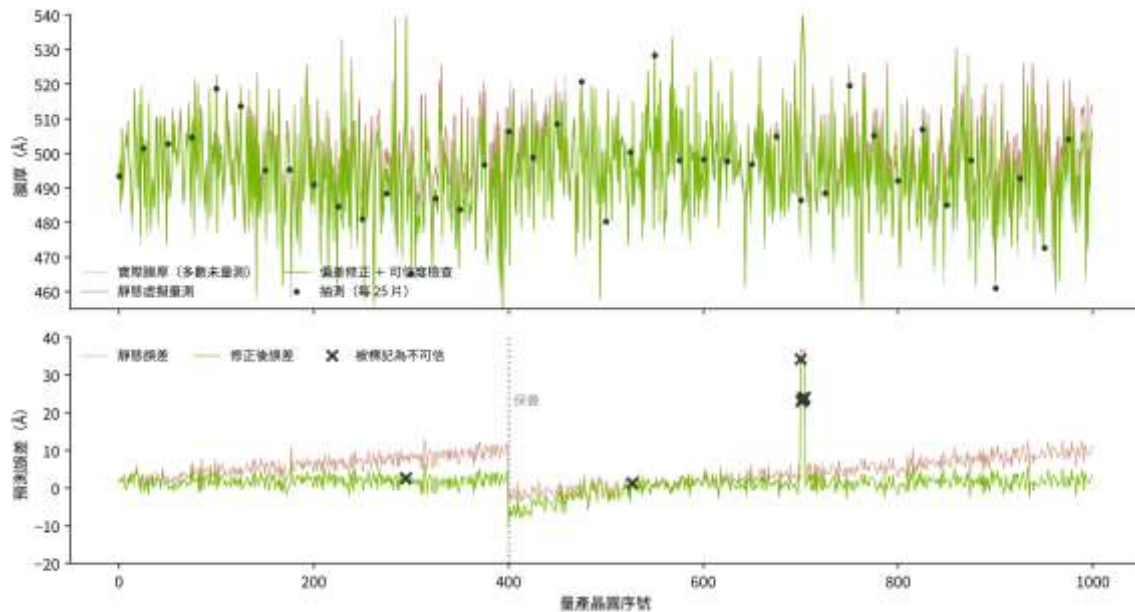


圖 21-7 虛擬量測：機台漂移、偏差修正與可信度檢查

這個例子展示了虛擬量測實用化的三個關鍵。第一，模型必須持續更新：機台會老化、保養、更換零件，一個不更新的模型很快就會失準，這與第 20 章的模型治理相呼應。第二，必須判斷預測是否可信：流量計故障時，模型仍然「認真地」給出一個預測值，但這個值錯了 20 Å 以上；如果沒有可信度檢查，這些晶圓會被當成合格而流到下一站。可信度檢查的本質，就是第 20 章的確認域：輸入超出建模資料的範圍時，模型不可信。第三，更新本身也要保護：第 700 片恰好被抽測，若不加檢查就拿它修正偏差，故障的資料會污染接下來數十片晶圓的預測。這些設計，正是全自動虛擬量測研究所強調的重點。

虛擬量測的應用不限於半導體。凡是量測昂貴、耗時或具破壞性，而製程資料豐富的場合，例如面板、太陽能電池、射出成形、熱處理與第 16 章的金屬積層製造，都可以考慮虛擬量測。它也是第 25 章閉環品質控制的重要基礎：當每一件產品都有一個即時的品質預測值，製程就可以逐件調整，而不必等抽測結果。

21.4 AR / VR、OpenUSD 與工業元宇宙

21.4.1 從螢幕到空間

前面幾節的模擬，結果多半呈現在螢幕上的圖表與數字。擴增實境、虛擬實境與混合實境，則讓人「走進」模擬與孿生之中：

表 21-6 AR、VR 與 MR 在製造中的應用

技術	特徵	典型應用	本書相關 章節
擴增實境 (AR)	在真實世界的影像上疊加數位資訊	組裝與維修的步驟指引、設備狀態顯示、遠端專家協助、品檢比對	第 26、 31 章
虛擬實境 (VR)	完全沉浸在電腦產生的環境中	產線佈置的審查、操作與安全訓練、人因工程評估	第 10、 31 章
混合實境 (MR)	虛擬物件與真實環境互動，例如放在真實地板上的虛擬機台	新設備在現場的擺放評估、與真實設備並行的訓練	第 31 章

這些技術的價值，在於把變生的資訊放到「需要它的人、需要它的位置」。維修人員戴上 AR 眼鏡看著一台泵浦，眼前就浮現它的運轉資料、上次的保養紀錄與這次的維修步驟；規劃人員戴上 VR 頭盔，就可以在還沒蓋好的廠房中走一圈，檢查走道寬度與操作空間。第 31 章將從人機協作的角度深入討論這些介面。

21.4.2 工業元宇宙

「元宇宙」一詞在 2021 年前後成為熱門話題，工業界隨之出現「工業元宇宙」的說法。它沒有統一的定義，一般指的是以即時 3D、物理模擬、數位孿生與即時資料為基礎，讓分散各地的人員、AI 與機器在共同的虛擬工業空間中協作、規劃與決策。與消費型的元宇宙相比，工業元宇宙更強調物理上的正確性與真實資料的連結：虛擬工廠中的機台必須與真實的機台尺寸相同、動作相同、資料同步。

依公開報導，BMW 集團在 2023 年 3 月表示，已在其位於匈牙利德布勒森、預定 2025 年啟用的新工廠完工前兩年多，就以 NVIDIA Omniverse 建立整座工廠的虛擬模型，讓世界各地的團隊即時協作，驗證佈置、機器人與物流的規劃，並把這種虛擬規劃方式作為未來工廠規劃的藍圖。依 BMW 集團的新聞稿，德布勒森廠已於 2025 年正式啟用，並於同年 10 月下旬開始量產 BMW iX3。第 19 章鴻海墨西哥新廠的例子，也屬於這一類。

工業元宇宙的技術基礎，正是前面各章介紹的元素：第 20 章的 OpenUSD 描述 3D 場景，資產管理殼與知識圖譜提供語意，OPC UA 與統一命名空間提供即時資料，物理模擬與離散事件模擬提供

行為，AR / VR 提供沉浸式的介面。換言之，工業元宇宙不是一項新的技術，而是把這些技術整合起來的一種使用方式。

21.4.3 以 OpenUSD 組工廠場景

第 20 章提到，OpenUSD 的分層與組合機制，讓不同團隊可以分別維護自己的圖層，再組合成完整的場景。下面的程式以 Python 的 OpenUSD 套件 (pxr) 建立三個圖層：

- 基礎圖層：廠房與兩台既有的 CNC 機台。
- 設備廠商圖層：新購的 MC-3 機台，由設備廠商提供。
- 規劃變更圖層：規劃人員把 CNC-09 往走道方向移動，這個圖層只「覆寫」CNC-09 的位置，不改動其他內容。

每一台機台都帶有一個屬性，連結到第 20 章的資產管理殼識別碼。組合後，程式計算每台機台的邊界，檢查它與 AMR 走道之間的淨空：

```
from pxr import Usd, UsdGeom, Sdf, Gf
import os, tempfile
# OpenUSD：廠房佈置（基礎圖層）+設備廠商圖層+規劃變更圖層，組合成工廠場景，再檢查 AMR 走道淨空
d = tempfile.mkdtemp()
def cube(stage, path, center, size, aas=None):
    c = UsdGeom.Cube.Define(stage, path); c.GetSizeAttr().Set(1.0)
    xf = UsdGeom.XformCommonAPI(c.GetPrim()); xf.SetTranslate(Gf.Vec3d(*center));
    xf.SetScale(Gf.Vec3f(*size))
    if aas: c.GetPrim().CreateAttribute("factory:aasId",
    Sdf.ValueTypeNames.String).Set(aas) # 連結到資產管理殼
base = Usd.Stage.CreateNew(os.path.join(d, "layout.usda")) # 基礎圖層：廠房與機
台初始位置
UsdGeom.Xform.Define(base, "/Factory")
cube(base, "/Factory/CNC_07", (2.0, 3.0, 1.0), (2.0, 2.5, 2.0), "urn:vendorA:aas:CNC-07")
cube(base, "/Factory/CNC_09", (6.0, 3.0, 1.0), (2.0, 2.5, 2.0), "urn:vendorA:aas:CNC-09")
base.GetRootLayer().Save()
vendor = Usd.Stage.CreateNew(os.path.join(d, "vendor_mc3.usda")) # 設備廠商圖層：新機
台 MC-3
cube(vendor, "/Factory/MC_3", (10.0, 3.0, 1.2), (2.4, 3.0, 2.4), "urn:vendorB:aas:MC-3");
vendor.GetRootLayer().Save()
change = Usd.Stage.CreateNew(os.path.join(d, "plan_change.usda")) # 規劃變更圖層：只覆
寫 CNC_09 的位置
over = change.OverridePrim("/Factory/CNC_09")
over.CreateAttribute("xformOp:translate", Sdf.ValueTypeNames.Double3).Set(Gf.Vec3d(6.0,
4.6, 1.0)); change.GetRootLayer().Save()
scene = Usd.Stage.CreateNew(os.path.join(d, "factory.usda")) # 組合：強的圖層在前
scene.GetRootLayer().subLayerPaths = ["plan_change.usda", "vendor_mc3.usda",
"layout.usda"]
scene.GetRootLayer().Save()
AISLE_Y, AMR_HALF = 6.0, 0.6 # AMR 走道中心 y = 6
m, 車寬 1.2 m
def check(stage):
```

```

cache = UsdGeom.BBoxCache(Usd.TimeCode.Default(), [UsdGeom.Tokens.default_])
for prim in stage.Traverse():
    if prim.GetTypeName() != "Cube": continue
    r = cache.ComputeWorldBound(prim).ComputeAlignedRange()
    gap = (AISLE_Y - AMR_HALF) - r.GetMax()[1]          # 機台外緣到 AMR 車身
的距離
    aas = prim.GetAttribute("factory:aasId").Get()
    print(f" {prim.GetName():<7} y 範圍 {r.GetMin()[1]:.2f}~{r.GetMax()[1]:.2f} m ,
淨空 {gap:+.2f} m"
        f"{' ← 干涉!' if gap < 0 else (' ← 不足 0.5 m' if gap < 0.5 else '')}
({aas}) ")
print("只有基礎圖層："); check(Usd.Stage.Open(os.path.join(d, "layout.usda")))
print("組合三個圖層後："); check(Usd.Stage.Open(os.path.join(d, "factory.usda")))

```

執行結果：只看基礎圖層時，兩台 CNC 與 AMR 車身之間都有 1.15 公尺的淨空。組合三個圖層之後，場景中多了廠商提供的 MC-3（淨空 0.90 公尺），而 CNC-09 因為規劃變更被移到 $y = 4.6$ 公尺，邊界延伸到 5.85 公尺，與 AMR 車身重疊 0.45 公尺，程式標示為干涉。規劃變更圖層本身的內容只有一行：覆寫 CNC-09 的 `xformOp:translate` 為 $(6, 4.6, 1)$ 。

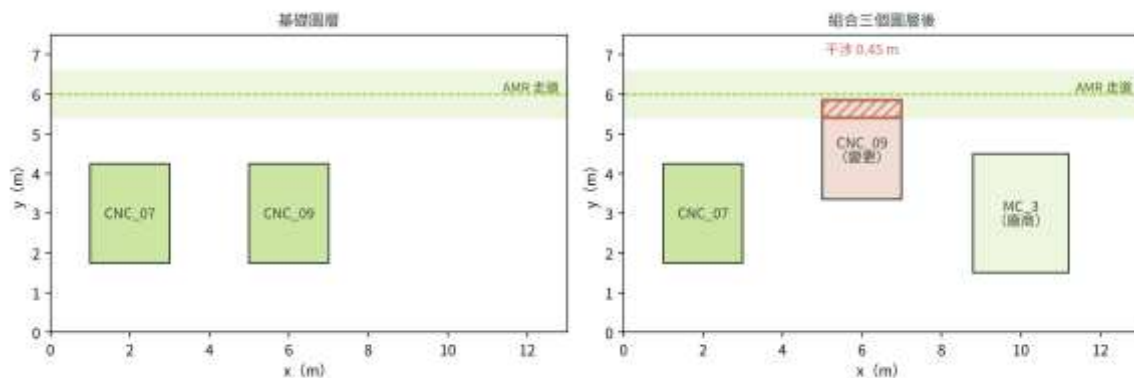


圖 21-8 以 OpenUSD 組合三個圖層的工廠場景與走道淨空檢查

這個例子展示了 OpenUSD 用於工廠規劃的幾個特點。第一，非破壞性的協作：規劃人員的變更放在自己的圖層，不會覆蓋廠房圖層或廠商的模型，任何時候都可以關掉變更圖層，回到原始的佈置比較。第二，圖層有強弱順序，強的圖層覆寫弱的圖層，讓「誰的修改優先」有明確的規則。第三，場景中的物件可以連結到資產管理殼等語意資料，讓 3D 場景不只是外觀。第四，組合後的場景可以被程式自動檢查，例如淨空、可達性與動線，而不必靠人在 VR 中一個個看。真實的工廠場景會以真正的 CAD 幾何取代程式中的方塊，檢查的項目也更多，但原理相同。

21.4.4 理性看待工業元宇宙

工業元宇宙描繪了誘人的願景，但導入時也要保持清醒：

- 從價值出發：沉浸式的 3D 環境很吸引人，但若只是「看起來很酷」，就會重蹈第 19 章叫獸開講中 3D 工廠的覆轍。工廠規劃、遠端協作與訓練，是目前效益較明確的應用。
- 資料與模型的品質：虛擬工廠的價值取決於它與真實工廠的一致程度；3D 模型的版本與現場不符，比沒有模型更危險。這需要第 20 章的數位主線與模型治理。
- 人因與成本：長時間配戴頭戴式裝置可能造成不適，裝置與內容製作的成本也不低。很多時候，平板上的 AR 或桌面上的 3D 檢視就已足夠。

21.5 合成資料、Sim-to-Real 與 AI 驅動學生

21.5.1 合成資料

第 24 章將介紹以深度學習做視覺品檢。深度學習需要大量標註好的資料，但製造業有一個先天的困難：好的產品很多，瑕疵品很少，而且每一種瑕疵的樣子又不一樣。要蒐集到足夠的瑕疵影像，可能要等上好幾個月；某些危險或罕見的狀況（設備碰撞、人員誤入、極端故障），根本不應該為了蒐集資料而讓它發生。

合成資料以模擬或生成模型產生訓練資料，是解決這個問題的方法之一：

- 以 3D 渲染產生影像：在模擬環境中擺放零件，隨機改變光線、角度、背景與瑕疵的位置與形狀，自動產生帶有標註的影像。由於影像是程式產生的，標註是「免費」而且完全正確的。
- 以物理模擬產生感測資料：以第 19 章的設備模型模擬各種故障下的振動、電流與溫度，用來訓練故障診斷模型。
- 以生成模型擴增資料：以第 14 章介紹的擴散模型等生成式 AI，從少量真實樣本生成更多變化。

合成資料的風險在於它和真實資料之間的差異。模型如果學到的是模擬的「習慣」，例如渲染的光線特徵，而不是真正的瑕疵特徵，到了真實世界就會失準。這種差異稱為模擬與真實的落差 (sim-to-real gap)。

21.5.2 Sim-to-Real 與領域隨機化

把在模擬中訓練的模型或控制策略移轉到真實世界，稱為 Sim-to-Real。它在機器人領域特別重

要：讓機器人在真實世界中試錯學習，既慢又可能損壞設備；在模擬中學習，則可以平行執行成千上萬次。第 19 章鴻海在虛擬環境中訓練機器手臂取放零件，就是一例。

模擬與真實的落差有幾個常見的來源：

表 21-7 模擬與真實落差的來源與對策

來源	例子	對策
物理參數	摩擦、質量、馬達常數、剛性與真實不同	系統鑑別、校正（第 20 章）、領域隨機化
感測器	雜訊、延遲、偏移、解析度	在模擬中加入感測器模型與雜訊
外觀	光線、反光、材質、背景	隨機化渲染參數、以真實影像微調
未建模的現象	線材拉扯、灰塵、磨耗、人員干擾	在真實環境中蒐集資料、持續學習
時間	控制週期、通訊延遲	硬體在迴路測試（21.2 節）

領域隨機化是一種簡單而有效的對策：與其想辦法讓模擬「完全正確」，不如在模擬中大幅隨機改變那些不確定的參數，讓模型見識足夠多的變化，使真實世界看起來只是其中的「另一種變化」。Tobin 等人於 2017 年在 IEEE / RSJ 國際智慧機器人與系統會議（IROS）發表的研究，以隨機化渲染的模擬影像訓練物體定位模型，在沒有使用真實影像訓練的情況下，成功移轉到真實的機器人上，是這個方法的代表性工作。

下面的程式以一個簡單的例子說明：AMR 想由馬達電流估計搬運的重量。馬達電流由摩擦（隨速度增加）與載重決定，再除以馬達常數；AMR 每次出發前，也會量一次空車移動的電流。程式以三種方式訓練線性迴歸模型，並在「真實」的機器人上測試，真實機器人的摩擦比模擬高 60%、馬達常數低 8%，雜訊也較大：

```
import numpy as np
from sklearn.linear_model import LinearRegression
# Sim-to-Real：以模擬資料訓練「由馬達電流估計搬運重量」的模型；比較固定參數模擬、領域隨機化與少量真實資料
rng = np.random.default_rng(8)
def robot(n, fric, kt, noise, rng, m_max=20, v_rng=(0.2, 1.0)):
    m = rng.uniform(0, m_max, n)           # 搬運重量 (kg)
    v = rng.uniform(*v_rng, n)             # 速度 (m/s)
    idle = fric * (1 + 0.5 * v) / kt        # 空車移動時的電流 (A)：只有
    摩擦
    cur = (fric * (1 + 0.5 * v) + 0.35 * m * (1 + 0.2 * v)) / kt + rng.normal(0, noise,
    n)
    idle_meas = idle + rng.normal(0, noise, n)           # 每次出發前量一次空車電流
```

```

    return np.c_[cur, idle_meas, v], m
def sim(n, randomize):
    if not randomize: return robot(n, 2.0, 1.0, 0.05, rng)    # 固定參數：摩擦 2.0、馬達常數 1.0
    Xs, ys = [], []
    for _ in range(n // 50):                                # 領域隨機化：每 50 筆換一組物理參數
        X, y = robot(50, rng.uniform(1.0, 4.0), rng.uniform(0.85, 1.15),
                      rng.uniform(0.02, 0.15), rng)
        Xs.append(X); ys.append(y)
    return np.vstack(Xs), np.concatenate(ys)
# 「真實」機器人：摩擦比模擬高 60%（輪子與地板），馬達常數低 8%，雜訊較大
Xr, yr = robot(2000, 3.2, 0.92, 0.10, rng)
mae = lambda mdl, X, y: np.mean(np.abs(mdl.predict(X) - y))
m_fix = LinearRegression().fit(*sim(5000, False))
m_dr = LinearRegression().fit(*sim(5000, True))
print(f"固定參數模擬：模擬中誤差 {mae(m_fix, *sim(2000, False)):.2f} kg，真實誤差 {mae(m_fix, Xr, yr):.2f} kg")
print(f"領域隨機化： 真實誤差 {mae(m_dr, Xr, yr):.2f} kg")
print(f"係數（電流、空車電流、速度）：固定 {np.round(m_fix.coef_, 2)}，隨機化 {np.round(m_dr.coef_, 2)}")
Xf, yf = robot(20, 3.2, 0.92, 0.10, rng, m_max=5, v_rng=(0.4, 0.6))    # 只有 20 筆真實資料，且只涵蓋輕載、中速
m_real20 = LinearRegression().fit(Xf, yf)
print(f"對照：只用 20 筆真實資料（輕載、中速）訓練：真實誤差 {mae(m_real20, Xr, yr):.2f} kg")

```

執行結果：以固定參數的模擬訓練的模型，在模擬中誤差只有約 0.21 公斤，看起來非常好；但到了真實機器人上，誤差暴增到約 5.77 公斤。以領域隨機化訓練的模型，同樣沒有用到任何真實資料，真實誤差卻只有約 0.58 公斤。

秘密藏在係數裡。在固定參數的模擬中，摩擦永遠不變，空車電流也幾乎是常數，模型因此學不到它的用途，係數約為 0；在領域隨機化的模擬中，摩擦一直在變，模型被迫學會「用空車電流扣掉摩擦的影響」，空車電流的係數約為 -2.59，與電流的係數約 2.44 大小相近、方向相反。換句話說，領域隨機化逼模型學到了正確的物理結構，而不是記住模擬的特定參數。

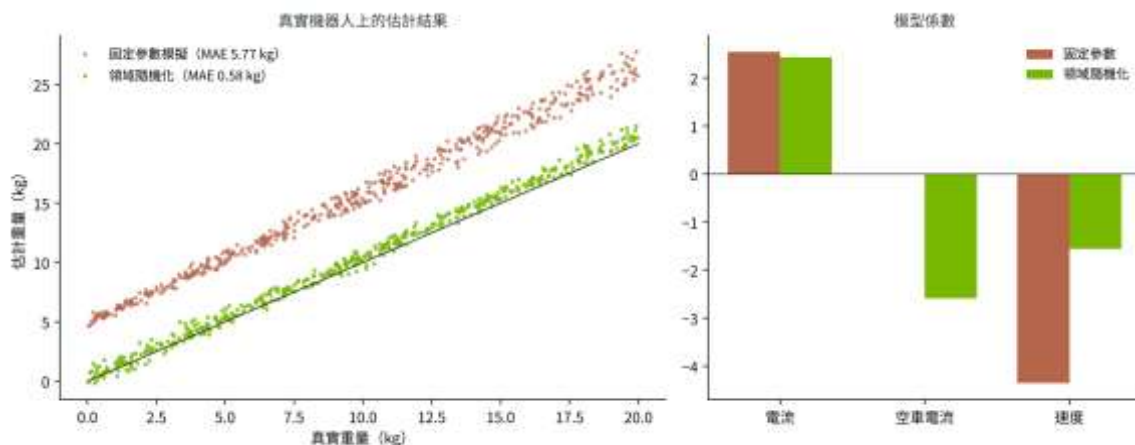


圖 21-9 Sim-to-Real：固定參數模擬與領域隨機化在真實機器人上的表現

程式最後的對照組也值得一提：只用 20 筆真實資料，而且只涵蓋輕載與中速，訓練出的模型真實誤差約 0.40 公斤，比領域隨機化還好。這是一個誠實的提醒：當問題的物理很簡單、真實資料又容易取得時，少量的真實資料可能就勝過大量的模擬。模擬與合成資料的價值，在於真實資料昂貴、危險或罕見的場合，例如瑕疵、故障、碰撞，以及需要大量試錯的機器人學習；而在實務上，最常見的做法是兩者結合：以模擬與領域隨機化預先訓練，再以少量真實資料微調與確認。無論哪一種，最後都必須以真實資料做第 20 章的確認。

21.5.3 AI 驅動學生

傳統的學生以物理模型為核心，AI 主要用於資料分析。近年來，AI 本身逐漸成為學生模型的一部分，常見的方向包括：

- 代理模型：以高保真度模擬的結果訓練神經網路或其他快速模型，讓原本要算數小時的模擬在毫秒內給出近似結果，使學生可以即時運算或進行大量的最佳化與不確定性分析。第 19 章的降階模型與第 14 章的生成式設計，都可以受益於代理模型。
- 物理資訊神經網路：在神經網路的訓練中加入物理方程式作為約束，讓模型在資料不足時仍遵守物理定律。Raissi、Perdikaris 與 Karniadakis 於 2019 年在《計算物理期刊》發表的論文，是這個方向被廣泛引用的代表作。
- 混合模型：以物理模型描述已知的部分，以機器學習補足未知或難以建模的部分，例如以物理模型預測熱變位，以機器學習修正殘差。第 19 章的熱變位學生與第 20 章的模型差異，都屬於這個思路。
- 生成式 AI 與基礎模型：以生成式 AI 產生測試情境與合成資料，或以大型的預訓練模型理解影像、文字與感測資料，作為學生的「感知」與「對話」介面。第 28 到 30 章將深入討論企業 AI、代理式 AI 與 Physical AI。

AI 驅動的學生帶來了速度與彈性，但也帶來新的挑戰：神經網路的代理模型在訓練資料範圍外可能給出不合理的結果，而且不易解釋。因此，第 20 章的驗證、確認、確認域與模型治理，對 AI 驅動的學生而言更加重要。一個實用的原則是：讓 AI 模型知道自己的不確定性，並在不確定時退回到物理模型或交給人員判斷，就像 21.3 節的虛擬量測在不可信時改用實際量測一樣。

21.5.4 第四篇的回顧

本章是第四篇的最後一章。回顧第四篇：第 17 章介紹了企業系統與製造營運管理，說明工廠的資訊如何流動；第 18 章比較了離散型與流程型的智慧工廠；第 19 章建立了數位孿生的基礎；第 20 章以數位主線、孿生組合與語意互通把孿生串起來，並以驗證與治理讓它值得信任；本章則以各種模擬讓孿生「動起來」，在實體存在之前就開始發揮作用。

第五篇將轉向製造資料科學與工業人工智慧：如何從孿生與系統累積的資料中，以統計、機器學習與深度學習找出知識，用於品質、維護與排程。

案例

案例 21-1 教學情境：在虛擬工廠裡先把錯誤犯完

背景：叫獸開講中的自動化整合團隊（綜合情境），第一次專案的現場試車花了將近一個月，發現大量 PLC 邏輯錯誤。

做法：下一個專案在設備出廠前，以輸送帶、感測器、推桿與機器人的行為模型建立虛擬產線，以軟體在迴路的方式連接真正的 PLC 程式；設計包含小間距、讀碼失敗、推桿故障與緊急停止的測試案例，自動化重複執行。

成果：數十個錯誤在虛擬環境中被找到並修正，現場試車時間大幅縮短，客戶的操作人員也提前在虛擬產線上受訓。

啟示：虛擬試車的價值在於系統性地測試邊界條件與異常情境；模型只需要描述控制程式看得、碰得到的部分。

討論：（1）哪些測試案例最容易在實機試車中被忽略？（2）虛擬試車的模型應由誰提供與維護？

案例 21-2 產業：以離散事件模擬規劃新產線

背景：一家電子組裝廠要為新產品建立產線，管理者在「多買一台貼片機」與「增加緩衝區與作

業員」之間猶豫。

做法：以設備規格、歷史故障資料與預估的產品組合建立離散事件模擬，比較多種方案在不同需求情境下的產出、在製品與交期，並以多次重複模擬估計信賴區間。

成果：模擬顯示瓶頸其實在檢測站而非貼片機，調整檢測站的人力與排班即可達到產出目標，延後了貼片機的投資。

啟示：模擬讓投資決策建立在系統行為的分析上；瓶頸常不在直覺認為的地方。

討論：（1）模擬的輸入資料哪些最容易出錯？（2）為什麼要以多次重複模擬估計信賴區間？

案例 21-3 台灣情境：半導體的全自動虛擬量測

背景：半導體晶圓廠的量測站產能有限，大多數晶圓只能抽測；成功大學鄭芳田教授的團隊發展全自動虛擬量測，相關研究發表於 2008 年與 2012 年的 IEEE 期刊。

做法：依公開的研究，全自動虛擬量測以製程資料預測每片晶圓的量測值，並以信心指標等機制自動判斷預測是否可信，在機台保養或製程改變後自動更新模型。

成果：依相關研究，全自動虛擬量測系統曾在台灣的 TFT-LCD 面板廠實作；虛擬量測讓每一片晶圓都有即時的品質預測，可支援逐片的品質監控與製程控制，並減少實際量測的需求。

啟示：虛擬量測實用化的關鍵，在於模型的自動更新與預測可信度的判斷；這也是 AI 模型在製造現場落地的共同課題。

討論：（1）信心指標與第 20 章的確認域有什麼關係？（2）除了半導體，還有哪些產業適合虛擬量測？

案例 21-4 產業：在工廠蓋好之前先開工

背景：依公開報導，BMW 集團於 2023 年 3 月表示，在其匈牙利德布勒森新工廠預定 2025 年啟用的兩年多前，就以 NVIDIA Omniverse 建立整座工廠的虛擬模型。

做法：依報導，世界各地的規劃團隊在共同的虛擬工廠中即時協作，驗證與最佳化佈置、機器人

與物流的規劃，並以擬真的模擬比較不同方案。

成果：BMW 表示這種方式讓規劃更快、更有效率，並把它作為未來工廠規劃的藍圖；該廠已於 2025 年正式啟用並開始量產。

啟示：工業元宇宙最明確的價值，是讓分散的團隊在實體存在之前就能協作與驗證；它建立在 3D 場景、模擬與資料標準的整合之上。

討論：（1）虛擬工廠與完工後的實體工廠如何保持一致？（2）哪些規劃錯誤最適合在虛擬工廠中發現？

案例 21-5 產業：以合成影像訓練瑕疵檢測

背景：一家金屬零件廠想以深度學習檢測表面刮痕與凹痕，但瑕疵品很少，蒐集半年也只有數十張影像。

做法：以零件的 CAD 模型在渲染軟體中產生影像，隨機改變光源、角度、表面紋理，以及刮痕與凹痕的位置、長度與深度，自動產生數萬張帶有標註的影像；先以合成影像預訓練模型，再以少量真實影像微調，並以另一批真實影像確認。

成果：模型在真實影像上的檢出率明顯優於只用少量真實影像訓練的模型；團隊也發現某些反光條件的合成影像不夠真實，持續改進渲染參數。

啟示：合成資料彌補了瑕疵樣本稀少的問題，但必須以真實資料確認；領域隨機化的範圍要涵蓋真實世界的變化。

討論：（1）合成影像中哪些因素最需要隨機化？（2）如何判斷模型是否學到了模擬的「習慣」？

案例 21-6 教學情境：AMR 要買幾台？

背景：一個學生專題團隊協助學校的實習工廠規劃 AMR 物流，老師的直覺是「車越多越快」。

做法：團隊以 21.1.5 節的代理人模擬為基礎，依實習工廠的走道與工作站建立模型，加入裝卸時

間與會車規則，比較不同車隊規模的送達件數與等待時間。

成果：模擬顯示超過某個台數後，產出不再增加，車輛大部分時間在站前排隊；團隊建議先改善裝卸站的作業時間，再決定車隊規模。

啟示：代理人模擬讓湧現的壅塞現象在投資前被看見；瓶頸會隨系統的改變而轉移。

討論：（1）若把裝卸時間縮短一半，最佳車隊規模會如何變化？（2）真實的 AMR 系統還有哪些規則需要加入模型？

實作活動

活動 21-1 排隊模擬

以 21.1.2 節的程式為基礎，擴充為兩個工作站串聯的產線，比較第二站加工時間的變異對整體等待時間的影響，並以多次重複模擬估計 95% 信賴區間。

活動 21-2 系統動態政策實驗

以 21.1.4 節的程式為基礎，改變交貨延遲、庫存調整時間與在途訂單的重視程度，找出讓庫存最快穩定、又不過度波動的參數組合，並以三階延遲與一階延遲比較結果。

活動 21-3 AMR 車隊

以 21.1.5 節的程式為基礎，加入第二個卸貨站與「空車讓重車」的會車規則，觀察最佳車隊規模的變化。

活動 21-4 虛擬試車

以 21.2.4 節的程式為基礎，加入「條碼讀取失敗」與「推桿卡住」兩種故障，修改 PLC 邏輯使其能安全處理，並把所有測試案例寫成自動化的回歸測試。

活動 21-5 虛擬量測

以 21.3.3 節的程式為基礎，比較不同的抽測頻率（每 10、25、50 片）與偏差修正的平滑係數，

畫出 RMSE 與量測成本的權衡曲線。

活動 21-6 OpenUSD 與 Sim-to-Real

以 21.4.3 節的程式為基礎，為你的實驗室建立 OpenUSD 場景並加入第二個規劃變更圖層；再以 21.5.2 節的程式為基礎，改變領域隨機化的參數範圍，觀察範圍太窄或太寬時對真實誤差的影響。相關範例程式將於本書的實驗倉庫中維護。

延伸閱讀

- Jay W. Forrester · Industrial Dynamics (MIT Press · 1961)：系統動態的開山之作。
- Averill M. Law · Simulation Modeling and Analysis：離散事件模擬的經典教材。
- VDI / VDE 3693 系列準則：虛擬試車的名詞、模型類型與導入。
- 鄭芳田等人關於全自動虛擬量測的系列論文，包括〈Developing an Automatic Virtual Metrology System〉(IEEE Transactions on Automation Science and Engineering · 2012)。
- Josh Tobin 等人，〈Domain Randomization for Transferring Deep Neural Networks from Simulation to the Real World〉，IROS 2017。
- Maziar Raissi、Paris Perdikaris 與 George Em Karniadakis，〈Physics-informed neural networks〉，Journal of Computational Physics · 2019。

表 21-8 本章思考地圖

本章概念	關鍵提問	後續延伸章節
四種模擬典範	問題的抽象層級是什麼？該用哪一種或哪幾種模擬？	第 27、29、33 章
虛擬試車	哪些情境在實機上難以測試？模型夠描述控制程式的需求嗎？	第 10、35 章
虛擬量測	預測可信嗎？模型如何隨機台變化更新？	第 22、25 章
AR / VR 與工業元宇宙	沉浸式介面帶來什麼具體價值？虛擬與實體一致嗎？	第 31 章
合成資料與 Sim-to-Real	模擬與真實的落差在哪裡？如何確認？	第 24、30 章

本章概念	關鍵提問	後續延伸章節
AI 驅動學生	AI 模型知道自己的不確定性嗎？超出範圍時怎麼辦？	第 28~30、36 章

章末小結

- 模擬有四種典範：離散事件、物理、系統動態與代理人。GPSS 與《工業動態學》都在 1961 年問世。離散事件模擬的結果與排隊理論相符，並顯示利用率與變異對等待時間的影響；系統動態顯示忽略在途訂單會讓需求小幅上升引發劇烈的庫存波動；代理人模擬顯示 AMR 車隊超過約 11 台後，瓶頸轉移到裝卸站，增加車輛只會增加排隊。
- 在迴路測試分為模型在迴路、軟體在迴路、硬體在迴路與實機試車。虛擬試車以模擬模型代替設備測試控制程式，VDI / VDE 3693 定義了其名詞與模型類型。範例中，間距縮短時暫存器被覆蓋的錯誤只在特定條件下出現，虛擬試車讓這類邊界條件可以被系統性地測試與回歸測試。
- 虛擬機台以模型描述機台行為，用於加工模擬、補償、品質預測與訓練。虛擬量測以製程資料預測測量測值；台灣在全自動虛擬量測上有重要的研究成果。範例中，抽測偏差修正把 RMSE 從 6.53 降到 3.50 Å，可信度檢查攔下流量計故障的晶圓，並避免故障資料污染模型更新，最後降到 2.52 Å。
- AR、VR 與 MR 把學生的資訊帶到需要的人與位置；工業元宇宙整合 3D 場景、模擬、語意與即時資料，最明確的價值在工廠規劃、遠端協作與訓練。以 OpenUSD 組合多個圖層，可以非破壞地協作並自動檢查佈置，範例中規劃變更造成的走道干涉被自動找出。
- 合成資料彌補罕見與危險情境的資料不足，但存在模擬與真實的落差。領域隨機化讓模型學到正確的結構：範例中真實誤差從 5.77 公斤降到 0.58 公斤；但少量真實資料有時更好，最終都必須以真實資料確認。AI 驅動學生包括代理模型、物理資訊神經網路、混合模型與生成式 AI，必須搭配不確定性估計與確認域。

習題

一、選擇題

1. 只在事件發生時更新系統狀態、適合模擬排隊與產線的是？(A) 系統動態 (B) 離散事件模擬
(C) 有限元素分析 (D) 計算流體力學
2. 以存量、流量與回饋迴路描述系統的模擬方法，由誰開創？(A) Geoffrey Gordon (B) Jay
Forrester (C) Joseph Orlicky (D) Michael Grieves
3. 實際的控制程式在電腦上執行，連接受控對象的模型，稱為？(A) MIL (B) SIL (C) HIL
(D) 實機試車
4. 實際的控制器硬體連接即時運算的模型，稱為？(A) MIL (B) SIL (C) HIL (D) AR
5. 專門討論虛擬試車的德國準則是？(A) VDI / VDE 3693 (B) ISA-95 (C) IEC 61131
(D) ISO 23247
6. 以製程與設備資料預測產品量測值的技術是？(A) 虛擬試車 (B) 虛擬量測 (C) 擴增實境
(D) 領域隨機化
7. 在真實世界的影像上疊加數位資訊的是？(A) VR (B) AR (C) DES (D) SD
8. 在模擬中大幅隨機改變不確定的參數，使模型能適應真實世界的方法是？(A) 資料協調 (B)
領域隨機化 (C) 蒙地卡羅積分 (D) 主成分分析
9. 在神經網路訓練中加入物理方程式作為約束的方法是？(A) 物理資訊神經網路 (B) 卷積神經
網路 (C) 決策樹 (D) 支持向量機
10. 依公開報導，BMW 以 NVIDIA Omniverse 預先規劃的新工廠位於哪一國？(A) 德國 (B) 美
國 (C) 匈牙利 (D) 墨西哥

二、簡答題

11. 比較四種模擬典範的描述方式與適用問題，各舉一個製造上的例子。
12. 為什麼離散事件模擬的結果要以多次重複估計信賴區間？

13. 以系統動態的觀點，說明為什麼忽略在途訂單會造成庫存波動。
14. 說明 MIL、SIL 與 HIL 的差異與適用時機。
15. 虛擬試車的模型需要描述哪些部分？為什麼不必有完美的物理保真度？
16. 說明虛擬量測實用化的三個關鍵。
17. 說明模擬與真實落差的來源，以及領域隨機化的原理。
18. AI 驅動孿生有哪些方向？它們共同面臨什麼挑戰？

三、計算題

19. 某工作站每分鐘可加工 1 件，工件平均每分鐘到達 0.8 件，到達與加工時間都是指數分布。平均等待時間為多少？若加工時間固定，平均等待時間為多少？
20. 承上題（指數分布），依利特爾定律，平均排隊件數為多少？
21. AMR 繞行一圈約需 70 步，每個裝卸站每 1,000 步約可服務 154 件，估計產出達到飽和所需的最小車隊規模。
22. 條碼讀取器與光電開關相距 0.7 公尺，輸送速度每秒 0.5 公尺，箱子間距 1.2 秒。兩者之間最多同時有幾個箱子？PLC 至少要能記住幾個箱子的類別？
23. 1,000 片晶圓若全部量測，每片 3 分鐘；改為每 25 片抽測 1 片並以虛擬量測補足，可節省多少量測時間？
24. 以自由度 5 的卡方分布 99.9% 分位數（約 20.5）作為可信度門檻，檢查 1,000 片正常的晶圓，預期會有幾片誤報？
25. 需求每週 120 件、交貨延遲 6 週，在途訂單的目標量為多少？
26. 模擬中摩擦固定為 2.0，真實摩擦為 3.2，真實比模擬高出多少百分比？若領域隨機化的範圍為 1.0~4.0，是否涵蓋真實值？

四、申論與設計題

27. 為一條包含輸送、機器人與檢測站的新產線，規劃虛擬試車的範圍、模型、測試案例與導入步

驟。

28. 評論以下說法：「只要模擬做得夠精細，就不需要真實資料。」
29. 為一家想導入「工業元宇宙」的工廠，評估三個最有價值的應用，並說明所需的資料、模型與標準，以及可能的風險。

五、是非題

30. 系統動態適合追蹤每一個工件的詳細路徑。
31. 軟體在迴路測試可以安全地測試在實體設備上危險的故障情境。
32. 虛擬量測的模型建立之後就不需要更新。
33. 虛擬量測在輸入超出建模範圍時，仍應直接採用其預測值。
34. 以固定參數模擬訓練的模型，在模擬中表現好，不代表在真實世界中也表現好。
35. 合成資料的標註通常是自動產生且精確的。

六、進階挑戰題

36. 以 SimPy 等套件建立一條三站串聯、有故障與換線的產線離散事件模型，結合第 19 章的產線孿生，評估三種改善方案並以統計檢定比較其差異。
37. 以 OPC UA 或 Modbus 模擬伺服器（第 11 章）把 21.2.4 節的 PLC 邏輯與模型分開執行，量測通訊延遲對分流結果的影響，並說明這是 SIL 還是 HIL 的範疇。
38. 以 Python 的渲染或影像處理套件產生帶有刮痕的合成影像，訓練一個簡單的分類器，比較有無隨機化光線與背景時，在少量真實影像上的表現。

七、補充題

39. 列出表 21-7 中模擬與真實落差的五個來源。
40. 列出在迴路測試的四個層級。

提示與參考解答

1. B。
2. B。
3. B。
4. C。
5. A。
6. B。
7. B。
8. B。
9. A。
10. C。
11. 參考：離散事件描述實體在資源間的流動，例如產線產能；物理以微分方程式描述物理現象，例如熱變形；系統動態以存量流量與回饋描述整體，例如供應鏈波動；代理人以個體規則與互動描述湧現行為，例如 AMR 車隊壅塞。
12. 參考：模擬的輸入是隨機的，結果也是隨機的；單次模擬可能偏高或偏低，尤其在高利用率時波動大，需以多次重複估計平均與信賴區間。
13. 參考：訂單有延遲，忽略在途訂單的人看到庫存低就持續多訂，等前面的貨陸續到達時庫存暴增，又大幅減少訂貨，形成反覆的波動。
14. 參考：MIL 控制器與對象都是模型，用於設計初期；SIL 以實際程式連接模型，可大量自動化測試；HIL 以實際控制器硬體連接即時模型，可測試時序、通訊與 I/O。
15. 參考：感測器、致動器、物料流與其他設備的行為。模型的目的是測出控制程式的錯誤，只要能正確產生控制程式看得到的訊號與狀態即可，不需模擬控制程式看不到的細節。
16. 參考：模型持續更新以追上機台變化；判斷預測是否可信，不可信時改用實測；保護更新過程，避免異常資料污染模型。

17. 參考：物理參數、感測器、外觀、未建模現象與時間。領域隨機化在模擬中大幅改變不確定的參數，讓模型學到不受這些參數影響的結構，使真實世界成為其中一種變化。
18. 參考：代理模型、物理資訊神經網路、混合模型、生成式 AI 與基礎模型。共同挑戰是在訓練範圍外可能不可靠、不易解釋，需要不確定性估計、確認域與模型治理。
19. $M / M / 1 : Wq = \rho \div (\mu - \lambda) = 0.8 \div 0.2 = 4$ 分鐘； $M / D / 1 : Wq = \rho \div [2\mu(1 - \rho)] = 0.8 \div 0.4 = 2$ 分鐘。
20. $Lq = \lambda \times Wq = 0.8 \times 4 = 3.2$ 件。
21. 每台每 1,000 步約送達 $1,000 \div 70 \approx 14.3$ 件； $154 \div 14.3 \approx 10.8$ ，約 11 台。
22. 行經時間 $0.7 \div 0.5 = 1.4$ 秒；間距 1.2 秒時最多同時有 $[1.4 \div 1.2] = 2$ 個箱子，至少要能記住 2 個箱子的類別（以佇列實作）。
23. 全量測 3,000 分鐘；抽測 40 片 120 分鐘；節省 2,880 分鐘，即 48 小時。
24. $1,000 \times 0.001 = 1$ 片。
25. $120 \times 6 = 720$ 件。
26. $(3.2 - 2.0) \div 2.0 = 60\%$ ；範圍 1.0~4.0 涵蓋 3.2。
27. 評分重點：範圍與模型的保真度合理；測試案例涵蓋正常、邊界與故障情境；有回歸測試與 SIL、HIL 的分層；導入考慮模型的提供與維護。
28. 評分重點：能指出模擬永遠有未建模的現象與參數不確定，必須以真實資料確認；說明模擬在罕見、危險情境的價值，以及模擬與真實資料結合的做法。
29. 評分重點：應用有明確價值；說明 3D 場景、語意、即時資料與模擬的整合；指出資料一致性、成本、人因與資安等風險。
30. X。系統動態描述整體的存量與流量，追蹤個別工件適合離散事件或代理人模擬。
31. ✓。
32. X。機台會老化、保養與改變，模型需要持續更新。
33. X。超出建模範圍時預測不可信，應改以實際量測或交由人員判斷。

34. ✓。

35. ✓。

36. 評分重點：模型正確反映故障與換線，結果經驗證；方案比較有重複模擬與統計檢定。

37. 評分重點：能量測並說明延遲對分流的影響；能正確區分：控制程式在電腦上執行屬於 SIL，若改用實際控制器硬體則為 HIL。

38. 評分重點：合成影像與標註正確；比較有無隨機化在真實影像上的表現，並討論落差的原因。

39. 物理參數、感測器、外觀、未建模的現象、時間。

40. 模型在迴路、軟體在迴路、硬體在迴路、實機試車。